SKALA

KDT Cloud

쿠버네티스 심화하기

2026.10

본 문서는 SK(주)C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

1일차

1. 실습환경 구성하기
2. k8s 기본 이해
3. label 과 Selector
4. 쿠버네티스 구조 이해
5. 전체 흐름 이해
6. init container
7. side car
8. replicaset 이해

2 일차

1. Persistent Voloume
2. Statefulset
3. Headless Service
4. Configmap
5. Secret
6. Http Ingress 구성하기
7. 무중단 배포하기

Self-Study

1. TLS 보안 메커니즘 이해
2. Job 과 CronJob
3. Graceful Shutdown
4. Pod Lifecycle 관리
5. Service 깊이 이해 하기
6. Pod SecurityContext
7. QoS Class & Priority Class

쿠버네티스 클러스터 접속을 위한 환경 구성

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

확인!!! - 실습 코드 내 {학번} 부분 변경 방법

sk{학번} → **sk000** 형태로 표시해주세요

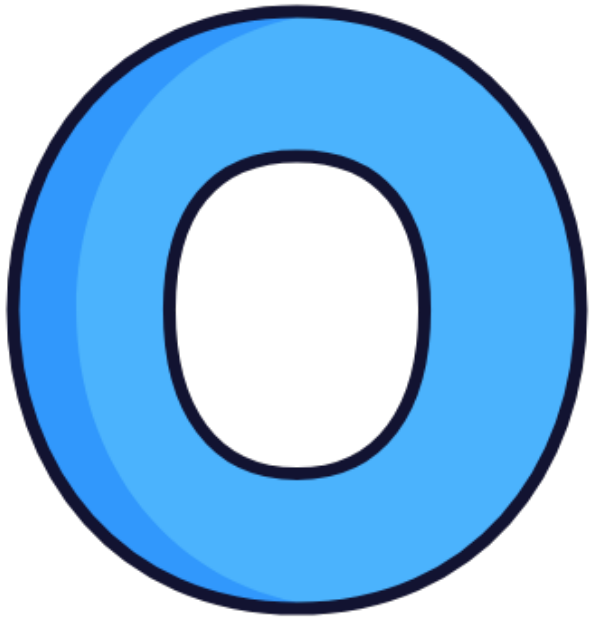
예를 들어, 99 번이면 **sk099**

확인!!! - 이렇게 하지 마세요.

p099 , u099, g099

sk0099

확인!!! - 이렇게 하세요!!!!



소문자 3자리 고유번호

sk099

kubernetes 실습 환경 구성 [판교]

kubernetes 실습을 위한 설치 및 설정 가이드

kubectl 설치

```
brew install kubectl
```

접속 인증 정보 다운로드

```
# 자신의 작업 디렉토리로 이동
# cd ~/workspace/cloud/

# 다운로드
> curl -f -O https://skala-fileserver.skala-ai.com/cloud/k8s-basic/setup-k8s-pg.tar
> tar xvf setup-k8s-pg.tar
```

또는 아래 링크로 접속후 다운로드 : **https://skala-fileserver.skala-ai.com**

kubernetes 인증 설정 확인하기 [판교]

```
# 디렉토리 이동
> cd ./setup-k8s-pg

# 자신의 클래스를 선택
> chmod +x setup.sh

# 자신의 클래스를 선택후 실행
# 1반 : class-1   교육생 번호: 0000
> ./setup.sh class-1  sk199

# kubernetes 접속을 위한 token & CA 정보가 포함되어 있는지 확인하기
> ls ~/.kube/config

# namespace 변경하기
#namespace 는 cloud: class-cloud     skala 1반: class-1,  2반: class-2   3반 : class-3
> kcns    class-1

# kubernetes 접속 인증 확인
> kubectl get serviceaccount

# namespace skala에 배포되어 있는 pod 검색
> k get pod -n skala

# gen-yaml.sh 파일 위치 확인하기
> which gen-yaml.sh
/Users/{계정}/.kube/gen-yaml.sh
```

kubernetes 실습 환경 구성 [울산]

kubernetes 실습을 위한 설치 및 설정 가이드

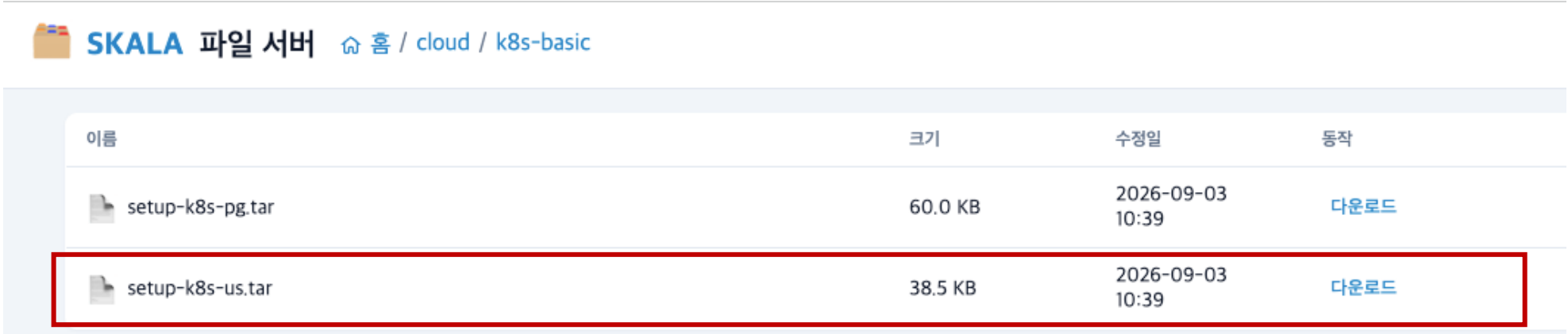
kubectl 설치

```
brew install kubectl
```

접속 인증 정보 다운로드

```
# 자신의 작업 디렉토리로 이동
# cd ~/workspace/cloud/

# 다운로드
> curl -f -O https://skala-fileserver.skala-ai.com/cloud/k8s-basic/setup-k8s-us.tar
> tar xvf setup-k8s-us.tar
```

또는 아래 링크로 접속후 다운로드 : **https://skala-fileserver.skala-ai.com**



kubernetes 인증 설정 확인하기 [울산]

```
# 디렉토리 이동
> cd ./setup-k8s-us

# 자신의 클래스를 선택
> chmod +x setup.sh

# 자신의 클래스를 선택후 실행
# 1반 : class-1   교육생 번호: 0000
> ./setup.sh class-1  sk199

# kubernetes 접속을 위한 token & CA 정보가 포함되어 있는지 확인하기
> ls ~/.kube/config

# namespace 변경하기
#namespace 는 cloud: class-cloud     skala 1반: class-1,  2반: class-2   3반 : class-3
> kcns    class-1

# kubernetes 접속 인증 확인
> kubectl get serviceaccount

# namespace skala에 배포되어 있는 pod 검색
> k get pod -n skala

# gen-yaml.sh 파일 위치 확인하기
> which gen-yaml.sh
/Users/{계정}/.kube/gen-yaml.sh
```

~/.kube/config 인증정보 확인하기

```
# 디렉토리 이동
> cat ~/.kube/config
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0t…klDQVRFLS0tLS0K
    server: https://F6CF0573B26EB0F515847B76DBFD3019.gr7.ap-northeast-2.eks.amazonaws.com
  name: skala-us
contexts:
- context:
    cluster: skala-us
    namespace: class-0
    user: skala-admin-sa
  name: skala-admin-sa@class-0
current-context: skala-admin-sa@class-0
kind: Config
users:
- name: skala-admin-sa
  user:
    token: eyJhbGciOiJSUzI1NiI_K…_ssEarilauQjH6A_mJez2Q
```

실습 코드 다운로드

실습 디렉토리 구성하기

```
# setup reload
> source ~/.zshrc

# home 디렉토리로 이동
> cd

# 작업 디렉토리가 없으면 아래처럼 만들어보자 있다면 스킵
> mkdir workspace
> cd workspace


# 소스 코드 git clone
> git clone https://github.com/himang10/skala-ddive-kube

> cd skala-ddive-kube
```

실습 코드 생성하기 (1/2)

kubernetes 실습 환경으로 들어가기

```
> ls env.properties
> cat env.properties
```

env.properties 내용

```
SERVICE_NAME="myfirst-api-server"

IMAGE_NAME="myfirst-api-server"
VERSION="1.0"

DOCKER_REGISTRY="${DOCKER_REGISTRY}"
CPU_PLATFORM=amd64
USER_NAME=${STUDENT_NUM}
NAMESPACE=${CLASS_NAME}
```

환경 변순 설정 여부 확인

```
> echo $CLASS_NAME
class-?

> echo $STUDENT_NUM
sk0???
```

실습 코드 생성하기 (2/2)

샘플코드를 생성해보자

```
> gen-yaml.sh
```

```
~/mydev/skala/skala-cloud/skala-4/cloud/02.skala-kube/ [master] gen-yaml.s
env.properties 파일: ./env.properties (현재 디렉토리)
=== 템플릿 스크립트 (create 모드) ===
1. 현재 디렉토리 및 하위 디렉토리에서 .t, .j 확장자 파일 검색 중...
발견된 템플릿 파일 개수: 31
  - ./03.kubectl/test-k8s/your-diff-attach-pod.t
  - ./03.kubectl/test-k8s/your-change-nginx-pod.t
  - ./03.kubectl/test-k8s/your-attach-pvc-pod.t
  - ./03.kubectl/common/webserver-service.t
  - ./03.kubectl/common/diff-attach-pod.t
  - ./03.kubectl/common/change-nginx-pod.t
  - ./03.kubectl/common/attach-pvc-pod.t
  - ./03.kubectl/common/resource.t
  - ./03.kubectl/common/webserver-deploy.t
  - ./01.pod/pod.t
  - ./01.pod/pod-with-generateName.t
  - ./01.pod/deploy.t
  - ./06.probe/service.t
  - ./06.probe/deploy.t
  - ./08.mariadb/springboot-k8s/deploy.t
  - ./08.mariadb/custom-values.t
  - ./07.blue-green/service-green.t
```

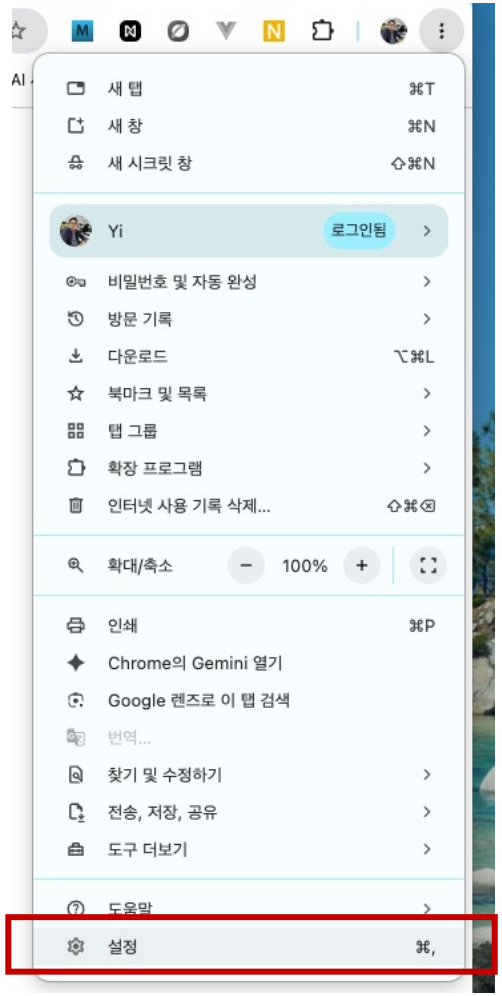
...

```
  ✓ ./05.pvc/deploy.t -> ./05.pvc/deploy.yaml
  ✓ ./02.deploy/service.t -> ./02.deploy/service.yaml
  ✓ ./02.deploy/deploy.t -> ./02.deploy/deploy.yaml
=== 템플릿 결과 파일 생성 완료 ===
31개의 .t/.j 파일이 성공적으로 변환되었습니다.
```

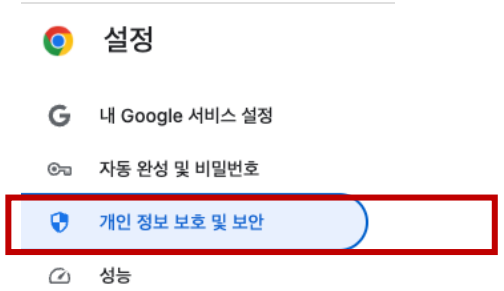
[참고] 예전 화면이 계속 나오는 경우

접속 시 새로운 html이 로딩되지 않을때

1. 브라우저 설정 클릭

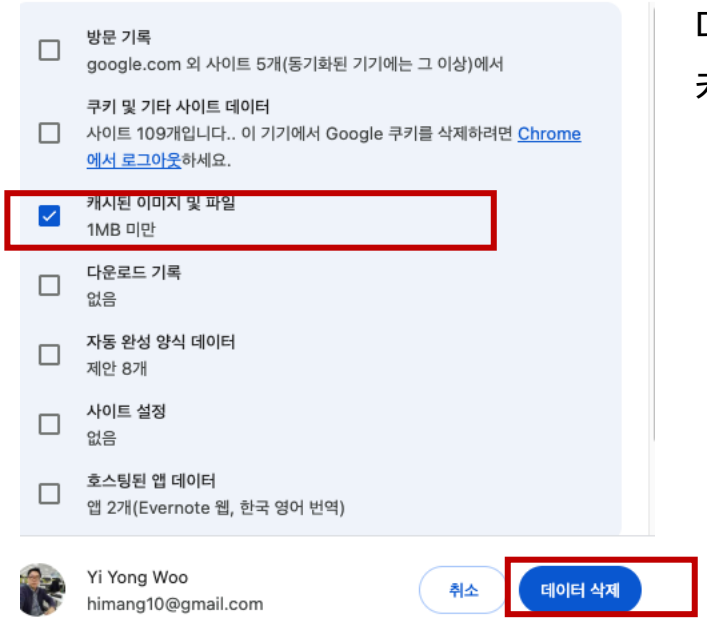


2. 개인정보 보호 및 보안 클릭



3. 인터넷 사용 기록 삭제

4. 캐시된 이미지 및 파일 삭제



다른 설정은 놓아두고
캐시된 이미지 및 파일만 삭제

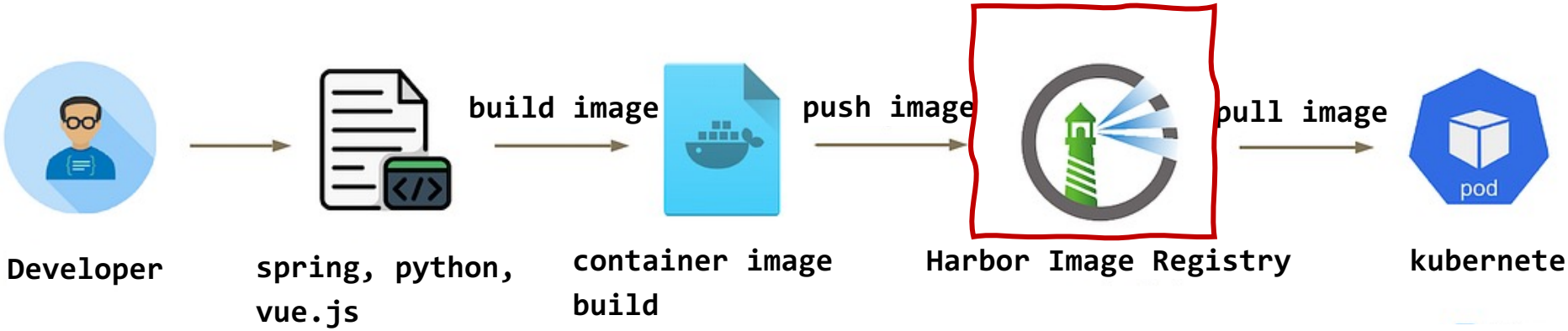
Public Registry에 이미지 Push 하기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

컨테이너 이미지를 Registry 등록하기

로컬에서 build한 container image를 Container Registry에 저장 (push)하기



[실습] 다양한 이미지를 빌드 및 push 해보자

목표: spring, python 기반의 API Server와 Index.html , vue.js 등 Frontend 를 push하기

소스 코드는 00.sample-container

실습 절차

1. Harbor Image Registry 접속해보기
2. 01.spring-backend-v1.0 의 이미지 생성 및 Registry 에 push 하기
3. 02.fastapi-backend-v2.0 의 이미지 생성 및 Registry 에 push 하기
4. 03.frontend 의 이미지 생성 및 Registry 에 push 하기
5. 04.vue-frontend 의 이미지 생성 및 Registry 에 push 하기

[실습] Harbor Registry 접속 하기

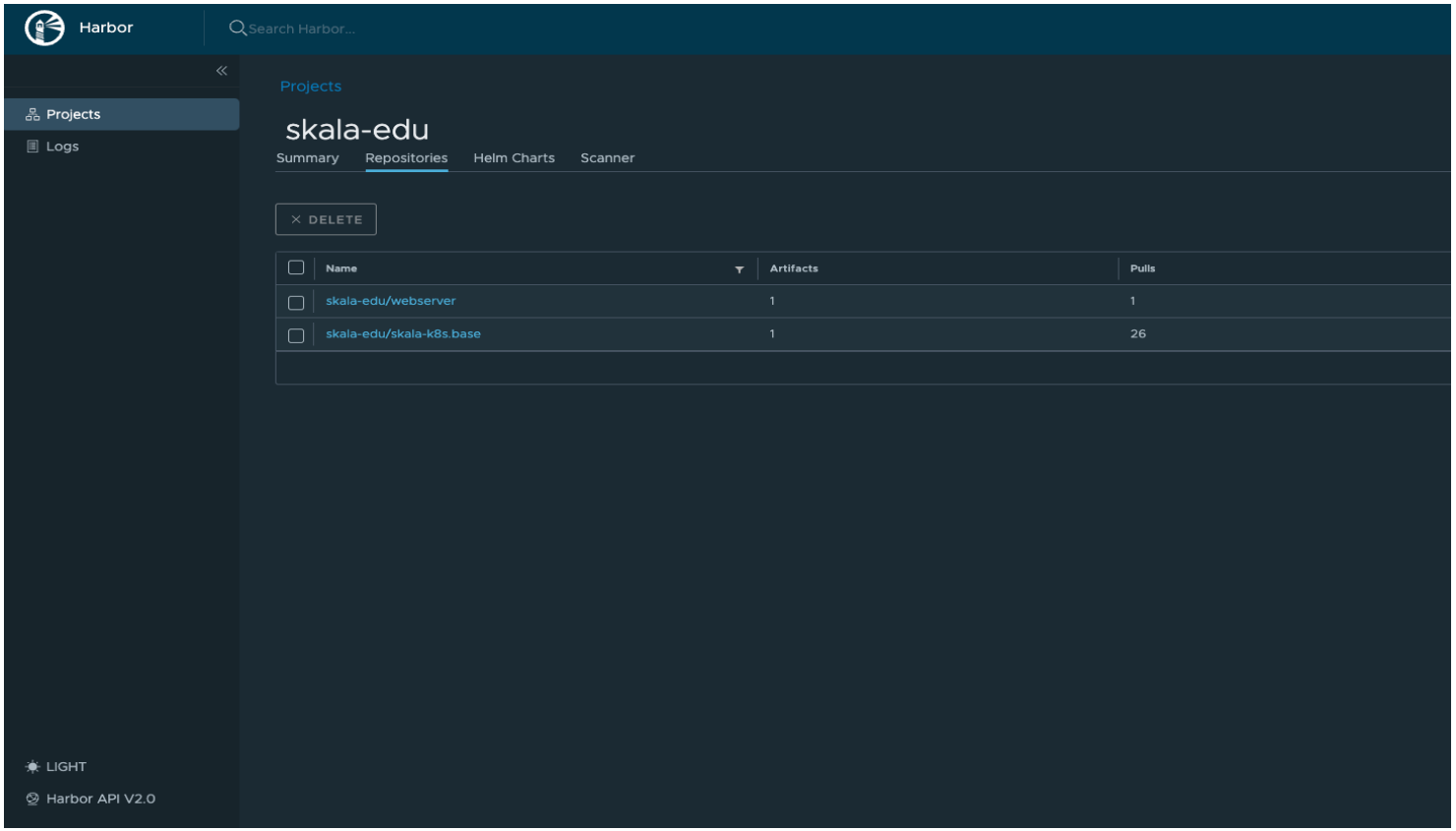
Hobor registry 접속하기

판교
https://skala-registry.skala-ai.com/

울산
https://skala-registry.skala-us.com/

Id: skala26b-st

pwd: Skala26b-st!23$



[코드 리뷰 및 실습] spring 용 Dockerfile

00.sample-container/01.spring-backend-v1.0

spring boot으로 만들어진 코드를 Registry에 push 하기

소스 코드 위치 : 00.sample-container/01.spring-backend-v1.0

--- Dockerfile

```
# FROM openjdk:17-jdk-slim
FROM openjdk:26-ea-21-jdk-slim

# 작업 디렉토리 설정
WORKDIR /app

# 외부에서 컨테이너의 8080 포트에 접근할 수 있도록 설정
EXPOSE 8080

#ENV JAVA_OPTS="-Xms256m -Xmx512m"

# 애플리케이션의 jar 파일을 컨테이너에 추가
ADD ./target/*.jar  app.jar

# 애플리케이션 실행 (JAVA_OPTS 환경변수 적용)
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

[실습] 빌드 및 push 하기

registry login

```
$ echo $DOCKER_REGISTRY_PASSWORD
oiPrruKtwhievCN1jFOLeZtl3Ge7xxx

$ docker login "skala-registry.skala-ai.com" -u "robot\$skala26b"
Password:
```

복사하기

1. 이미지 빌드

```
$ docker buildx build -t sk000-webserver:1.0 --platform linux/amd64,linux/arm64 .
```

2. 이미지 push하기

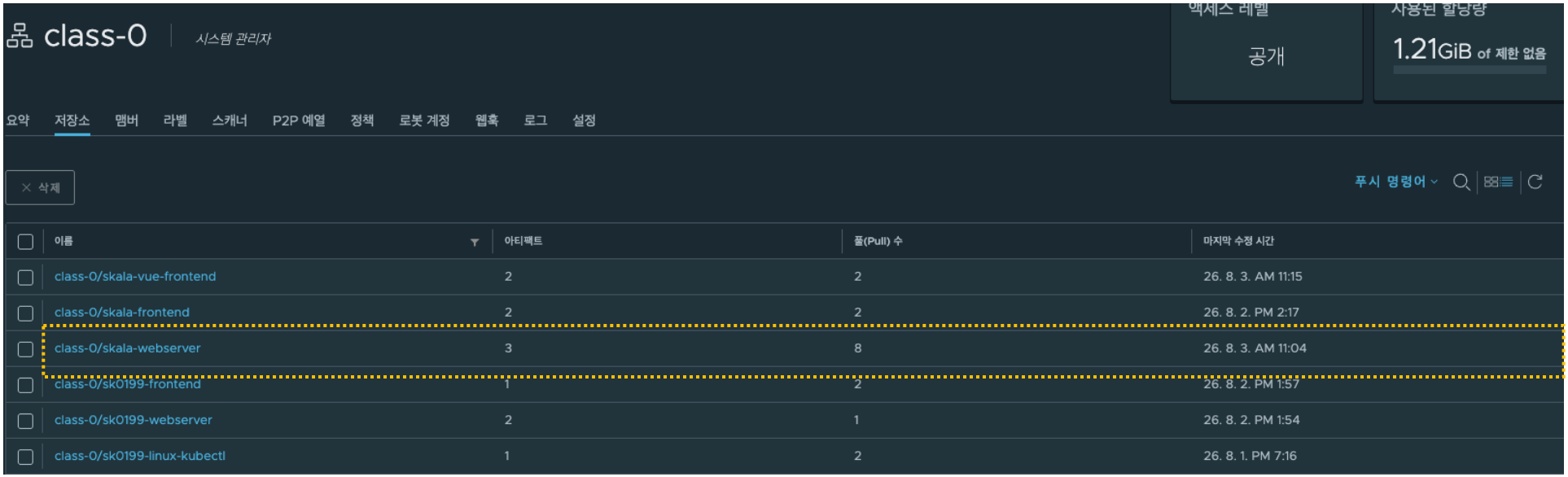
```
$ docker tag sk000-webserver:1.0 skala-registry.skala-us.com/class-0/sk000-webserver:1.0
$ docker push skala-registry.skala-us.com/class-0/sk000-webserver:1.0
```

빌드/푸시 한번에 하기

```
$ docker buildx build \
  --tag skala-registry.skala-us.com/class-0/sk000-webserver:1.0 \
  --platform linux/amd64,linux/arm64 \
  --push .
```

[실습] Harbor Registry 접속 후 생성 이미지 확인하기

Harbor registry 접속 후 생성 이미지 확인



[실습] script로 이미지 빌드하기

이미지 빌드 : docker-build.sh

```
#!/bin/bash
USER_NAME=${STUDENT_NUM}
IMAGE_NAME="webserver"
VERSION="1.0"
#IS_CACHE="--no-cache"

# Docker 이미지 빌드
docker buildx build \
  --tag ${USER_NAME}-${IMAGE_NAME}:${VERSION} \
  --file Dockerfile \
  --platform linux/amd64,linux/arm64 \
  ${IS_CACHE} .
```

```
### 1. docker-build.sh 실행해서 빌드
$./docker-build.sh
[+] Building 1.5s (13/13) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 853B
 => [linux/arm64 internal] load metadata for docker.io/library/openjdk:26-ea-21-jdk-slim
 => [linux/amd64 internal] load metadata for docker.io/library/openjdk:26-ea-21-jdk-slim
 => [auth] library/openjdk:pull token for registry-1.docker.io
```

[실습] script로 이미지 푸시하기

2. Harbor 에 이미지 push하기 : docker-push.sh

```
#!/bin/bash

 USER_NAME=${STUDENT_NUM}
 IMAGE_NAME="webserver"
 VERSION="1.0"
 PROJECT_NAME=$CLASS_NAME

DOCKER_REGISTRY=${DOCKER_REGISTRY}        # "skala-registry.skala-us.com"

 # 1. Docker 레지스트리에 로그인 (옵션: 이 스크립트를 실행하기 전에 미리 로그인해두어도 됩니다)
 echo ${DOCKER_REGISTRY_PASSWORD} | docker login ${DOCKER_REGISTRY} -u ${DOCKER_REGISTRY_USER}  --password-stdin \
     || { echo "Docker 로그인 실패"; exit 1; }

 # 2. harbor 로 push 하기 위해 tag 추가
 docker tag  ${USER_NAME}-${IMAGE_NAME}:${VERSION} ${DOCKER_REGISTRY}/${PROJECT_NAME}/${USER_NAME}-${IMAGE_NAME}:${VERSION}

# Docker 이미지 푸시
 docker push ${DOCKER_REGISTRY}/${PROJECT_NAME}/${USER_NAME}-${IMAGE_NAME}:${VERSION}
```

```
> ./docker-push.sh
Login Succeeded
The push refers to repository [skala-registry.skala-ai.com/class-0/sk0199-webserver]
2f866615507c: Mounted from class-0/skala-webserver
a16e55119267: Layer already exists
…
b1b9f4a73dc1: Pushed
1.0: digest: sha256:15683b1c8b10bc90b7af3546d8bccdc2db381a8a1a52eb9ad632dc40042dec52 size: 1609
```

[실습] 로컬에서 실행해보기

로컬에서 실행해보기

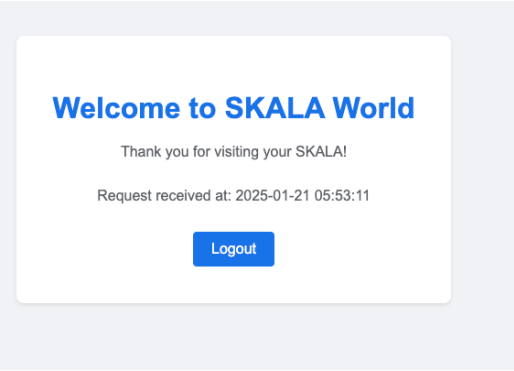
1 컨테이너 실행

```
$  docker run --rm -it --name webserver -p 8888:8080 sk{학번}-webserver:1.0
Starting server on port 8080...
Access the server at http://localhost:8080/login
```

2 PORT Tab 선택

http://localhost:8888 접속

3. “welcome to SKALA World 가 뜹니다



실습이 끝나면

CTRL + C 를 해서 실행중인 Conainer를 종료 시켜주세요

[코드 리뷰 및 실습] python용 Build/Push

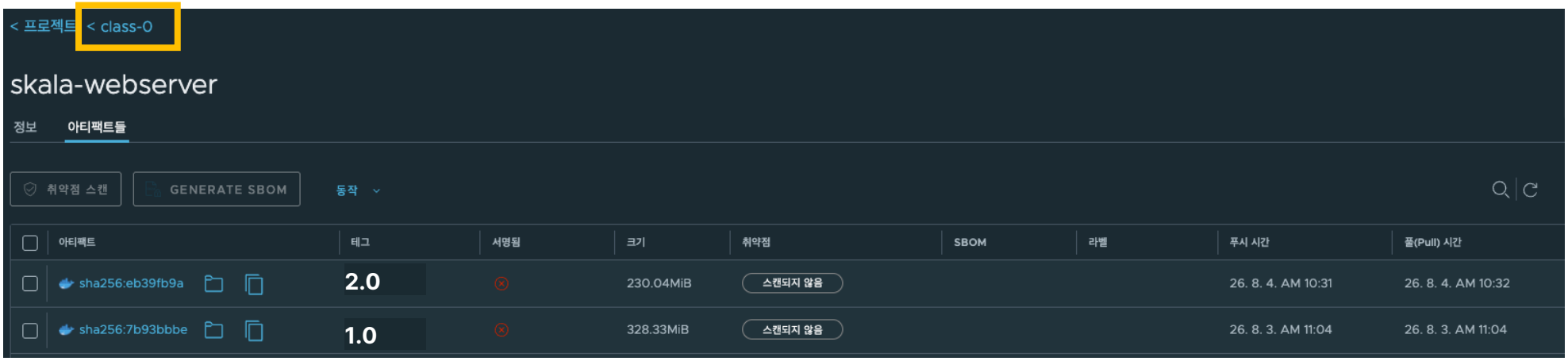
pythond fastapi로 만들어진 코드를 Registry에 push 하기

소스 코드 위치 : 00.sample-container/02.fastapi-backend-v2.0

1. Dockerfile 검토 하기
2. docker-build.sh , docker-push.sh 실행하기
3. skala-registry.skala-us.com 접속하기
   - 프로젝트: {자신의 클래스} # 예 1반은 class-1, 10반은 class-10

실습용 컨테이너 확인하기

아래의 sk{학버}-webserver 1.0 과 2.0 이 있는지 확인한다.



없다면

00.sample-container 아래의 디렉토리에서 이미지를 build & push

- 01.spring-backend-v1.0
- 02.fastapi-backend-v2.0

[코드 리뷰 및 실습] frontend 용 Build/Push

nginx 기반의 frontend를 Registry에 push 하기

소스 코드 위치 : 00.sample-container/03.frontend

1. Dockerfile 검토 하기
2. docker-build.sh , docker-push.sh 실행하기
3. skala-registry.skala-us.com 접속하기
   - 프로젝트: {자신의 클래스} # 예 1반은 class-1, 10반은 class-10

[코드 리뷰 및 실습] vue.js 용 Build/Push

nginx 기반의 vue.js를 Registry에 push 하기

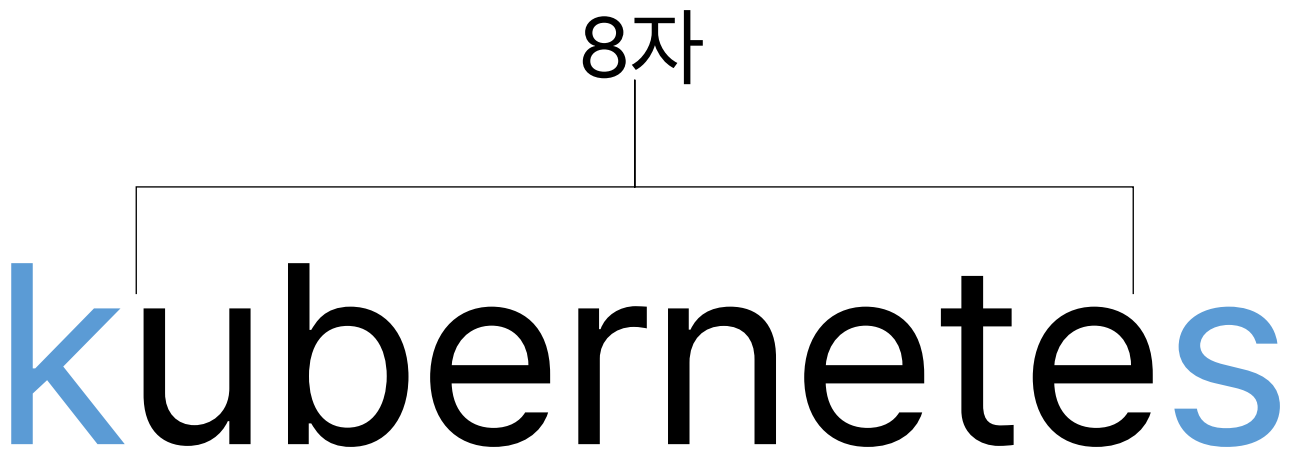
소스 코드 위치 : 00.sample-container/04.vue-frontend

1. Dockerfile 검토 하기
2. docker-build.sh , docker-push.sh 실행하기
3. skala-registry.skala-us.com 접속하기
   - 프로젝트: {자신의 클래스} # 예 1반은 class-1, 10반은 class-10

쿠버네티스 리마인드

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C



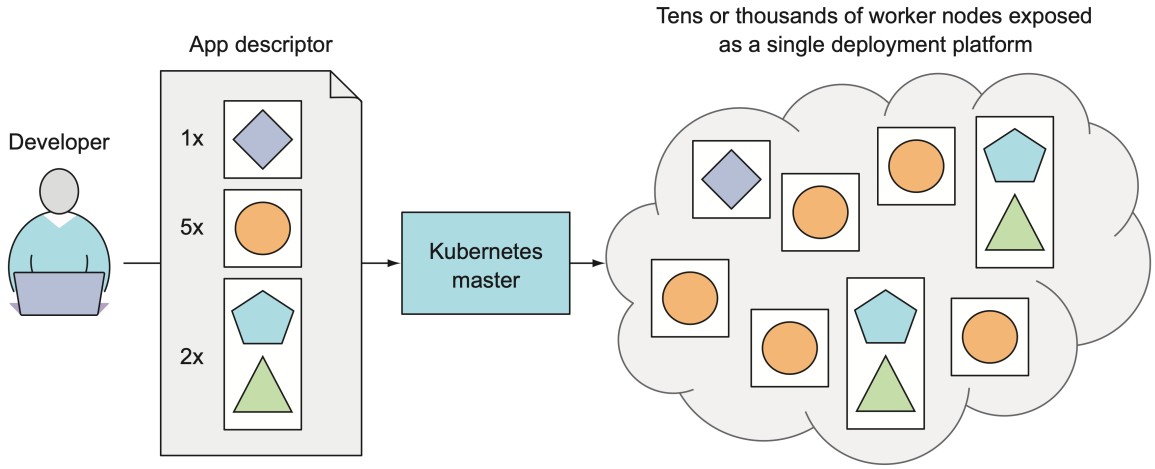
k8s

애플리케이션을 위한 인프라 구성 요청서 .. 그리고 유지

쿠버네티스는 전체 물리 서버를 하나의 배포 가능 플랫폼으로 논리적으로 통합하여 사용자에게 제공

요청 선언/명세서(Manifest File)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 5 # 5개의 Pod가 있어야 함
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.21.6 # 컨테이너 이미지
        ports:
        - containerPort: 8080
```



요청 선언서는 Kubernetes에게 무엇을 해야 하는지를 정의하는 문서
주황색 image를 namespace “nginx-deployment”에 5개 만들어줘
그리고 port는 8080으로 열여줘

kubernetes-in-action book 참고

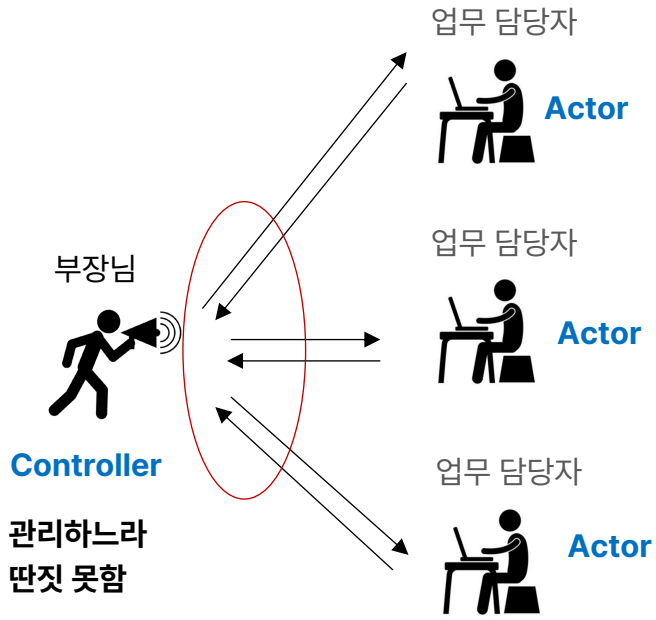
쿠버네티스의 선언적 방식

Kubernetes의 핵심은 기존의 명령적 (imperative) 방식이 아닌

선언적 (Declarative) 운영 방식을 지원한다고 정의한다.

쿠버네티스의 선언적 방식

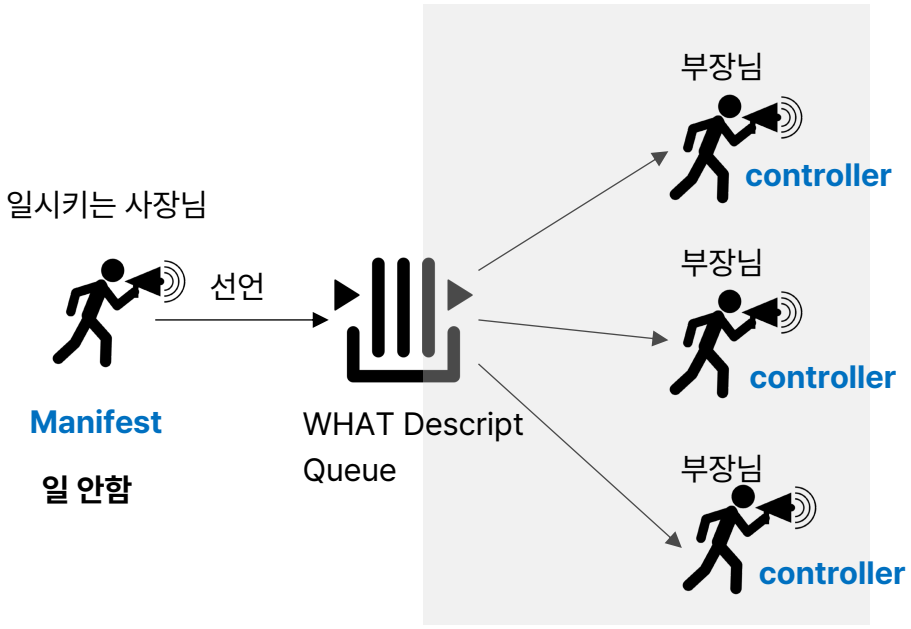
명령적 (Imperative)



업무를 지시하고 계속해서 지켜보면서 조율해야 함
문제가 발생하면 업무 재할당, 업무 담당자 추가, 보고 등

부장님은 무슨일이 있어도 일을 제대로 수행해야 함

선언적 (Declarative)



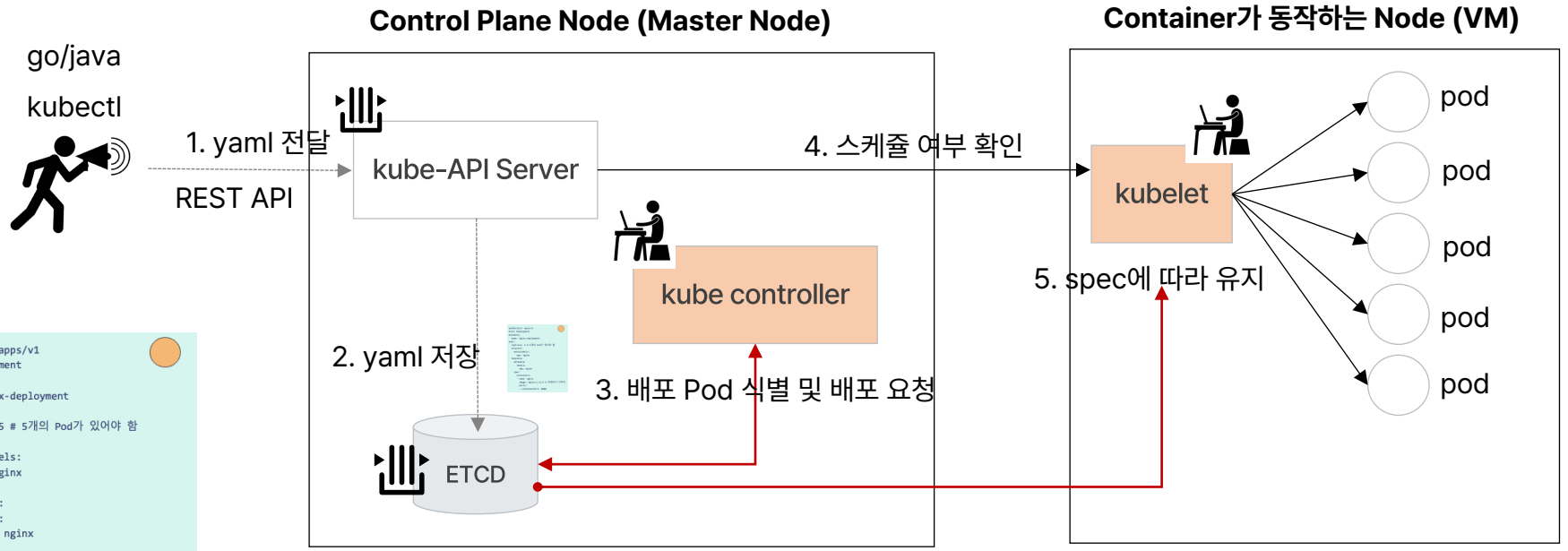
Manifest는 명세서 (job descriptor)를 전달하고
더이상 신경쓰지 않는다 (Controller가 알아서 운영)

요청자는 지시받은 업무는 무슨일이 있더라도 지켜야함

kubernetes Manifest, Tekton, ArgoCD

쿠버네티스 선언적 구조 동작 방식

쿠버네티스 Manifest 작성 후 REST API로 전달 (kubectl CLI 이용) 시 내부 controller가 중심이 되어 지속적으로 운영




```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 5 # 5개의 Pod가 있어야 함
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.21.6 # 컨테이너 이미지
        ports:
        - containerPort: 8080
```

명세서 (Manifest)

ETCD 내에 Manifset 가 저장되면 이것을 기준으로
Kubernetes는 SPEC을 유지하기 위해 계속해서 알아서 일한다

Kubernetes의 SPEC과 STATUS 설계 철학

- SPEC: 사용자가 쿠버네티스에게 "이렇게 되기를 원한다"고 선언하는 목표/요구 사항(Desired State)
- STATUS: 쿠버네티스 시스템이 관찰하고 계산한 현재 상태 (Current State)

무엇을 원하는지 선언 (deploy.yaml)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3 # 원하는 상태: 3개의 Pod가 있어야 함
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.21.6 # 원하는 컨테이너 이미지
        ports:
        - containerPort: 80
```

manifest의 spec 정의

확인

Kubernetes Deployment Controller

Sync

status 가 spec을 따라 유지되는지 확인

spec으로 따르도록 지속적 조정

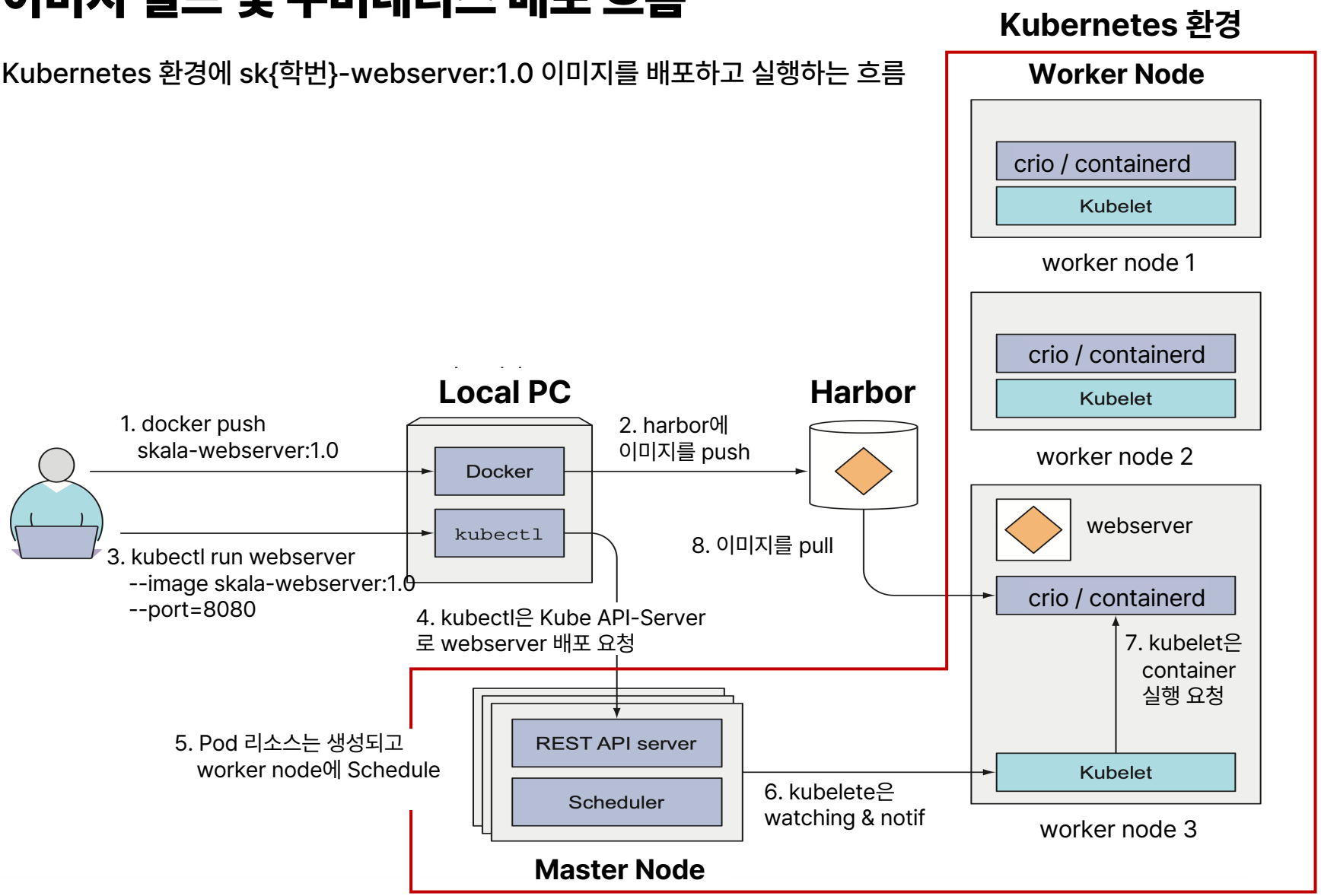
쿠버네티스 내부

```yaml
status:
  availableReplicas: 3 # 현재 가용한 Pod수
  readyReplicas: 3 # 현재 준비된 Pod수
  replicas: 3 # 전체 생성된 Pod수
  updatedReplicas: 3 # 최신템플릿으로 업데이트된 Pod수
  conditions:
  - type: Available
    status: "True"
    lastUpdateTime: "2025-01-16T10:00:00Z"
  - type: Progressing
    status: "True"
    lastUpdateTime: "2025-01-16T10:00:00Z"
```

manifest 의 status

이미지 빌드 및 쿠버네티스 배포 흐름

Kubernetes 환경에 sk{학번}-webserver:1.0 이미지를 배포하고 실행하는 흐름

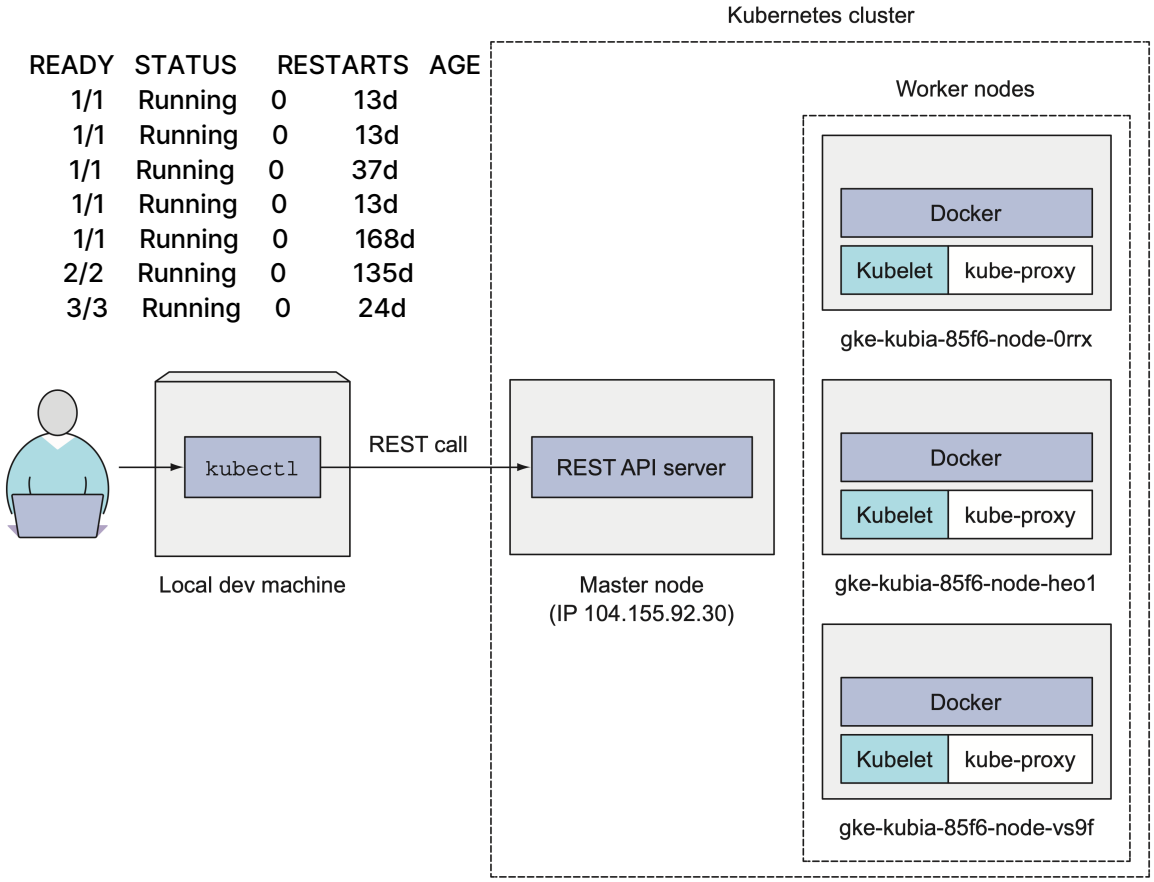


kubernetes-in-action book 참고

kubectl 검색

$ kubectl get pod –n class-{자신의 반 번호}

NAME	READY	STATUS	RESTARTS	AGE
devplace-app-proxy-deployment-6777cd97dc-rfbg4	1/1	Running	0	13d
frontend-rde-deployment-9cb4c44cd-9lr9t	1/1	Running	0	13d
new-rde-proxy-deployment-6766945489-b6mg7	1/1	Running	0	37d
new-rdeconfig-operator-55f89bb4fd-6cld8	1/1	Running	0	13d
pdf-prompt-ollama-deployment-c86d567f9-jmbh6	1/1	Running	0	168d
ssh-for-rde-new-6879d89f96-f6p6z	2/2	Running	0	135d
ywyi-arm64-extend-rde-0	3/3	Running	0	24d

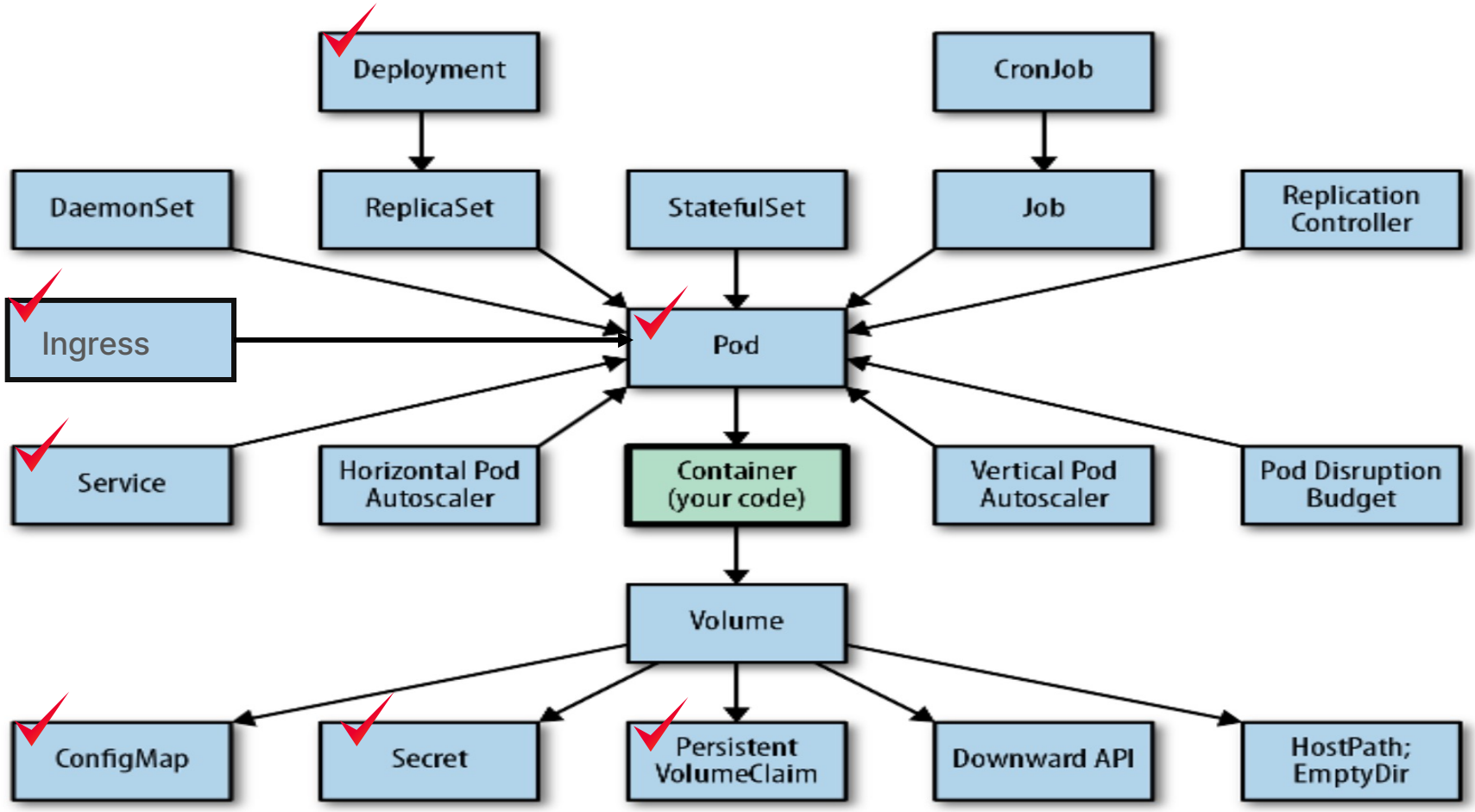


-

kubernetes-in-action book 참고

개발자가 알아야할 K8S Resource 유형

개발자가 알아야할 쿠버네티스 리소스 유형 종류



POD

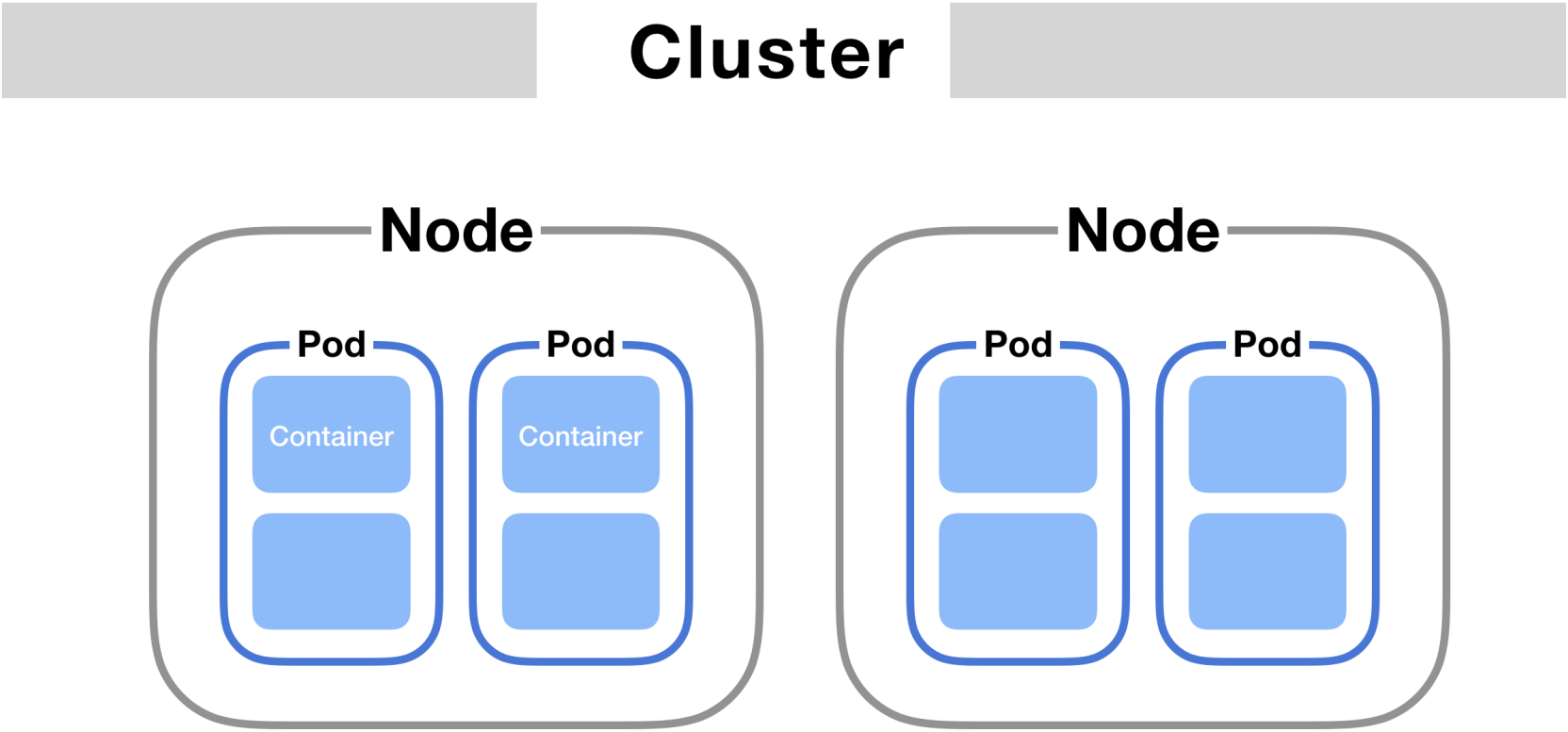
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

POD 정의

k8s에서 배포·실행되는 가장 작은 단위로, 하나 이상의 컨테이너를 묶어 동일한 실행 환경에서 함께 실행

컨테이너를 감싸는 실행 단위이자, **스케줄링의 최소 단위**



Pod yaml 구조

Pod는 하나의 독자적인 실행 단위이며, kubernetes에서 관리하는 최소 단위

skala-kube/01.pod/webser-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sk0199-webserver
  namespace: class-0
  labels:
    app: sk0199-webserver
spec:
  containers:
  - name: webserver
    image: skala-registry.skala-ai.com/skala25a/sk0199-webserver:1.0
    imagePullPolicy: Always
    env:
    - name: USER_NAME
      value: sk0199
    - name: CLASS_NAME
      value: class-0
```

POD 정의

1. 컨테이너 실행의 최소 단위

- k8s는 컨테이너를 직접 실행하지 않고 Pod를 실행
- Scheduler는 Pod를 기준으로 노드에 배치

```
Node ← Pod ← Container
```

2. 네트워크 공유

- Pod 단위로 IP 하나
- 모든 컨테이너가 동일한:
  - ✓ IP
  - ✓ Port 공간
  - ✓ localhost

3. 컨테이너 재시작 ≠ Pod 재생성

- 노드에서 실행은 컨테이너 단위
- 컨테이너가 죽을 경우 kublet이 컨테이너 재시작
  - IP, Storage는 변경되지 않음
- Pod 삭제 시 Pod 새로운 Pod 생성

4. 컨테이너 간 강결합 실행

- 같은 Pod 안 컨테이너들은 항상 함께 배포/종료
- 한 애플리케이션을 구성하는 컨테이너 묶음

5. 스토리지 볼륨 공유

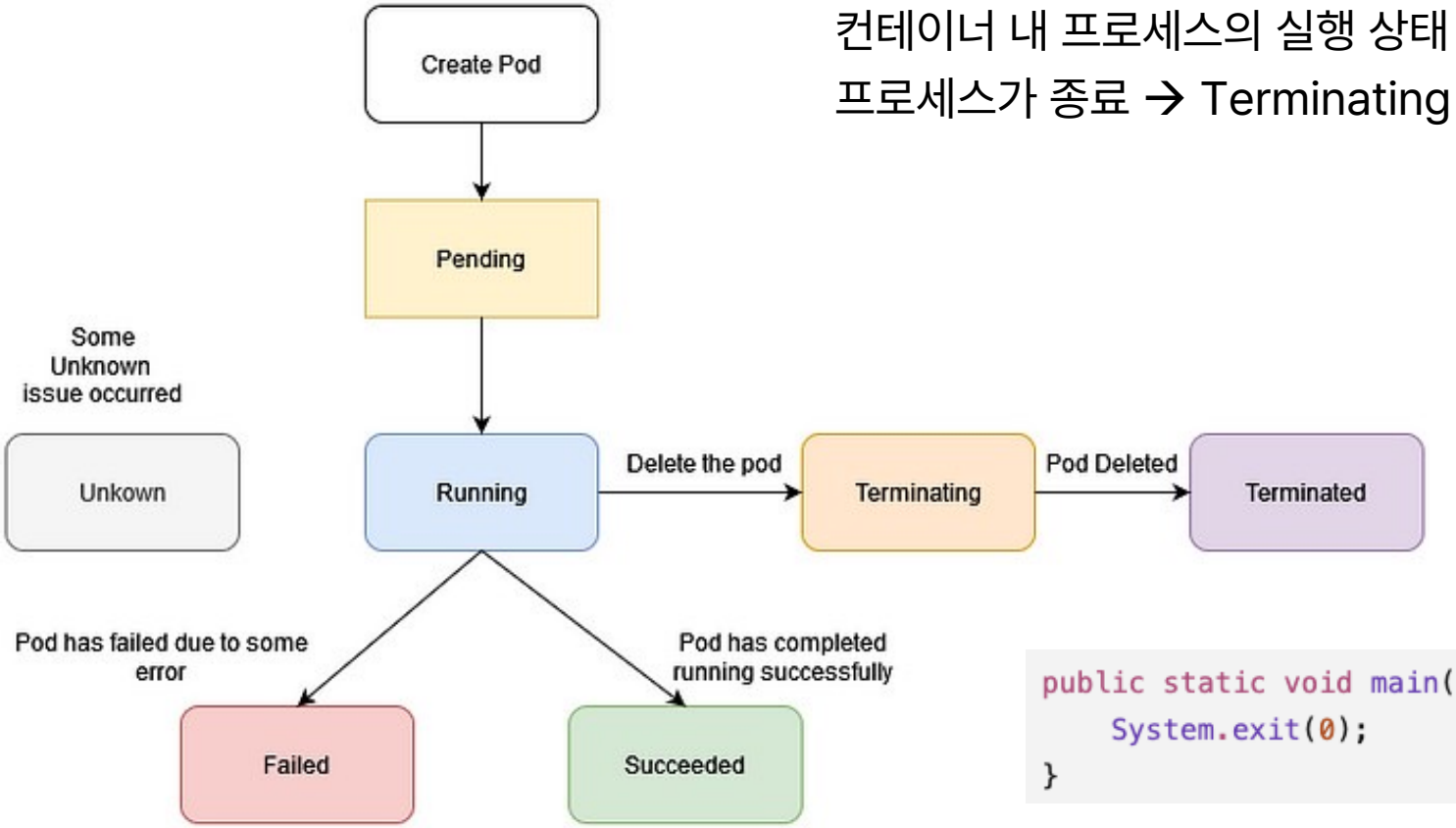
- Pod에 정의된 Volume을 모든 컨테이너가 공유 가능
- 컨테이너간 파일 교환 가능

```
volumes:
- name: shared-data
  emptyDir: {}
```

6. Pod IP는 불안정

- Pod 재 생성 시 IP 변경
- Pod IP를 직접 호출은 금물
- 서비스의 존재 이유

Pod 라이프사이클



exitCode=0

exitCode !=0

컨테이너 내 프로세스의 실행 상태

프로세스가 종료 → Terminating

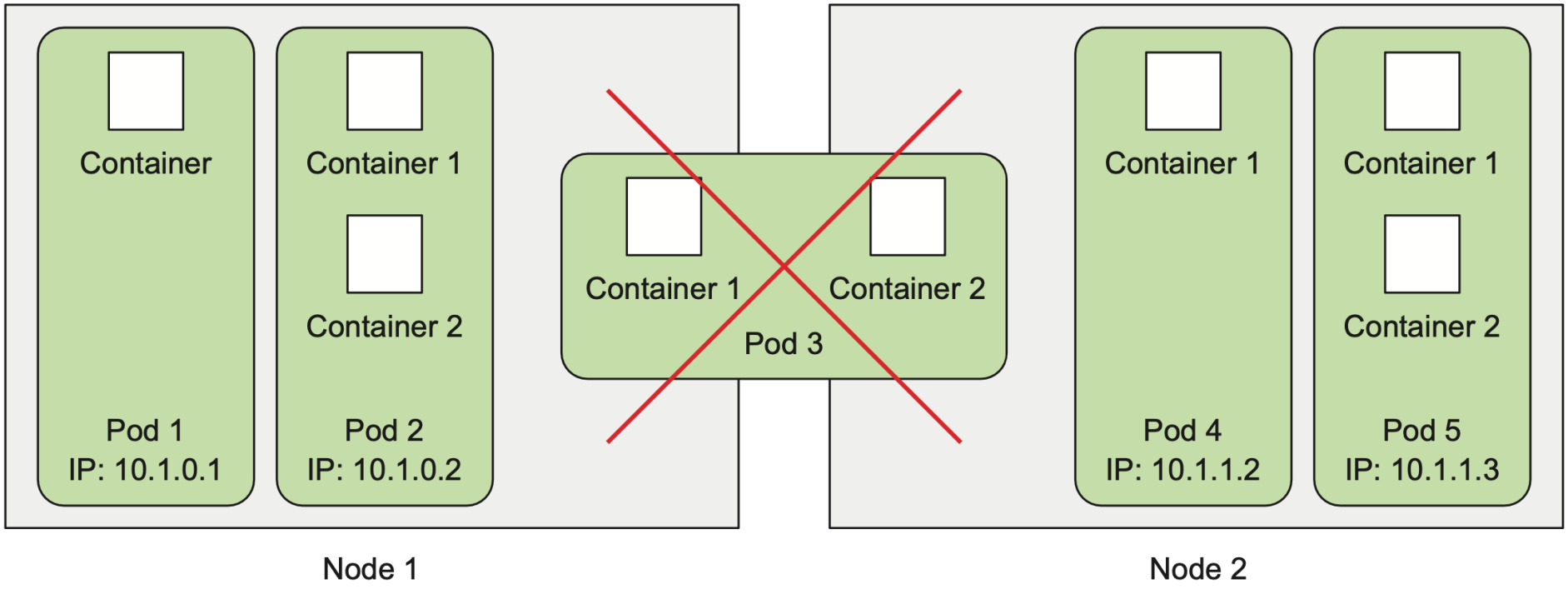
```
public static void main(String[] args) {
    System.exit(0);
}
```

```
System.exit(1);
```

SIGKILL : 137

POD 의 실행 범위

Pod는 컨테이너 실행을 위한 명세이며, kubelet은 Container 중심으로 실행 및 관리



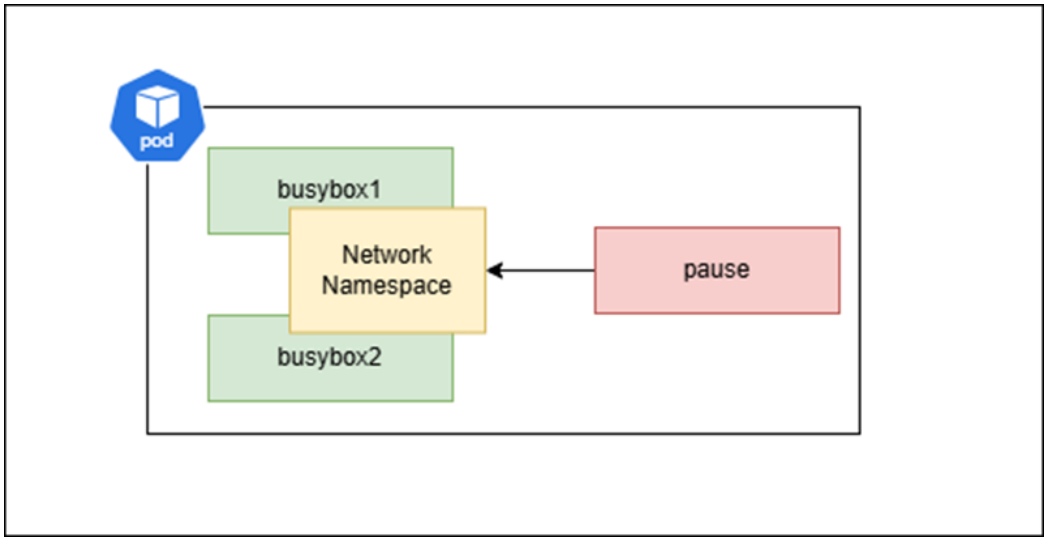
POD 내 컨테이너 구조

- Pod는 밀접하게 연관된 컨테이너(프로세스)를 함께 실행하고 마치 하나의 환경에서 동작하는 것처럼 보임
- 그러나 동일한 환경을 제공하면서 다소 격리된 상태로 유지

pause container

아무일도 하지 않는 컨테이너

- Pod 전용 Linux namespace 생성
- 네트워크 인터페이스(veth) 연결
- Pod IP 유지
- 다른 컨테이너들이 setns()로 들어올 수 있는 기준점 제공

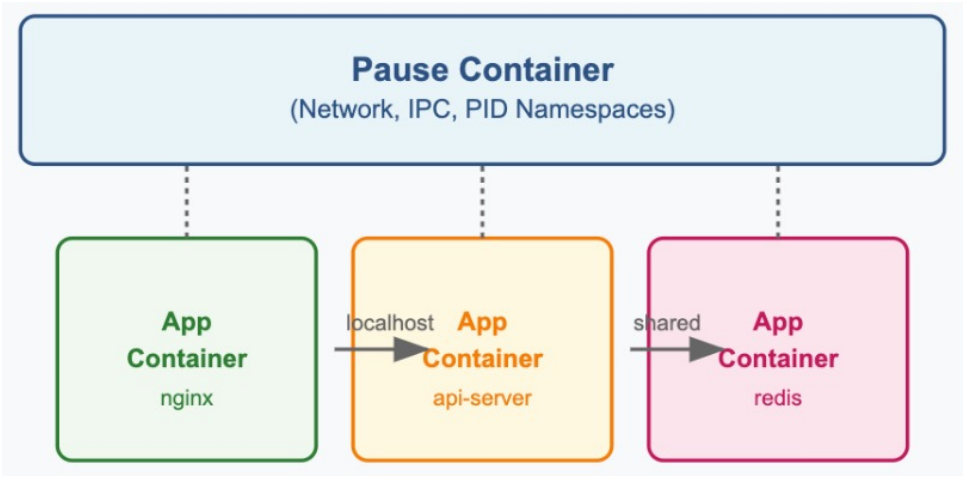


pause 목적

- namespace 수명 = Pod 수명
- 컨테이너 수명과 분리

POD 내 컨테이너 공유 네임스페이스

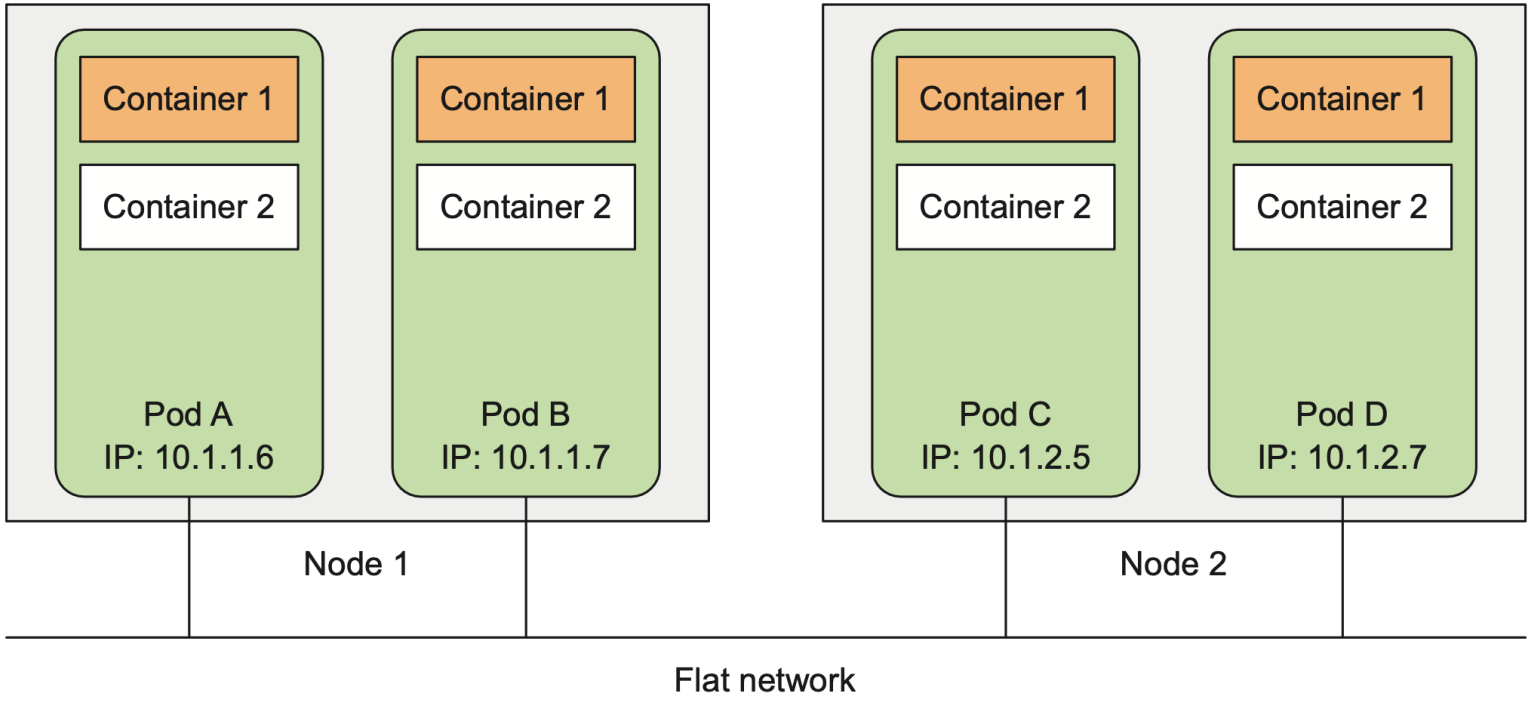
네임스페이스	공유 여부	비고
Network (net)	**공유 (O)**	동일한 IP, 포트 공간, localhost 통신 공유 (Pause 컨테이너 기준)
UTS (Hostname/Domain)	**공유 (O)**	호스트네임(기본 Pod 이름) 및 도메인명 공유
IPC (ipc)	**공유 (O)**	공유 메모리(POSIX/SysV), 세마포어 등 공유 가능
PID (pid)	**격리 (X)**	기본적으로는 각자 분리. spec.shareProcessNamespace: true 설정 시 공유 가능
Mount (mnt)	**격리 (X)**	기본 컨테이너 루트 파일시스템은 분리. volumeMounts를 통해서만 파일/디렉터리 공유
User (user)	**격리 (X)**	개별 UID/GID 매핑 격리 (User Namespaces 기능 활용 시)



POD 네트워크 구조

- 쿠버네티스 클러스터의 모든 포드는 공유된 단일 플랫, 네트워크 주소 공간에 위치
- 파드 사이에는 NAT 게이트웨이가 존재 하지 않음
- 파드내 컨테이너 간 통신은 local host 기반을 통신하면 모든 자원을 공유

모든 Pod IP는 L3 로 동일 subnet에 존재하는 것처럼 보인다.



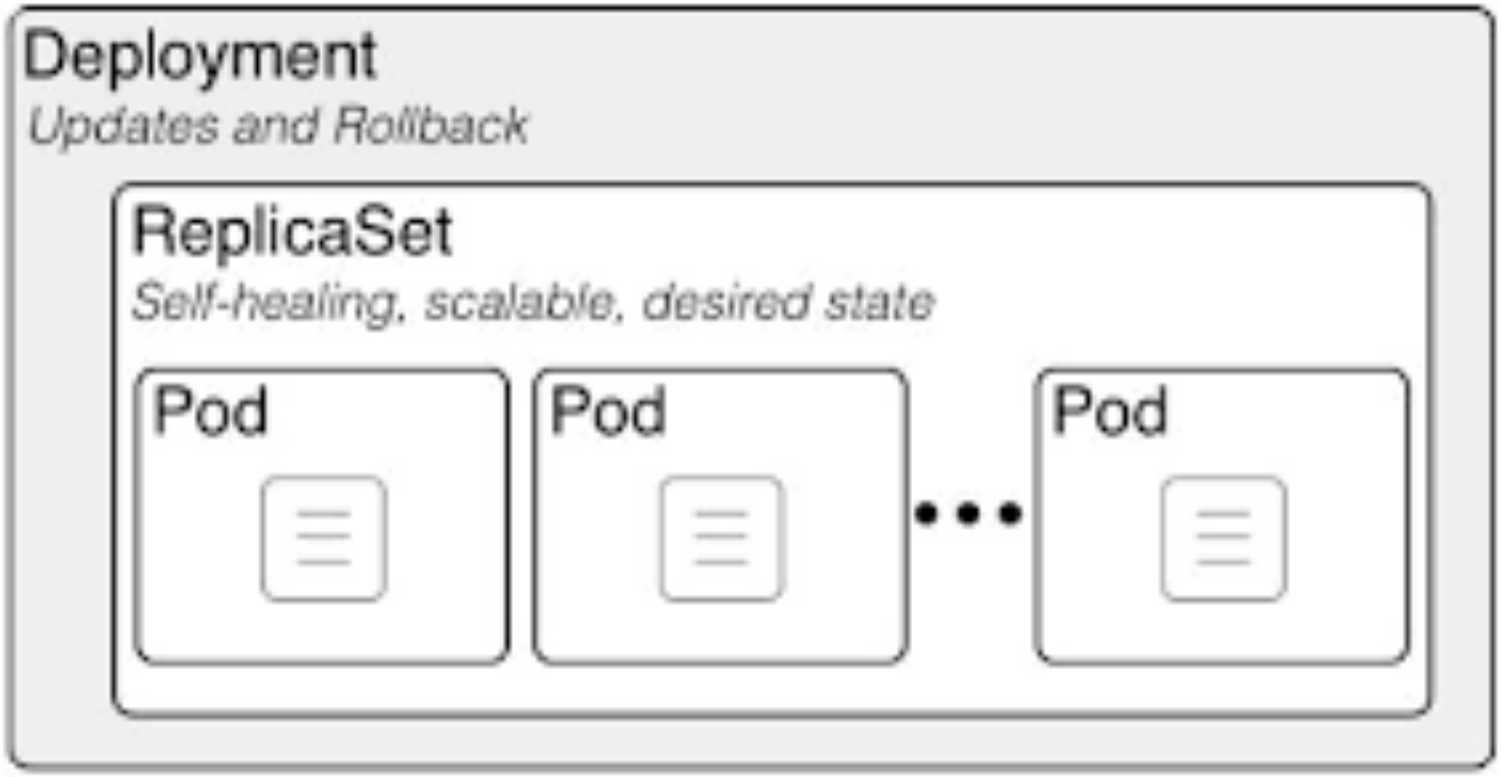
Deployment

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Deployment

애플리케이션의 원하는 상태(desired state)를 선언하면, 그 상태를 지속적으로 유지·업데이트해주는 컨트롤러




```
Deployment
 └─ ReplicaSet v1 (replicas=3)
```

```
Deployment
 ├─ ReplicaSet v1 (replicas=3 → 2 → 1 → 0)
 └─ ReplicaSet v2 (replicas=0 → 1 → 2 → 3)
```

Deployment

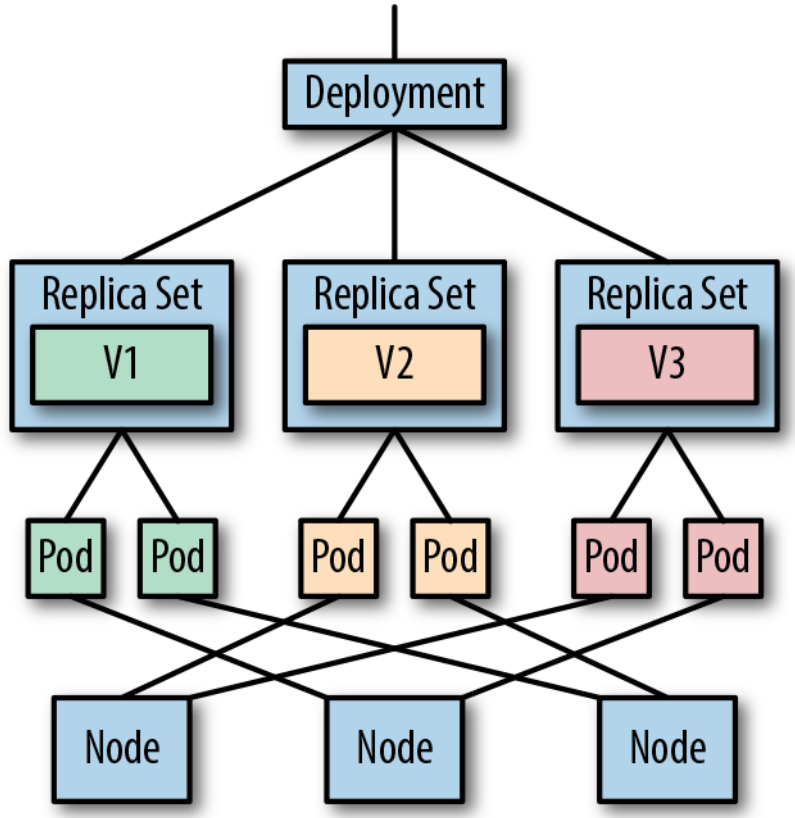
새로운 이미지를 가지는 동일한 Deployment를 배포하는 경우 기존 운영중인 Replica Set Pod와 신규 버전 Replica Set Pod가 점진적 업데이트되는데 이를 통합 관리하는 것이 Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-replicaset-test
  namespace: skala-practice
spec:
  replicas: 2
  selector:
    matchLabels:
      app: sk000-replicaset-test
  template:
    metadata:
      labels:
        app: sk000-replicaset-test
    spec:
      serviceAccountName: default
      containers:
      - name: webserver
        image: xxx/sk000-webserver:1.0.0
        imagePullPolicy: Always
```

2.0.0

update 시

New ReplicaSet 생성



Deployment, ReplicaSet, Pod

Deployment

name: sk000-repliaset-test

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-replicaset-test
  namespace: skala-practice
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sk000-replicaset-test
  template:
    metadata:
      labels:
        app: sk000-replicaset-test
    spec:
      serviceAccountName: default
      containers:
      - name: webserver
        image: xxxx/sk000-webserver:2.0.0
        imagePullPolicy: Always
        env:
        - name: LOGGING_LEVEL
          value: DEBUG
        - name: USER_NAME
          value: sk000
        - name: NAMESPACE
          value: skala-practice
```

ReplicaSet

name: sk000-repliaset-test-**xxxxxx**

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: sk000-replicaset-test
  namespace: skala-practice
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sk000-replicaset-test
  template:
    metadata:
      labels:
        app: sk000-replicaset-test
    spec:
      serviceAccountName: default
      containers:
      - name: webserver
        image: xxx/sk000-webserver:2.0.0
        imagePullPolicy: Always
        env:
        - name: LOGGING_LEVEL
          value: DEBUG
        - name: USER_NAME
          value: sk000
        - name: NAMESPACE
          value: skala-practice
```

Pod

name: sk000-repliaset-test-**yyyyy**

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sk000-replicaset-test-pod
  namespace: skala-practice
  labels:
    app: sk000-replicaset-test
spec:
  serviceAccountName: default
  containers:
  - name: webserver
    image: xxxx/sk000-webserver:2.0.0
    imagePullPolicy: Always
    env:
    - name: LOGGING_LEVEL
      value: DEBUG
    - name: USER_NAME
      value: sk000
    - name: NAMESPACE
      value: skala-practice
```

DESIRED vs CURRENT

$k get deploy {학번}-myfirst-api-server –o yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
```

DESIRED

```
status:
  availableReplicas: 2
  conditions:
  - lastTransitionTime: "2025-01-17T08:58:14Z"
    lastUpdateTime: "2025-01-17T08:58:14Z"
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
    type: Available
  - lastTransitionTime: "2025-01-17T08:58:12Z"
    lastUpdateTime: "2025-01-17T10:09:17Z"
    message: ReplicaSet "himang10-deployment-test-
6bf78b7ccb" has successfully progressed.
    reason: NewReplicaSetAvailable
    status: "True"
    type: Progressing
```

CURRENT

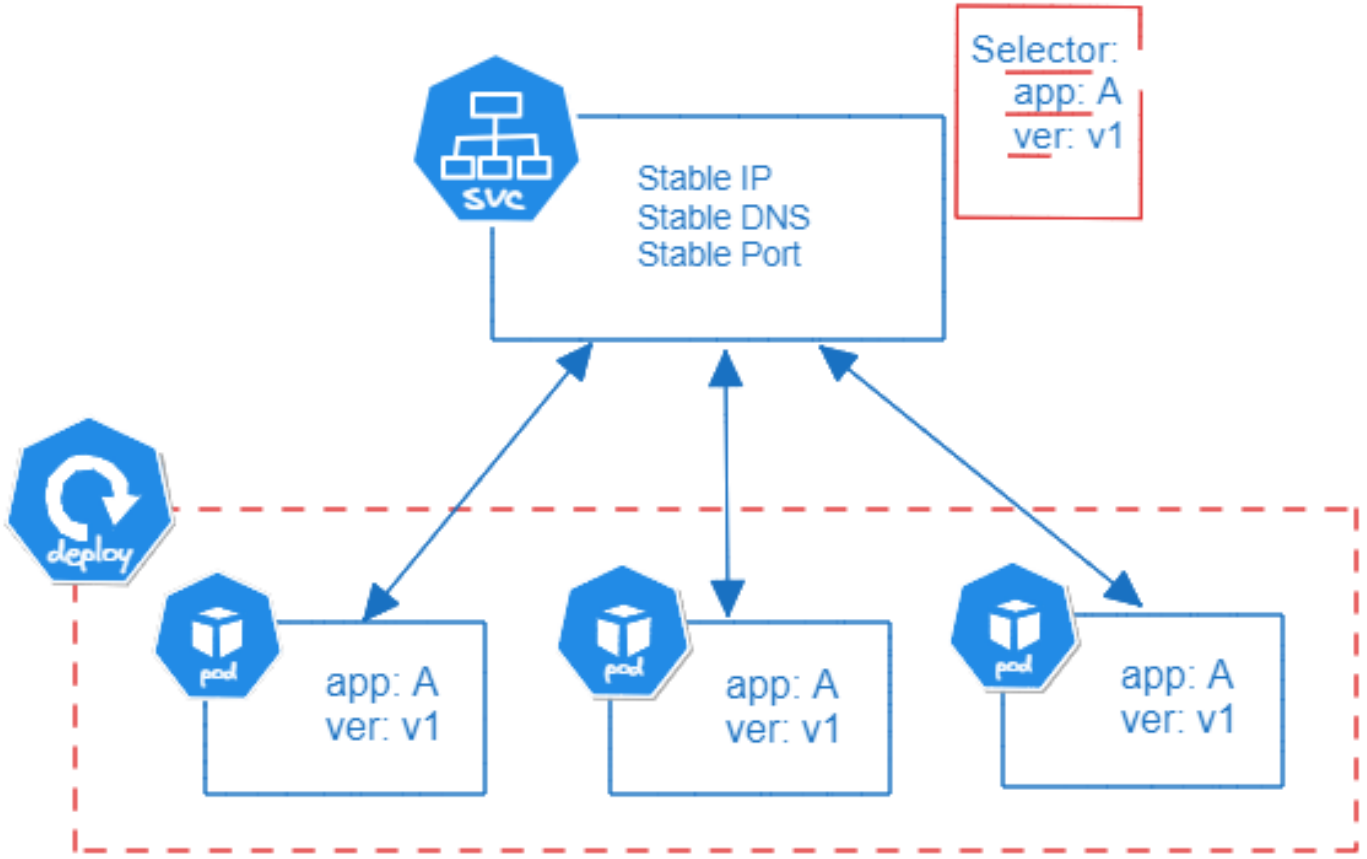
Service

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Service 정의

동적으로 생성되고 사라지는 **Pod 집합**에 **단일 엔드포인트(고정 IP or DNS 이름)**를 제공하여 지속적인 통신이 가능하도록 연결해 주는 네트워크 추상화 리소스

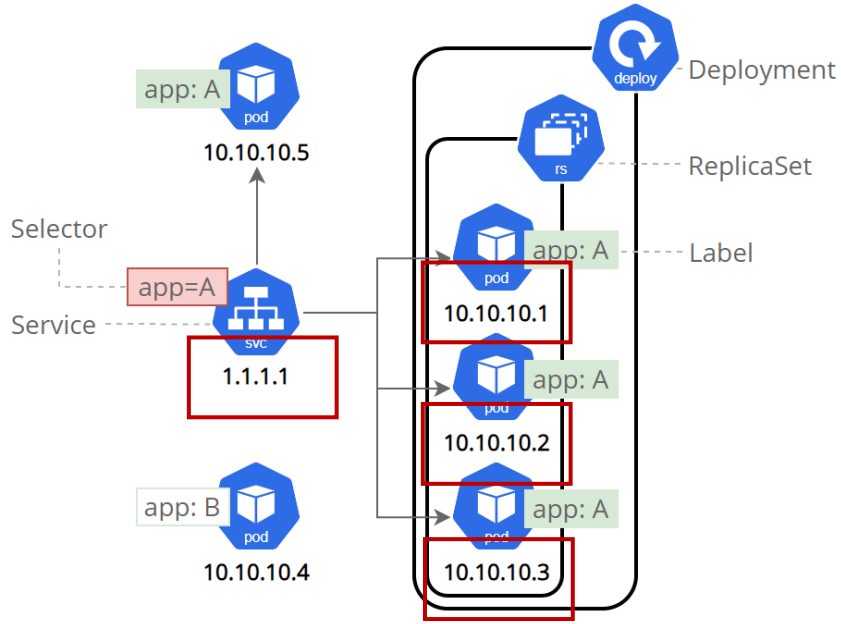


Service: cluster IP 통신

Service 를 이용한 Pod 간 통신 방식 중 가상의 Cluster IP를 할당한 Pod IP 검색 방식

예) LoadBalancer, Node IP, External Name, Cluster IP 방식 이 존재

```yaml
apiVersion: v1
kind: Service
metadata:
  name: skala-webserver
  namespace: class-0
  labels:
    app: skala-webserver
spec:
  type: ClusterIP
  selector:
    app: skala-webserver
  ports:
    - name: http
      port: 8080
      targetPort: 8080
    - name: mgmt
      port: 8081
      targetPort: 8081
```




```
> kubectl get service -o wide
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE   SELECTOR
skala-webserver     ClusterIP   10.100.103.22    <none>       8080/TCP   9h    app=skala-webserver
```

kubernetes in action book 내용 참고

Endpoints

Service Resource 생성 시 Service의 이름과 동일한 Endponts Resource가 생성

Service Yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: skala-webserver
  namespace: class-0
  labels:
    app: skala-webserver
spec:
  type: ClusterIP
  selector:
    app: skala-webserver
  ports:
    - name: http
      port: 8080
      targetPort: 8080
    - name: mgmt
      port: 8081
      targetPort: 8081
```

Endpoints Yaml

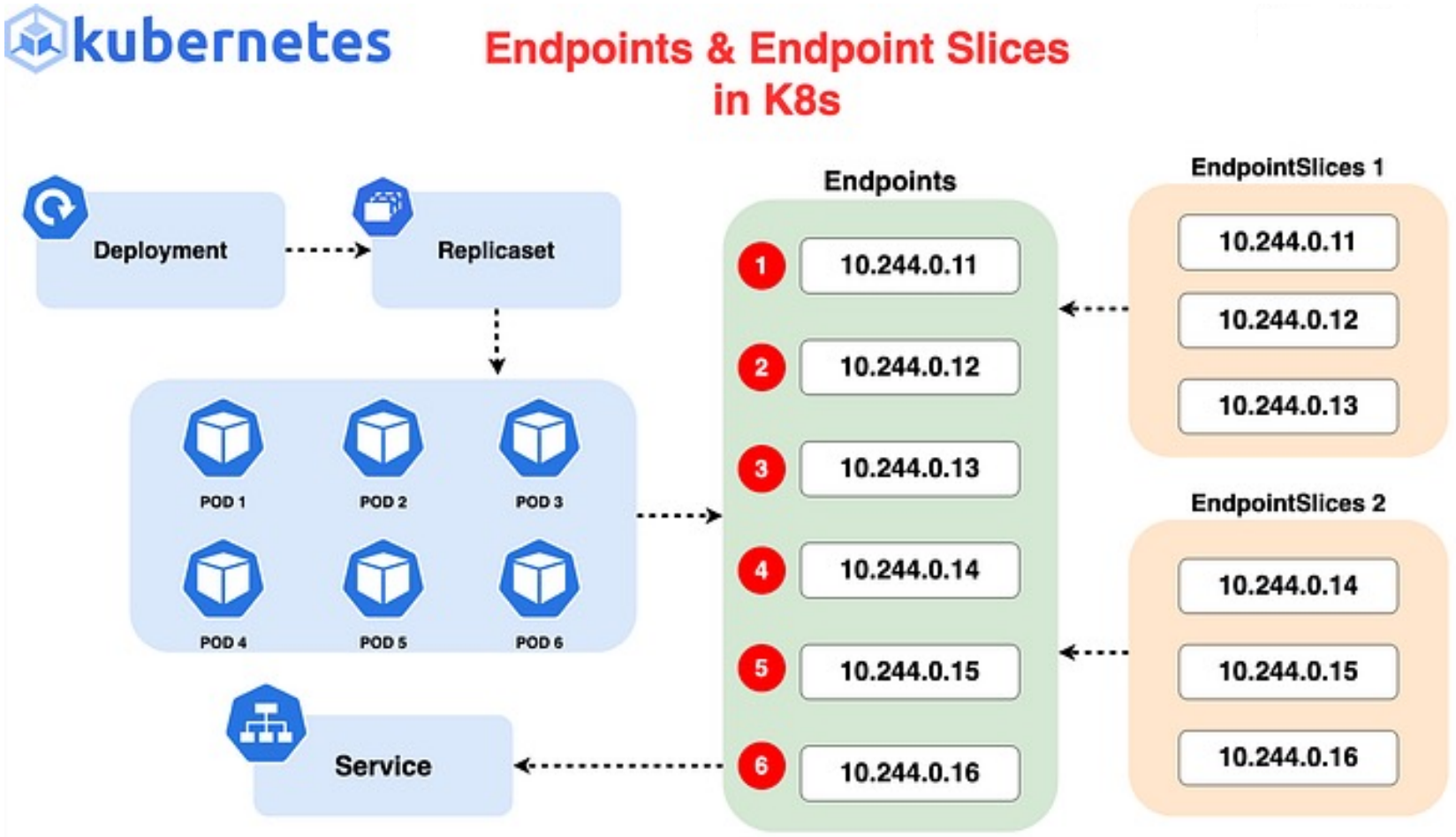
```yaml
apiVersion: v1
kind: Endpoints
metadata:
  name: skala-webserver
  namespace: class-0
subsets:
- addresses:
  - ip: 172.31.24.237
    nodeName: ip-172-31-19-8.ap-northeast-2.compute.internal
    targetRef:
      kind: Pod
      name: skala-webserver-76f8bb5f67-dls7q
      namespace: class-0
      uid: 8a55506e-9793-4d52-9984-5fb9c61061c8
  - ip: 172.31.27.16
    nodeName: ip-172-31-19-8.ap-northeast-2.compute.internal
    targetRef:
      kind: Pod
      name: skala-webserver-76f8bb5f67-kg4fp
      namespace: class-0
      uid: aa092362-e140-450c-9622-5c4c5aa88c1c
```

kubernetes in action book 내용 참고

[참고] EndpointSlice

현재 내부 구조에서는 Endpoint 대신 EndpointSlice로 동작

```
> k get endpointslice -l app=web-server
NAME                     ADDRESSTYPE   PORTS       ENDPOINTS                   AGE
skala-webserver-dlr6x    IPv4          8080,8081   172.31.24.237,172.31.27.16   52m
```



KDT Cloud – 5. 쿠버네티스 심화하기

Namespace

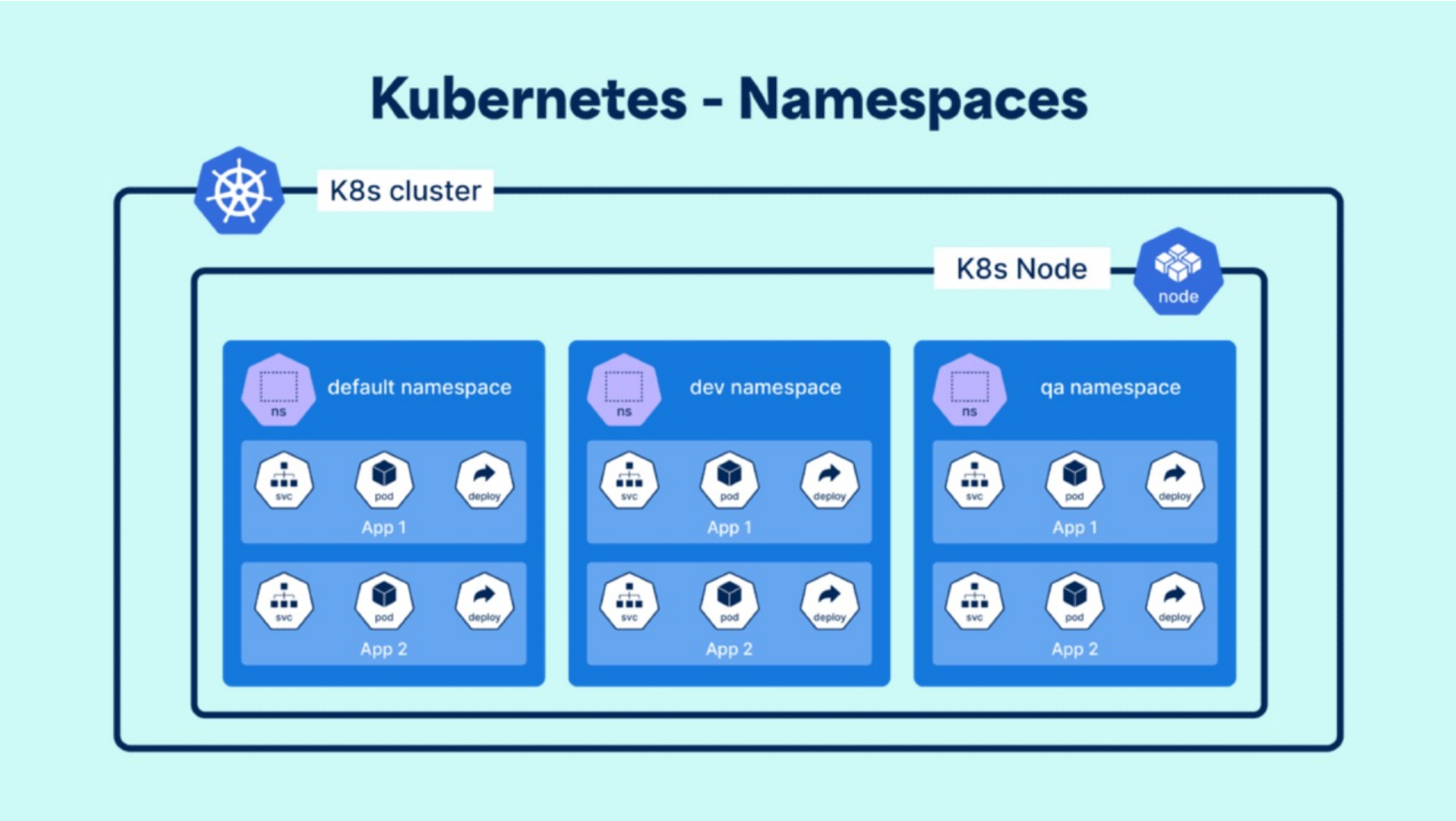
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Namespaces 란?

하나의 Kubernetes 클러스터 안에서 리소스를 논리적으로 분리하기 위한 가상 공간

같은 클러스터를 쓰되, 리소스 **이름·권한·정책·가시성**을 서로 격리해서 관리



Namespaces 란?

리소스 격릭 (Isolation)

- Pod, Service, ConfigMap, Secret 등 대부분의 리소스는 Namespace 단위로 소속
- 같은 이름의 리소스도 Namespace가 다르면 공존 가능 (dev/payment-service, prod/payment-service)

멀티 테넌시 (Multi-tenancy)

- 하나의 클러스터를 여러 팀 / 프로젝트 / 조직이 공유
- 팀별 Namespace 할당
- 인프라 비용 절감 + 운영 일관성 유지

※ 여러 고객/조직(세입자들) 이 동시에 빌려 쓰는 구조

접근 제어 (RBAC의 경계)

- Role / RoleBinding은 Namespace 스코프
- 사용자는 특정 Namespace에만 접근 가능
- 실수로 prod namespace 날리는 사고 방지

운영 단위 관리

- 배포 / 삭제 / 조회를 Namespace 단위로 수행
- dev namespace 일괄 정리 (삭제)

리소스 관리 (Quota & Limit)

- Namespace 단위로 리소스 상한 설정 가능
- 특정 팀/조직에서 클러스터 자원 독식 방지

```
ResourceQuota:
  cpu: 10
  memory: 32Gi
  pods: 50
```

```
LimitRange:
  type: Container
  default:
  defaultRequest
```

Namespaces 특징

Namespace-scoped 리소스

다음 리소스들은 반드시 Namespace에 속함

```
kubectl get pods -n 또는 --namespace dev
```

- Pod
- Service
- Deployment / StatefulSet / DaemonSet
- ConfigMap / Secret
- Ingress
- Role / RoleBinding
- PVC
- NetworkPolicy / LimitRange / ResourceQuota

Cluster-scoped 리소스

Namespace에 속하지 않고 클러스터 전역에 존재

```
-n option 없음
```

- Node
- PersistentVolume (PV)
- Namespace 자체
- ClusterRole / ClusterRoleBinding
- StorageClass
- CustomResourceDefinition (CRD)

네트워크 격리

NetworkPolicy를 통한 간접 격리

- Namespace는 네트워크를 자동으로 분리하지 않음
- Pod 간 통신은 Namespace가 달라도 가능

DNS 네이밍 규칙

Namespace는 서비스 DNS 이름의 일부

```
service-name.namespace.svc.cluster.local

예) mysql.dev.svc.cluster.local
```

Namespaces 특징

Namespace-scoped 리소스

다음 리소스들은 반드시 Namespace에 속함

`kubectl get pods -n 또는 --namespace dev`

- Pod
- Service
- Deployment / StatefulSet / DaemonSet
- ConfigMap / Secret
- Ingress
- Role / RoleBinding
- PVC
- NetworkPolicy / LimitRange / ResourceQuota

Cluster-scoped 리소스

Namespace에 속하지 않고 클러스터 전역에 존재

`-n option 없음`

- Node
- PersistentVolume (PV)
- Namespace 자체
- ClusterRole / ClusterRoleBinding
- StorageClass
- CustomResourceDefinition (CRD)

네트워크 격리를 직접 하지 않음

NetworkPolicy를 통한 간접 격리

- Namespace는 네트워크를 자동으로 분리하지 않음
- Pod 간 통신은 Namespace가 달라도 가능

[실습] Namespaces 확인하기

kubernetes 접속을 위해 현재 namespace를 확인하고 변경하는 방법은 다음과 같다.

현재 namespace 확인하기

```
kubectl config get-contexts
kubectl config get-contexts | sed ‘1d’ | awk ‘{print $5}’
```

namespace 변경하기

```
kubectl config set-context $(kubectl config current-context) --namespace skala-practice
```

alias를 통한 namespace 변경 단축 명령어 자성

```
alias kcns='kubectl config set-context $(kubectl config current-context) --namespace'
```

[실습] Namespaces

각 네임스페이스 상세 내용 확인

- kubectl get 을 namespace 지정없이 사용하면 현재 네임스페이스에 질의
- 다른 사용자와 분리된 환경으로 타인의 접근을 제한
- 네임스페이스 별로 리소스 접근 허용과 리소스 양도 제어 가능
- --namespace나 -n을 사용하여 네임스페이스 별로 확인이 가능

```
$ kubectl get pod -n kube-system
NAME                                        READY     STATUS    RESTARTS   AGE
event-exporter-v0.2.4-5f7d5d7dd4-szrgm      2/2       Running   0          7h17m
fluentd-gcp-scaler-7b895cbc89-qxd5g         1/1       Running   0          7h16m
fluentd-gcp-v3.2.0-qtwf7                    2/2       Running   0          7h16m
...
```

[데모] Namespaces

ClusterRole 생성 권한은
관리자 권한 필요

YAML 파일로 네임스페이스 만들기

● test_ns.yaml 파일을 생성하고 create를 사용하여 생성

```yaml
apiVersion: v1
kind: Namespace
metadata:
  # 네임스페이스 이름
  name: test-ns
```

kubectl 명령어로 yaml 없이 바로 네임스페이스 생성 가능

$ kubectl create namespace “test-namespace”

$ kubectl create -f test_ns.yaml

namespace/test-ns created

$ kubectl get ns

NAME	STATUS	AGE
application-system	Active	7h44m
default	Active	7h54m
kube-public	Active	7h54m
kube-system	Active	7h54m
test-ns	Active	4m17s

[실습] Namespaces

전체 네임스페이스 조회

- 전체 네임스페이스를 대상으로 kubectl을 실행하는 방법
- $ kubectl get pod -A (full argument: --all-namespaces)

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
application-system	application-controller-manager-0	1/1	Running	0	7h50m
default	http-go-gpu	1/1	Running	0	6h47m
default	http-go	1/1	Running	0	7h10m
default	http-go-v2	1/1	Running	0	7h11m
kube-system	event-exporter-v0.2.4-5f7d5d7dd4-szrgm	2/2	Running	0	8h
kube-system	fluentd-gcp-scaler-7b895cbc89-qxd5g	1/1	Running	0	7h59m
kube-system	fluentd-gcp-v3.2.0-qtwf7	2/2	Running	0	7h59m
kube-system	fluentd-gcp-v3.2.0-xdtjg	2/2	Running	0	7h59m
kube-system	fluentd-gcp-v3.2.0-xwgw5	2/2	Running	0	7h59m
kube-system	heapster-v1.6.0-beta.1-54cb65c898-xg4p2	3/3	Running	0	7h59m
kube-system	kube-dns-autoscaler-76fcd5f658-d7kll	1/1	Running	0	7h59m
kube-system	kube-dns-b46cc9485-9w2fj	4/4	Running	0	7h59m
kube-system	kube-dns-b46cc9485-lxwc7	4/4	Running	0	8h
kube-system	kube-proxy-gke-standard-cluster-1-default-pool-b1e2cd6b-3r3q	1/1	Running	0	7h59m
kube-system	kube-proxy-gke-standard-cluster-1-default-pool-b1e2cd6b-cb83	1/1	Running	0	8h
kube-system	kube-proxy-gke-standard-cluster-1-default-pool-b1e2cd6b-z2nd	1/1	Running	0	8h
kube-system	l7-default-backend-6f8697844f-xtrt7	1/1	Running	0	8h
kube-system	metrics-server-v0.3.1-5b4d6d8d98-ggjr9	2/2	Running	0	7h59m
kube-system	prometheus-to-sd-5pfw5	1/1	Running	0	7h59m
kube-system	prometheus-to-sd-g66mn	1/1	Running	0	8h
kube-system	prometheus-to-sd-j64x2	1/1	Running	0	8h

label 과 Selector

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

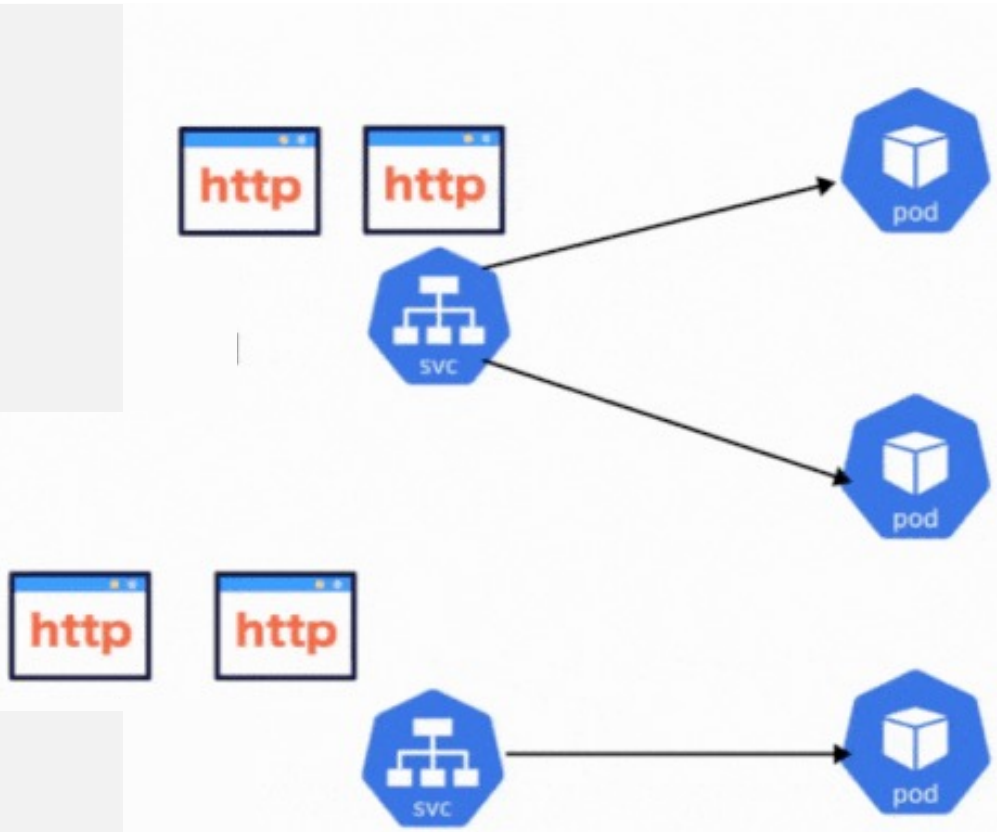
SK 주식회사 C&C

label 과 Selector

Label과 Selector는 리소스를 느슨하게 연결하기 위한 핵심 메커니즘

이 Pod가 누구 소속인지, 이 Service가 어떤 Pod를 대상으로 하는지를 메타데이터로 결정

```yaml
apiVersion: v1
kind: Service
metadata:
  name: prod-svc-1
spec:
  selector:
    app: prod
  ports:
  - port: 80
    targetPort: 80
```




```yaml
apiVersion: v1
kind: Pod
metadata:
  name: prod-pod-1
  labels:
    app: prod
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: prod-pod-2
  labels:
    app: prod
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: prod-svc-2
spec:
  selector:
    app: test
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: prod-pod-3
  labels:
    app: test
```

label 과 Selector

Label

리소스가 스스로를 설명하는 테그

나는 web 앱이고, 환경이 Prod이며, v1 버전이다

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: prod-pod-1
  labels:
    app: web
    env: prod
    version: v1
```

특징

- **의미 기반 분류**: app, env, tier, version 등
- **중복 허용**: 여러 리소스가 같은 label을 가질 수 있음
- **동적 변경 가능**: 실행 중에도 수정 가능
- **쿼리 대상**: Selector로 검색·연결의 기준이 됨

Selector

Label을 기준으로 대상을 선택하는 조건

나는 web 앱이면서, 환경이 prod인 Pod를 선택한다

```yaml
apiVersion: v1
kind: Service
metadata:
  name: prod-svc-1
spec:
  selector:
    app: web
    env: prod
```

특징

- **의미 기반 분류**: app, env, tier, version 등
- **중복 허용**: 여러 리소스가 같은 label을 가질 수 있음
- **동적 변경 가능**: 실행 중에도 수정 가능
- **쿼리 대상**: Selector로 검색·연결의 기준이 됨

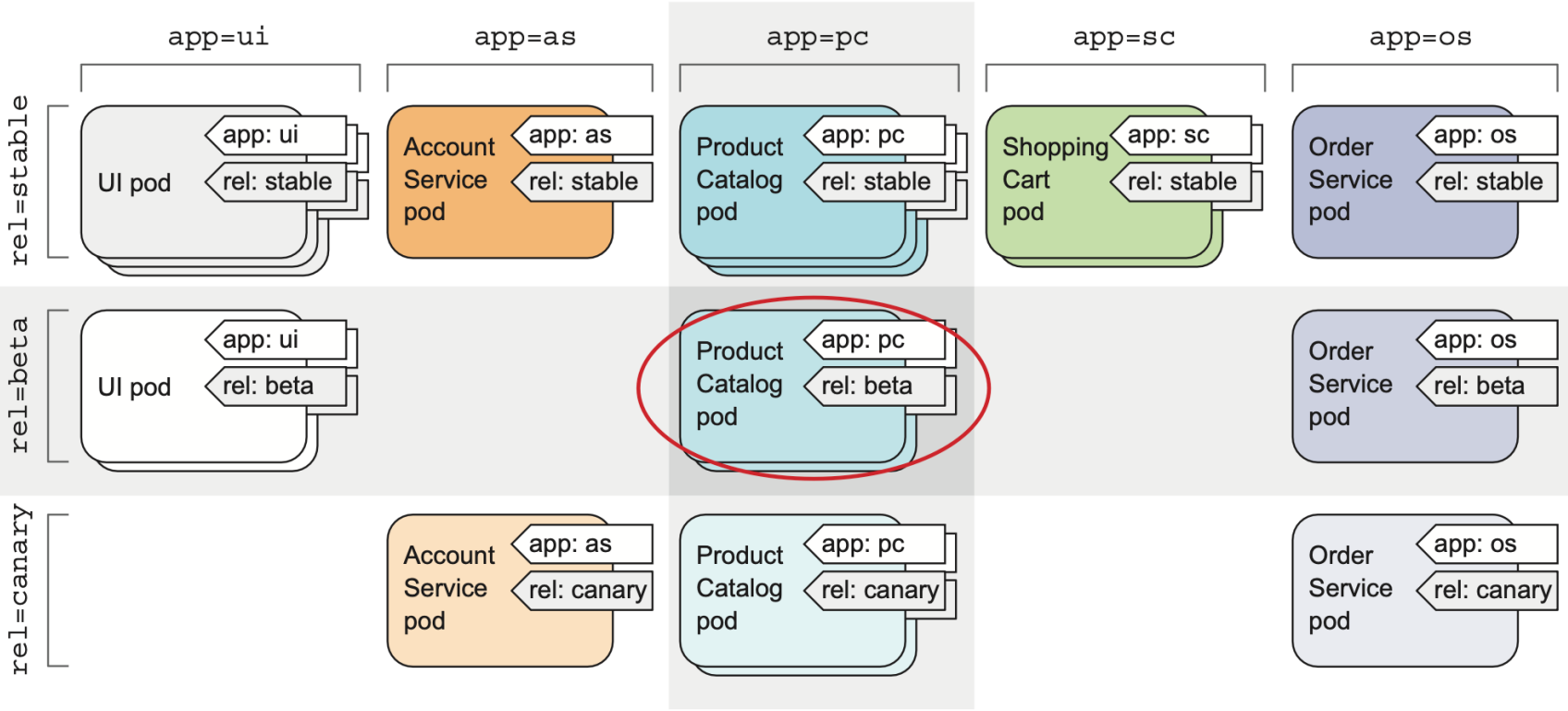
Label로 선택하기

Label을 통한 Select는 배포된 Pod와 상관없이 동일 Label을 가지고 있는 모든 Pod가 선택

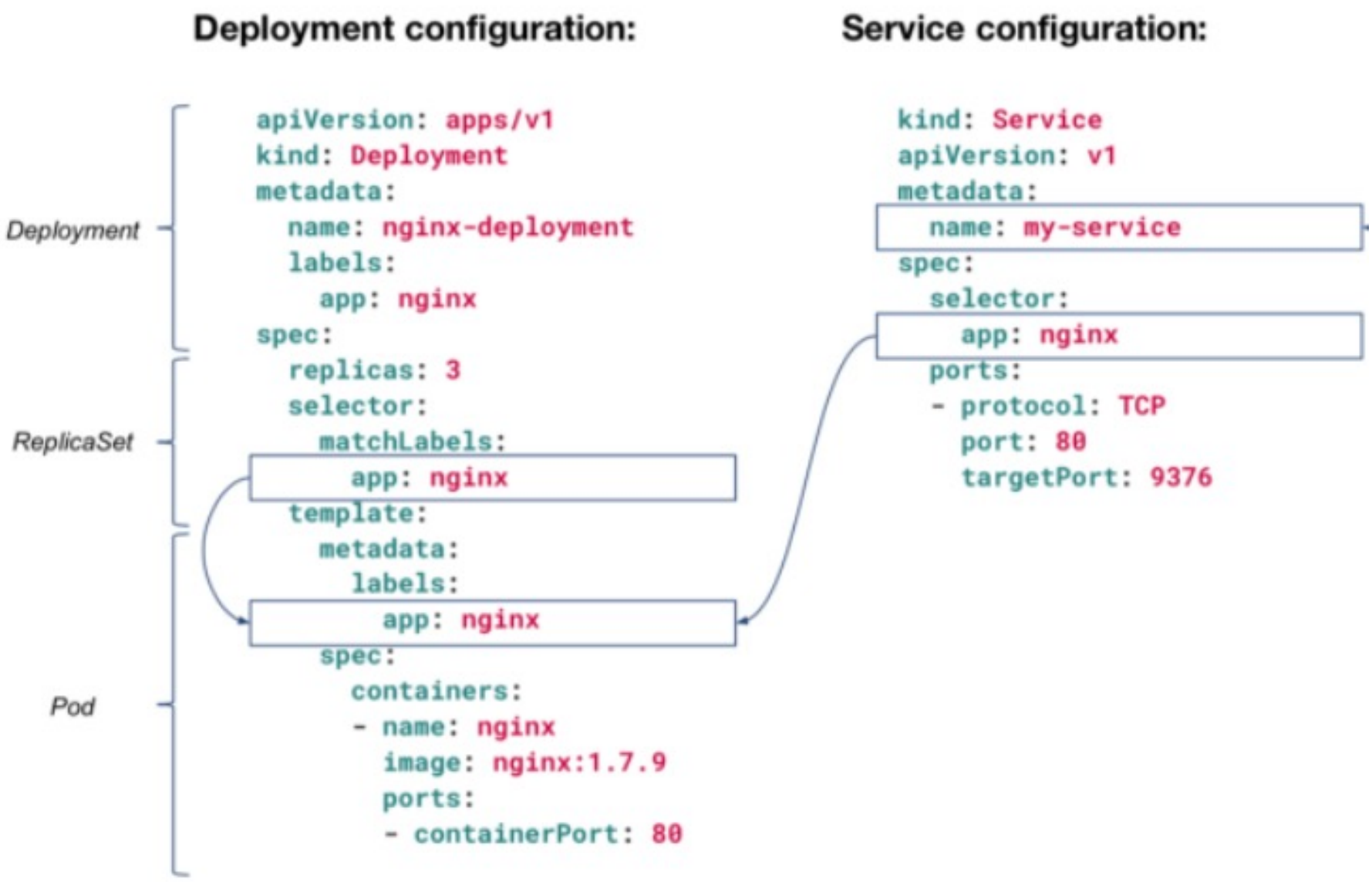
```
spec:
  selector:
    rel: beta
  ports:
  - port: 80
    targetPort: 80
```

```
spec:
  selector:
    app: pc
  ports:
  - port: 80
    targetPort: 80
```

```
spec:
  selector:
    app: pc
    rel: beta
  ports:
  - port: 80
    targetPort: 80
```



Deployment Label



[실습] Label 확인하기

Label 확인하기

- Label 보여주기

$ kubectl get pod --show-labels

```
NAME                                    READY   STATUS    RESTARTS   AGE   LABELS
file-explorer-744b6b95cb-mfx69          1/1     Running   0          19h   app=file-explorer,pod-template-
hash=744b6b95cb
himang10-deployment-test-6bf78b7ccb-khvtl   1/1     Running   0          30m   app=himang10-deployment-test,pod-template-
hash=6bf78b7ccb
himang10-deployment-test-6bf78b7ccb-ns6tm   1/1     Running   0          30m   app=himang10-deployment-test,pod-template-
hash=6bf78b7ccb
```

- 특정 Label 컬럼으로 확인

$ kubectl get pod –Lapp, pod-template-hash

```
NAME                                    READY   STATUS    RESTARTS   AGE   APP                    POD-TEMPLATE-
HASH
file-explorer-744b6b95cb-mfx69          1/1     Running   0          19h   file-explorer          744b6b95cb
himang10-deployment-test-6bf78b7ccb-khvtl   1/1     Running   0          32m   himang10-deployment-test   6bf78b7ccb
himang10-deployment-test-6bf78b7ccb-ns6tm   1/1     Running   0          32m   himang10-deployment-test   6bf78b7ccb
```

[실습]

- 새로운 레이블을 추가할 때는 label 명령어를 사용

```
$k label pod sk000-myfirst-api-server-6bf78b7ccb-khvtl skala=sk000
$k get pod –show-labels
```

```
NAME                                   READY   STATUS    RESTARTS   AGE   LABELS
sk000-myfirst-api-server-6bf78b7ccb-khvtl   1/1     Running   0          37m   app=himang10-deployment-test,pod-
template-hash=6bf78b7ccb,skala=sk000
```

- 기존 레이블의 값을 업데이트하기 위한 --overwrite 적용

```
$ k label pod sk000-myfirst-api-server-6bf78b7ccb-khvtl skala=b --overwrite
```

```
pod/sk000-myfirst-api-server-6bf78b7ccb-khvtl labeled
```

```
k get pod himang10-deployment-test-6bf78b7ccb-khvtl --show-labels
```

```
NAME                                   READY   STATUS    RESTARTS   AGE   LABELS
sk000-myfirst-api-server-6bf78b7ccb-khvtl   1/1     Running   0          42m   app=himang10-deployment-test, skala=b
```

- 레이블 삭제

```
$ k label pod sk000-myfirst-api-server-6bf78b7ccb-khvtl skala-
```

```
pod/himang10-deployment-test-6bf78b7ccb-khvtl labeled
```

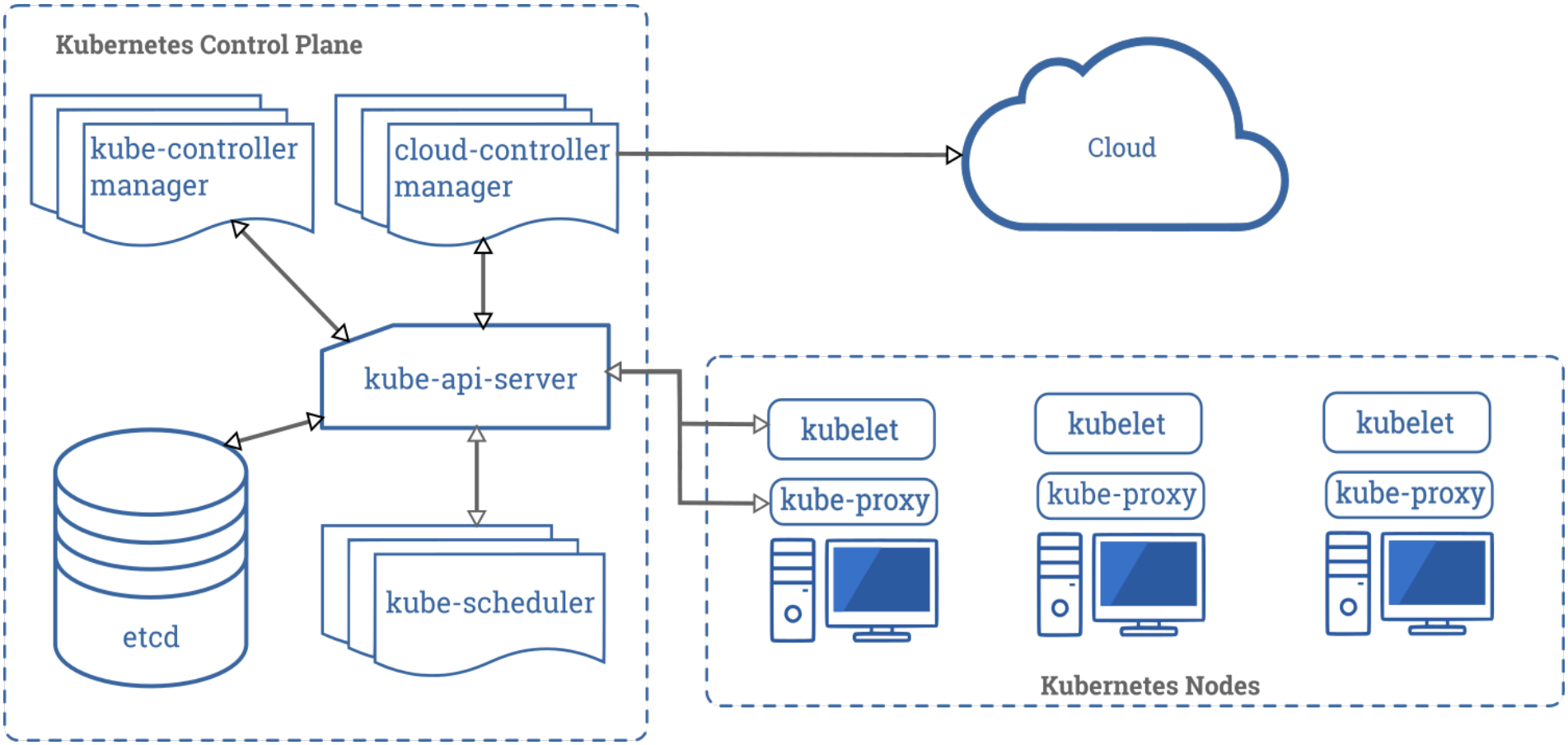
Kube-system 구조

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

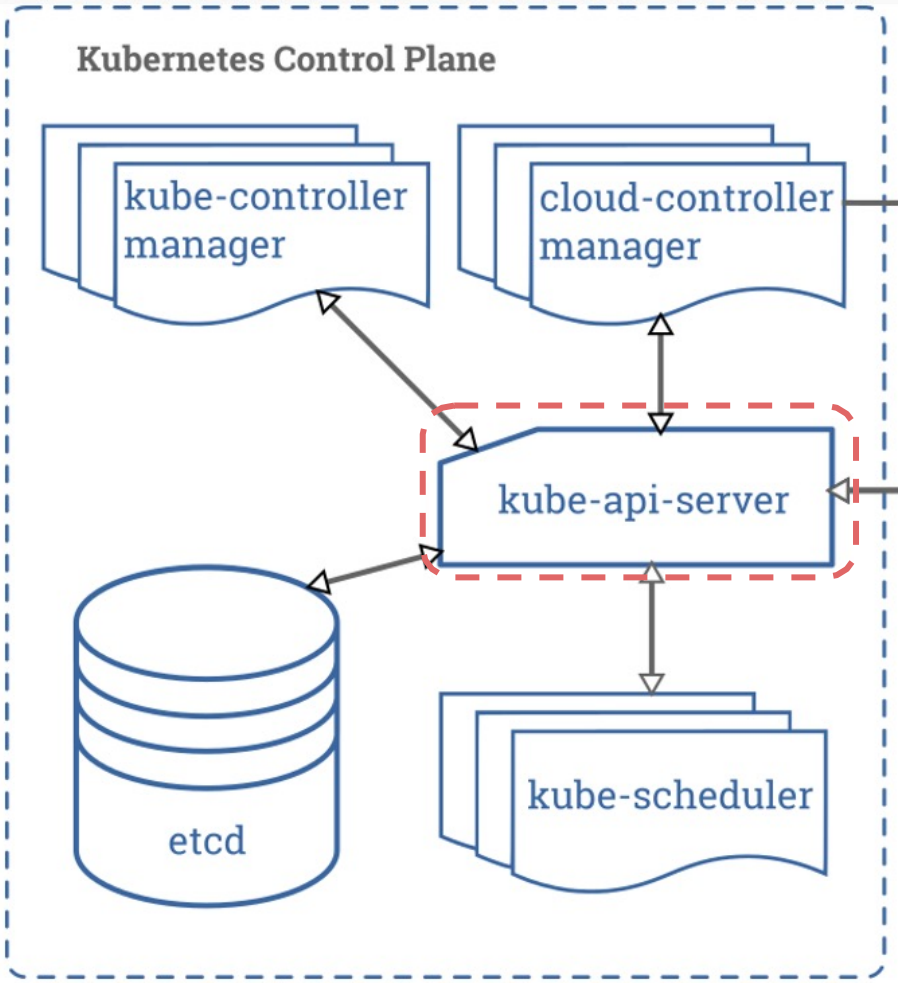
Kubernetes Cluster 구성

구성도



Kubernetes Cluster 구성

Control Plane

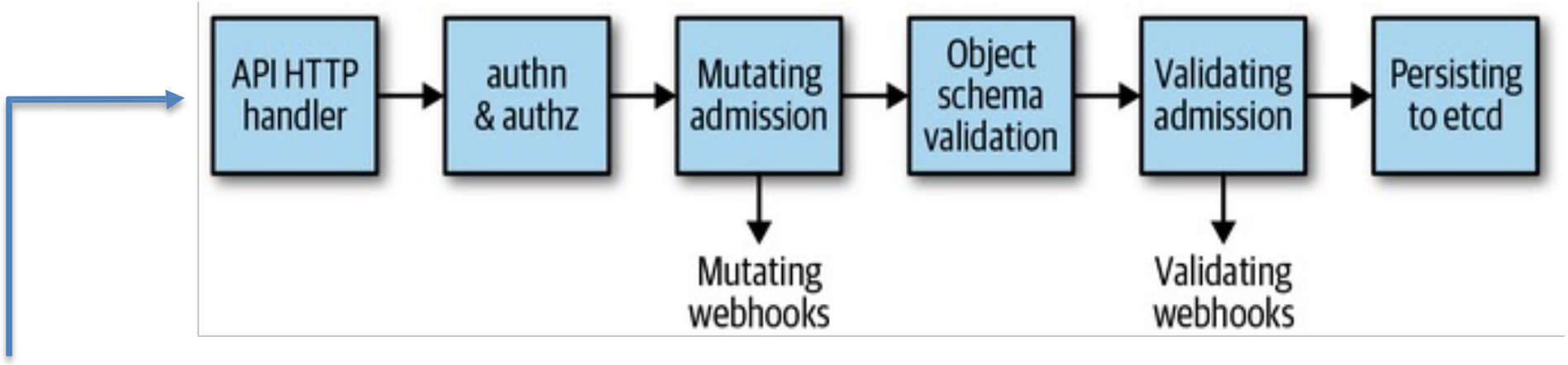


Kube-api-server

- **API 요청 처리**
  클러스터와의 모든 요청의 입구 역할 수행
- **ETCD와의 통신**
  클러스터 상태 데이터를 ETCD에 저장하고 읽어오는 작업을 수행
- **보안 기능**
  인증, 권한 부여, 데이터 암호화, 네트워크 보안을 처리
- **유효성 검사**
  리소스 정의가 유효한지 확인하고 기본값을 설정하거나 필요한 변환을 수행

kube api-server의 Admission Control Flow

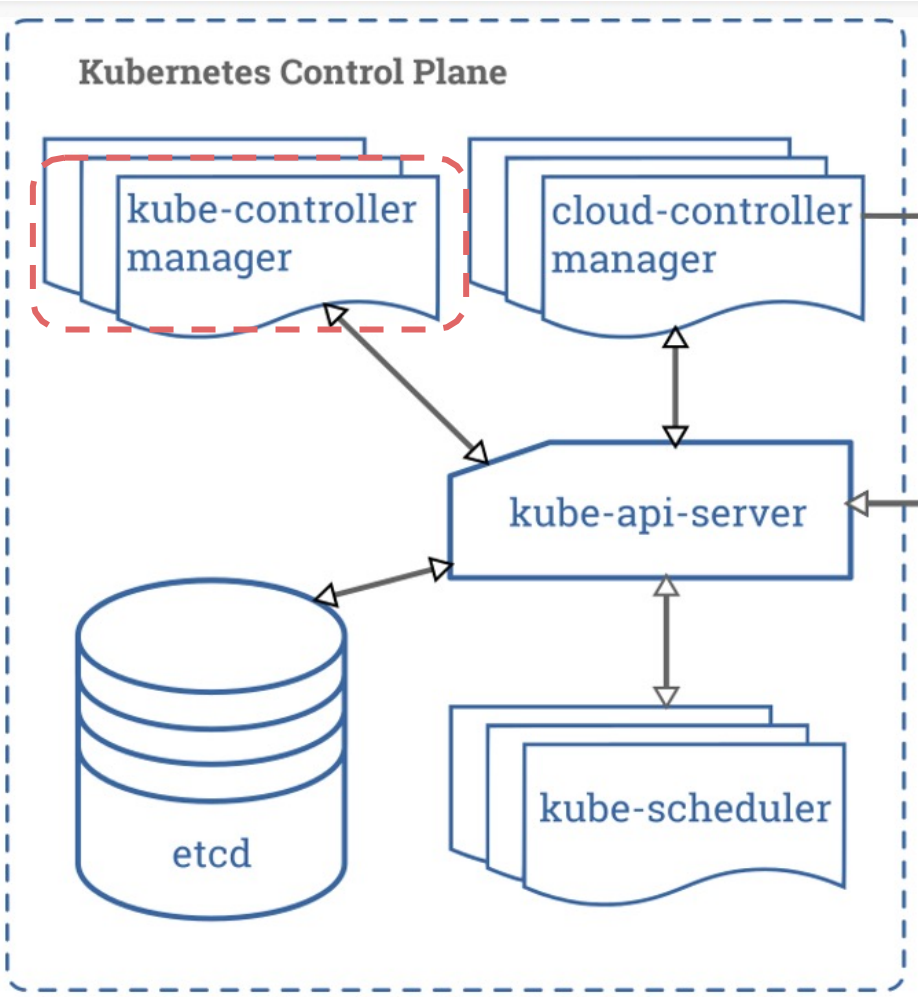
```
> kubectl apply -f deploy.yaml
```



- **API HTTP Handler**: 요청 수신
- **Authn & Authz**: 너는 누구냐 (Authn), 권한 있어?(Authz) -- 인증 및 권한 처리 (Token 확인, RBAC 권한 확인)
- **Mutating Admission**: 요청에 추가적인 작업 수행
  (service account 미 포함시 default 추가, istio sidecar 자동 주입 등)
- **Object Schema Validation**: Resource Schema 와 CRD Schema 검증
- **Validating Admission**: 네임스페이스 정책 적용, 리소스 검증, 보안정책 적용 등 의 허용/거부 (변경 불가)
  (Resource Quota 초과시 요청 거부 등)
- **Persisting to etcd**: 요청이 모든 검증 단계를 통과하면 etcd에 객체로 저장

Control Plane Component

Control Plane

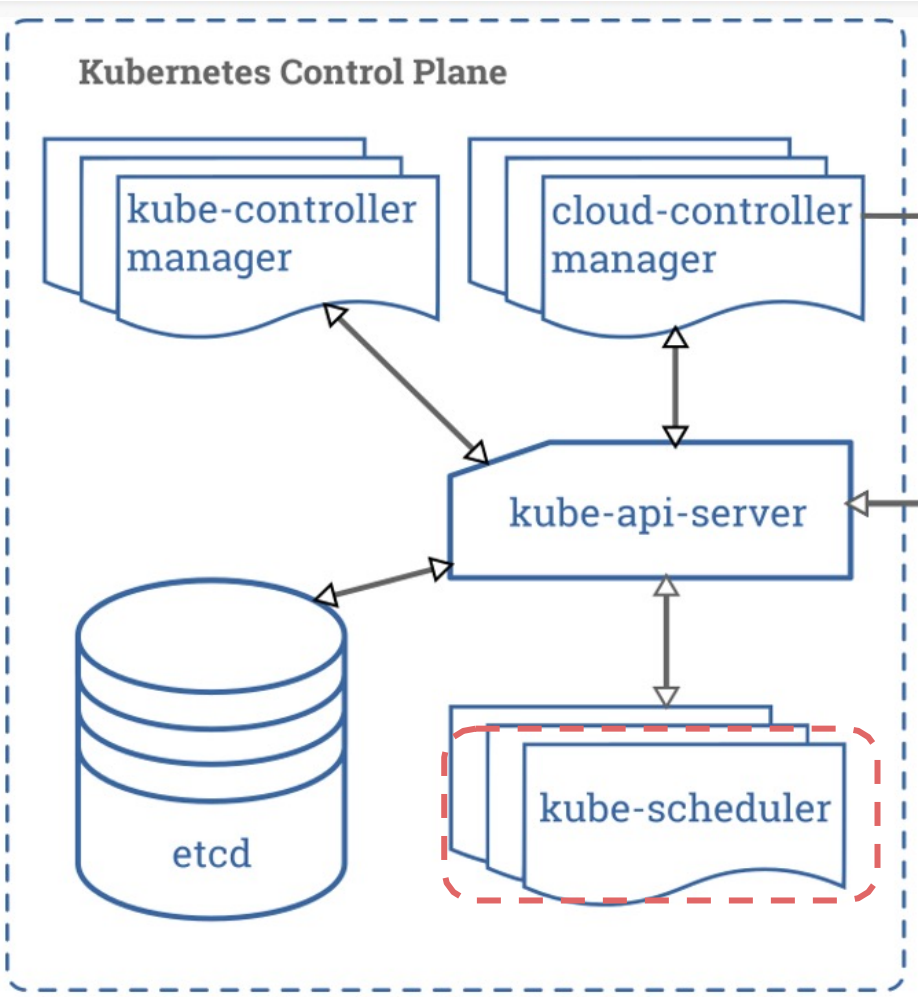


kube-controller-manager

- 클러스터 내의 다양한 컨트롤러(controller)를 실행하여 클러스터 상태를 원하는 상태로 유지
- 컨트롤러는 Kubernetes 리소스를 지속적으로 모니터링
- 실제(Actual) 상태가 선언된(desired) 상태와 일치하도록 코디네이트하는 역할 수행
  - 디플로이먼트 컨트롤러
  - 레플리카셋, 데몬셋, 잡 컨트롤러
  - 스테이트풀셋 컨트롤러,
  - 노드 컨트롤러
  - 서비스 컨트롤러
  - 엔드포인트 컨트롤러
  - 네임스페이스 컨트롤러
  - 영구 볼륨 컨트롤러

Control Plane Component

Control Plane



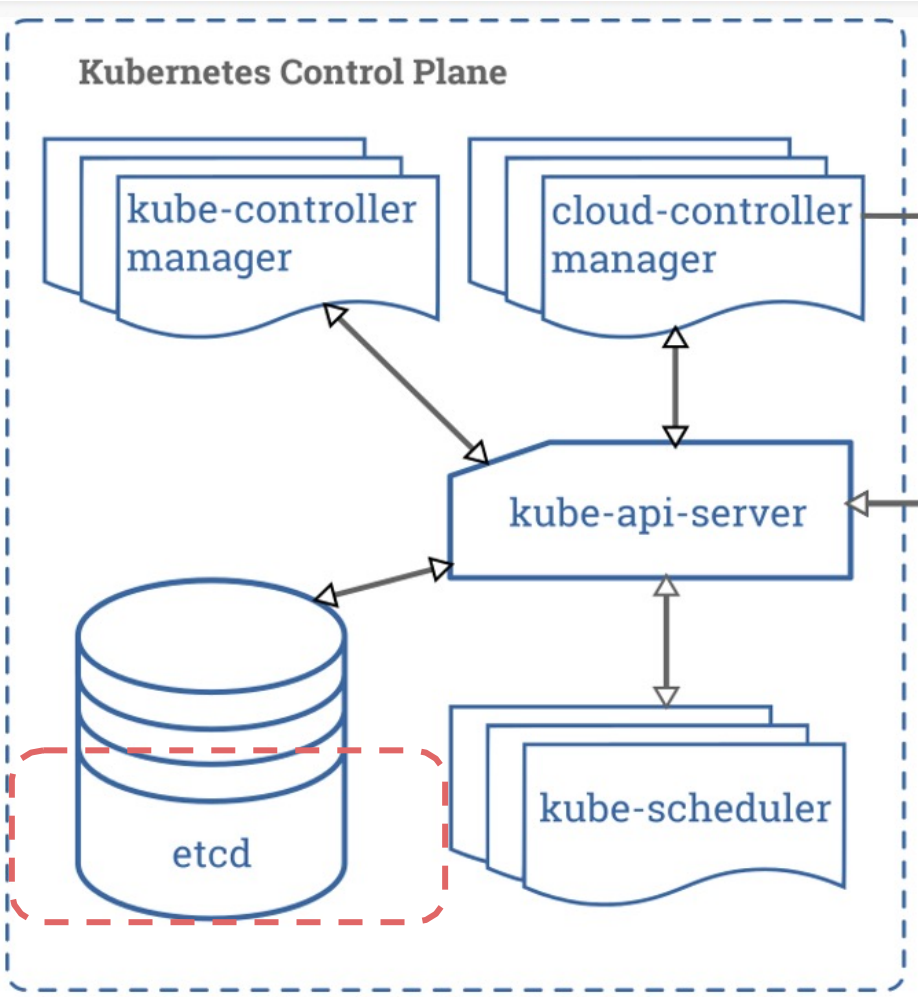
kube-scheduler

- **새로 생성된 Pod을 실행할 적절한 노드(Node)를 선택하는 역할 수행**
- **Pod이 생성되면 kube-scheduler가 해당 Pod의 스케줄링을 담당하며, 다양한 기준을 바탕으로 적합한 노드를 결정**

  - Pod-Node 매칭
  - 스케쥴링 제약 조건 검토
    - 하드 제약 (Node Selector, Taint)
    - 소프트 제약 (Affinity)
    - podTopologySpreadConstraint
  - 리소스 사용량평가
  - 노드 상태 확인
  - 스케쥴링 알고리즘 적용

Control Plane Component

Control Plane



etcd

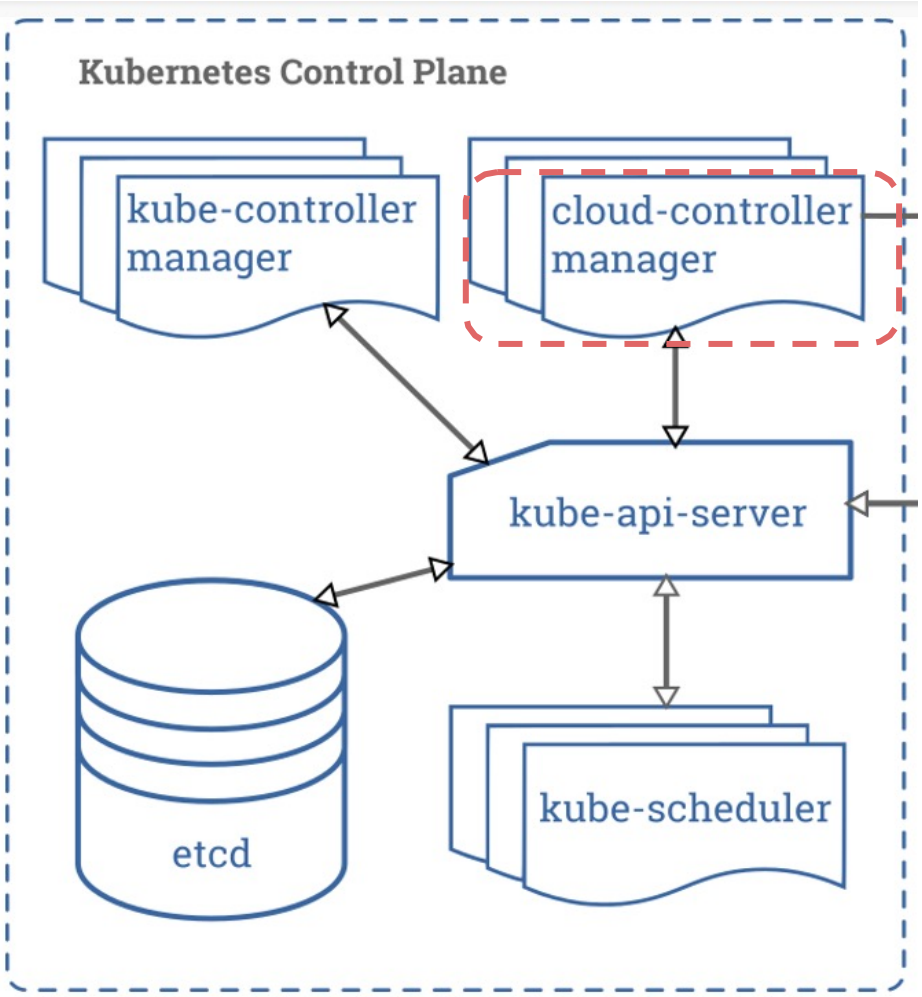
- **etcd는 클러스터의 분산 키-값 저장소 역할 수행**
- **K8s의 선언적 구조의 핵심 역할 수행**
- **etcd는 클러스터의 상태를 영구적으로 저장하고 관리하는 핵심 컴포넌트**
- **모든 Kubernetes Control Plane 컴포넌트가 etcd를 사용하여 데이터를 저장하고 조회**

  - 클러스터 상태 저장
  - 클러스터 설정 및 모든 리소스 데이터 관리
  - 모든 주요 컴포넌트는 ETCD 상태를 watching 하면서 상태 데이터 중심으로 관리

```
KEY                                    VALUE
/registry/pods/default/pod-a           → [Pod A 전체]
/registry/pods/default/pod-b           → [Pod B 전체]
/registry/pods/dev/pod-c               → [Pod C 전체]
/registry/services/default/service-a   → [Service A 전체]
```

Control Plane Component

Control Plane



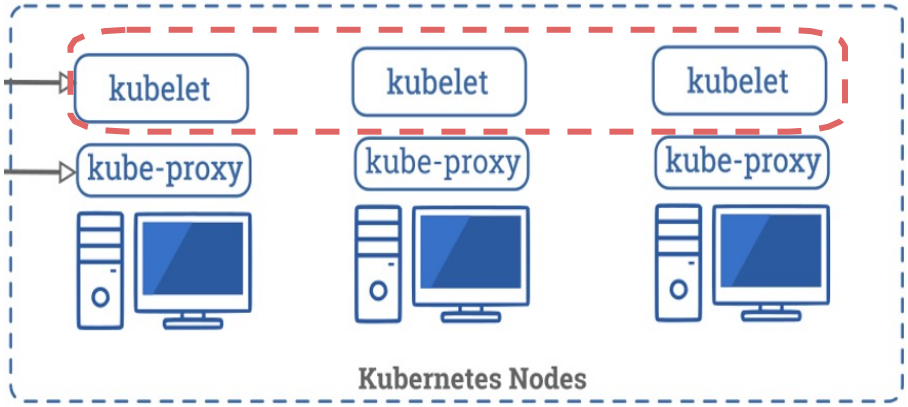
cloud-controller manager

Kubernetes ↔ 클라우드 제공자(AWS) 사이의 "클러스터 수준 통합" 담당

역할

- Node 등록 / 삭제 (AWS EC2)
- LoadBalancer(Service Type=LB)
- Route Table
- 일부 네트워크 / 인프라 동기화

Data Plane Component



kubelet

- 각 워커 노드(worker node)에 실행되는 Agent로 **노드의 컨테이너와 리소스를 관리**하는 역할 수행
- kubelet은 Control Plane과 통신하며, Pod을 실행하고 관리하기 위해 반드시 필요한 컴포넌트
- kube-api-server 로 watch 통신을 통해 ETCD의 리소스 CUD 모니터링해서 Node Resource와 동기화
- containerd, cri-o 등 컨테이너 런타임과 통신하여 컨테이너 라이프사이클 관리

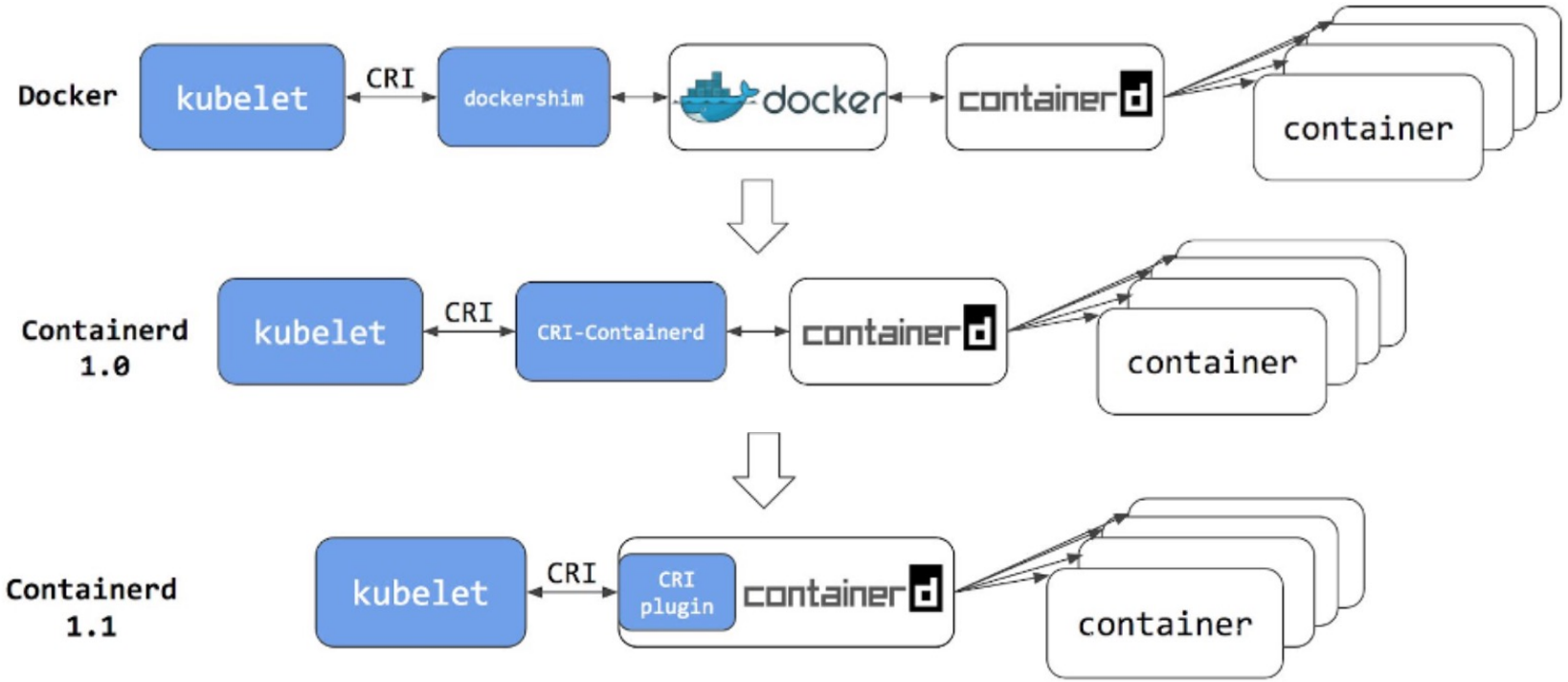
  - Pod 실행 및 관리
  - 노드 상태 보고
  - Pod Spec 준수 확인
  - volume 및 네트워크 관리
  - Pod Health Check 및 Self Healing

[참고] kubelet에서 컨테이너 실행

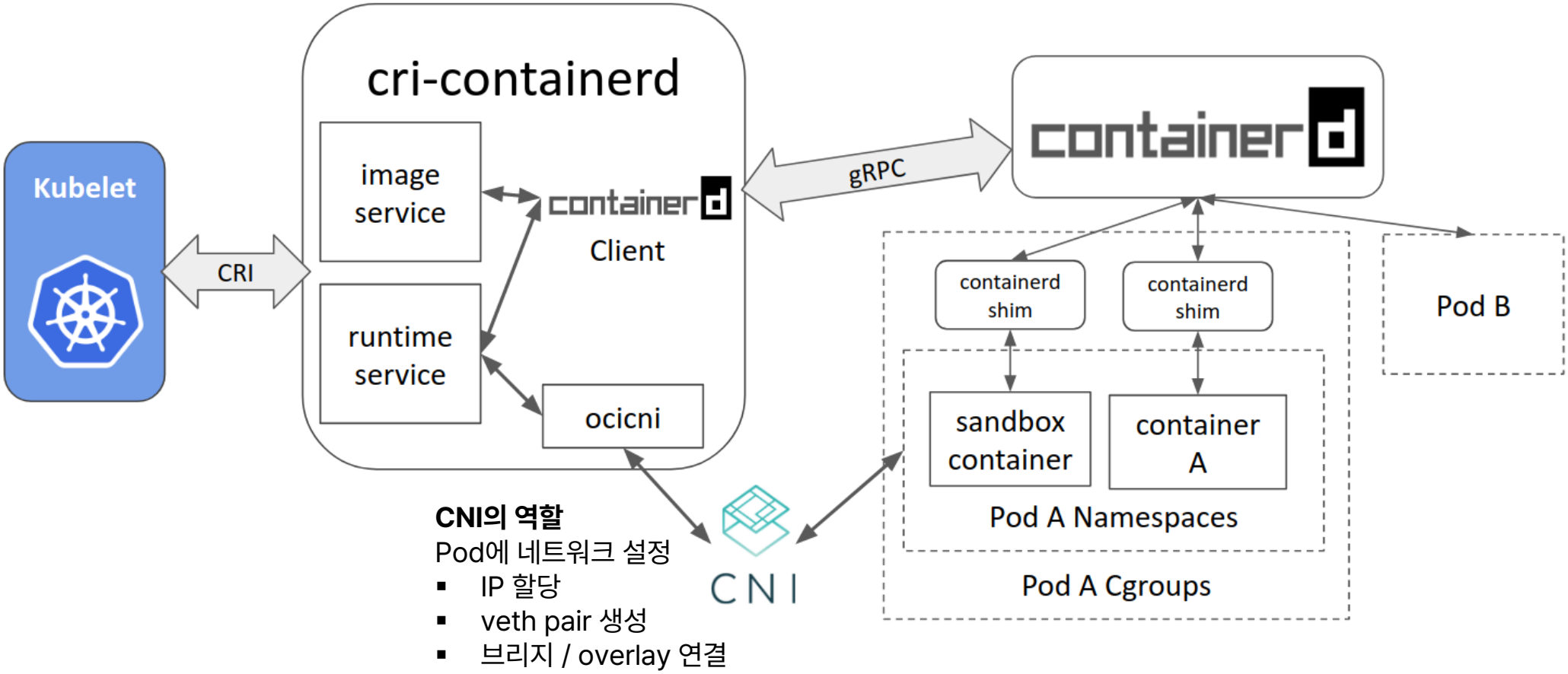
CRI : Container Runtime Interface

kubernetes가 다양한 컨테이너 런타임과 통신하기 위해 정의한 표준 인터페이스 (API)

Docker 환경에서 containerd | crio 전환



[참고] kubelet에서 컨테이너 실행



cri-containerd는 kubelet 과 containerd간 CRI Wrapper 컴포넌트이며,
최근에는 containerd 자체가 cri를 지원하여 cri-containerd가 필요없음

참고) **Sandbox container**: Pod 단위로 하나씩 생성, Pause 이미지로 실행
pause 컨테이너는 거의 아무 일도 하지 않고 단순히 **Pod 내 컨테이너에게 공통된 Network/IPC/UTC namespace**를 제공

[참고] 주요 프로세스 확인하기

개별 worker Node에 동작하고 있는 프로세스와 파일 구조를 확인해보자

```
> kubectl apply -f check-process.yaml

> kubectl exec –it sk000-pod-as-pid -- /bin/sh

# containerd가 포함된 프로세스 찾기
> pgrep -a containerd

> ps aux

> ps aux | grep kubelet
> ps aux | grep containerd

> chroot   /host  systemctl status kubelet
> chroot   /host  systemctl status containerd


# containerd가 포함된 프로세스 찾기
> pgrep -a containerd

> chroot /host ctr -n k8s.io container info <container-ID>

> chroot /host ctr -n k8s.io containers list

> chroot /host ctr -n k8s.io tasks list
```

```
apiVersion: v1
kind: Pod

metadata:
  name: skala-pod-as-pid
  namespace: class-0

spec:
  hostPID: true
  hostNetwork: true

  containers:
    - name: main
      image: alpine

      command:
        - /bin/sh
        - -c
        - |
          apk add procps
          exec sleep infinity

      volumeMounts:
        - name: host-root
          mountPath: /host
          readOnly: true

  volumes:
    - name: host-root
      hostPath:
        path: /
        type: Directory
```

- ✓ controller 와 kube-proxy는 Pod로 기동
- ✓ containerd 와 kubelet은 노드에 프로세스로 기동

[참고] containerd 실행 방법

docker 처럼 디렉토리를 마운트하고 실행하는 방법은 다음과 같다

```
> ctr namespaces create skala

> ctr -n skala run \
  --mount type=bind,src=/data,dst=/app/data,options=rbind:rw \
  docker.io/library/alpine:latest \
  my-alpine \
  sleep infinity
```

전체 흐름 이해해보기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Deployment 배포 시 내부 흐름 따라가보기

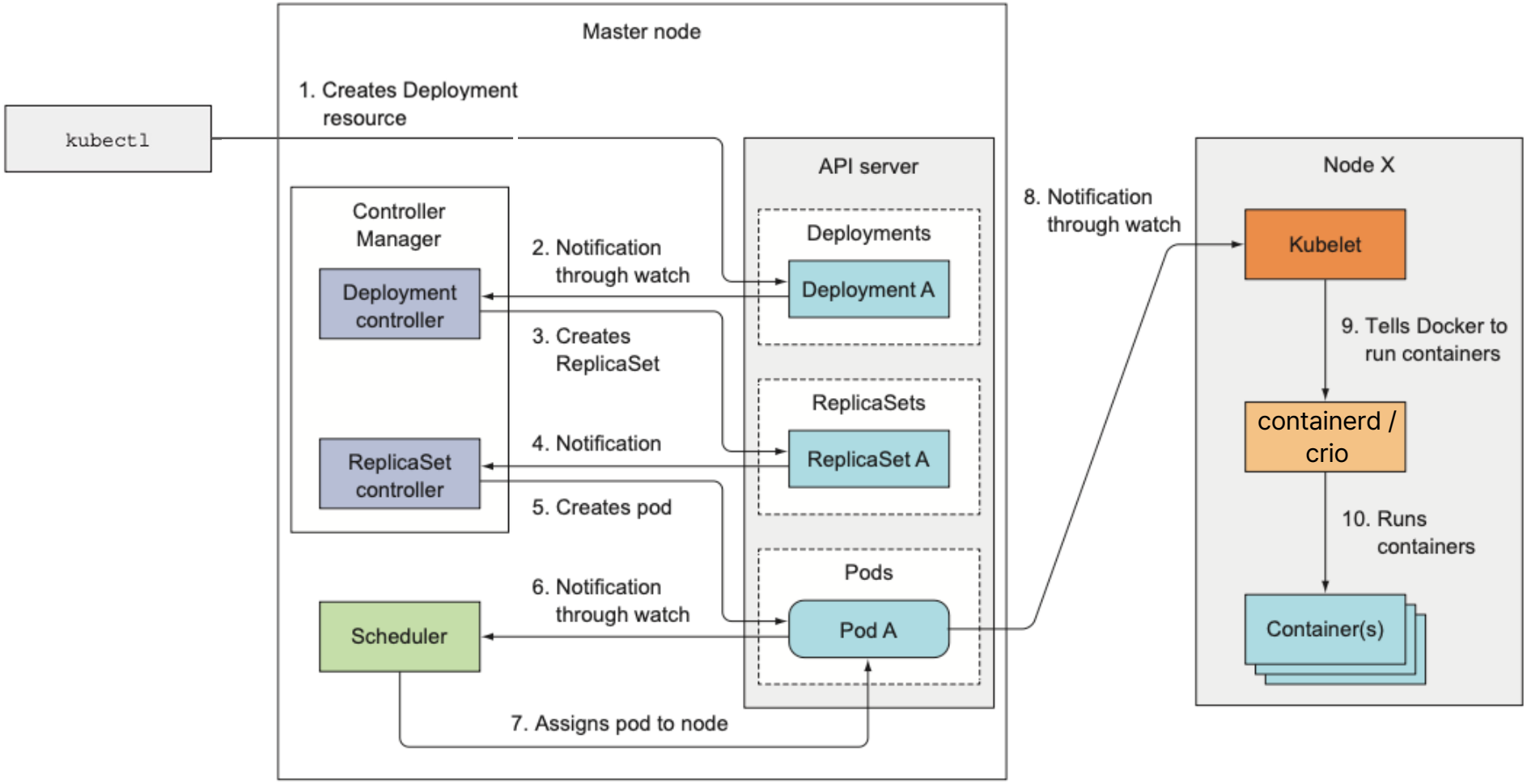
```
# 배포하기
> kubectl apply -f  deployment.yaml
```

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: skala-webserver
  namespace: class-0
  labels:
    app: skala-webserver
spec:
  replicas: 3
  selector:
    matchLabels:
      app: skala-webserver
  template:
    metadata:
      labels:
        app: skala-webserver
    spec:
      containers:
      - name: webserver
        image: skala-registry.skala-ai.com/class-0/skala-webserver:1.0
        imagePullPolicy: Always
        env:
        - name: USER_NAME
          value: skala
```

컨테이너(애플리케이션) 배포 흐름

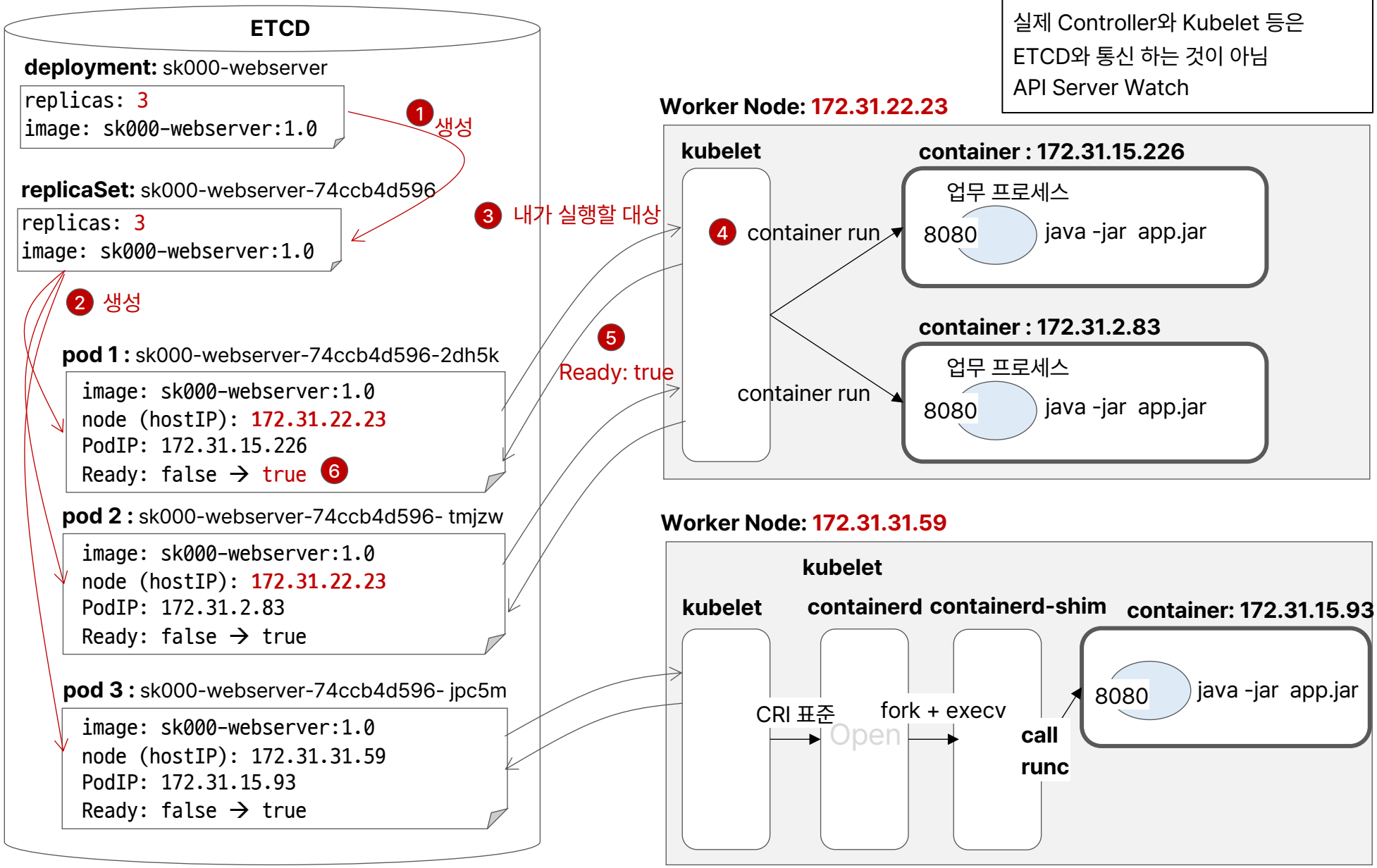
Deployment를 apply 할 경우 내부적 호출 흐름 파악



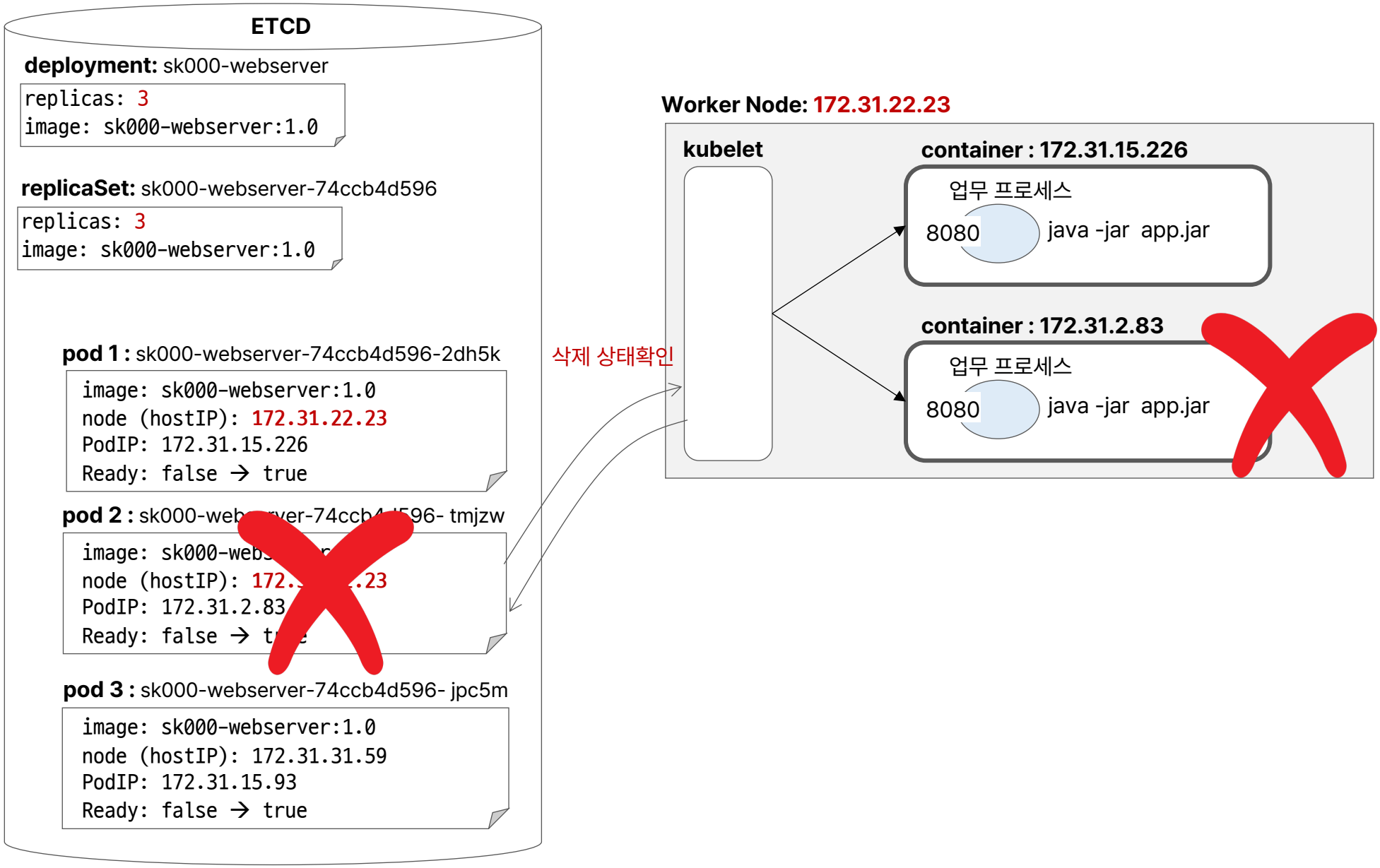
kubernetes-in-action book 참고

Deployment 기반 컨테이너 실행 흐름

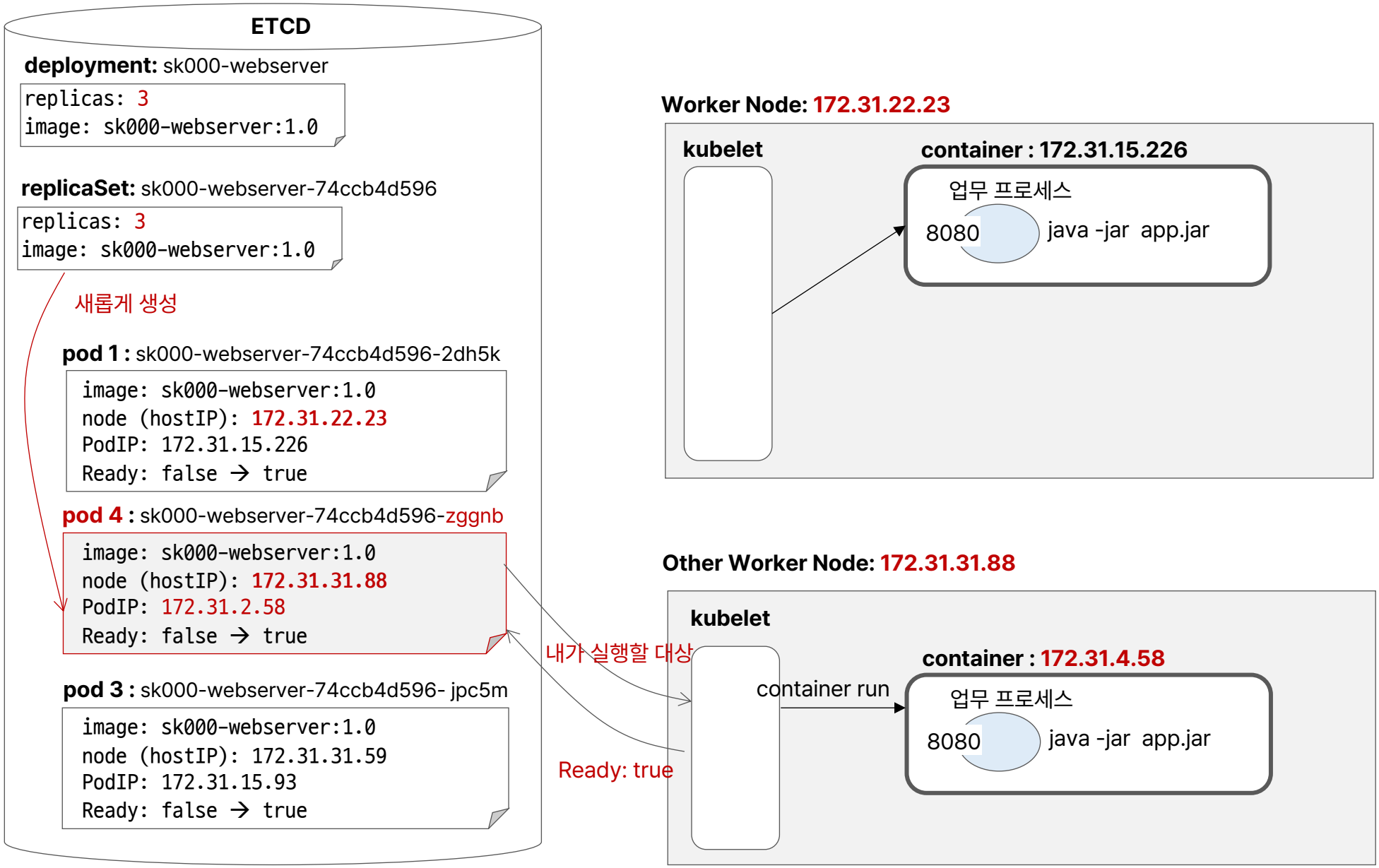
참고:
실제 Controller와 Kubelet 등은
ETCD와 통신 하는 것이 아님
API Server Watch



Deployment 기반 컨테이너 실행 흐름 : 특정 컨테이너 장애로 Pod 삭제 시

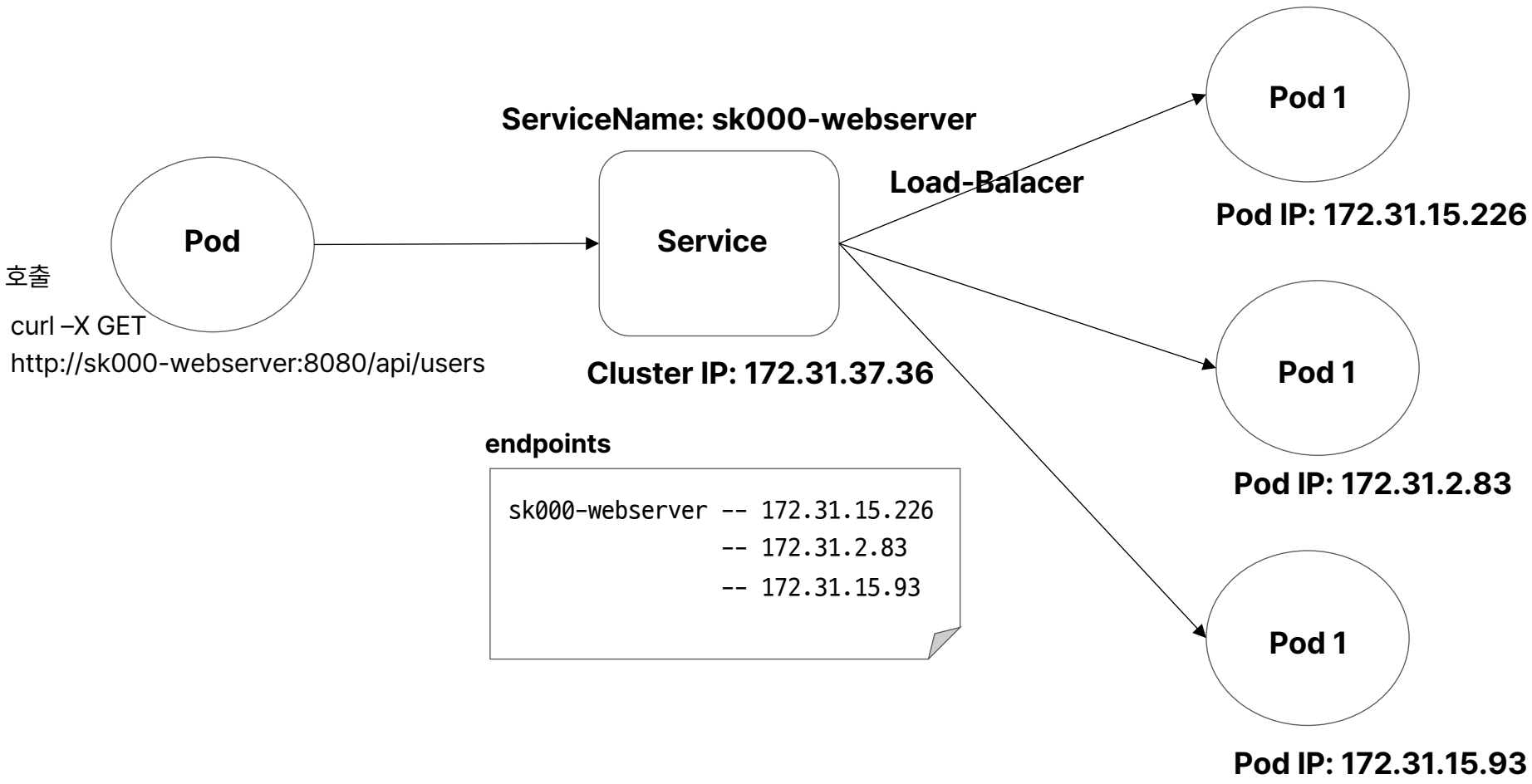


Deployment 기반 컨테이너 실행 흐름 : 새로운 Pod 생성 및 Container 실행

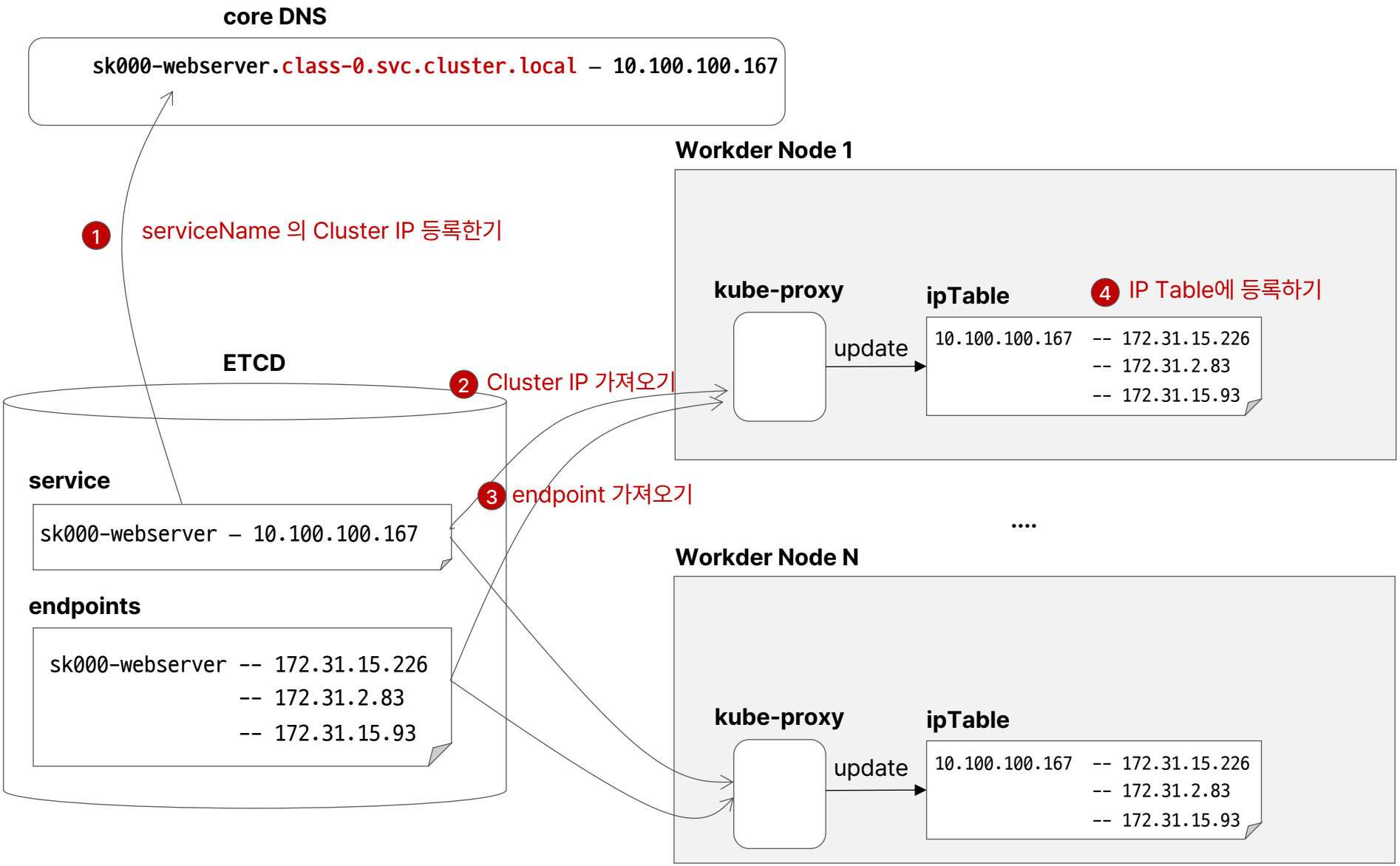


Service 기반 논리적 Pod 호출 흐름

Pod 내에서 서비스 호출 시 흐름은 다음과 같다

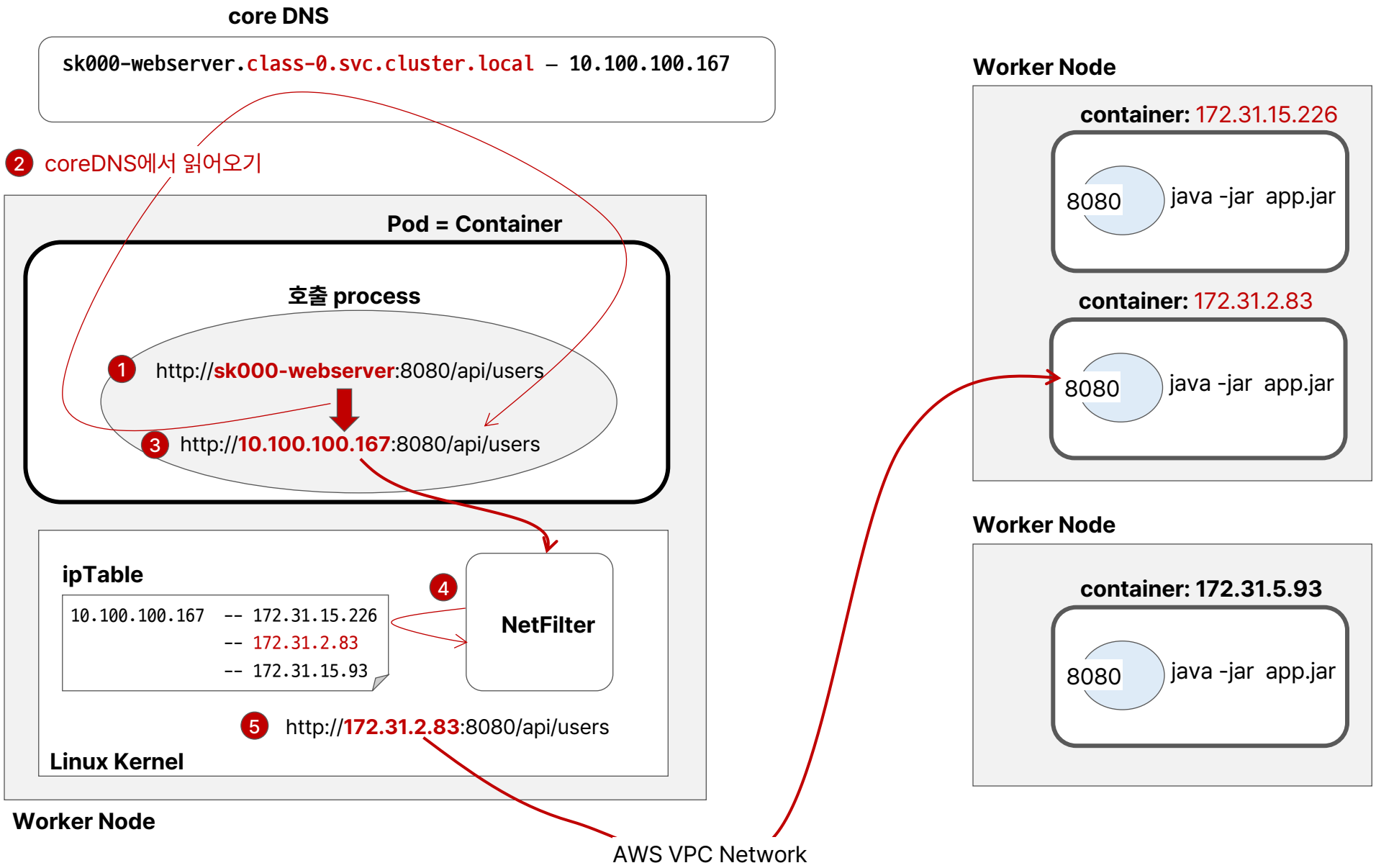


Service 기반 실제 Pod 호출 흐름 : 네트워크 구성



Service 기반 실제 Pod 호출 흐름 : Target Container 찾는 방법

Service를 배포할때 실제 Network를 구성하는 흐름은 다음과 같다



KDT Cloud – 5. 쿠버네티스 심화하기

Pod Argument 전달

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

초기 명령어 및 Argument 전달과 실행

컨테이너에 실행 명령과 인자(args)를 전달하는 방법 정의

command: 컨테이너에서 실행할 명령 (ENTRYPOINT 대체)
args: command에 전달할 인자 (CMD 대체)

```
containers:
- name: webserver
  image: amdp-registry.skala-ai.com/skala25a/sk000-webserver:2.0.0
  imagePullPolicy: Always
  command:
  - python3
  args:
  - fastserver.py
  - --host
  - 0.0.0.0
  - --port
  - "8080"
```

Kubernetes	Docker
command	ENTRYPOINT 또는 CMD
args	CMD

초기 명령어 및 Argument 전달과 실행

초기 실행 시 명령어와 아규먼트를 전달

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-first-deploy-test
  namespace: skala-practice
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sk000-first-deploy-test
  template:
    metadata:
      labels:
        app: sk000-first-deploy-test
    spec:
      containers:
      - name: webserver
        image: amdp-registry.skala-ai.com/skala25a/sk000-webserver:2.0.0
        imagePullPolicy: Always
        command:
          - python3
        args:
          - fastserver.py
          - --host
          - 0.0.0.0
          - --port
          - "8080"
```

Dockerfile의 ENTRYPOINT와 CMD는

→ **K8s command가 대체**

Dockerfile의 CMD는

→ k8s args가 대체

실제 실행

python3 fastserver.py --host 0.0.0.0 --port 8080

작성 방식 (대안)

```yaml
command: ["python3", "fastserver.py"]
args: ["--host", "0.0.0.0", "--port", "8080"]
```

초기 명령어 및 Argument 전달과 실행

컨테이너에 실행 명령과 인자(args)를 전달하는 방법 정의

args만 오버라이드 (일반적 방식)

- Dockerfile ENTRYPOINT 유지
- 실행 옵션만 변경

```
containers:
- image: myapp:1.0
  args: ["--port", "8080"]
```

command + args 명확 분리

- command는 Dockerfile 내 ENTRYPOINT 교체
- args는 CMD는 교체

```
command: ["python3"]
args:
- app.py
- --debug
```

쉘 이용하는 방식

- 여러 명령 실행 이나 환경 변수 처리
- PID 1 SIGNAL 처리 주의 필요

```
command: ["sh", "-c"]
args:
- |
  echo "starting..."
  python3 app.py
```

pid 1 = sh

exec 이용하는 방식

- 현재 프로세스를 교체 (새로운 프로세스를 fork 하지 않음)
- PID 1 SIGNAL 처리 해소

```
command: ["sh", "-c"]
args:
- |
  echo "starting..."
  exec python3 app.py
```

pid 1 = python3

환경 변수 전달 방식

초기 실행 시 명령어와 아규먼트를 전달

- 환경 변수를 활용하여 출력할 때는 $를 사용하여 명령 내용 변경 가능

```
command:
  - /bin/sh
  - -c
args:
  - exec python3 "$EXEC_PYTHON_CODE" --port "$SERVER_PORT"
env:
  - name: EXEC_PYTHON_CODE
    value: fastserver.py
  - name: SERVER_PORT
    value: "9090"
```

환경 변수를 주입하기 위해서는
/bin/sh –c 를 이용해서 command로 주입

이를 통해 실행 단계에서
환경 변수를 주입할 수 있는 구조

참고
/bin/sh –c 는 뒤에 오는 연결된 string을 command로 인식해서 주입해주게 됨. 그러므로 환경 변수로 인식 가능
exec를 사용하지 않으면 container 실행 시 sh 가 1번 프로세스로 생성되고 sh 의 child로 python3가 기동
exec를 사용하게 되면 python3를 바로 1번 프로세스로 동작하게 적용.
이는 SIGTERM 등 signal 처리시 python3가 직접 받도록 지원 하여 안정적인 종료 처리 지원

[참고] PID 1 문제

컨테이너 안에서 PID 1은 그 컨테이너의 최초 프로세스이며, 일반 프로세스와 **행동 규칙**이 다름

Signal 기본 동작 방식

- 일반 프로세스:
  - SIGTERM → 기본 종료
- PID 1:
  - signal Handler가 없으면 무시됨

```
kubectl delete pod my-app
```

실제로 발생하는 시나리오

```
command: ["sh", "-c"]
args:
- python3 app.py
```

```
PID 1: sh
  └─ PID 7: python3 app.py
```

내부에서 실제로 벌어지는 일

① kubectl delete pod 실행 시
② kubelet 이 SIGTERM을 PID 1(sh)에 전다
③ sh:
  - SIGTERM 처리 하지 않음
  - Child Process에 전달하지 않음
④ python3 계속 실행
⑤ graceful shutdown 실패
⑥ SIGKILL 타임아웃 후 강제 종료

결과

```
로그 유실, 연결 끊김, 데이터 손상 발생
```

[실습]

참고 코드 위치: cloud/k8s-ddive/02.cmdargs

1. Dockerfile 과 fastserver.py 확인해보기
2. deployment yaml 생성 후 argument 확인하기 **(deploy.yaml 수정필요)**
3. deployment 배포 후 배포 현황 확인
4. port-forward pod/xxxx 9090 으로 접속해서 확인하기
5. k exec -it skala-myfirst-api-server-xxx -- /bin/bash 접속 후
   > ps –ef

EXEC_PYTHON_CODE 환경 변수와 SERVER_PORT가 실행 단계에서 환경 변수로 연결되어서 동작

KDT Cloud – 5. 쿠버네티스 심화하기

Init-Containr

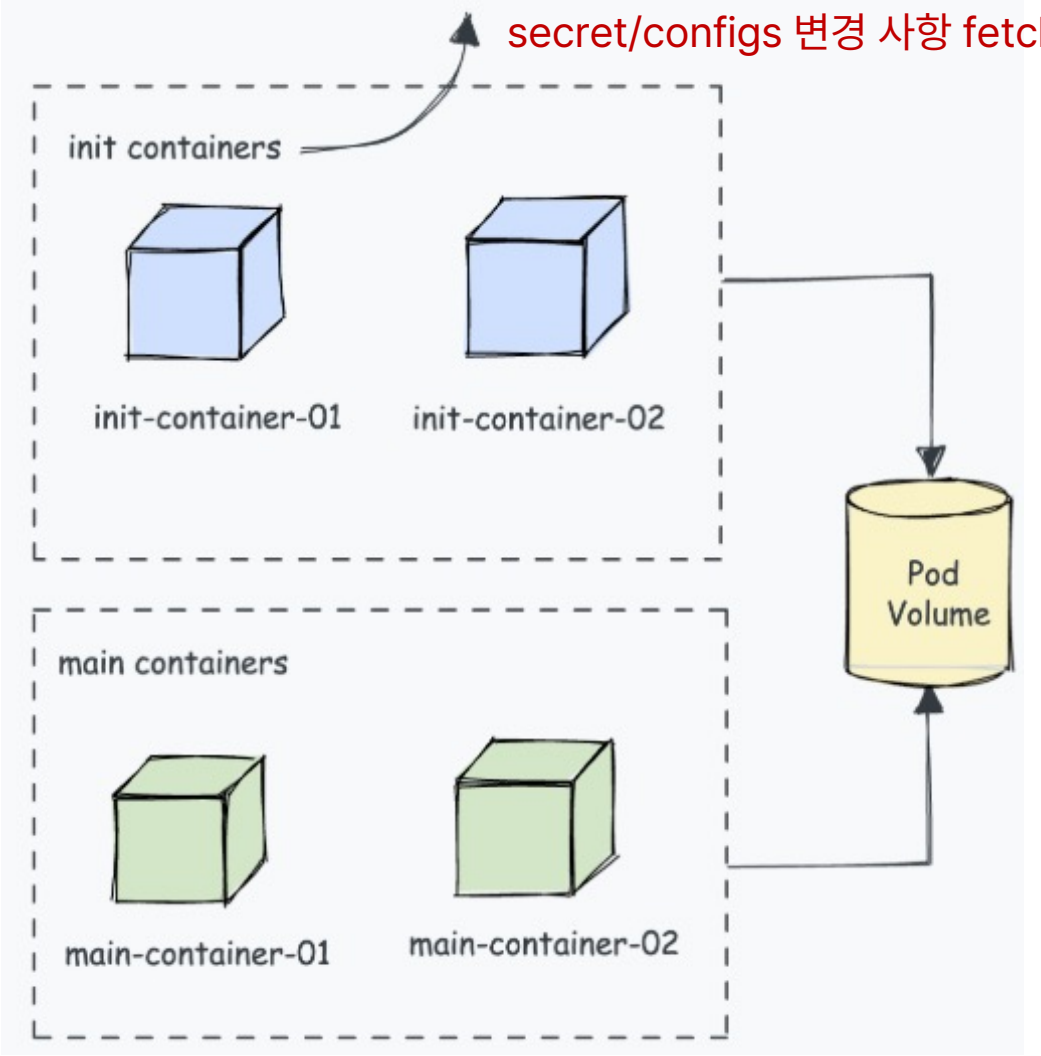
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Init Container

Pod가 시작되기 전에 반드시 먼저 실행되고, 성공적으로 종료되어야만 Main Container가 시작되는 선행 작업용 컨테이너

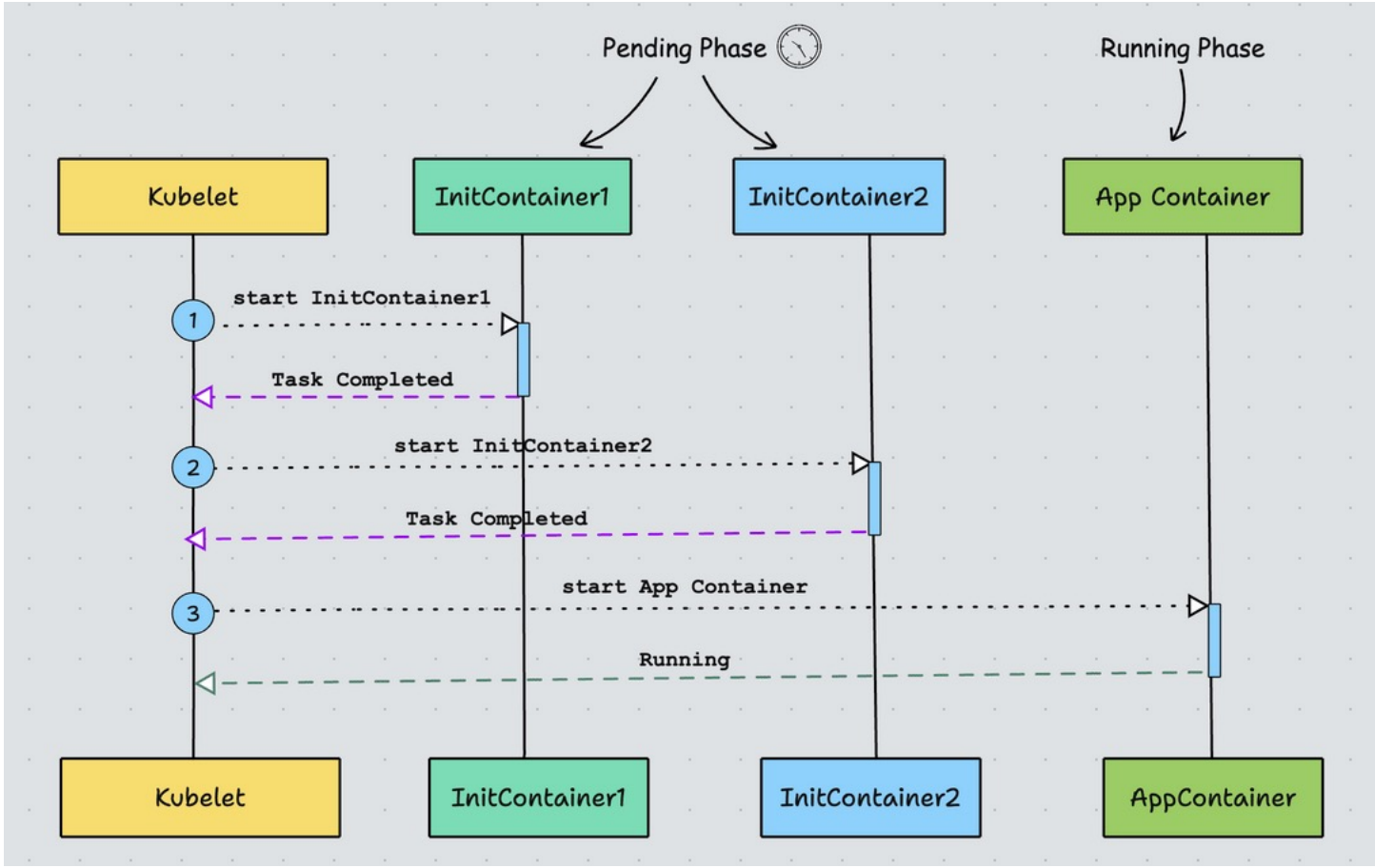
사전 설정, 의존관계 다운로드
secret/configs 변경 사항 fetch 등



init container

Pod의 실행되기 전 가장 먼저 초기에 실행되어 초기화 및 사전 작업을 수행하는 컨테이너로 Pod 내 포함

- Pod 실행 시 최초 실행, Init-container가 정상적으로 종료시 다음 절차가 실행
- 일정 시간 이상 내에 정상 처리되지 않으면 재시작 정책에 따라 재시도



init-container 구조

Pod는 하나의 독자적인 실행 단위이며, kubernetes에서 관리하는 최소 단위

위치: cloud/k8s-ddive/03.init-container/init-pod.yaml

```
 initContainers:
  - name: check-active-enabled
    image: busybox
    command:
    - sh
    - -c
    - |
      timeout 180 sh -c "
      while [ ! -f /root/active.enabled ]; do
        echo 'Waiting for active.enabled file...'
        sleep 5
      done
      echo 'File active.enabled found. Initialization complete.'
      "
    volumeMounts:
    - name: root-volume
      mountPath: /root
…
  volumes:
  - name: root-volume
    emptyDir: {}
```

[실습]

위치: 06.init-container

본 실습은 kubernetes 환경에서 배포를 실행

terminal 1

```
# yaml을 생성한후 생성된 Pod 확인
$ k apply -f init-pod.yaml
$ watch -n 1 "kubectl get pod -l app=sk000-first-api-server"
NAME                          READY   STATUS     RESTARTS
sk000-first-api-server        0/1     Init:0/1   0
--> 현재 empdir의 /root/active.enabled  파일이 존재하지않음



--> active.enabled 파일이 생성되면서 init 완료
sk000-first-api-server       0/1     PodInitializing   0
sk000-first-api-server       0/1     Running           0

--> readinessProbe 동작
sk000-first-api-server       1/1     Running           0
```

terminal 2

```
# init-container (check-active-enabled) 에 접속
$k exec -it sk000-first-api-server -c check-active-
enabled -- /bin/sh
/ # cd /root
~ # ls
~ # touch active.enabled
~
```

[실습]

동작 과정

1. init-container 내부에서 “touch active.enabled” 수행 시
   init-container 정상 종료

```
sk099-init-container-test   0/1     Init:0/1   0          10s
sk099-init-container-test   0/1     PodInitializing   0          85s
```

2. readinessProbe 에서 30초 대기

```
sk099-init-container-test   0/1     Running            0          90s
```

3. 30초 후 readinessProbe는 /root/active.enabled 파일 존재 확인해서 있으면 OK
   외부에서 트래픽이 들어오도록 Pod IP 를 Endpoints에 등록

```
sk099-init-container-test   1/1     Running            0          101s
```

KDT Cloud – 5. 쿠버네티스 심화하기

Sidecar container

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

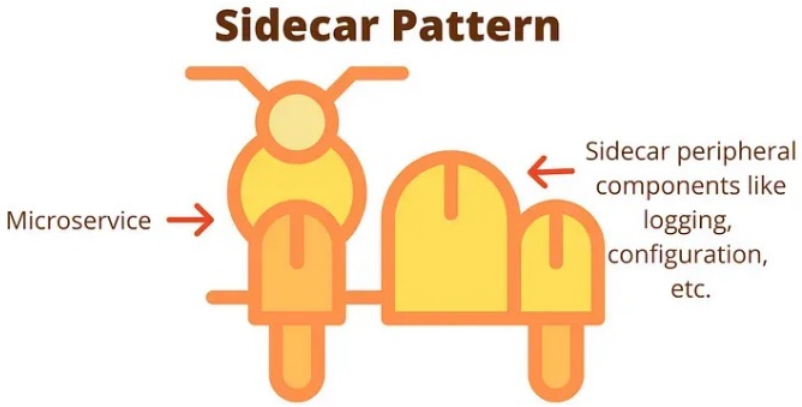
side-car container

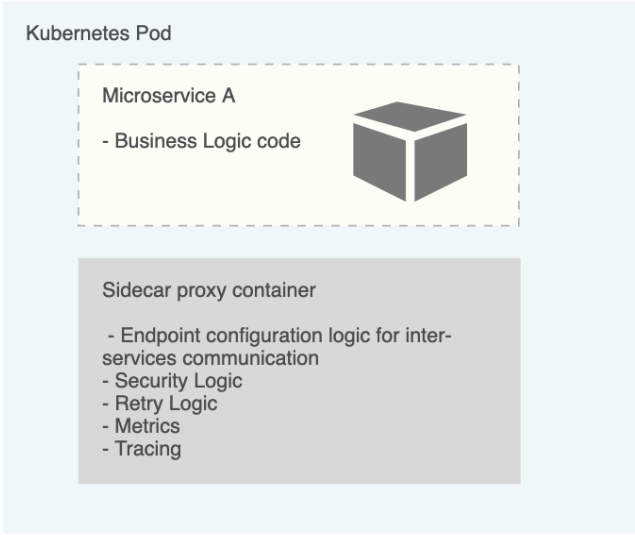
일반적으로 Pod는 주 컨테이너 와 지원 컨테이너인 한 개 이상의 side car 컨테이너로 이루어져 있음
Pod 내 모든 컨테이너가 주 컨테이너로 구성하지 않는 이유는 다음과 같다

- Pod내 컨테이너는 한배를 탄 동지 죽어도 같이 죽고 살아도 같이 산다
- Kubernetes의 핵심은 컨테이너의 탄력적 확장성 지원인데, Pod 내 컨테이너는 지원되지 않는다.

그럼 왜 ? Pod 내 컨테이너를 두개 이상 포함하는 가?

일반적으로 공통 기능을 옆에 같이 포함 (side car) 시켜서 컨테이너의 안정성, 네트워크, 로그, 모니터링, 자가 복구를 지원하도록 한다
예) EKF 의 fluentd, istio(service mesh)의 envoy, APM (Watap) 의 metrics 수집 agent,





side car의 예시 service mesh istio

service간 트래픽 연결을 그물망 처럼 자유롭게 연결

Pod 내 주 컨테이너로 들어오고 나가는 모든 트래픽을 side car container인 proxy가 모두 통제함으로써 유연하게 트래픽 제어

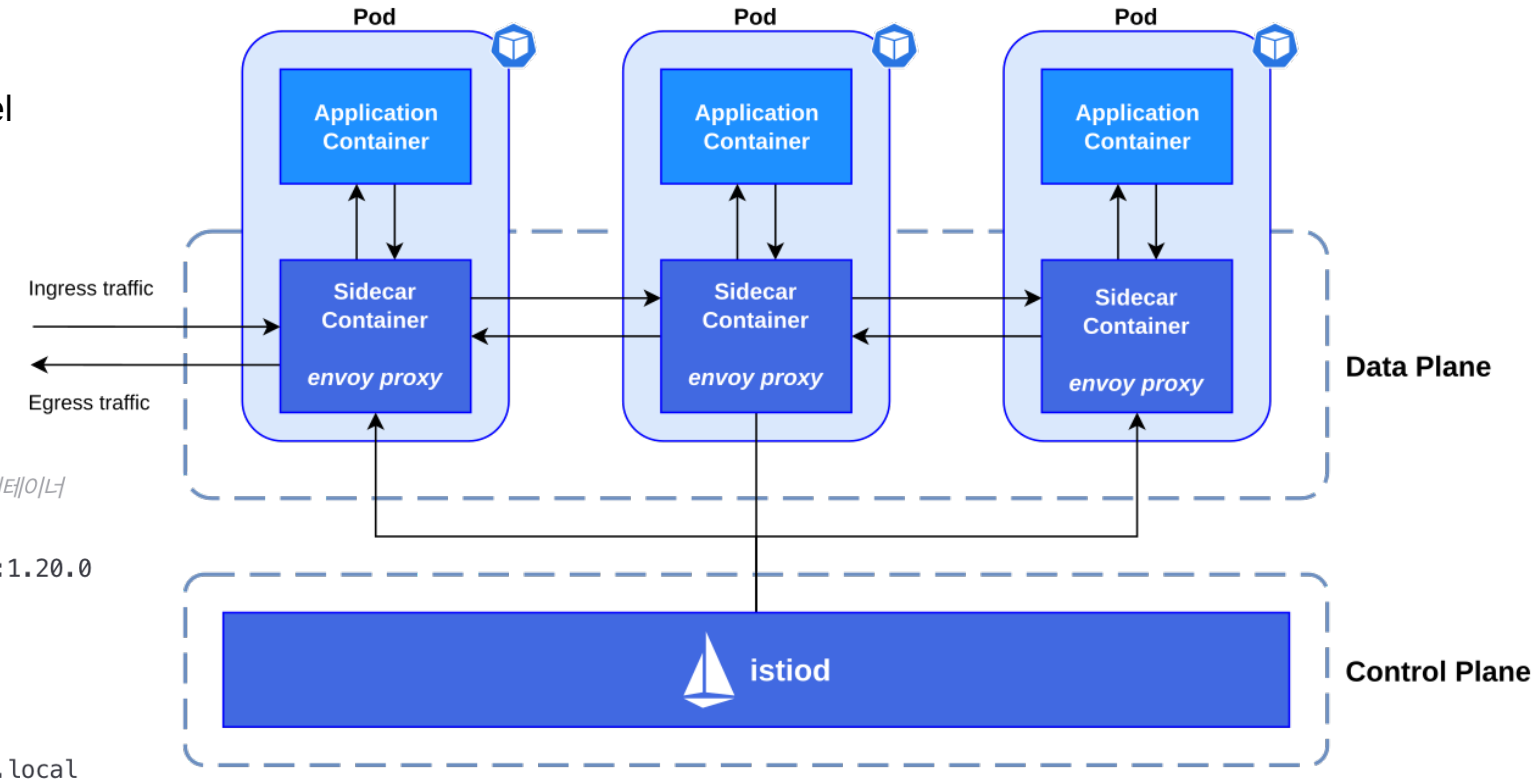
Traffic Flow Control & Traffic Resilient

namespace

istio-injection=enabled label

Pod annotation

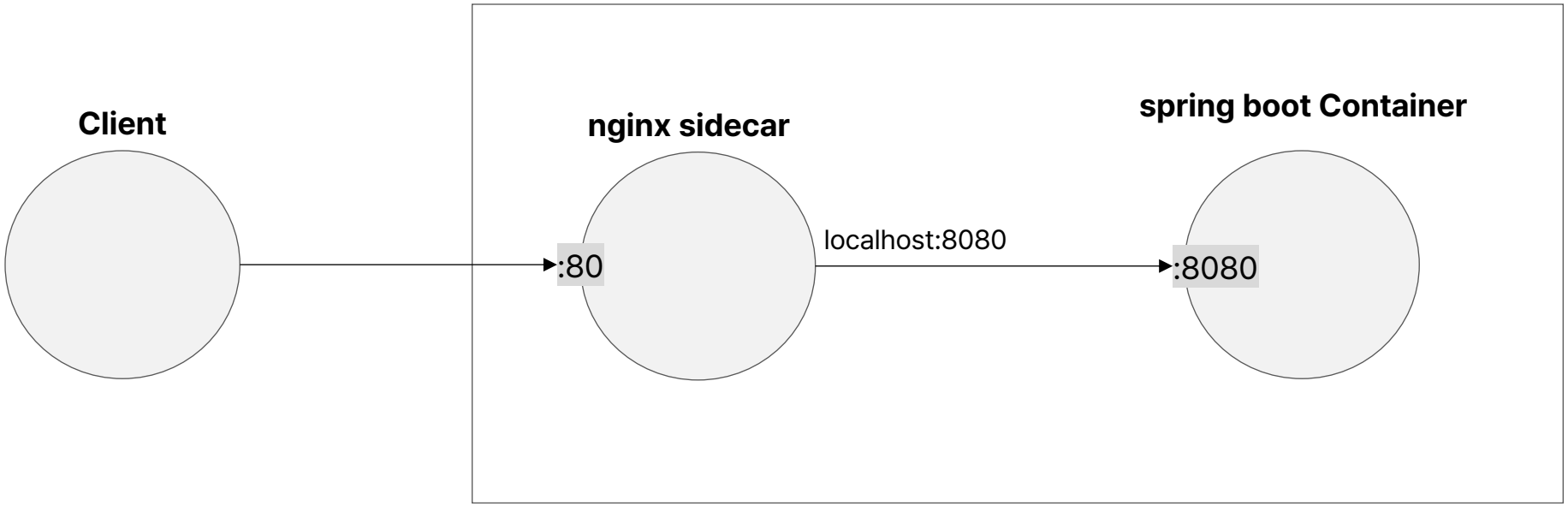
sidcar.isito.io/inject: “true




```yaml
# Istio가 자동으로 주입하는 sidecar 컨테이너
- name: istio-proxy
  image: docker.io/istio/proxyv2:1.20.0
  args:
  - proxy
  - sidecar
  - --domain
  - $(POD_NAMESPACE).svc.cluster.local
  - --proxyLogLevel=warning
  - --proxyComponentLogLevel=misc:error
  - --log_output_level=default:info
  env:
  - name: JWT_POLICY
    value: third-party-jwt
```

[실습] nginx sidecar 적용하기

nginx를 sidecar로 붙여서 Main Spring boot 앞의 Reverse Proxy 역할을 수행하도록 적용해보자



Nginx sidecar의 역할

- 하나의 Pod 안에 Spring Boot + Nginx 두 개의 Container
- 두 Container는 **Network Namespace**를 공유
- Nginx에서 Spring Boot를 **localhost:8080**으로 호출 가능
- 외부에서는 Spring Boot를 직접 호출하지 않고 Nginx :80 → Spring Boot :8080
- Nginx에서 **Header 추가, 로깅 같은 부가 기능**을 수행

[실습] nginx sidecar 적용하기

위치: 07.sidedar/

04.sidecar 내에 deploy.yaml과 nginx-configmap.yaml 내용을 검토해보자

nginx-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: skala-nginx-sidecar-config
  namespace: class-0
data:
  default.conf: |
    server {
        listen 80;

        location / {
            proxy_pass http://127.0.0.1:8080;

            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;

            # Sidecar를 통과했다는 것을 확인하기 위한 Header
            add_header X-Sidecar-Proxy "nginx-sidecar" always;
        }

        location /nginx-health {
            return 200 "nginx sidecar is running\n";
        }
    }
```

deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: skala-myfirst-api-server
…
    spec:
      containers:
      # ------------------------------------
      # Main Container
      # ------------------------------------
      - name: myfirst-api-server
        image: skala-registry.skala-ai.com/class-0/skala-
myfirst-api-server:1.0
…
      # ------------------------------------
      # Sidecar Container
      # ------------------------------------
      - name: nginx-sidecar
        image: nginx:1.27-alpine

        volumeMounts:
        - name: nginx-config
          mountPath: /etc/nginx/conf.d/default.conf
          subPath: default.conf

      volumes:
      - name: nginx-config
        configMap:
          name: skala-nginx-sidecar-config
```

[실습] kubectl sidecar 적용하기

configmap 배포하기

```
> k apply -f nginx-configmap.yaml
> k get configmap | grep sk000
```

deploy 배포하기

```
> k apply –f deploy.yaml
> k rollout status –f deploy.yaml
```

내부 접속해서 접속하기

```
> k exec -it deploy/sk000-myfirst-api-server -c nginx-sidecar -- /bin/sh

# container 내부
cat /etc/nginx/conf.d/default.conf

# main container 접속하기
curl –i localhost:8080/api/users
exit
```

[실습] kubectl sidecar 적용하기

Fortforward로 nginx로 접속 후 spring boot으로 접속해보자

port-forward 연결하기 (Terminal 1)

```
> kubectl port-forward deploy/sk000-myfirst-api-server 8081:80
```

다른 터미널에서 curl 로 접속하기 (Terminal 2)

```
> curl -i http://localhost:8081/api/users
HTTP/1.1 200
Server: nginx/1.27.5
Date: Sat, 26 Sep 2026 08:01:20 GMT
Content-Type: application/json
Content-Length: 149
Connection: keep-alive
X-Sidecar-Proxy: nginx-sidecar
```

```
> curl -i http://lcalhost:8081/nginx-health
HTTP/1.1 200 OK
Server: nginx/1.27.5
Date: Sat, 26 Sep 2026 08:02:28 GMT
Content-Type: application/octet-stream
Content-Length: 25
Connection: keep-alive

nginx sidecar is running
```

KDT Cloud – 5. 쿠버네티스 심화하기

ReplicaSet

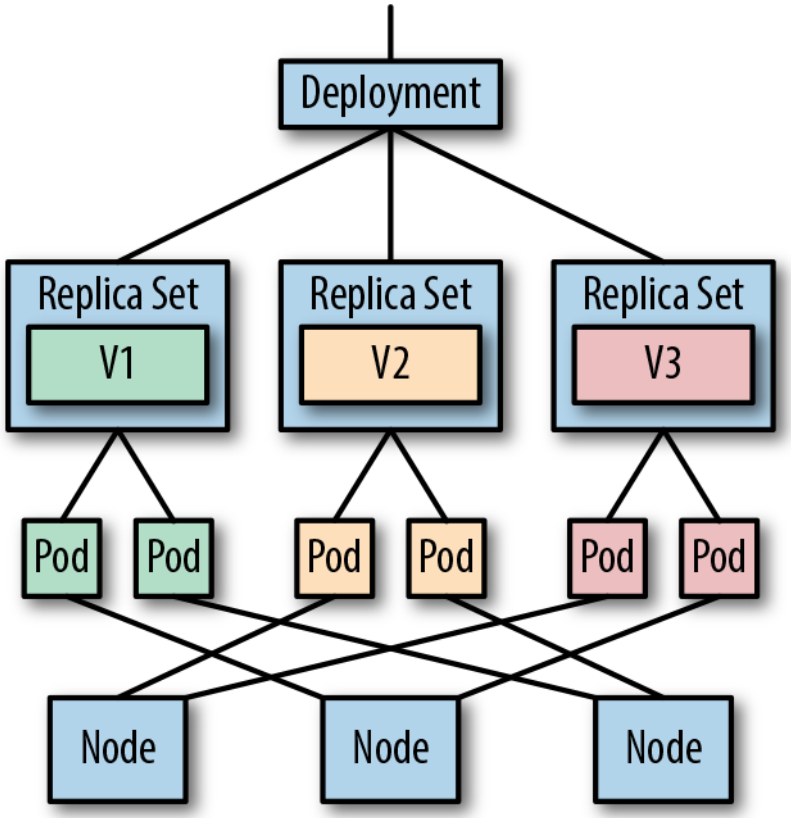
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Deployment

새로운 이미지를 가지는 동일한 Deployment를 배포하는 경우 기존 운영중인 Replica Set Pod와 신규 버전 Replica Set Pod가 점진적 업데이트되는데 이를 통합 관리하는 것이 Deployment

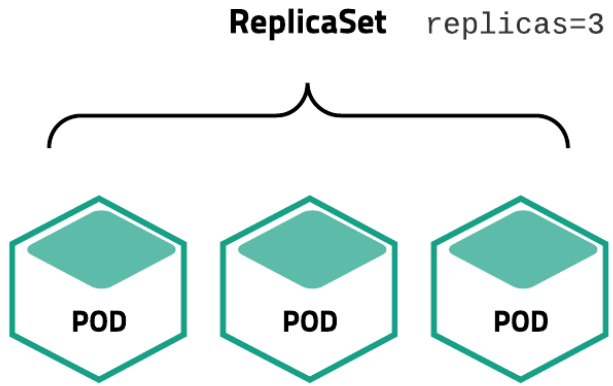
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-myfirst-api-server
  namespace: class-0
spec:
  replicas: 2

  selector:
    matchLabels:
      app: sk000-myfirst-api-server
  template:
    metadata:
      labels:
        app: sk000-myfirst-api-server
    spec:
     containers:
      - name: webserver
        image: xxx/sk000-webserver:1.0
        imagePullPolicy: Always
```



ReplicaSet

ReplicaSet은 Kubernetes에서 지정한 수의 Pod 복제본을 항상 유지하도록 도와주는 컨트롤러
만약 어떤 Pod가 종료되거나 삭제되면, ReplicaSet이 자동으로 새로운 Pod를 생성해서 지정된 수를 유지

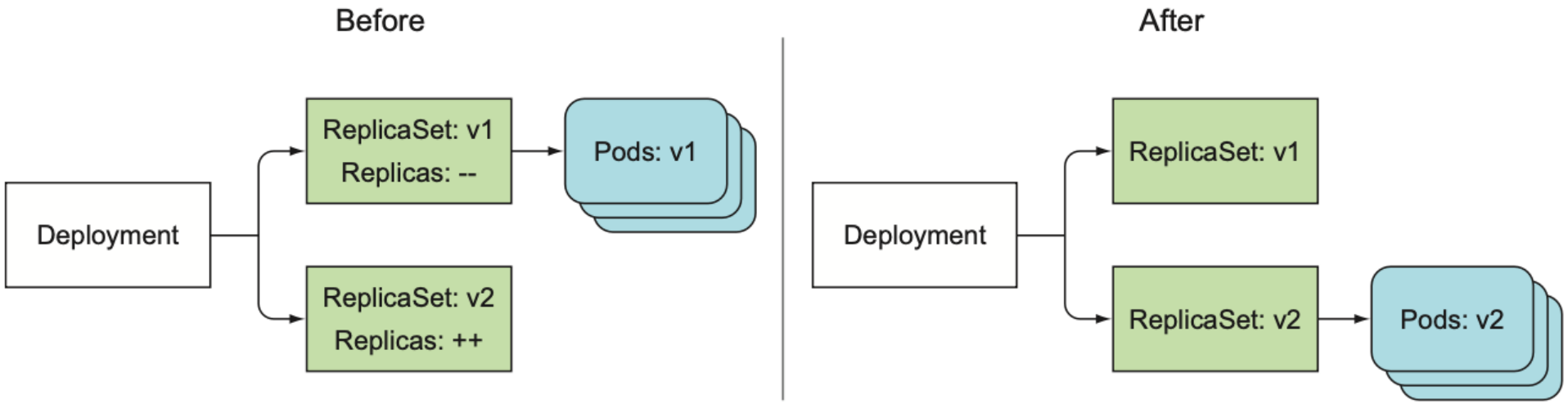
```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: sk000-myfirst-api-server
  namespace: skala-practice
spec:
  replicas: 3

  selector:
    matchLabels:
      app: sk000-myfirst-api-server
  template:
    metadata:
      labels:
        app: sk000-myfirst-api-server
    spec:
     containers:
      - name: webserver
        image: xxx/sk000-webserver:1.0
        imagePullPolicy: Always
```



kubernetes in action book 내용 참고

ReplicaSet

ReplicaSet을 사용자가 생성하지 않고 Deployment 적용 시 ReplicaSet Controller가 RepllicaSet을 자동 구성
Deployment는 여러 개의 ReplicasSet을 제공해서 버전 변화에 따라 Rolling Update 및 Rollback 실행

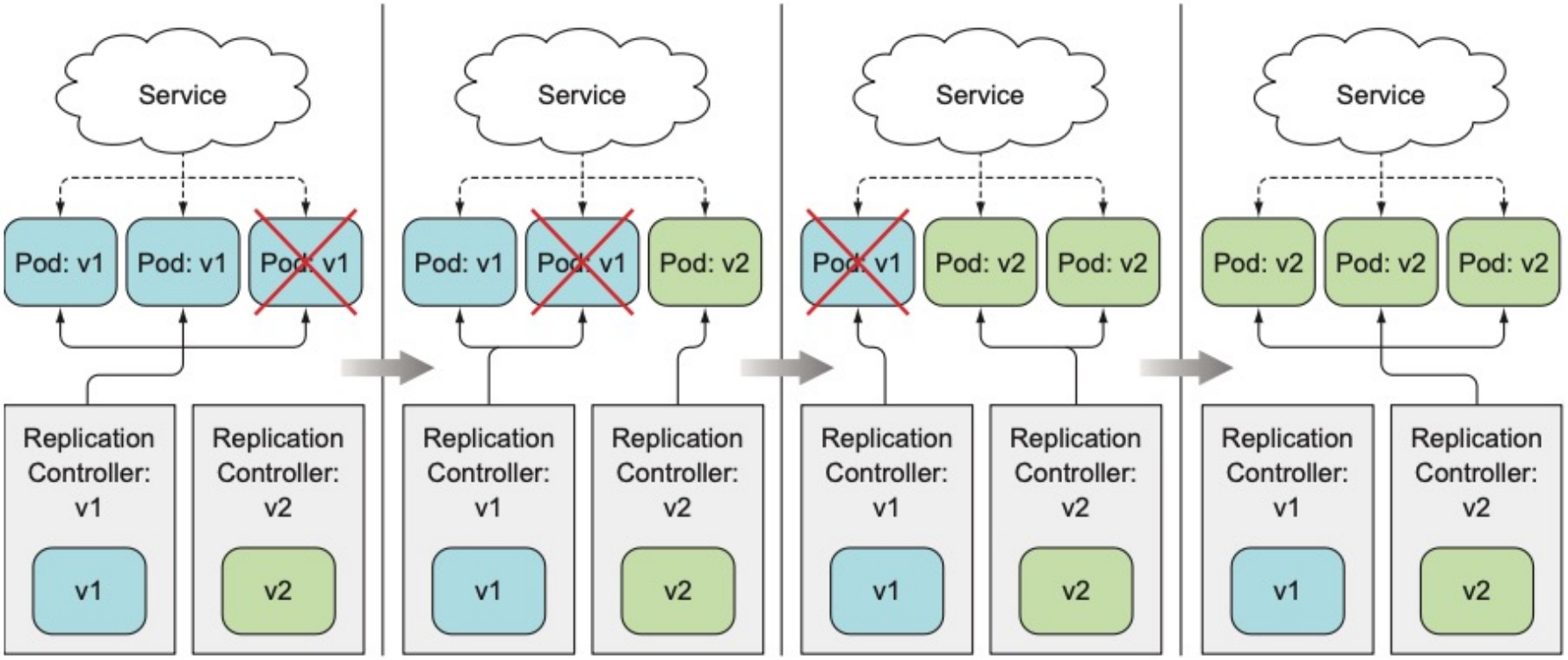


kubernetes in action book 내용 참고

ReplicaSet를 통한 Rolling Update

Rolling Update 방법

- pod 이미지 업데이트 시 새로운 ReplicaSet이 만들어지고 Old 와 New Pod가 점진적 업데이트 (Rolling update) 지원
- ReplicaSet의 선언적 정의는 Deployment 내에 포함되어 있음



ReplicaSet를 통한 Rolling Update

ReplicaSet v1 이에서 ReplicaSet v2로 롤링 업데이트 시 Roll out 속도를 통제하기 위한 방법이 존재

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0
    maxSurge: 1
```

maxSurge: (최대 넘침)

Rolling Update 시 기존 desired replica 개수보다 추가로 생성될 수 있는 Pod 수

maxUnavailable:

업데이트 중에 기존 desired replica 개수 기준 동작하지 않아도 되는 Pod 수

Pod가 3일때 가정해보면

maxSurge 1은

Desired 개수 3개보다 최대 1개 더, 즉 4개까지 임시로 만들 수 있다

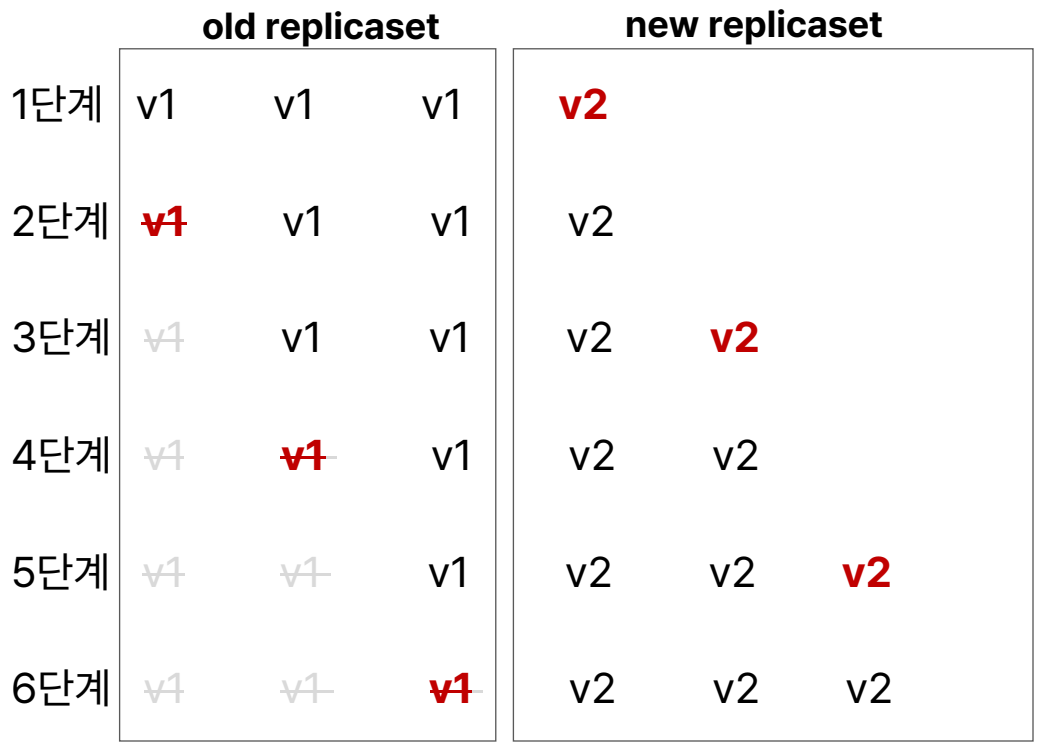
maxUnavailable 0은

업데이트 중에도 **기존 서비스 가능한 Pod 3개를 절대 2개로 줄이지 않음**

kubernetes in action book 내용 참고

Rolling Update 동작 방식 : CASE 1 (1/2)

설정	Pod 3개일 때 의미
replicas: 3	목표 Pod 수 3개
maxUnavailable: 0	Ready Pod를 3개 아래로 줄이지 않음 (무조건 3 유지)
maxSurge: 1	업데이트 중 최대 4개까지 허용 (Pod 3 + 1)



kubernetes in action book 내용 참고

Rolling Update 동작 방식 : CASE 1 (2/2)

replicas=3, maxSurge = 1 and maxUnavailable = 0

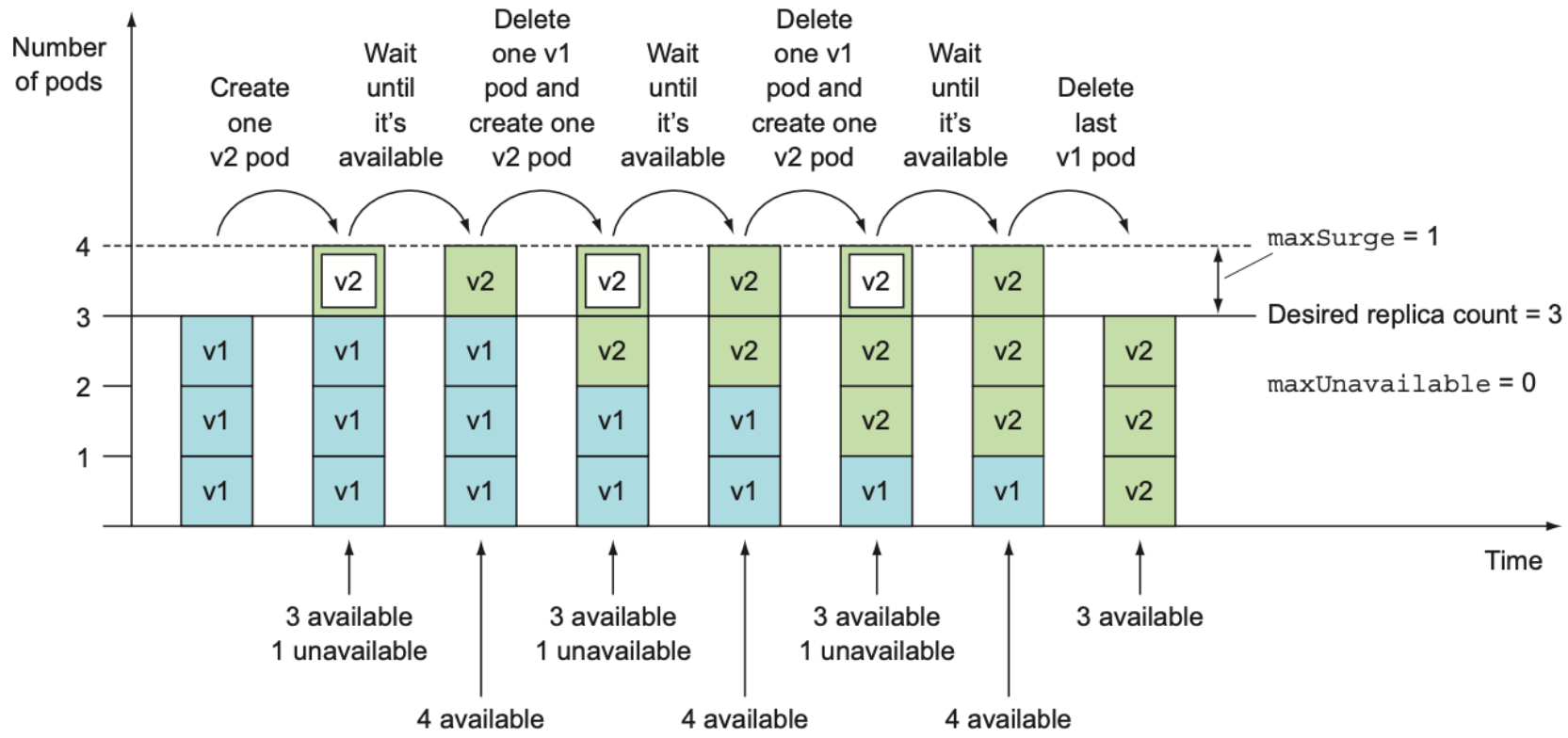
최소 3개는 항상 살아 있어야 한다.

- 최소 Pod 수 : 3 (replicas - maxUnavailable)
- 최대 Pod 수 : 4 (replicas + maxSurge)

실행 절차

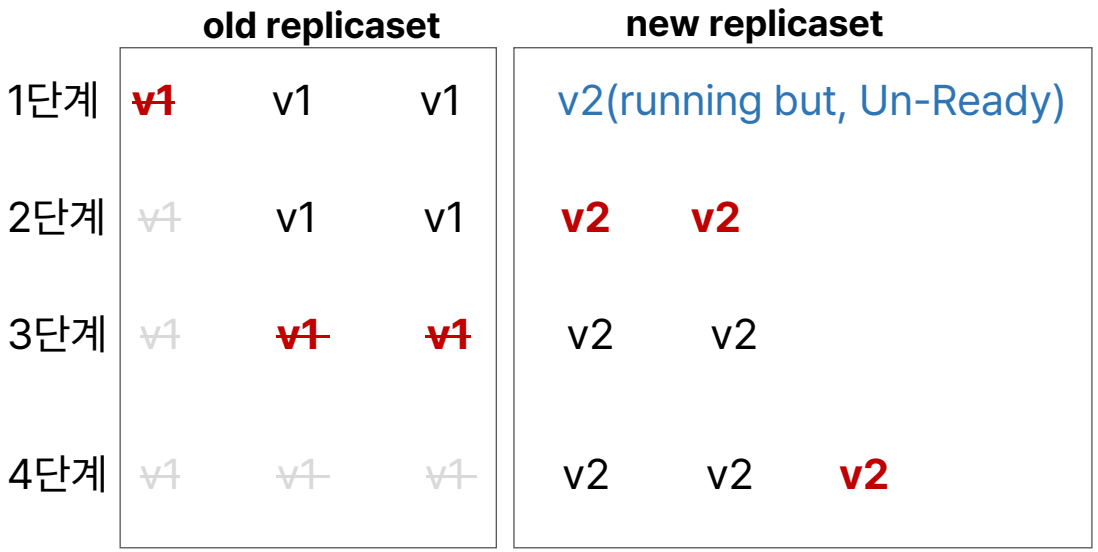
- 1단계: v1.0 Pod 수를 최소 Pod 수만큼 제거 (3 → 3)
- 2단계: v2.0을 최대 Pod 수만큼 추가 (3 → 4)
- 3단계: v1.0을 최소 파드 수만큼 제거 (4 → 3)
- 4단계: v 2.0을 최대 Pod 수만큼 추가 (3 → 4)

....



Rolling Update 동작 방식 : CASE 2 (1/2)

설정	Pod 3개일 때 의미
replicas: 3	목표 Pod 수 3개
maxUnavailable: 1	최소 Available Pod 수 = 3 - 1 = 2
maxSurge: 1	최대 전체 Pod 수 = 3 +1 = 4



자원이 총 3개만 띄울 수 있을때
즉, 자원이 부족할때 유용

주의 : 실제는 maxUnavalable이 무조건 제거되기 보다는 최악의 경우 maxUnavailable을 맞춰야 한다는 의미
기본 스케쥴리은 Replicas= 3을 맞추려고 노력함

kubernetes in action book 내용 참고

Rolling Update 동작 방식 : CASE 2 (2/2)

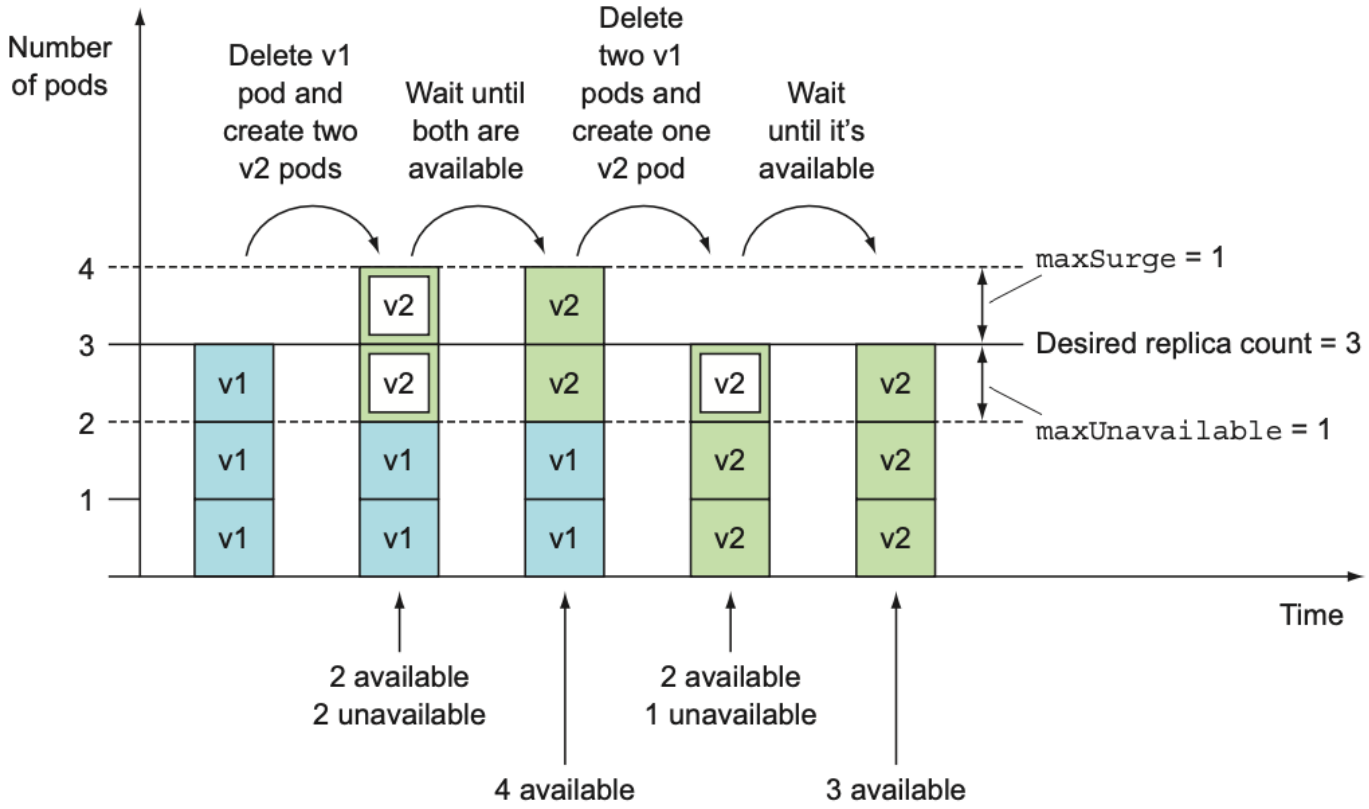
replicas=3, maxSurge = 1 and maxUnavailable = 1

최소 2개는 항상 살아 있어야 한다.

- 최소 Pod 수 : 2 (replicas - maxUnavailable)
- 최대 Pod 수 : 4 (replicas + maxSurge)

실행 절차

- 1단계: v1.0 Pod 수를 최소 Pod 수만큼 제거 (3 → 2)
- 2단계: v2.0을 최대 Pod 수만큼 추가 (2 → 4)
- 3단계: v1.0을 최소 파드 수만큼 제거 (4 →2)
- 4단계: 모든 v1.0제거 되었으므로 Replicas=3으로 v2.0 추가 (2 → 3)



[참고] 설정하지 않는 경우 기본값

default 값으로 동작

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 25%
    maxSurge: 25%
```

필드	기본값	설명
strategy.type	RollingUpdate	기본 업데이트 방식 (other: Recreate)
maxSurge	25%	전체 replica 수의 25%까지 새 파드를 초과 생성
maxUnavailable	25%	전체 replica 수의 25%까지 기존 파드 unavailable 허용

kubernetes in action book 내용 참고

Strateg.type 종류

전략 타입 (type)	동작 방식	다운타임	특징 및 사용 시점
RollingUpdate (기본값)	구버전 Pod를 하나씩 점진적으로 제거하면서 새 버전 Pod를 띄움	**무중단 (Zero Downtime)**	일반적인 무중단 배포의 표준
Recreate	실행 중인 구버전 Pod를 먼저 완전히 종료 뒤 새 버전 Pod를 일괄 생성	**다운타임 발생**	구버전과 신버전이 동시에 실행되면 안 되는 경우 (예: DB 스키마 단독 변경, ReadWriteOnce 볼륨 중복 마운트 불가 시)

kubernetes in action book 내용 참고

[사전 작업] 기존의 deploy 를 제거

```
# 사전에 배포했던 deployment를 제거하자

> k delete deploy sk000-myfrist-api-server
```

[실습] Rolling Update & Roll back (1/)

위치: 08.replicaset

myfrist-api-server:1.0 과 2.0를 기준으로 Rolling update 와 Rollback을 진행

```
# VERSION = 1.0으로
> kubectl apply –f deploy-v1.0.yaml


# VERSION = 2.0으로
> kubectl apply –f deploy-v2.0.yaml


> k get replicaset -l  app=sk000-myfirs-api-server
NAME                                DESIRED   CURRENT   READY   AGE
sk000-myfirst-api-server-5b4dc7bb86   1         1         1       16m
sk000-myfirst-api-server-657bcbc866   0         0         0       17m


> k describe replicaset sk000-myfirst-api-server-657bcbc866
Name:           skala-myfirst-api-server-7c78dc97f5
Namespace:      class-0
…
  Containers:
   webserver:
    Image:      skala-registry.skala-us.com/class-0/sk000-webserver:2.0

> k describe replicaset sk000-myfirst-api-server-657bcbc866
```

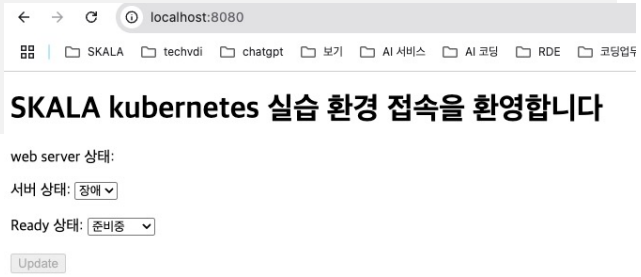
kubernetes in action book 내용 참고

[실습] Rolling Update & Roll back

위치: 08.replicaset

1. 만들어 놓은 revision 목록 검색

```
> kubectl rollout history deploy sk000-myfirst-api-server
REVISION  CHANGE-CAUSE
1         <none>
2         <none>


> kubectl rollout history deploy sk000-myfirst-api-server   --revision=2
deployment.apps/sk000-my-app with revision #3
Pod Template:
  Labels:     app=sk000-myfirst-api-server
              pod-template-hash=57f694b86f
  Annotations:              prometheus.io/path: /actuator/prometheus
              prometheus.io/port: 8081
              prometheus.io/scrape: true
              update: 96699fbc6c0893f3dbd0f87e32a1291f
  Service Account:          default
  Containers:
   my-app:
    Image:    amdp-registry.skala-ai.com/skala25a/sk000-myfirst-api-server:2.0.0-NEW
    Port:     <none>
    Host Port:              <none>
```



2. 접속 해보기

```
> kubectl port-forward   deploy/sk000-myfirst-api-server  8080:8080
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
```

kubernetes in action book 내용 참고

[실습] Rolling Update & Roll back

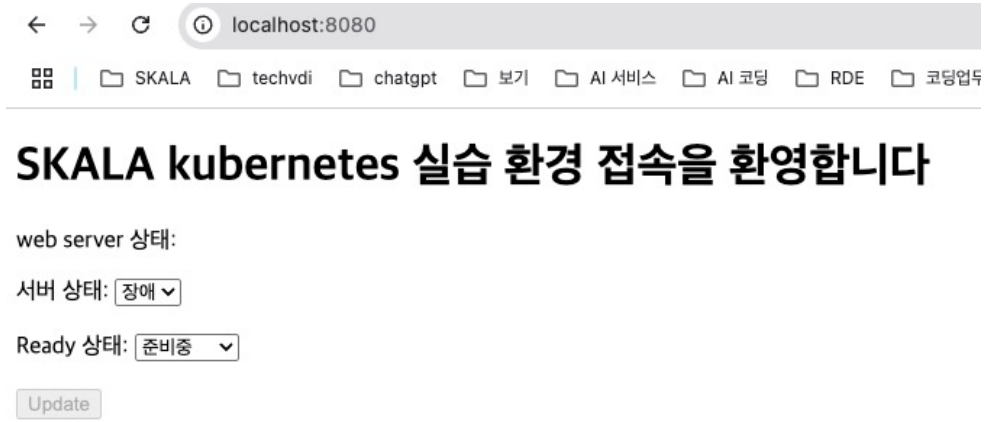
위치: 08.replicaset

2. 2.0 version 접속하기

```
> kubectl port-forward   deploy/sk000-myfirst-api-server  8080:8080
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
```

3. 브라우저로 접속

http://localhost:8080



[실습] Rolling Update & Roll back

위치: 08.replicaset

1. 직전 버전으로 롤백하기

```
> kubectl rollout undo deploy  sk000-myfirst-api-server
REVISION  CHANGE-CAUSE
2         <none>
3         <none>
```

2. Rollback 이미지 확인

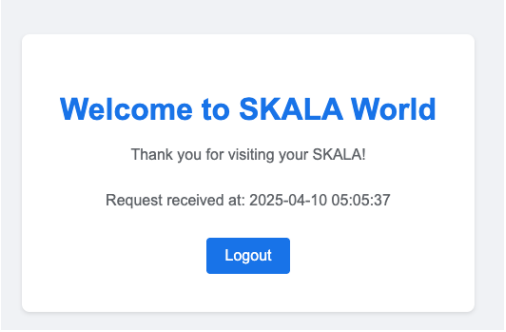
```
> kubectl rollout history deploy sk000-myfirst-api-server   --revision=3
```

3. 롤백 후 정상적으로 넘어 갔는지 확인해보자

```
> kubectl port-forward   deploy/sk000-myfirst-api-server   8080:8080
```

4. 브라우저로 접속

http://localhost:8080



kubernetes in action book 내용 참고

[실습] Rolling Update & Roll back

위치: 08.replicaset

1. 특정 버전으로 롤백 하기

```
> kubectl rollout undo deploy sk000-myfirst-api-server  --to-revision=2
REVISION  CHANGE-CAUSE
3     <none>
4     <none>
```

2. Rollback 이미지 확인

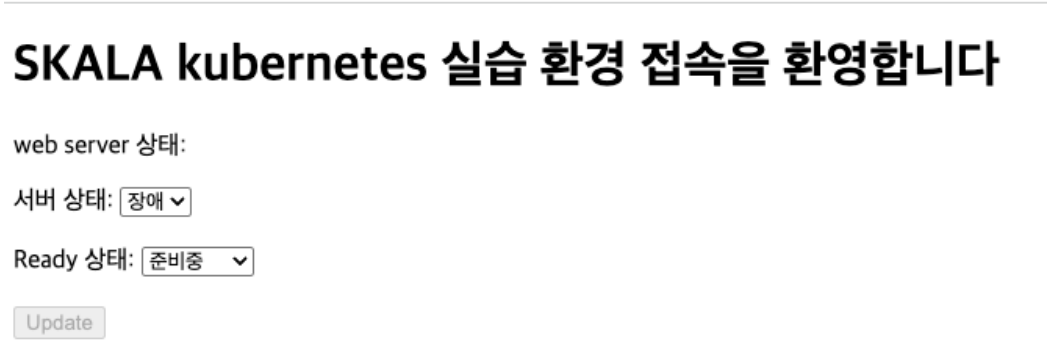
```
> kubectl rollout history deploy sk000-myfirst-api-server  --revision=4
```

3. 롤백 후 정상적으로 넘어 갔는지 확인해보자

```
> kubectl port-forward deploy/sk000-myfirst-api-server   8080:8080
```

4. 브라우저로 접속

http://localhost:8080



[완료 작업] 기존의 deploy 를 제거

```
> k delete -f deploy-v1.0.yaml
> k delete –f deploy-v2.0.yaml
```

kubernetes in action book 내용 참고

KDT Cloud – 5. 쿠버네티스 심화하기

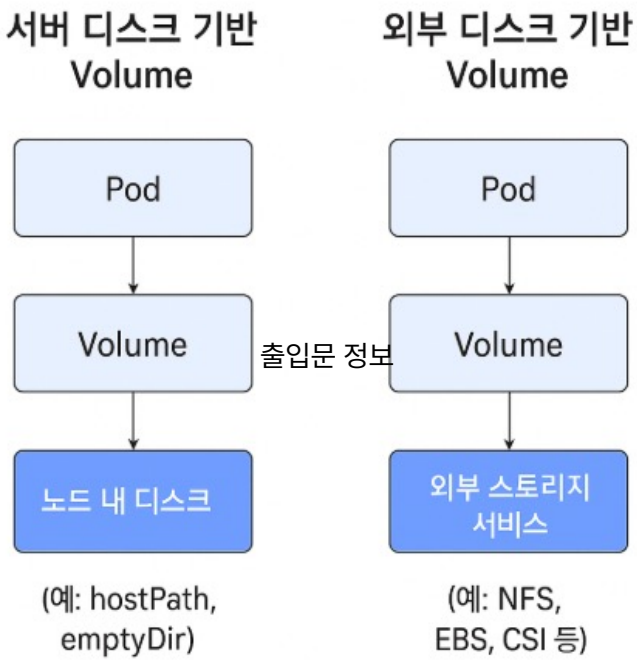
Persistent Volumes

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Volume이란?

볼륨은 컨테이너가 데이터를 저장하고 공유할 수 있는 스토리지를 **할당/연결한 단위**를 의미
(스토리지: 창고, 볼륨은 창고에서 내자리로 배정한 공간)
쿠버네티스는 기본적으로 컨테이너가 종료되면 해당 컨테이너 내부의 파일 시스템도 함께 사라지기 때문에,
지속적인 데이터 저장이나 여러 컨테이너 간의 데이터 공유가 필요할 경우 볼륨을 사용



항목	로컬 볼륨	외부 볼륨
예시	**hostPath, emptyDir**	**nfs, EBS, EFS 등**
저장 위치	노드(서버)의 로컬 디스크	외부 스토리지 시스템
데이터 영속성	Pod 종료 시 소멸하거나, 노드 삭제 시 소멸 가능	Pod 삭제와 무관하게 데이터 유지 가능
공유 가능성	동일 노드의 Pod끼리만 공유 가능	여러 노드 또는 클러스터에서 공유 가능
성능	빠른 I/O (로컬 디스크 접근)	네트워크를 통한 접근 I/O 지연 가능성
클라우드 지원	일반적으로 클라우드 비의존적	클라우드/스토리지 플랫폼과 연계
유연성	스케줄링 제약 (같은 노드에서만)	스토리지 분리로 스케줄링 유연함
관리 편의성	간단하지만 위험 부담 있음	초기 설정 필요하지만 안정적

kubernetes in action book 내용 참고

Volume 종류

Pod에서 사용 가능한 볼륨 종류

구분	emptyDir	hostPath	EBS (CSI)	EFS (CSI)
분류 축	Pod 임시 Volume	Node 로컬 Volume	Block Storage	Network File Storage
정의	Pod 생명주기 동안만 존재하는 임시 디렉터리	Node의 특정 경로를 Pod에 직접 마운트	AWS의 블록 스토리지를 PV로 사용	AWS의 공유 파일 시스템을 PV로 사용
데이터 영속성	Pod 삭제 시 소멸	Node에 종속	블럭 스토리지	파일 스토리지
Node 종속성	없음	Node 디스크 사용	AZ 내 사용	네트워크 연결 디스크
다중 Pod 공유	불가	Node 내에서만	불가	가능
주 사용 목적	캐시, 임시 파일	로컬 테스트, 특수 케이스	단일 Pod DB	다중 Pod 공유 스토리지
운영 환경	운영 중 상태 임시 저장	비 권장	권장	권장

emptyDir:

-v option 또는 Dockerfile의 VOLUME 을 통해 특정 디렉토리 가인 볼륨 이름으로 마운트하는 경우
임시 디렉토리를 할당해서 제공하는 방식과 동일

hostPath:

-v ${Local Directory Path}:${CONTAINER_DIR_PATH} 와 동일한 방식

kubernetes in action book 내용 참고

EBS 와 EFS

EBS와 EFS의 비교

구분	EBS (Elastic Block Store)	EFS (Elastic File System)	NFS (Network File System)
스토리지 타입	Block Storage (블록 단위)	File Storage (파일 단위)	File Storage (파일 단위)
관리 주체	클라이언트 OS가 파일시스템 관리 (ext4, xfs 등 직접 포맷)	AWS 관리형 파일시스템 (POSIX 호환)	자체 NFS 서버가 파일시스템 관리
접근 단위	단일 AZ 내	멀티 AZ (VPC 내 전체)	멀티 AZ
마운트 방식	한 번에 하나의 EC2 인스턴스에만 연결 (RWO)	여러 EC2, EKS Pod에서 동시에 마운트 (RWX)	여러 클라이언트에서 동시에 마운트 가능 (RWX)
접근 모드	ReadWriteOnce (RWO)	ReadWriteMany (RWX)	ReadWriteMany (RWX)
성능 특성	고성능, 낮은 지연시간 (DB, 트랜잭션 적합)	확장성 뛰어남, 동시접속 많을수록 효율적 (공유 스토리지 적합)	성능은 네트워크/서버에 의존, 대규모 환경에선 한계
확장성	단일 볼륨 최대 16TiB (EC2당 연결 제한 있음)	자동 확장 (페타바이트 단위)	서버 디스크 용량에 따라 제한
용도	데이터베이스, 로그 저장소	공유파일 저장, 워크로드 간 공유 볼륨	공유파일 저장, 워크로드 간 공유 볼륨
비용 모델	프로비저닝 용량 기반 (GB/월 + IOPS)	사용량 기반 (GB/월)	직접 서버/스토리지 운영 비용

kubernetes in action book 내용 참고

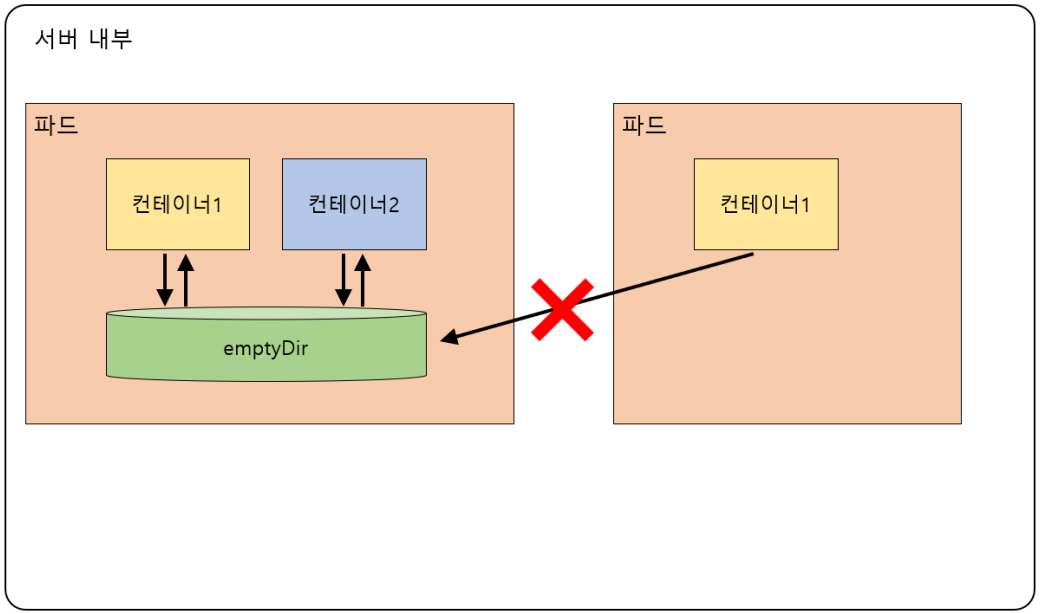
볼륨 종류: emptydir

Pod가 시작될 때 Empty Directory로 생성되며, Pod 내 모든 컨테이너가 공유하도록 지원

Pod 종료 시 kubelet은 임시로 만들어 놓은 emptydir도 함께 삭제

사용 사례

- 임시 캐시
- 로그 버퍼
- 임시 작업 공간
- Pod 내 컨테이너 간 임시 데이터 공유




```yaml
spec:
  containers:
  - name: busybox
    image: busybox
    command: ["/bin/sh", "-c"]
    args:
      - echo "Hello from busybox"
    volumeMounts:
    - name: emptydir-volume
      mountPath: /mnt/data
  volumes:
  - name: emptydir-volume
    emptyDir: {}
```

볼륨을 어느 디렉토리에 마운트할지 선언 (**container 수준**)

어떤 볼륨을 사용할지 선언 (**Pod 수준**)

이것과 동일

```
docker run --rm -v /mnt/data busybox
```

kubernetes in action book 내용 참고

볼륨 종류: emptydir 예시

적용 방식

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: busybox-emptydir-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: busybox-emptydir
  template:
    metadata:
      labels:
        app: busybox-emptydir
    spec:
      containers:
      - name: busybox
        image: busybox
        command: ["/bin/sh", "-c"]
        args:
          - echo "Hello from busybox" > /mnt/data/hello.txt && sleep 3600
        volumeMounts:
        - name: emptydir-volume
          mountPath: /mnt/data
      volumes:
      - name: emptydir-volume
        emptyDir: {}
```

disk

```yaml
emptyDir: {}
```

memory

```yaml
emptyDir:
  medium: "Memory"
```

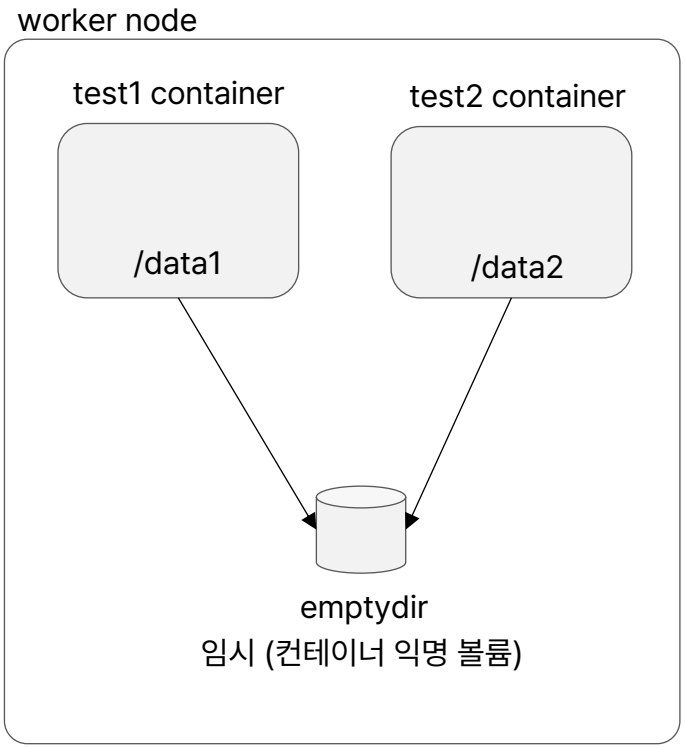
```yaml
volumes:
- name: emptydir-volume
  emptyDir:
    medium: "Memory"
    sizeLimit: "64Mi"
```

kubernetes in action book 내용 참고

[실습] emptydir (1/2)

위치: 13.volume/01.emptydir

emptydir을 통해 Pod 간 즉 동일 노드내에 컨테이너 간 데이터를 공유하는 볼륨이 만들어졌는지 확인



Pod 내 컨테이너
즉, 동일 노드내에 컨테이너 간 데이터 공유 가능한
익명 볼륨이 만들어짐

```
spec:
     containers:
     - name: test1
       image: busybox
       imagePullPolicy: IfNotPresent
       command: ["/bin/sh", "-c"]
       args:
         - echo "Hello from busybox" > /data1/hello1.txt && sleep infinity
       volumeMounts:
       - name: emptydir-volume
         mountPath: /data1
     - name: test2
       image: busybox
       imagePullPolicy: IfNotPresent
       command: ["/bin/sh", "-c"]
       args:
         - echo "Hello from busybox" > /data2/hello2.txt && sleep infinity
       volumeMounts:
       - name: emptydir-volume
         mountPath: /data2
     volumes:
     - name: emptydir-volume
       emptyDir: {}
```

[실습] emptydir (2/2)

위치: 13.volume/01.emptydir

emptydir을 통해 Pod 간 즉 동일 노드내에 컨테이너 간 데이터를 공유하는 볼륨이 만들어졌는지 확인

```
> kubectl apply -f deploy.yaml

> k get pod -l app=sk000-volume-test
NAME                                 READY   STATUS    RESTARTS   AGE
sk000-volume-test-688b57db9c-jv4cx   2/2     Running   0          77m

--> test1 container 로 접속하기
> k exec   -it sk000-volume-test-688b57db9c-jv4cx  -c test1 -- /bin/sh
# cd data1
/data1 # ls
hello1.txt  hello2.txt

/data1 # touch skala
/data1 # exit

→ 컨테이너 volume-test2 로 접속해보자
> k exec   -it sk000-volume-test-688b57db9c-jv4cx  -c test2 -- /bin/sh
# cd data1
/data1 # ls
hello1.txt  hello2.txt skala

/data1 # exit
```

생성된 자원을 삭제하자

```
> k delete -f 01.emptydir
```

볼륨 종류: hostPath

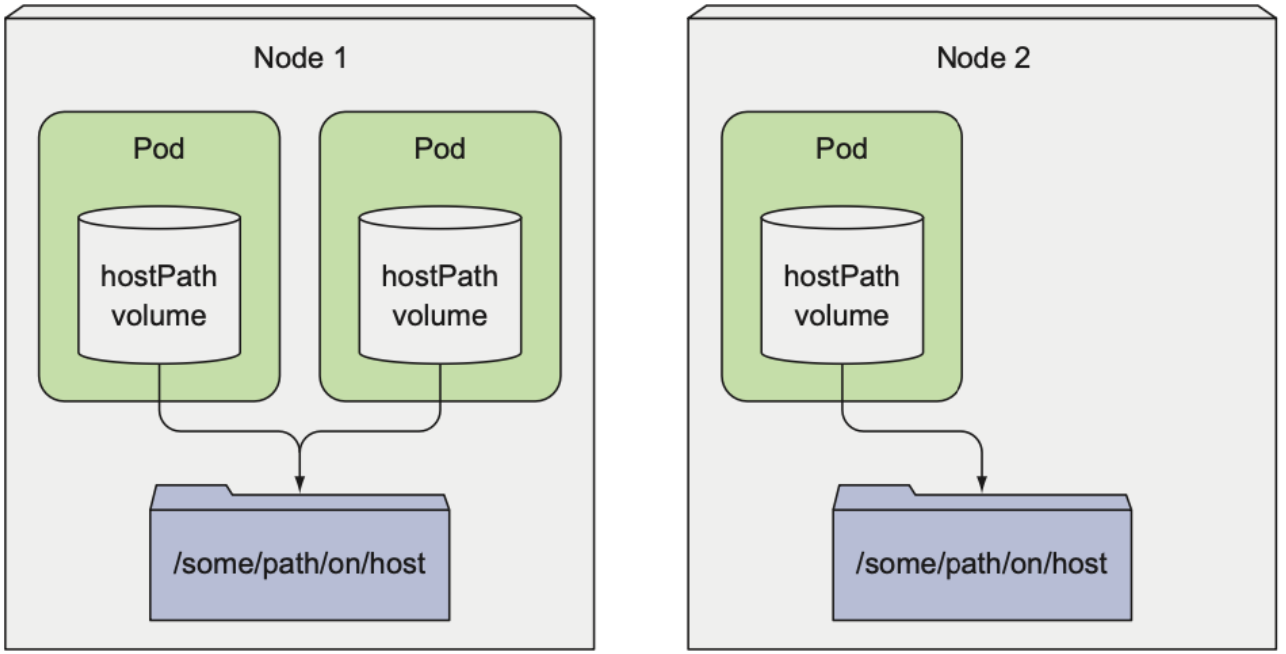
노드의 파일 시스템을 Mount. 테스트 목적 또는 특정 호스트 디렉토리에 의존적으로 사용 가능하며,

동일 Node 재 기동 시 지속적으로 데이터 유지 가능 (별도 삭제하지 않음)

사용 사례

- 개발, 디버깅, 호스트 로그 파일 접근 등
- 동일 노드로 배포 가능한 경우

```
volumes:
- name: hostpath-volume
  hostPath:
    path: /some/path
    type: DirectoryOrCreate
```



kubernetes in action book 내용 참고

볼륨 종류: hostPath 예시

적용 방식

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-volume-test
  namespace: skala-practice
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sk000-volume-test
  template:
    metadata:
      annotations:
        update: 2ca50ac397be556d9bf031374430ce20
      labels:
        app: sk000-volume-test
    spec:
      serviceAccountName: default
      containers:
      - name: storage-test
        image: busybox
        imagePullPolicy: IfNotPresent
        command: ["/bin/sh", "-c"]
        args:
          - echo "Hello from busybox" > /data/hello.txt && sleep infinity
        volumeMounts:
        - name: hostpath-volume
          mountPath: /data
      volumes:
      - name: hostpath-volume
        hostPath:
          path: /tmp/hostpath-sk000-volume-test
          type: DirectoryOrCreate
```

hostPath.type

타입	설명
`Directory`	해당 경로가 디렉토리여야 함. 없으면 오류 발생.
`DirectoryOrCreate`	디렉토리가 없으면 자동 생성. 있음 → 사용.
`File`	파일이어야 함. 없으면 오류 발생.
`FileOrCreate`	파일이 없으면 빈 파일로 생성.
`Socket`	소켓 파일이어야 함.
`CharDevice`	문자 디바이스여야 함 (예: `/dev/null`).
`BlockDevice`	블록 디바이스여야 함 (예: 디스크).

default: 아무것도 확이하지 않고 마운트 시도
없으면 그냥 마운트안됨

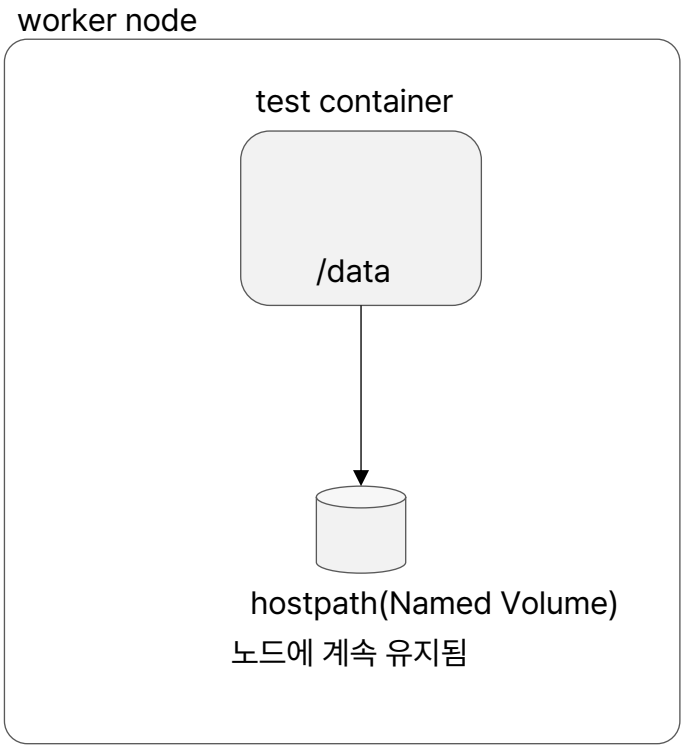
이것과 동일

```
docker run -v /tmp/hostpath-sk000-volume-test:data busybox
```

kubernetes in action book 내용 참고

hostpath

컨테이너가 종료되더라도 Host(Node)에 계속 유지 가능한 볼륨을 생성하고 관리

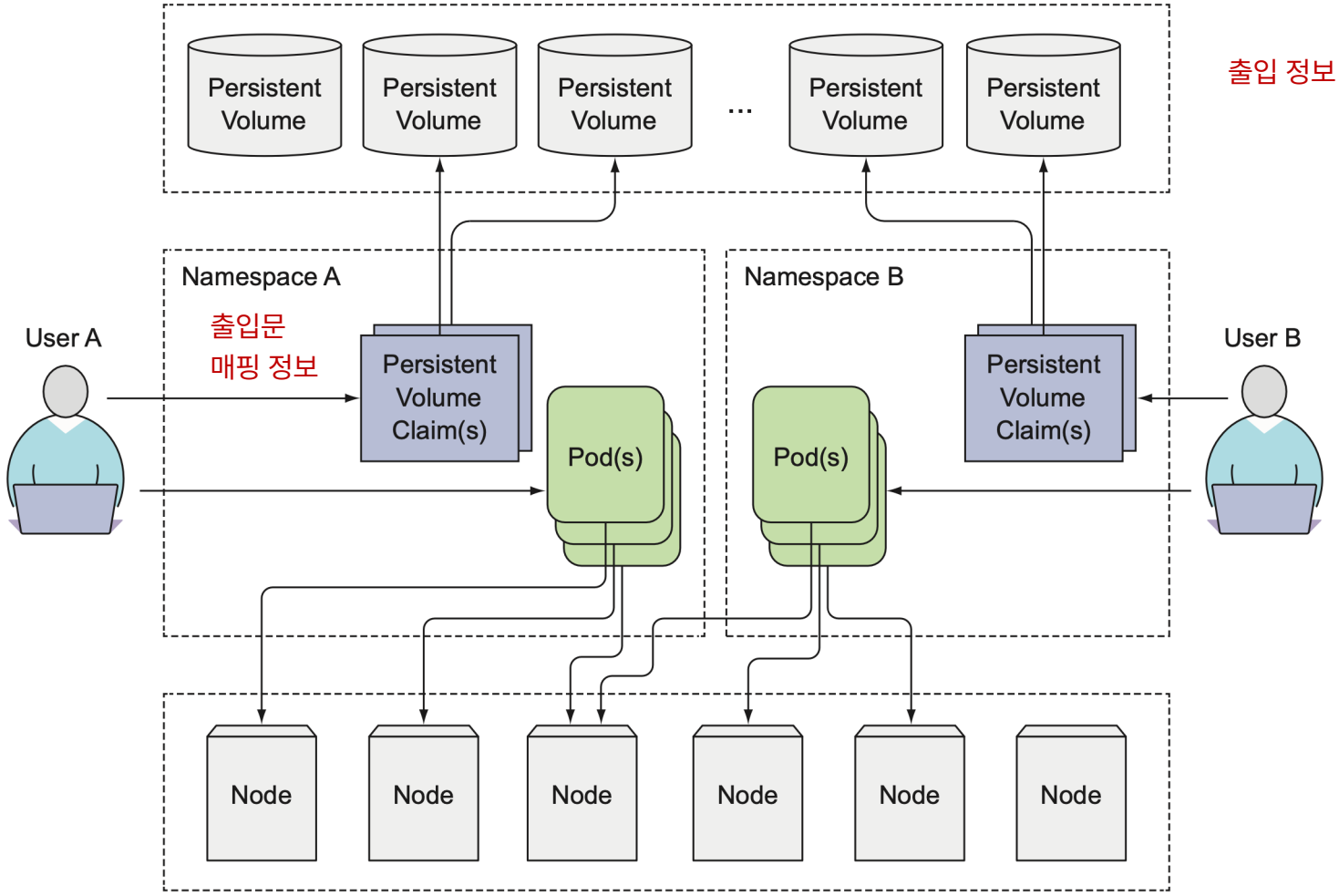



```
spec:
     containers:
     - name: test
       image: busybox
       imagePullPolicy: IfNotPresent
       command: ["/bin/sh", "-c"]
       args:
         - echo "Hello from busybox" > /data/hello.txt && sleep infinity
       volumeMounts:
       - name: ebs-volume
         mountPath: /data
     volumes:
     - name: ebs-volume
       persistentVolumeClaim:
         claimName: skala-ebs-pvc
```

컨테이너가 종료 후 동일 노드에 다시 실행되는 경우
데이터 유지됨

PV & PVC

PV는 실제 스토리지를 가리키는 리소스, PVC는 이 스토리지를 Pod에 연결시키기 위한 요청 리소스



kubernetes in action book 내용 참고

Persistent Volume(PV)

PV는 실제 스토리지(EBS, EFS 등)에 접근하기 위한 **접속 정보와 특성을 가진 리소스** (메타 정보)

PV 수동 생성 과정은 사전에 EFS/EBS를 생성한 후 스토리지 접속을 위한 **EBS Volume ID 또는 FileSystem ID**기반 PV 생성

"공급자" 모델 PV

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: efs-pv-volume-test
spec:
  capacity:
    storage: 100Mi  # 논리적 크기, 실제 EFS는 무제한
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  csi:
    driver: efs.csi.aws.com
    volumeHandle: fs-0e24afe9e2bc46ee5::fsap-066c4369ed1b14b8f
  claimRef:
    namespace: skala-practice
    name: efs-pvc-volume-test
```

PV 생성:
물리적인 디스크의 규격(SEPC)을 정의하는 선언적 객체
(생성되어 있는 볼륨 접속 정보
File System ID, fs Access Point 존재)

실제 물리적 스토리지 접속 정보

[참고] File System ID와 Access Point

- File System ID: EFS를 할당 후 File 접속을 위한 접속 정보 ID
- File System Access Point(fsap): 파일 시스템 안의 특정 디렉토리를 진입점으로 삼아 해당 경로, POSIX UID/GID, 권한을 함께 캡슐화한 객체 (Access Point는 특정 디렉토리를 가르킴)

```
fs-xxxxxxxx    → File system ID
├─ /
├─ /skala-code
├─ /shared
└─ /amdp
```

fs-0e24afe9e2bc46ee5

이름 또는 ID로 액세스 포인트 검색

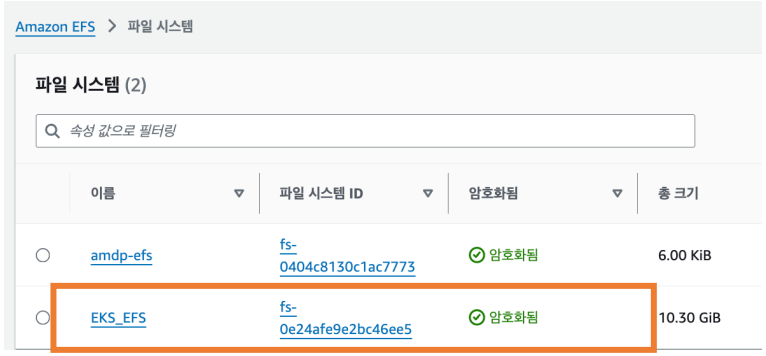
1

이름	액세스 포인트 ID	경로	POSIX 사용자	생성 정보
amdp-access-point	fsap-066d01a676012f324	/amdp	-	1000 : 1000 (0775
-	fsap-0487e971040bb84ba	/pvc-6d552825-3e32-49bc-a3d0-85d8...	50000 : 50000	50000 : 50000 (77
-	fsap-066c4369ed1b14b8f	/skala-code	1000 : 1000	1000 : 1000 (077
skala-professor	fsap-06ed5732e80e6243a	/skala-profressor	1000 : 1000 (1000)	1000 : 1000 (077
-	fsap-000166b5d09e5352e	/shared/pvc-52f7ca6a-1254-493c-883...	0 : 0	0 : 0 (777)
-	fsap-06c1b250f653a2266	/shared/pvc-46e98c5a-4bfe-4a16-b9e...	0 : 0	0 : 0 (777)
-	fsap-01756f1002e5f64c4	/shared/pvc-22adbee5-4199-40e1-a7c...	0 : 0	0 : 0 (777)

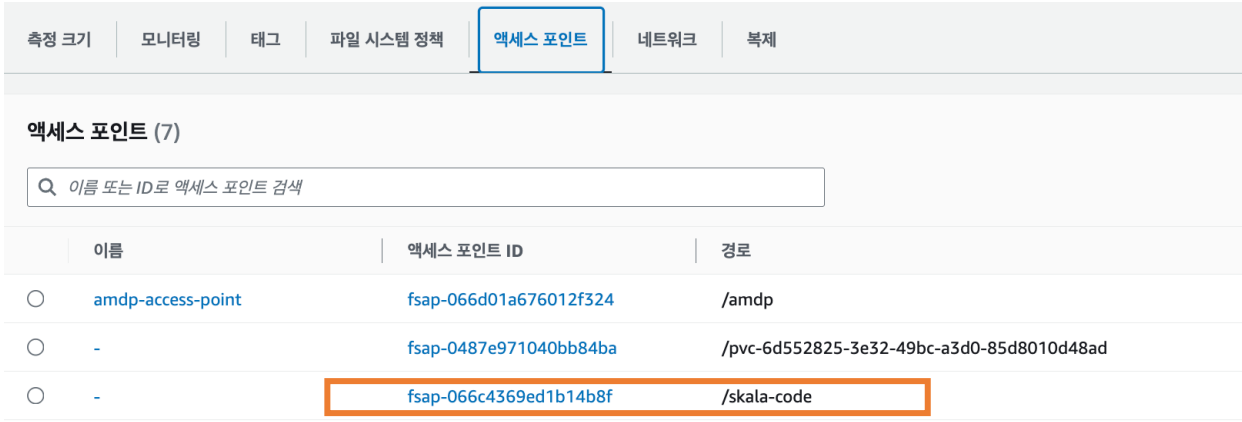
https://881490135253.signin.aws.amazon.com/console

[참고] AWS File System 정보 가져오기

AWS 의 EFS File System의 volumeHandler의 fs 및 fsap 정보 가져오기



file system Id



Access Point

항목	설명	역할
File System ID	`fs-xxxxxxxx`	EFS 전체를 지칭하는 고유 ID
Access Point ID	`fsap-xxxxxxxx`	EFS 내의 특정 경로 + 권한을 설정한 마운트 포인트
조합 예시	`fs-1234::fsap-5678`	CSI Driver가 이 경로만 Pod에 연결하게 함

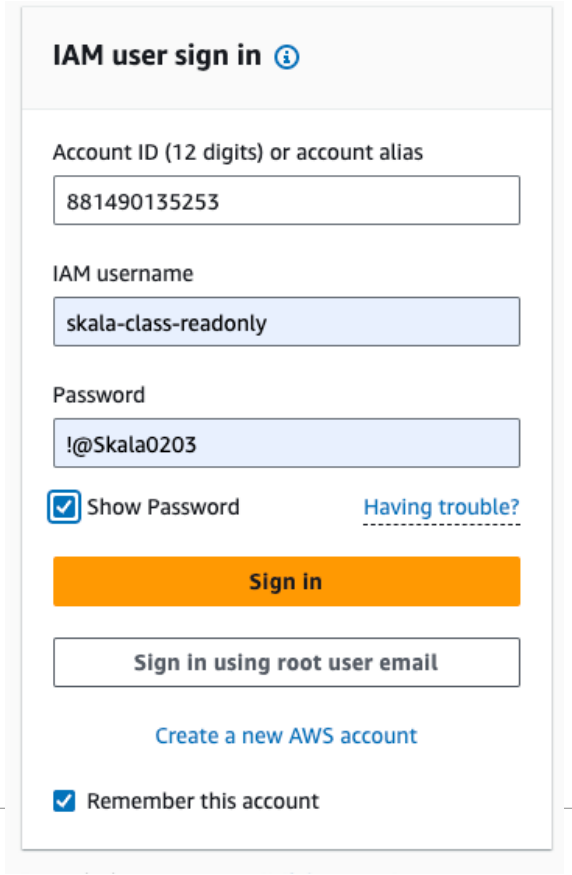
[참고] AWS 접속 정보 (판교)

https://881490135253.signin.aws.amazon.com/console

접속 정보

Account ID	881490135253
ID	skala-class-readonly
PWD	!@Skala0203

접속 위치



[참고] AWS 접속 정보 (울산)

https://430986759212.signin.aws.amazon.com/console

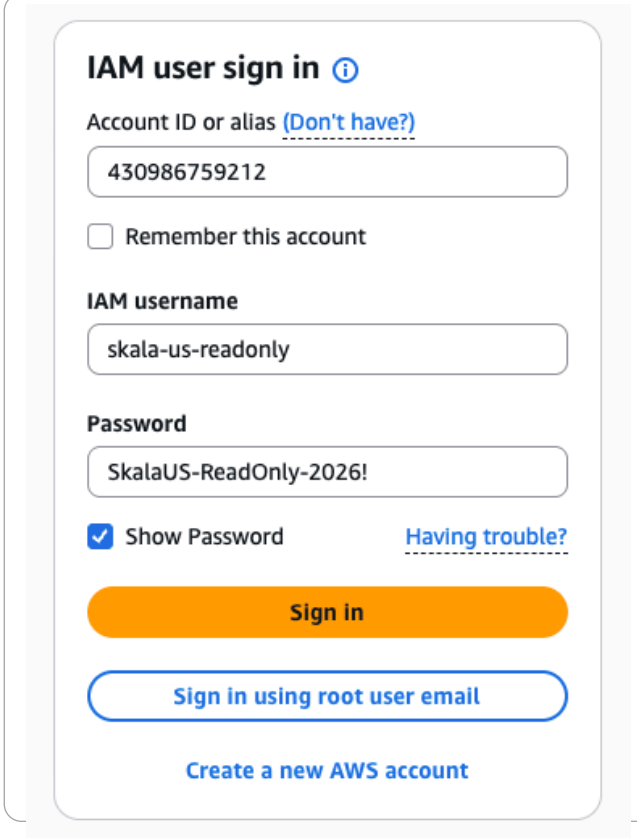
접속 정보

Account ID 430986759212

ID skala-us-readonly

PWD SkalaUS-ReadOnly-2026!

접속 위치



PV Claim (PVC)

PVC는 해당 PV (스토리지 구성 + 정책 스팩)를 **사용하겠다라고 선언하는 요청 객체**

PVC를 생성시 PV와 Binding 과정을 수행 (Bound는 논리적으로 연결, 실제 마운트는 컨테이너 배포시)

```
> kubectl get pvc
NAME                STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS    VOLUMEATTRIBUTESCLASS
nginx-storage-pvc   Bound    pvc-22adbee5-4199-40e1-a7c6-75035c71d97c   100Gi      RWX            efs-sc-shared   <unset>
                             PV Name

> kubectl get pv pvc-22adbee5-4199-40e1-a7c6-75035c71d97c
```

"소비자" 모델 PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: efs-pvc-volume-test
  namespace: skala-practice
spec:
  accessModes:
    - ReadWriteMany
  volumeMode: Filesystem
  resources:
    requests:
      storage: 100Mi
  volumeName: efs-pv-volume-test
```

사용자가 PVC를 생성하면,
PV Controller가 PVC 조건에 맞는 PV를 찾아 자동 바인딩

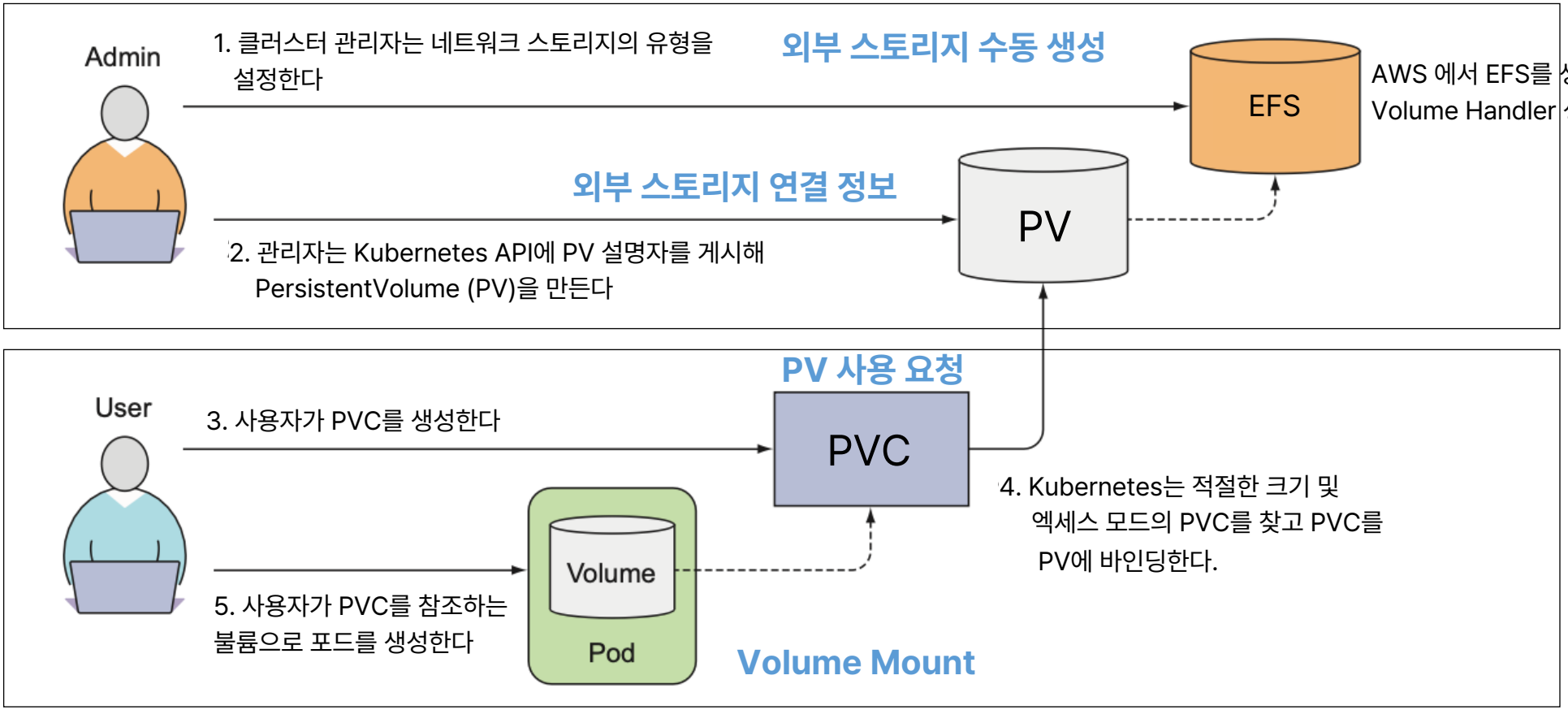
Retain : PVC가 삭제 후 다시 다른 PVC로 연결 시 기존 데이터가 PV에 존재
Delete: PVC 삭제 후 다시 PVC 로 연결 시 데이터 존재하지 않음

PV & PVC 흐름

PV는 물리 디스크나 외부 스토리지를 추상화한 선언적 개체 (실제 물리 노드에 마운트시 사용하는 정보)

PVC는 Pod가 **얼마나 많은 저장소가 필요한지, 어떤 방식으로 접근해야 하는지**를 요청하는 객체

클러스터 관리자가 볼륨을 프로비저닝하고 Persistent Volume Claim을 통해 포드가 사용



[참고] Volume 연결 과정

K8S에서 볼륨을 연결하는 것은

① Storage를 먼저 할당하고, 스토리지 구성/접속를 담은 PV를 생성

② PVC 생성 단계엇 사용 가능한 PV를 검색하고 논리적으로 Binding하고

③ Pod를 스케쥴링된 해당 Node에 스토리지를 Attach(실장) 및 Node Mount

④ 컨테이너의 디렉토리로 Mount

EBS (Bock Storage)

attach는 노드에 Block storage에 논리적 실장의미

```
EBS
 ↓ (Attach)
Node Block Device (/dev/nvmeX)
 ↓
① Node Mount (ext4 mount)
 ↓
② Container Bind Mount (/data)
```

EFS (NFS)

Attach 단계 필요없음

```
EFS (NFS Server)
 ↓
① Node Mount (nfs mount)
 ↓
② Container Bind Mount (/data)
```

kubernetes in action book 내용 참고

[참고] PV 와 PVC 내부 흐름

2. volume을 계약 – 나중에 Pod에서 사용

PVC : sk000-efs-pvc

PVC는 계약서 (Claim)
- 난 어떤 유형의 파일 시스템 (File, Block)
- 필요 Size (100Mi)

bound
계약됨

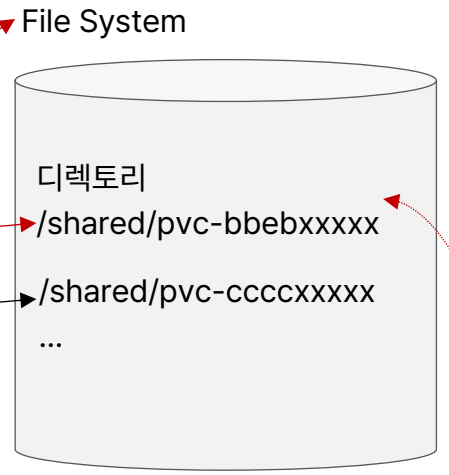
1. 미리 만들어놓은 Volume Market

pv: efs-pv

PV는 연결을 위한 정보
- 어떤 파일 시스템 (filesystemid)
- 어느 디렉토리 (fsap)

pv2

...

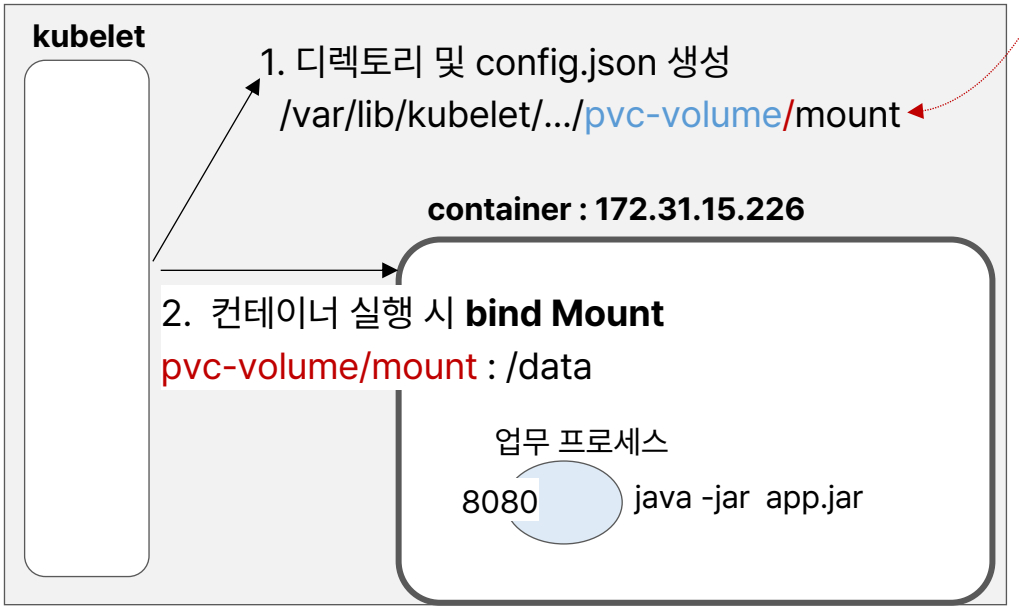


3. pod 배포 시 이 볼륨을 노드에 연결해서 컨테이너에 마운트 해라고 요청

Pod

```
        volumeMounts:
        - name: pvc-volume
          mountPath: /data
      volumes:
      - name: pvc-volume
        persistentVolumeClaim:
          claimName: sk000-efs-pvc
```

4. 요청에 따라 노드에 Attach/Mount 컨테이너에 Bind Mount



[참고] PV 기반 볼륨 마운트 흐름

PV 생성부터 최종 Pod 바인딩까지의 흐름

1. PV 생성:
물리적인 디스크를 규격을 정의하는 선언적 객체
(EFS는 기 생성되어 있어야 한다)

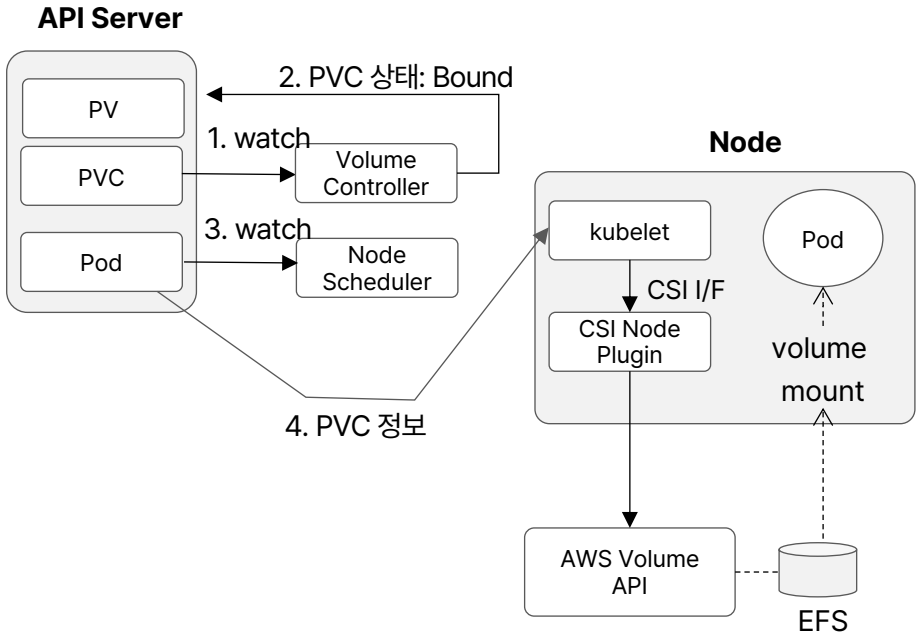
2. PVC가 생성:
사용자가 PVC를 ETCD에 등록하면,
Volume Controller가 PVC 조건에 맞는 PV를 찾아 자동 바인딩

3. Pod가 배포될 때:
kube-scheduler가 Pod를 노드에 스케줄

4. 해당 노드의 kubelet:

- 자신에게 배포된 Pod에 관계된 바인딩된 PV/PVC 정보를 읽고,
- 거기 있는 csi driver, volumeHandle 값을 기반으로
- CSI Node Plugin에게 마운트 요청

5. CSI Node Plugin:
해당 노드에 설치된 컨테이너 (DaemonSet 형태)
gRPC로 호출되어 실제로 AWS EFS, EBS, NFS 등의 FS를 마운트
마운트가 끝나면 kubelet이 Pod의 /data 등에 연결 완료



EBS PVC인 경우 CSI Controller가 Attach 요청함

참고
Static은 이미 생성되어 있는 볼륨을 마운트하는 행위만 수행

kubernetes in action book 내용 참고

[참고] CSI (Container Storage Interface)

CSI(Container Storage Interface) 는
Kubernetes가 어떤 스토리지든 동일한 방식으로 사용할 수 있게 해주는 표준 인터페이스

CSI Driver는 이 인터페이스를 구현한 실제 드라이버(프로그램)
CSI Driver는 Controller Plugin 와 Node Plugin으로 구분

CSI Controller Plugin (Master Node)

VolumeAttachment Resource 생성/삭제

- 볼륨 생성/삭제(Provision/Delete)
- 볼륨 Attach/Detach(어떤 노드에 디스크를 붙일지/떼어낼지)
- 동적 프로비저닝 지원
- ReadWriteOnce, ReadWriteMany 같은 모드 관리 등

CSI Node Plugin (개별 Worker Node)

- 노드에 attach된 디바이스를 포맷/마운트
- 노드에 볼륨 마운트 / 언마운트

kubernetes in action book 내용 참고

[참고] CSI Driver (Container Storage Interface)

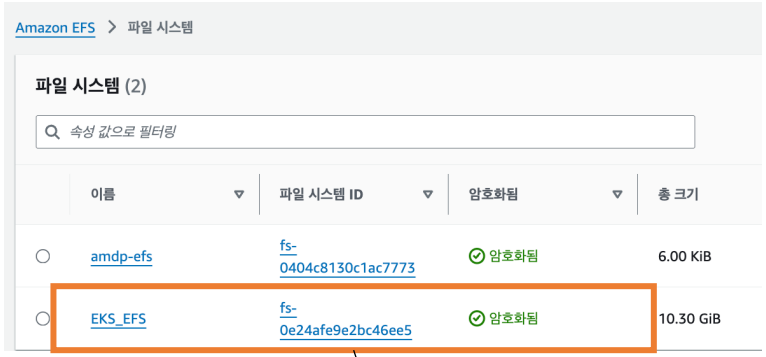
PV/PVC를 외부 스토리지에 연결할 때 사용하는 **CSI 드라이버 종류**를 스토리지별로 정리

스토리지 종류	CSI Driver 이름	특징
AWS EBS	ebs.csi.aws.com	RWO(ReadWriteOnce), 블록 스토리지
AWS EFS	efs.csi.aws.com	RWX(ReadWriteMany), 공유 네트워크 파일 시스템
GCP PD	pd.csi.storage.gke.io	Google Persistent Disk
Azure Disk	disk.csi.azure.com	RWO, 블록 스토리지
Azure File	file.csi.azure.com	RWX, 공유 파일 스토리지
OpenEBS	cstor.csi.openebs.io 등	오픈소스 로컬 또는 분산 스토리지
Ceph (RBD)	rbd.csi.ceph.com	블록 기반 분산 스토리지
CephFS	cephfs.csi.ceph.com	공유 파일 시스템
NFS	nfs.csi.k8s.io	단순한 NFS 볼륨 사용
Longhorn	driver.longhorn.io	Lightweight distributed block storage
NetApp, Dell, Portworx, VMware	각자 전용 CSI Driver 제공	엔터프라이즈급 기능 포함

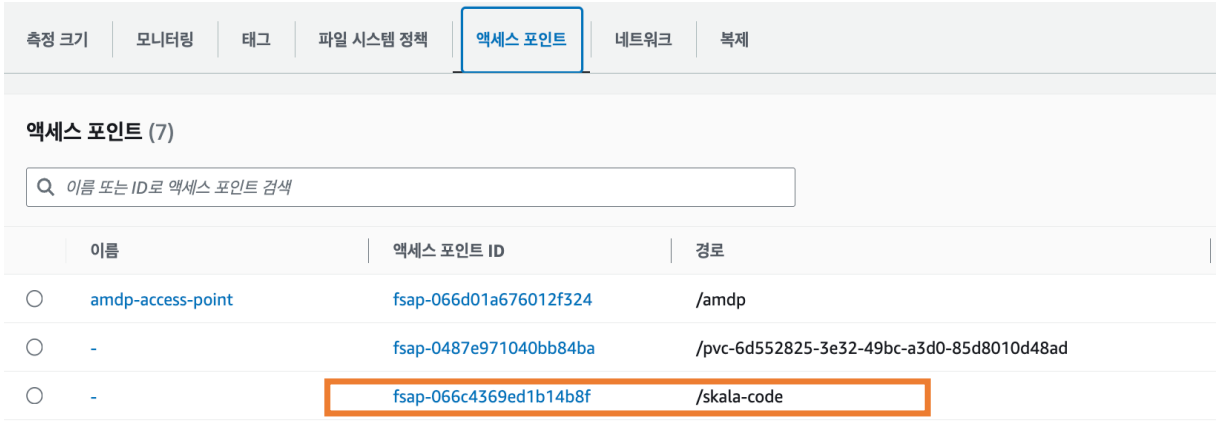
[사전 작업] PV 구성하기

위치: 13.volum/03.pvc

먼저 AWS에 접속해서 File system ID와 Access Point 값을 추출한다.



file system Id



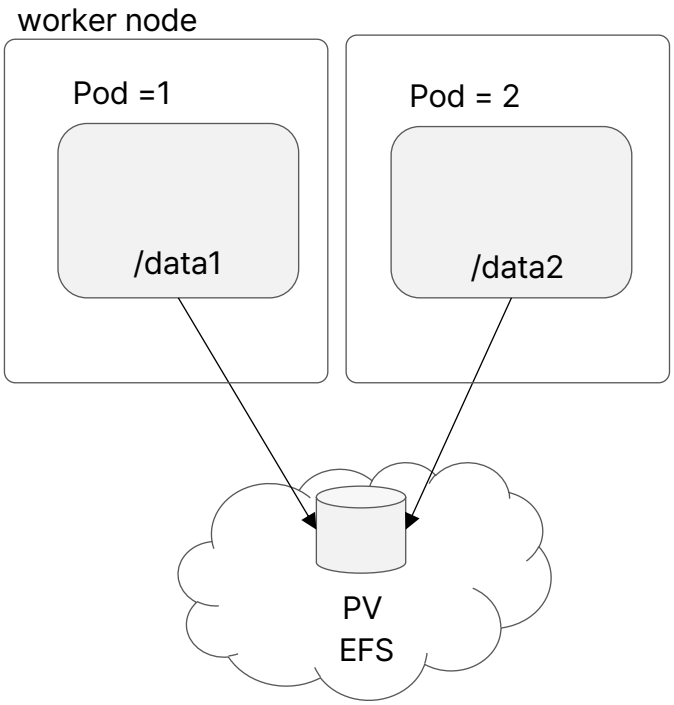
Access Point

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: skala-efs-pvc
spec:
  capacity:
    storage: 100Mi  # 논리적 크기, 실제 EFS는 무제한
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  csi:
    driver: efs.csi.aws.com
    volumeHandle: fs-0e24afe9e2bc46ee5::fsap-066c4369ed1b14b8
  claimRef:
    namespace: class-0
    name: skala-efs-pvc
```

[실습] PV & PVC 생성하기 (1/3)

위치: 13.volum/03.pvc

replicas=2 로 두개의 복제 Pod 가 원격에서 스토리지를 공유하는 형식으로 구성해보자



Pod 내 컨테이너
즉, 동일 노드내에 컨테이너 간 데이터 공유 가능한
익명 볼륨이 만들어짐

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: skala-volume-test
  namespace: class-0
spec:
  replicas: 2
…
    spec:
      terminationGracePeriodSeconds: 0
      containers:
      - name: storage-test
        image: busybox
        imagePullPolicy: IfNotPresent
        command: ["/bin/sh", "-c"]
        args:
         - echo "Hello from busybox" > /data/skala-hello.txt && sleep infinity
        volumeMounts:
        - name: pvc-volume
          mountPath: /data
      volumes:
      - name: pvc-volume
        persistentVolumeClaim:
          claimName: skala-efs-pvc
```

[실습] PV & PVC 생성하기 (2/3)

위치: 13.volum/03.pvc

emptydir을 통해 Pod 간 즉 동일 노드내에 컨테이너 간 데이터를 공유하는 볼륨이 만들어졌는지 확인

```
> kubectl apply -f pv.yaml
NAME           CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS      CLAIM                   STORAGECLASS
skala-efs-pvc   100Mi      RWX            Retain           Available   class-0/skala-efs-pvc

> k apply -f pvc.yaml

> k get pv sk000-efs-pvc
NAME           CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS      CLAIM                   STORAGECLASS
skala-efs-pvc   100Mi      RWX            Retain           Bound       class-0/skala-efs-pvc

> k get pvc sk000-efs-pvc
NAME           STATUS   VOLUME          CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
skala-efs-pvc   Bound    skala-efs-pvc   100Mi      RWX                           <unset>                 2m16s
```

[실습] PV & PVC 생성하기 (3/3)

위치: 13.volum/03.pvc

emptydir을 통해 Pod 간 즉 동일 노드내에 컨테이너 간 데이터를 공유하는 볼륨이 만들어졌는지 확인

deploy 배포하기

```
> k apply -f deploy.yaml
> k get pod -l app=sk000-volume-test
NAME                                  READY   STATUS    RESTARTS   AGE
sk000-volume-test-84f5c577cc-pgbr7    1/1     Running   0          15m
sk000-volume-test-84f5c577cc-tbtrm    1/1     Running   0          15m
```

Terminal 1

```
> k exec -it sk000-volume-test-84f5c577cc-pgbr7 -- /bin/sh

/# cd /data
/# ls
sk000-hello.txt

/# touch skala

/# ls
sk000-hello.txt skala




/# ls
sk000-hello.txt skala skala2
```

Terminal 2

```
> k exec -it sk000-volume-test-84f5c577cc-tbtrm -- /bin/sh

/# cd /data









/# ls
sk000-hello.txt skala

/# touch skala2
```

[후속 처리] 실습 자원을 삭제해보자

위치: 13.volum/03.pvc

```
# 13.volume 으로 이동

> k delete -f 03.pvc
```

Static & Dynamic Volume Provisioning

Static Provisioning

운영자가 **미리 Persistent Volume(PV)** 을 수동으로 생성해두고, 사용자가 PVC(PersistentVolumeClaim)를 만들어서 **이미 만들어진 PV에 바인딩**하는 방식 (지금까지 배운 방식)

Dynamic Provisioning

사용자가 PVC를 만들면, **Kubernetes가 자동으로 PV를 생성**하여 바인딩하는 방식.
이때 어떤 스토리지를 사용할지 명시하기 위해 **StorageClass**를 지정하면 자동적으로 PV 생성

kubernetes in action book 내용 참고

Storage Class

StorageClass는 어떤 CSI Driver를 통해, 어떤 파라미터로 Storage를 동적 프로비저닝(Dynamic Provisioning) 할지 정의한 템플릿

AWS의 CSI Driver는 EBS와 EFS 별도로 구성되며, Controller, Nolde Plugin으로 K8S 환경에 배포

```
CSI Driver k8s 배포
    ├─ Controller (Deployment)   → 프로비저닝/관리 담당
    └─ Node Plugin (DaemonSet)   → 실제 마운트 담당
```

Elastic File System (EFS)

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: efs-sc-shared
provisioner: efs.csi.aws.com
reclaimPolicy: Delete
volumeBindingMode: Immediate
allowVolumeExpansion: true
```

Elastic Block Store (EBS)

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp2
provisioner: ebs.csi.aws.com
parameters:
  fsType: ext4
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

kubernetes in action book 내용 참고

Storage Class : Elastic Block Store (EBS)

Kubernetes에서 볼륨을 어떤 스토리지(EBS, EFS 등)로, 어떤 정책으로 생성할지 정의한 템플릿
PVC는 StorageClass를 참조해 동적 프로비저닝을 트리거하고, 실제 스토리지는 CSI 드라이버가 생성·관리

Elastic Block Store (EBS)

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:                    PVC에 storageClassName이 없으면 gp3  자동
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
parameters:
  type: gp3  →   ebs-csi-controller가 AWS EBS API 호출 시 지정하는 EBS Volume Type
  fsType: ext4  → 어떤 파일 시스템으로 포맷할지 지정
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true  → 사이즈 확장 가능
```

volumeBindingMode

PVC를 언제 PV와 바인딩할지를 결정하는 정책

- **Immediate**
  - PVC가 생성되는 즉시 PV를 생성/바인딩
  - 스토리지가 먼저 만들어짐
  - EBS는 AZ 종속
- **WaitForFirstConsumer**
  - Pod 스케쥴링 후 AZ 확정
  - EBS 사용시 정석

kubernetes in action book 내용 참고

Storage Class : Elastic File System(EFS)

Kubernetes에서 볼륨을 어떤 스토리지(EBS, EFS 등)로, 어떤 정책으로 생성할지 정의한 템플릿
PVC는 StorageClass를 참조해 동적 프로비저닝을 트리거하고, 실제 스토리지는 CSI 드라이버가 생성·관리

Elastic File System (EFS)

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: efs-sc-shared
provisioner: efs.csi.aws.com
parameters:
  provisioningMode: efs-ap
  fileSystemId: fs-0e24afe9e2bc46ee5
  basePath: /shared
  ensureUniquePaths: "true"
  directoryPerms: "777"
  uid: "0"
  gid: "0"
reclaimPolicy: Delete
volumeBindingMode: Immediate
allowVolumeExpansion: "true"
```

필드	값 (M/O)	의미
provisioningMode	**필수**	동적 프로비저닝 방식. efs-ap는 EFS Access Point 기반으로 PVC마다 AP를 만들어서 마운트
fileSystemId	**필수**	사용할 EFS File System ID
basePath	선택 /	Access Point가 가리킬 디렉토리의 상위 경로(prefix)
ensureUniquePaths	선택 "false"	PVC마다 생성되는 디렉토리 경로를 유니크하게 강제할지 여부
directoryPerms	선택 "700"	새로 생성되는 디렉토리의 권한 (8진수 문자열)
uid	선택 "0"	Access Point를 통해 접근할 때 적용되는 POSIX 사용자 ID
gid	선택 "0"	Access Point를 통해 접근할 때 적용되는 POSIX 그룹 ID

kubernetes in action book 내용 참고

Storage Class기반 PVC 구성

storage Class를 지정해서 PVC 구성시 자동으로 PV 를 생성하고 바인드되도록 구성

PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-efs-pvc
  namespace: skala-practice
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: efs-sc-shared
  resources:
    requests:
      storage: 5Gi
```

AccessModes

accessMode 값	의미 (한 줄 정의)	동시 접근	대표 스토리지
ReadWriteOnce (RWO)	하나의 노드에서 읽기/쓰기 가능	동일 노드만 가능	EBS, GCE PD
ReadOnlyMany (ROX)	여러 노드에서 읽기 전용	읽기만 가능	NFS, EFS
ReadWriteMany (RWX)	여러 노드에서 읽기/쓰기 가능	가능	EFS, NFS, CephFS
ReadWriteOncePod (RWOP)	단 하나의 Pod만 읽기/쓰기 가능	(Pod 단위 단일)	EBS (고정밀 제어)

kubernetes in action book 내용 참고

[참고] Storage Class의 주요 필드 정의

필드	설명
provisioner	CSI 드라이버나 스토리지 백엔드를 지정합니다. • **efs.csi.aws.com** : AWS EFS용 CSI 드라이버 • **ebs.csi.aws.com** : AWS EBS용 CSI 드라이버
parameters	스토리지 클래스가 스토리지를 생성할 때 필요한 추가 정보입니다. • **fsType** : 파일 시스템 형식 (예: ext4, xfs 등) • **type** : EBS 볼륨 타입 (gp2, gp3, io1, sc1 등)
reclaimPolicy	PVC가 삭제되었을 때 PV를 어떻게 처리할지를 설정 • **Delete** : PVC 삭제 시 PV 및 실제 볼륨도 삭제 • **Retain** : PV는 삭제하지 않고 보존 (수동으로 정리 필요)
volumeBindingMode	볼륨을 생성할 시점을 제어 • **Immediate** : PVC가 생성되자마자 바로 PV 생성 시도 • **WaitForFirstConsumer** : PVC를 사용하는 Pod가 실제로 어떤 노드에 배치될 때까지 기다렸다가 볼륨 생성 → 노드 위치에 따라 스토리지를 만들어야 **하는 EBS와 같은 AZ 종속 스토리지에 필수**
allowVolumeExpansion	PVC의 용량을 늘리는 것이 가능한지 여부 • **true 로 설정 시**, PVC 사이즈 변경이 가능 (스토리지 드라이버도 이 기능을 지원해야 함)

kubernetes in action book 내용 참고

Access Mode

Kubernetes에서 사용하는 Access Modes는 볼륨이 **하나 이상의 파드에 어떤 방식으로 마운트될 수 있는지를 정의**

Access Mode	설명	EBS	EFS
ReadWriteOnce (RWO)	하나의 노드에서 읽기/쓰기 가능한 방식. 하나의 파드만 쓸 수 있음.	✅	✅
ReadOnlyMany (ROX)	여러 노드에서 읽기 전용으로 동시에 마운트 가능. 공유는 가능하지만 쓰기 불가.	❌	✅
ReadWriteMany (RWX)	여러 노드에서 읽기/쓰기 모두 가능. 다수의 파드에서 동시에 쓰기 가능.	❌	✅
ReadWriteOncePod (RWOP)	단일 파드에서만 읽기/쓰기 가능 (Kubernetes v1.22+) RWO보다 더 강한 제한.	버전별	❌

kubernetes in action book 내용 참고

Static vs Dynamic Provisioning 볼륨 생성 방식 차이

Storage Class를 이용하는 경우 PVC 선언에 따라 자동적으로 PV를 생성하고 바인딩 진행

Static PV : PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: efs-volume-test
  namespace: skala-practice
spec:
  accessModes:
    - ReadWriteMany
  volumeMode: Filesystem
  resources:
    requests:
      storage: 100Mi
  volumeName: efs-volume-test
```

Storage Class : PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: efs-sc-volume-test
  namespace: skala-practice
spec:
  accessModes:
    - ReadWriteMany
  volumeMode: Filesystem
  resources:
    requests:
      storage: 100Mi
  storageClassName: efs-sc
```

kubernetes in action book 내용 참고

[참고] Storage Class 기반 PVC 흐름

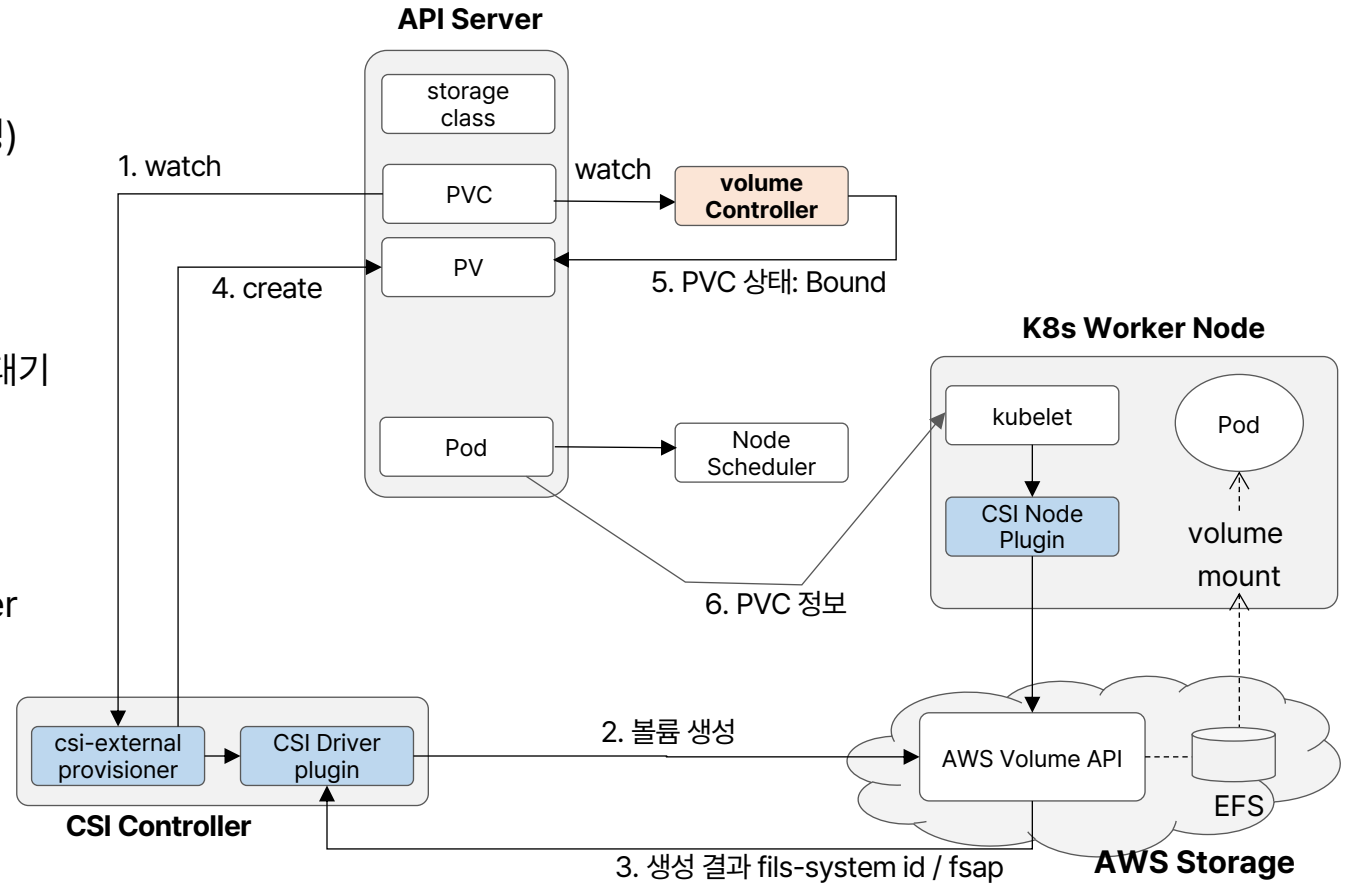
PV 생성부터 최종 Pod 바인딩까지의 흐름

1. PVC 생성:
사용자가 PVC 생성 (Storage Class 지정)

2. Volume Controller의 PVC 감지:
- Volume Controller는 대응되는 Storage Class 조회
- 동적 프로비전인것을 인지하고 PV 생성대기

3. Pod가 배포될 때:
- CSI Driver Controller의 sidecar인 csi-external provisioner가 PVC 감지
- csi-external provisioner는 CSI Driver plugin에 볼륨 생성 요청
- csi-provisioner는 PV 를 생성

4. Pod 생성 시점
정적 프로비전과 동일하게 동작



참고 1) AWS csi driver plugin
- aws-ebs-csi-driver
- aws-efs-csi-driver

참고 2) FileSystemId: EFS를 생성하면 자동으로 발급되는 고유 식별자 예) fs-12345678

참고 3) FSAP(File System Access Point): 특정 디렉토리, 권한, 사용자그룹/그룹 ID를 사전에 정의한 진입점
예) fsap-abcd1234efgh5678

kubernetes in action book 내용 참고

[참고] AWS CSI Driver Pod

CSI(Container Storage Interface) driver는 namespace kube-system 에 아래와 같은 Pod로 배포되어 있음

항목	동적 프로비저닝	정적 프로비저닝
`efs-csi-controller`	✅ AWS API 호출 (`CreateVolume`)	❌ 역할 없음
`efs-csi-node`	✅ 마운트 수행	✅ 마운트 수행
PV 생성 방식	자동 (CSI Driver)	수동 (관리자가 YAML로 정의)
AWS API 호출	✅ Access Point 또는 FS 생성	❌ 이미 존재하므로 호출 없음

```
~/ k get pod -n kube-system | grep csi
ebs-csi-controller-8fc78947-78qjh                6/6     Running   0          16h
ebs-csi-controller-8fc78947-pq8cf                6/6     Running   0          16h
ebs-csi-node-5x59q                               3/3     Running   0          6h26m
ebs-csi-node-l29cs                               3/3     Running   0          6h26m
ebs-csi-node-p25jx                               3/3     Running   0          6h25m
ebs-csi-node-tmxcw                               3/3     Running   0          6h25m
efs-csi-controller-58c96d96c5-77xt8              3/3     Running   0          16h
efs-csi-controller-58c96d96c5-cncwx              3/3     Running   0          16h
efs-csi-node-2fgqz                               3/3     Running   0          6h25m
efs-csi-node-qdtfg                               3/3     Running   0          6h25m
efs-csi-node-qkftk                               3/3     Running   0          6h26m
efs-csi-node-wt4lh                               3/3     Running   0          6h26m
(⎈|skala-2025:skala-practice)
```

kubernetes in action book 내용 참고

[실습] StorageClass 기반 EFS 동적볼륨 할당

위치: 13.volume/04.efs-sc

storage Class = efs-sc-shared 를 PV 동적 할당해보자

```
> k apply -f pvc.yaml
> k apply -f deploy.yaml
> k get pod -l app=sk000-volume-test

> k exec  -it  sk000-volume-test-694477f485-2g4gf -- /bin/sh
# cd data
/data # touch sk000

/data # exit
```

2. 배포한 deployment를 제거하고 다시 실행 (delete, apply 실행)

3. pod로 다시 연결 후 (k exec –it {pid name} -- /bin/sh)
/data/sk000 가 존재하는지 확인

4. deploy pod 개수 증가
k scale --replicas=2 deploy sk000-volume-test

5. 두개의 Pod 에서 각각 접속해서 데이터가 공유되는지 확인
k exec –it {pod-name} -- /bin/sh

[실습] 짜꿍끼리 공유해보기

위치: 13.volume/04.efs-sc

짜꿍의 PVC를 마운트해서 사용해보자

옆의 짜꿍의 PVC를 deployment에 직접 변경해서 실행한후

kubectl exec –it 를 통해 집접 접속하고

kubectl cp 를 이용해서 파일을 넣어보자

이를 통해 파일 공유가 되는지 확인해보자

KDT Cloud – 5. 쿠버네티스 심화하기

Statefulset

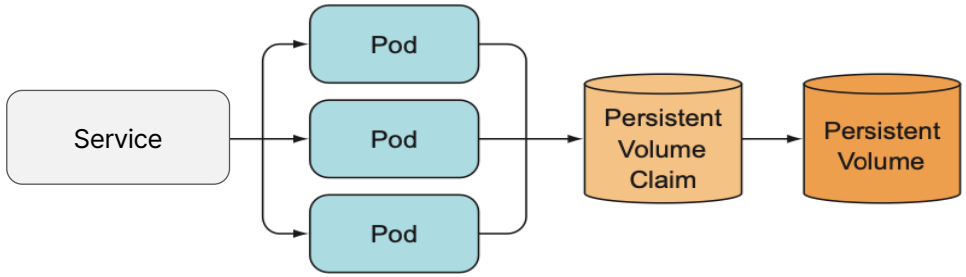
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Deployment 의 한계

Deployment는 Stateless 애플리케이션에 적합한 구조

- Pod들은 서로 구분할 필요가 없으며, 하나가 문제가 생겨도 다른 복제본이 대체 (ReplicaSet 구조)
- Pod 각각의 고유한 정체성이 중요하지 않음
- Pod 이름은 계속해서 변경
- 어느 Pod가 요청을 처리해도 동일 처리를 가정
- Pod 별 고유 데이터 저장을 전제로 구조화되지 않음
- Pod 생성 순서나 종료 순서가 중요하지 않음



Deployment 의 한계

- 특정 Pod의 이름이 계속 유지되어야 함
- 각 Pod가 자기 전용 저장공간을 가져야 함
- Pod가 재생성되어도 기존 데이터를 다시 사용해야 함
- 각 Pod를 고정된 이름으로 찾아야 함
- 생성 순서가 중요할 수 있음Pod 별로 서로 다르게 동작하기 위한 로직 개발 필요한 구조
- Client 가 지속적으로 특정 Pod 와 통신해야 하는 경우

kubernetes in action book 내용 참고

Statefulset 이란

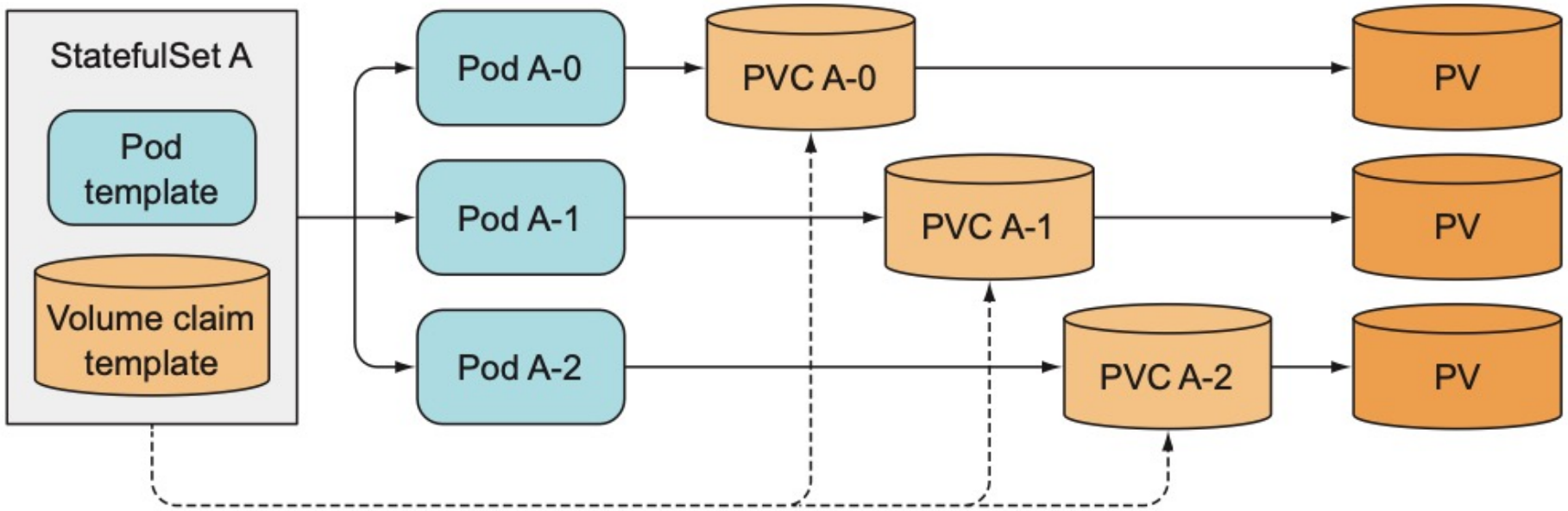
각 Pod의 고유한 이름·저장공간·네트워크 식별자를 유지해야 하는
Stateful 애플리케이션을 관리하기 위한 Kubernetes 워크로드 컨트롤러

구분	Deployment	StatefulSet
목적	Stateless	Stateful
Pod 이름	변경 가능	고정
Pod 고유성	중요하지 않음	중요
Pod별 저장소	기본 개념 아님	Pod별 PVC 관리
네트워크 이름	개별 Pod 식별이 중요하지 않음	Pod별 구분해서 호출
생성 순서	보장하지 않음	기본적으로 순서 보장
대표 사용	Web, API	DB, Kafka

kubernetes in action book 내용 참고

Statefulset 구조

Statefulset은 복제를 위한 Pod Template 뿐만 아니라 VPC를 생성하기 위한 VolumeClaimTemplate도 포함

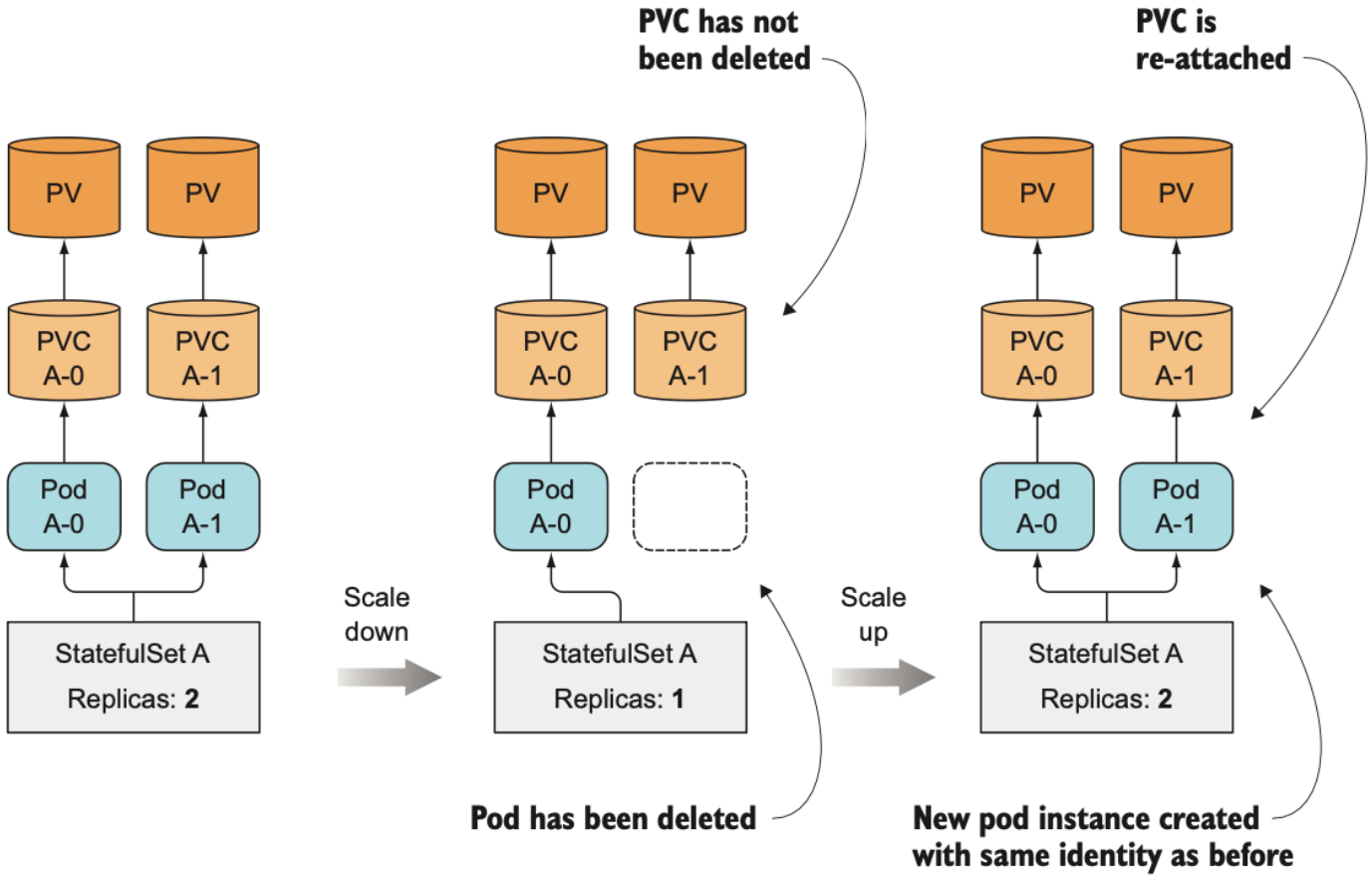


참고 : EBS는 AZ 간 마운트가 넘어가지 않는다. 그로인해 Statefulset Pod가 EBS를 사용하는 경우 AZ 의존관계 존재
AZ 의존관계를 없애기 위해서는 EFS를 사용하도록 권고

kubernetes in action book 내용 참고

Statefulset 의 Scal-down/Scale-up

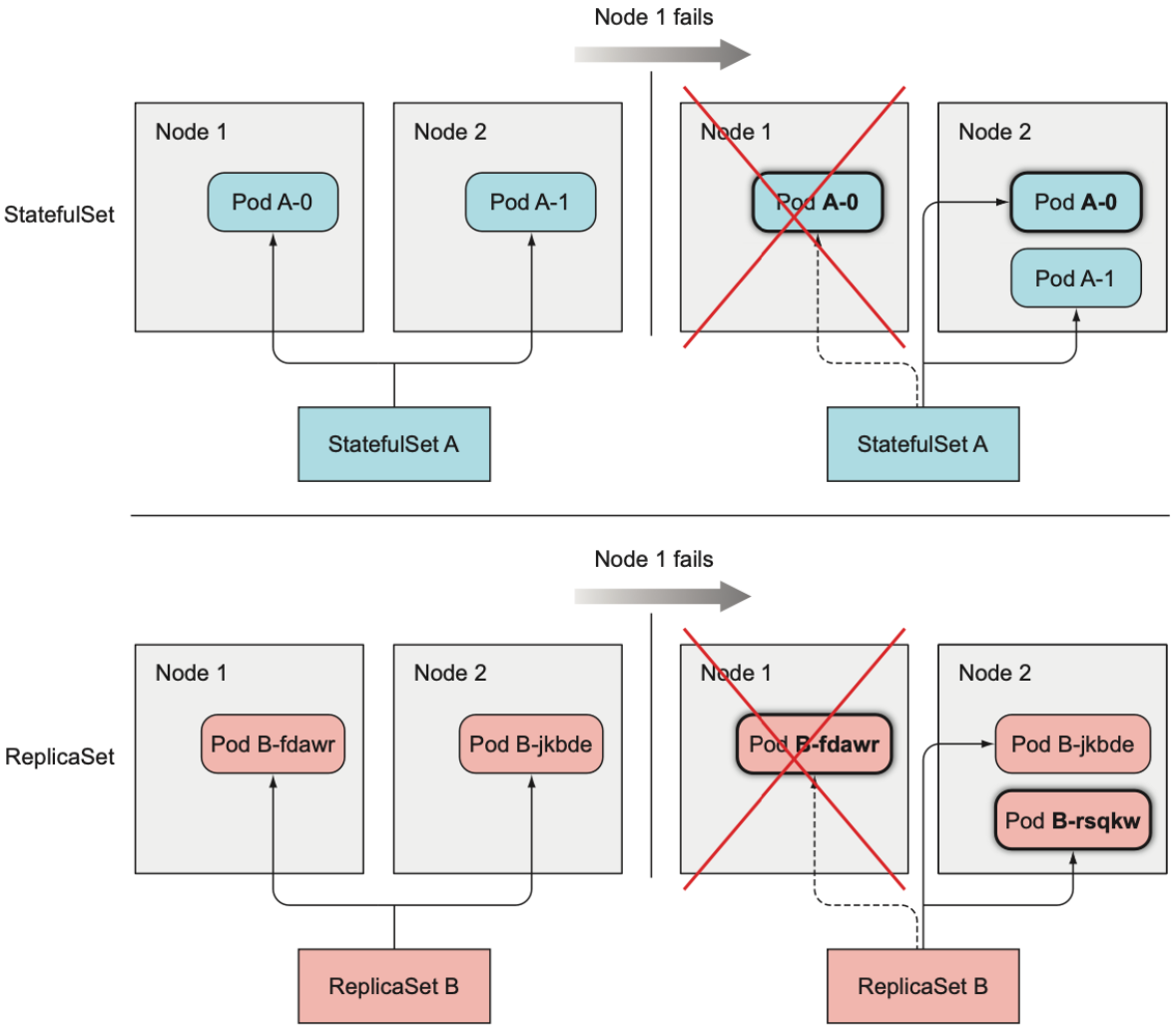
Statefulset은 복제를 위한 Pod Template 뿐만 아니라 VPC를 생성하기 위한 VolumeClaimTemplate도 포함



kubernetes in action book 내용 참고

Statefulset 의 잃어버린 Pod 교체 방법

Node shutdown으로 인한 일시적으로 중단된 Pod의 재 실행 방법



Node Failed 또는 Network 장애로 인한 실패 시 (Unknown 상태)
Pod A-0가 정상적으로 다른 Node에 Replace 안되는 현상 발생 할 수 있음

강제 삭제 필요
k delete pod podA-0 --force

Node Failed 또는 Network 장애로 인한 실패 시
Pod A-0가 정상적으로 다른 Node에 Replace

-

kubernetes in action book 내용 참고

Statefulset의 Rolling Update 방식

Statefulset은 동일한 Pod 여러개를 실행할 수 없음

rolling update 전환 방식

- 기존 Pod를 삭제
- 같은 ordinal 이름으로 새 Pod를 재생성
- 새 Pod가 Running/Ready가 될 때까지 다음 Pod는 안 건드림

```
기존:
app-0 v1
app-1 v1
app-2 v1

업데이트 시작:
app-2 삭제  → 이 순간 Ready Pod 수 감소
app-2 v2 생성 → Ready 될 때까지 대기
app-1 삭제
app-1 v2 생성
app-0 삭제
app-0 v2 생성
```

kubernetes in action book 내용 참고

왜 ReplicaSet을 사용하지 않는가?

Deployment는 내부적으로 ReplicaSet을 생성해서 Pod를 관리
StatefulSet은 ReplicaSet을 사용하지 않고 직접 Pod를 제어

ReplicaSet의 특성

상태없는 Pod의 복제 개념

- 기본 무상태 개념으로 전환 시 빠른 전환 가능
- Pod 별 세밀한 제어가 필요 없는 구조에 적합
- 일시적으로 Pod 복제 기능을 늘어나더라도 문제 없는 구조에 적합

Statefulset의 특성

상태있는 Pod로 인한 대체 불가

- Pod의 순서를 엄격히 지켜야 함
- Pod의 고유한 스토리지 및 네트워크 ID 필요
- Pod를 한나씩 순서대로 안전하게 교체 필요
- Pod는 고유한 상태를 가지고 있으므로 단순 Pod 복제로 인한 데이터 Crash 되는 문제 고려 필요

kubernetes in action book 내용 참고

[실습] Statefulset

위치: 09.statefulset

headless Service는 사전에 먼저 배포되는 것을 권고

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: sk199-myfirst-api-server
spec:
  replicas: 1
  serviceName: sk000-myfirst-api-server-headless      → headless service 구성을 위한 방법
  selector:
    matchLabels:
      app: sk199-myfirst-api-server
  template:
    metadata:
     labels:
        app: sk199-myfirst-api-server
    spec:
      containers:
      - name: myfirst-api-server
        image: amdp-registry.skala-ai.com/skala25a/sk199-myfirst-api-server:2.0
        imagePullPolicy: Always
        volumeMounts:
        - name: data
          mountPath: /data
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteMany"]
      storageClassName: efs-sc-shared
      resources:
        requests:
          storage: 100Mi
```

kubernetes in action book 내용 참고

[실습] Statefulset

위치: 09.statefulset

기존 Deployment를 Statefulset로 전환해보자

statefulset 적용하기

```
> kubectl apply –f statefulset.yaml

> kubectl get pvc    # 생성된 PVC를 확인해보자

> kubectl get pod    # statefulset 과 deployment Pod의 이름을 확인해보자
```

statefulset 고정된 이름 사용

```
> k exec -it sk000-myfirst-api-server-0 -- /bin/bash
> hostname

> cat /etc/hosts
```

```
> k exec -it sk000-myfirst-api-server-5cdf7d8ddf-xvcgk
-- /bin/bash
> hostname

> cat /etc/hosts
```

가장 큰 차이는 pod 이름, hostname, PVC

statefulset 확장하기 : 확장되거나 축소될때 **pod와 pvc 이름 변화**를 확인해보자

```
> k scale --replicas=2 -f statefulset.yaml

> k get pod  -l app=sk000-myfirst-api-server
> k get pvc -l app=skala-myfirst-api-server
```

```
> k scale --replicas=1 -f statefulset.yaml

> k get pod  -l app=sk000-myfirst-api-server
> k get pvc -l app= sk000-myfirst-api-server
```

kubernetes in action book 내용 참고

KDT Cloud – 5. 쿠버네티스 심화하기

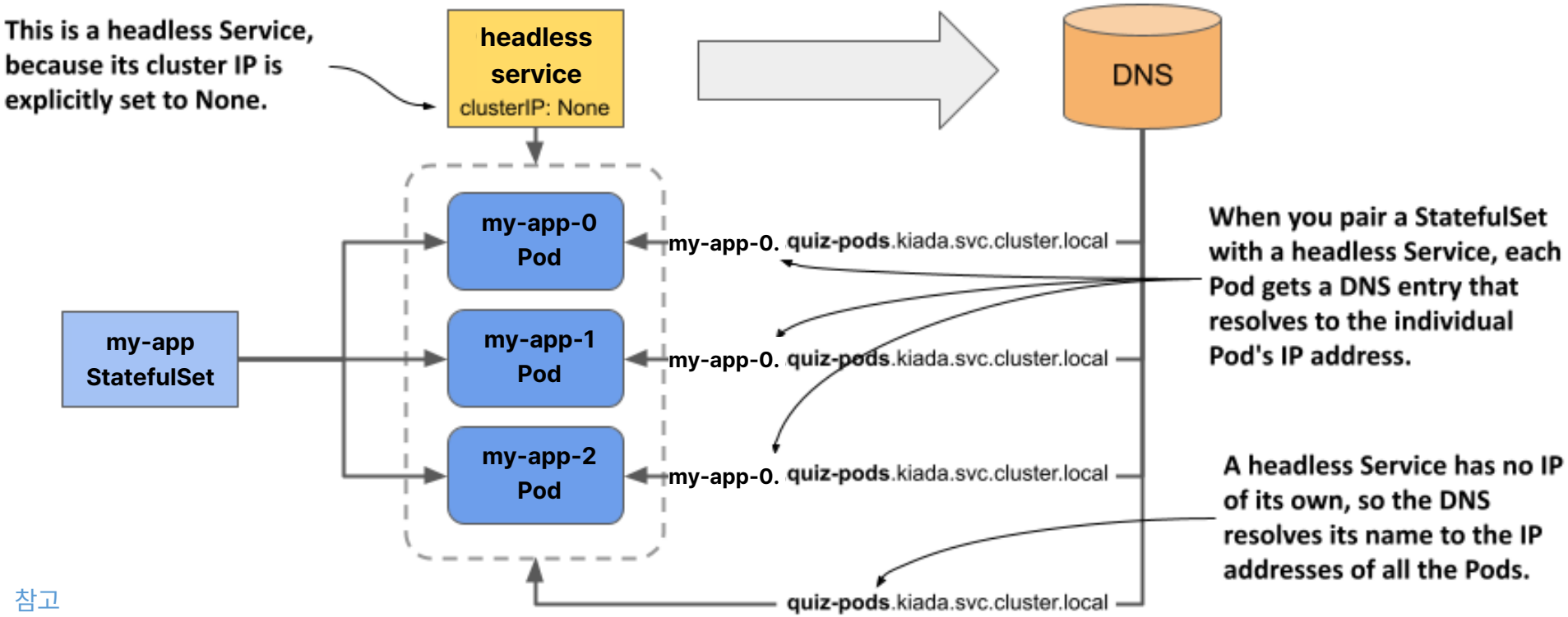
headless service

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Headless Service

k8s에서 클러스터 내에서 **파드에 직접 접근할 수 있도록 해주는 서비스 유형**이며, 일반적인 Service와는 달리, CluserIP를 부여하지 않고, DNS를 통해 파드들의 실제 IP 목록을 반환하는 방식
주로 StatefulSets 기반의 데이터베이스나 혹은 직접 파드와 통신해야 하는 경우에 사용



참고
Service head(머리)는 ClusterIP 이다.

kubernetes in action book 내용 참고

Headless Service

Headless Service는 기존 Service와 다르게 ClusterIP: None 설정을 통해 구성할 수 있음

이름 구성: {Pod-Name}.{Headless-Service-Name}.{NAMESPACE-NAME}.svc.cluster.local

```
apiVersion: v1
kind: Service
metadata:
  name: my-headless-service
  namespace: class-0
spec:
  clusterIP: None
  selector:
    app: sk000-my-app-statefulset
  ports:
    - name: http
      protocol: TCP
      port: 8080
      targetPort: 8080
    - name: mgmt
      protocol: TCP
      port: 8081
      targetPort: 8081
```

core DNS 에 등록된 정보

my-app-0.my-headless-service.skala-practice.svc.cluster.local -> 10.0.0.11

my-app-1.my-headless-service.skala-practice.svc.cluster.local -> 10.0.0.12

my-app-2.my-headless-service.skala-practice.svc.cluster.local -> 10.0.0.12

curl http:// my-app-2.my-headless-service.skala-practice.svc.cluster.local
호출 시 core DNS를 통해 Pod IP를 바로 찾아서 호출 (기존 Cluster IP 처리 x)

참고:
ClusterIP 용 기존 Service를 호출해도 동작하므로 두가지 모두 사용 가능
단, 목적이 다름

kubernetes in action book 내용 참고

Statefulset에 ServiceName Field

headless Service는 사전에 먼저 배포되는 것을 권고

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: sk199-myfirst-api-server
spec:
  replicas: 1
  serviceName: sk000-myfirst-api-server-headless
  selector:
    matchLabels:
      app: sk199-myfirst-api-server
  template:
    metadata:
     labels:
        app: sk199-myfirst-api-server
    spec:
      containers:
      - name: myfirst-api-server
        image: amdp-registry.skala-ai.com/skala25a/sk199-myfirst-api-server:2.0
        imagePullPolicy: Always
        volumeMounts:
        - name: data
          mountPath: /data
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteMany"]
      storageClassName: efs-sc-shared
      resources:
        requests:
          storage: 100Mi
```

kubernetes in action book 내용 참고

[실습] Headless Test

위치: 09.statefulset

Headless로 직접 Pod를 호출해보자

headless service 배포하기

```
# 배포된 statefulset pod  검색
> k get pod -l app=sk000-myfirst-api-server
sk100-my-app-statefulset-0   1/1     Running   0          8m15s
sk100-my-app-statefulset-1   1/1     Running   0          7m14s

> k apply -f headless.yaml

> k get service sk100-myfirst-api-server-headless
```

Pod에 직접 접속후 pod name으로 접속해보기

```
# curl-pod 로 접속하기
> kubectl run --rm -it sk000-curl-test --restart=Never --image=curlimages/curl -- /bin/sh

# Curl -pod 내부 podName.servieName:8080/python/hostname
$ curl http://sk000-myfirst-api-server-0.sk100-myfirst-api-server-headless:8080/python/hostname
{"hostname":"sk000-myfirst-api-server-0"}

$curl http://sk000-myfirst-api-server-1.sk100-myfirst-api-server-headless:8080/python/hostname
{"hostname":"sk000-myfirst-api-server-1"}
```

kubernetes in action book 내용 참고

대표적인 사용 시나리오

Headless는 일반적인 사용 방식은아니며, 아래와 같은 경우에 사용 가능

대표적인 사용 시나리오:

관계형 데이터베이스
- MySQL, PostgreSQL
- 각 Pod가 자신의 데이터 저장소를 유지해야 함
- Primary / Replica 구조에서 Pod의 고유 식별이 중요

분산 데이터베이스
- MongoDB, Cassandra
- 각 노드가 클러스터의 고유 멤버로 동작
- Pod 이름과 저장 데이터가 지속적으로 유지되어야 함

메시징 / 스트리밍 시스템
- Kafka
- 각 Broker가 고유한 ID와 저장 데이터를 가짐
- 특정 Broker와 저장소의 관계가 유지되어야 함

검색 / 분산 저장 시스템
- - Elasticsearch
- - 각 노드가 자신의 데이터와 클러스터 역할을 유지
- - Pod 재생성 시 기존 저장공간을 다시 연결할 필요가 있음

캐시 클러스터
- - Redis Cluster
- - 각 Redis 노드가 고유한 역할과 데이터를 가질 수 있음
- - 특정 노드 식별과 안정적인 네트워크 주소가 필요

-

kubernetes in action book 내용 참고

KDT Cloud – 5. 쿠버네티스 심화하기

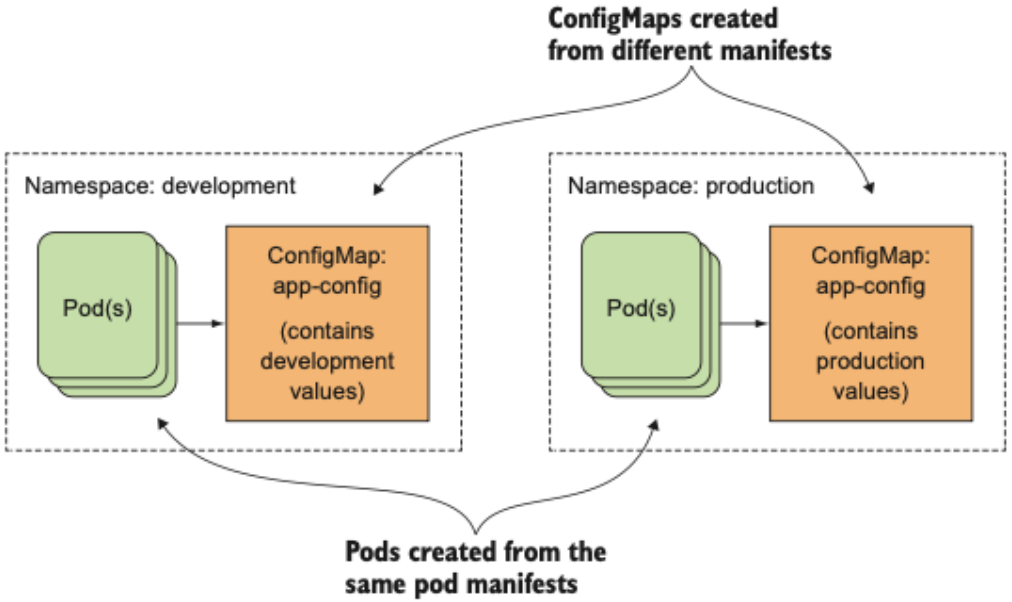
configmap

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

configmap

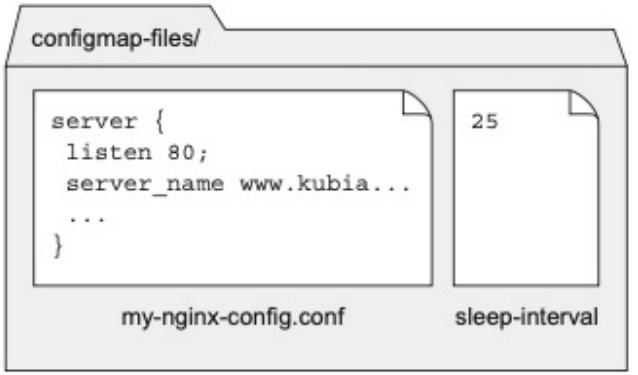
ConfigMap은 Key-Value 형식의 설정 데이터를 저장하는 Kubernetes 객체이며, 애플리케이션이 동작하는 환경 설정을 코드와 분리하여 관리할 수 있도록 지원



정의

- 배포 환경 (개발계, 검증계, 운영계) 에 따라 애플리케이션이 필요로하는 환경 및 설정 정보를 주입하는 방식
- 환경 변수 와 파일 형태로 환경 설정 지원

지원 방식

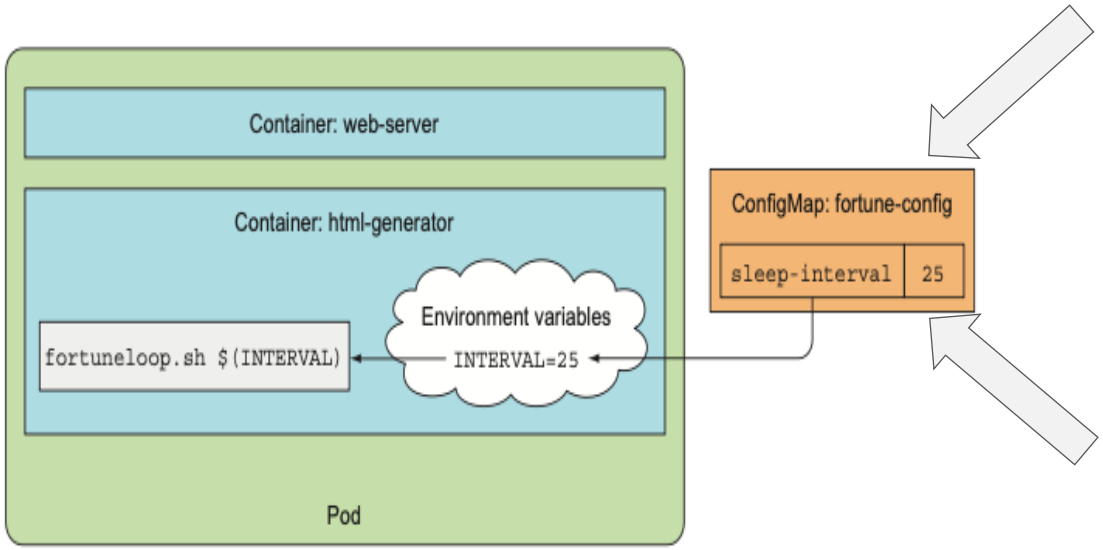


kubernetes in action book 내용 참고

환경 변수 주입 방식

Pod 내에서 사용하는 환경 변수 INTERVAL은 개발 환경 (dev) 과 운영 환경 (production)별도 다르게 동작하도록 구성하여 업무 로직과 환경에 따라 다르게 설정되는 변수를 분리

sleep-interval 을 환경 변수 INTERVAL 에 연결
프로그램 내에서 INTERVAL 사용 할 수 있음



develop-sleep-configmap.yaml

```yaml
# configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: sleep-config
  namespace: dev
data:
  sleep-interval: 25
```

production-sleep-configmap.yaml

```yaml
# configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: sleep-config
  namespace: prod
data:
  sleep-interval: 100
```

운영 환경의 Pod는 루프 실행 주기가
100초 마다 로핑 동작

kubernetes in action book 내용 참고

[실습 코드] Key-Value 환경 변수 주입 (1/3)

위치: 11.configmap/01.key-value

동일 Pod 이미지에 대해 configmap을 변경해서 환경 변수가 어떻게 바뀌는지 확인한다.

11.configmap/01.key-value/deploy.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-myfirst-api-server
  namespace: class-0
  labels:
    app: sk000-myfirst-api-server
spec:
…
        - name: DATABASE_URL
          valueFrom:
            configMapKeyRef:
              name: sk000-db-config
              key: database.url
        - name: DATABASE_USER
          valueFrom:
            configMapKeyRef:
              name: sk000-db-config
              key: database.user
        - name: DATABASE_PASSWORD
          valueFrom:
            configMapKeyRef:
              name: sk000-db-config
              key: database.password
```

11.configmap/01.key-value/dev-key-configmap.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: sk000-db-config
data:
  database.url: jdbc:mysql://localhost:3306/develop-db
  database.user: developer
  database.password: developer
```

11.configmap/01.key-value/prod-key-configmap.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: sk000-db-config
data:
  database.url: jdbc:mysql://localhost:3306/production-db
  database.user: productor
  database.password: productor
```

kubernetes in action book 내용 참고

[실습 코드] Key-Value 환경 변수 주입 (2/3)

위치: 11.configmap/01.key-value

동일 Pod 이미지에 대해 configmap을 변경해서 환경 변수가 어떻게 바뀌는지 확인한다.

```
> k apply -f dev-key-configmap.yaml

> k apply -f deploy.yaml

> k get pod -l app=sk000-myfirst-api-server
NAME                                        READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-d9fbf5b5d-rv6z5    1/1     Running   0          19m

> k exec -it sk000-myfirst-api-server-d9fbf5b5d-rv6z5 -- /bin/bash

/app# echo $DATABASE_URL
jdbc:mysql://localhost:3306/develop-db

/app# echo $DATABASE_USER
developer

/app# echo $DATABASE_PASSWORD
developer

/app# exit
```

kubernetes in action book 내용 참고

[실습 코드] Key-Value 환경 변수 주입 (3/3)

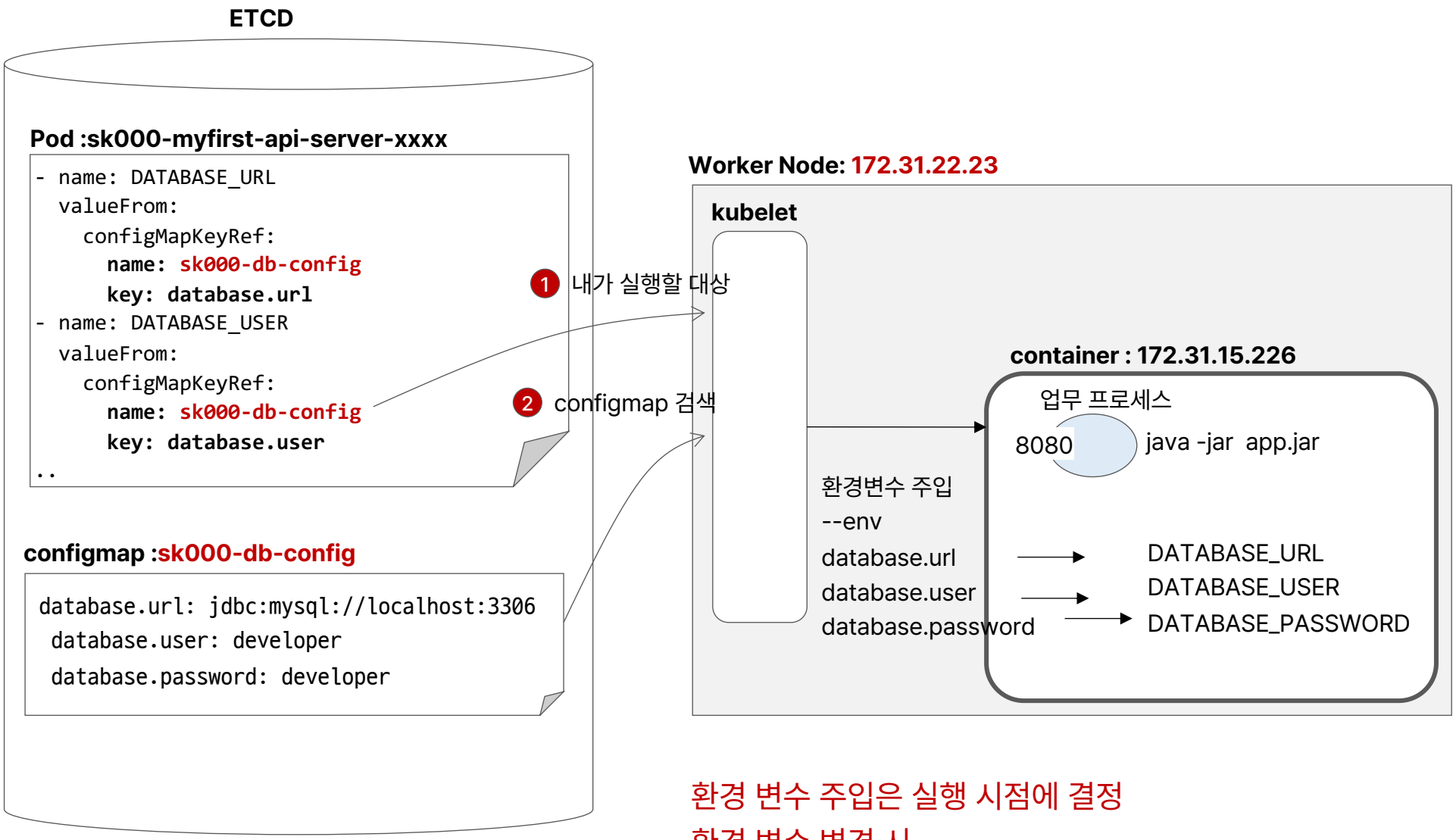
위치: 11.configmap/01.key-value

동일 Pod 이미지에 대해 configmap을 변경해서 환경 변수가 어떻게 바뀌는지 확인한다.

```
> k apply -f prod-key-configmap.yaml

# Rollout restart 명령을 통해 pod 를 restart
> k rollout restart -f deploy.yaml

> k get pod -l app=sk000-myfirst-api-server
NAME                                      READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-b9bddfb68-2dpv2   1/1     Running   0          3m28s

> k exec -it sk000-myfirst-api-server-b9bddfb68-2dpv2 -- /bin/bash

/app# echo $DATABASE_URL
jdbc:mysql://localhost:3306/production-db

/app# echo $DATABASE_USER
productor

:/app# echo $DATABASE_PASSWORD
productor

:/app# exit
```

kubernetes in action book 내용 참고

환경 변수 주입 흐름



환경 변수 주입은 실행 시점에 결정

환경 변수 변경 시

컨테이너 재 실행이 필수

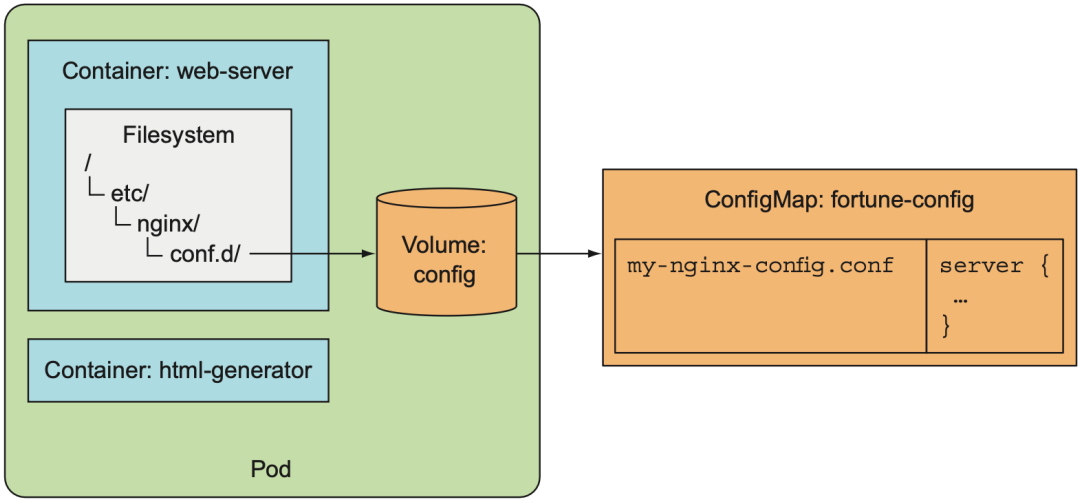
[후속 처리] 실습 자원을 삭제해보자

위치: 11.configmap/01.key-value

```
# 11.configmap으로 이동

> k delete -f 01.key-value
```

환경 파일 주입 방식

Pod 내에서 사용하는 환경 변수 INTERVAL은 개발 환경 (dev) 과 운영 환경 (production)별도 다르게 동작하도록 구성하여 업무 로직과 환경에 따라 다르게 설정되는 변수를 분리



devlop-fortune-config.yaml

```
# configmap.yaml
data:
  my-nginx-config.conf: |
    server {
      listen        8080;
      server_name   dev.skala25a.skala-ai.com

      gzip   on;
      …
```

production-fortune-config.yaml

```
# configmap.yaml
data:
  my-nginx-config.conf: |
    server {
      listen        80;
      server_name   prod.skala25a.skala-ai.com

      gzip   on;
      …
```

kubernetes in action book 내용 참고

[실습 코드] Configmap File Mount (1/3)

위치: 11.configmap/02.file

동일 Pod 이미지에 대해 configmap을 변경해서 마운트되는 파일 정보가 어떻게 바뀌는지 확인한다.

11.configmap/02.file/deploy.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-myfirst-api-server
  namespace: class-0
  labels:
    app: sk000-myfirst-api-server
 spec:
      containers:
      - name: webserver
        image: xxx:1.0
        imagePullPolicy: Always

          volumeMounts:
          - name: config-volume
            mountPath: /etc/config
        volumes:
        - name: config-volume
          configMap:
            name: sk000-file-config
```

11.configmap/02.file/dev-file-configmap.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: sk000-file-config
data:
  config.json: |
  {
    "database.url": "jdbc:mysql://localhost:3306/develop-db",
    "database.user": "developer",
    "database.password": "developer"
  }
```

11.configmap/02.file/prod-file-configmap.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: sk000-file-config
data:
  config.json: |
  {
    "database.url": "jdbc:mysql://localhost:3306/prod-db",
    "database.user": "productor",
    "database.password": "productor"
  }
```

kubernetes in action book 내용 참고

[실습 코드] Configmap File Mount (2/3)

위치: 11.configmap/01.key-value

동일 Pod 이미지에 대해 configmap을 변경해서 마운트되는 파일 정보가 어떻게 바뀌는지 확인한다.

```
> k apply -f dev-file-configmap.yaml

> k apply -f deploy.yaml

> k get pod -l app=sk000-myfirst-api-server
NAME                                        READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-d9fbf5b5d-rv6z5    1/1     Running   0          19m

> k exec -it sk000-myfirst-api-server-d9fbf5b5d-rv6z5 -- /bin/bash

/app# cd /etc/config

/app# ls
config.json

/app# cat config.json
{
  "database.url": "jdbc:mysql://localhost:3306/develop-db",
  "database.user": "developer",
  "database.password": "developer"
}
```

현재 Terminal 은 그대로 두고 새로운 터미널을 오픈한다

```
/app#
```

kubernetes in action book 내용 참고

[실습 코드] Configmap File Mount (3/3)

위치: 11.configmap/01.key-value

새로운 Terminal

```
> k apply -f prod-file-configmap.yaml
```

앞의 기존 터미널 (k exec –it 실행을 통해 이미 열러있는 터미널)

```
/app# ls
config.json

# 일정 시간 지난후 (30초 내외)
/app# cat config.json
{
  "database.url": "jdbc:mysql://localhost:3306/production-db",
  "database.user": "productor",
  "database.password": "productor"
}
```

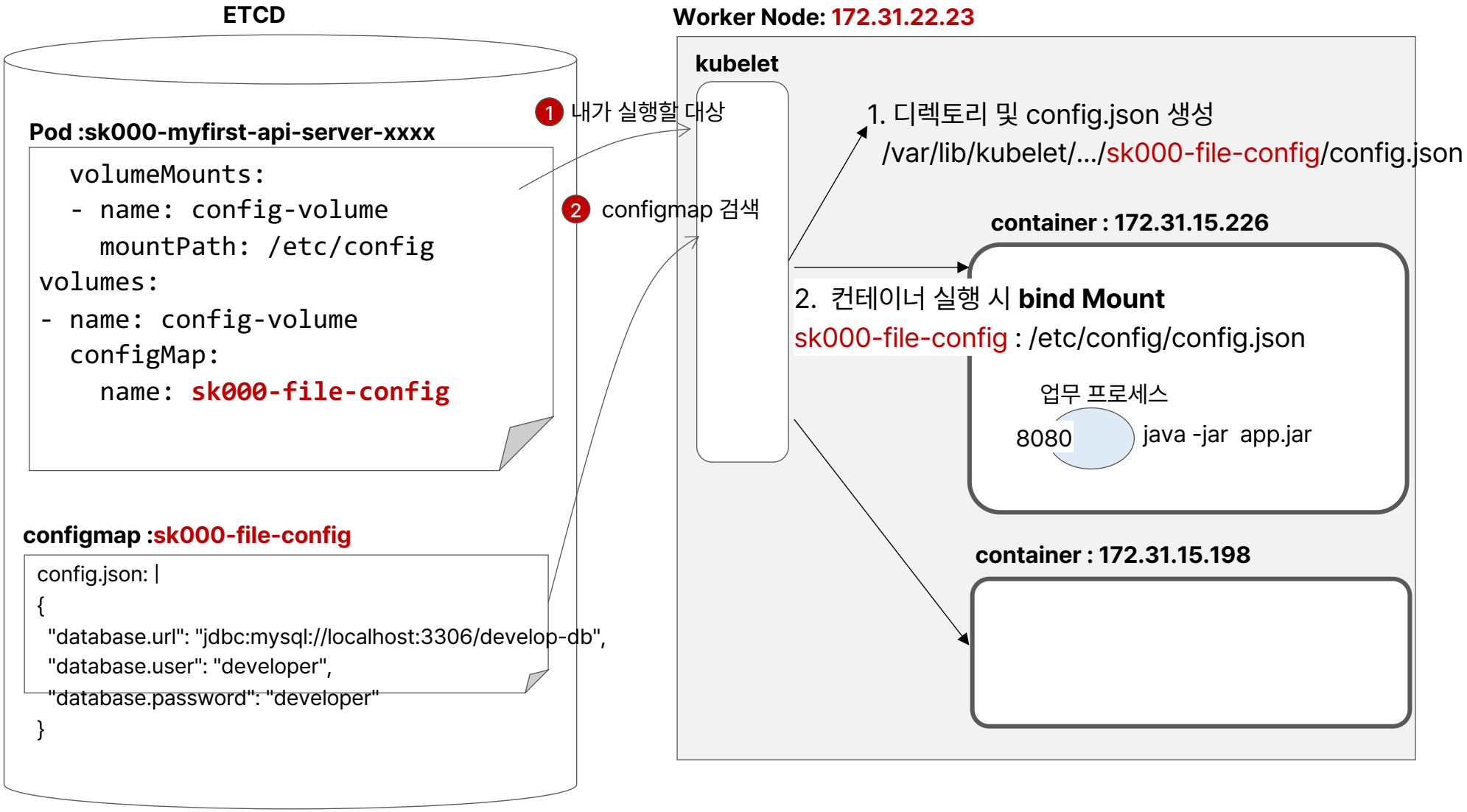
Volume 을 연결하는 방식은 Pod 재 기동과 상관없이 Pod에 반영됨

단, Application이 반영된 파일을 로딩하는 것은 별개의 문제

kubernetes in action book 내용 참고

파일 마운트 적용 흐름

/var/lib/kubelet/pods/<Pod-UID>/volumes/kubernetes.io~configmap/<볼륨이름>/



configmap volume의 data 변경 시
/var/lib/ ... /sk000-file-config/config.json에 바로 반영

[후속 처리] 실습 자원을 삭제해보자

위치: 11.configmap/01.key-value

```
# 11.configmap으로 이동

> k delete -f 02.file
```

KDT Cloud – 5. 쿠버네티스 심화하기

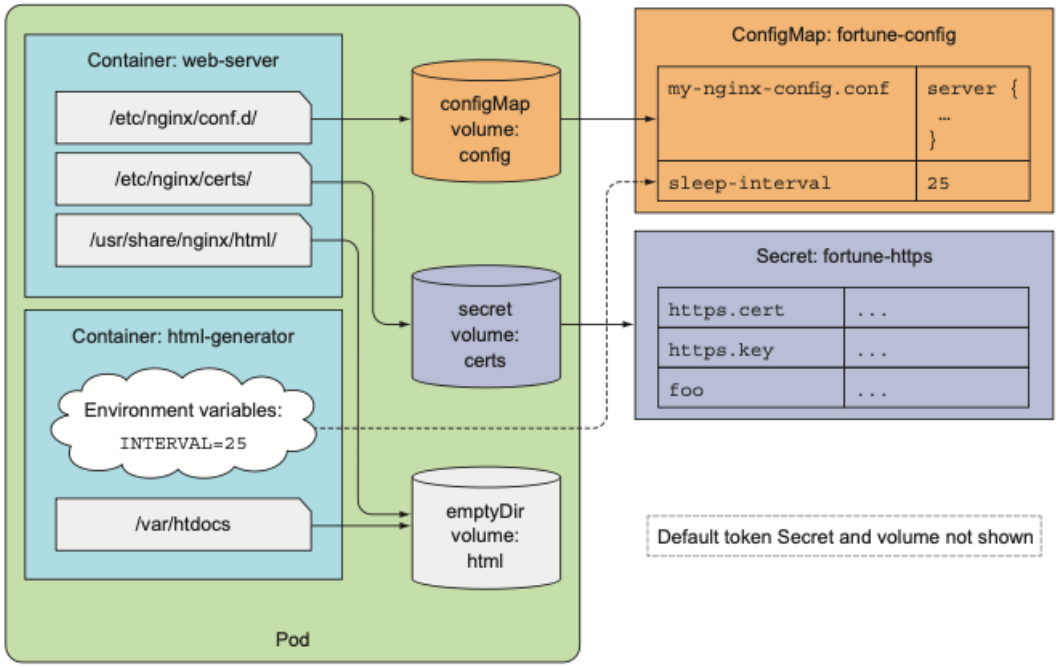
secret

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

secret

Secret은 k8s상에 민감한 데이터를 저장하고 관리하기 위한 객체로 비밀번호, Oauth 토큰, SSH key 등을 저장
Configmap은 비 민감 데이터를 저장한다면 Secret은 Base64로 인코딩해서 저장



정의

- 민감한 데이터 암호화 저장
- RBAC을 통한 접근 제어
- 컨테이너에서 환경 변수, 볼륨 마운트로 참조 지원
- 데이터베이스 접근 정보, 인증 정보등을 저장관리

지원 방식

- Opaque Secret (일반 텍스트)
- Docker Registry Secret
- TLS Secret
- Basic Auth Secret

kubernetes in action book 내용 참고

secret 유형

Secret은 포함되는 필드가 고정되어 제공되어야 하므로 다양한 유형 설정 필요

타입	주요 용도	주요 필드
Opaque	일반 키-값 데이터 저장	사용자 정의 key-value
kubernetes.io/dockerconfigjson	프라이빗 Docker 레지스트리 인증	.dockerconfigjson
kubernetes.io/basic-auth	HTTP Basic 인증	username, password
kubernetes.io/ssh-auth	SSH 키 인증	ssh-privatekey
kubernetes.io/tls	TLS 인증서 저장	tls.crt, tls.key
kubernetes.io/service-account-token	ServiceAccount 토큰 저장	token, ca.crt, namespace

실습 예정

devOps 시 사용 예정

ingress TLS 생성 시

Service Account 시 설명

-

kubernetes in action book 내용 참고

secret 유형 : Opaque

사용자가 원하는 임의의 키 이름을 자유롭게 정의해서 사용
Configmap 과 동일하지만 data field에 base64 Encoding 실행된 구조

```
apiVersion: v1
kind: Secret
metadata:
  name: example-opaque
type: Opaque
stringData:
  MY_API_KEY: "<임의의 API 키 값>"
  DB_PASSWORD: "<데이터베이스 비밀번호>"
```

kubernetes in action book 내용 참고

data 와 stringData

기본 Secret data 추가 필드는 data
사용자가 평문(Plaintext)를 그대로 넣을 수 있는 작성 편의용 (write-only) 필드 stringData를 제공

제공 방식: StringData (Plaintext) --> kue-api-server → data (Base64 Encoding)

```
apiVersion: v1
kind: Secret
metadata:
  name: my-app-secret
type: Opaque
stringData:
  DB_PASSWORD: "mysecretpassword123!"
  API_TOKEN: "Bearer abc-123-xyz"
```

```
apiVersion: v1
kind: Secret
metadata:
  name: my-app-secret
type: Opaque
data:
  DB_PASSWORD: "mysecretpassword123!"
  API_TOKEN: "Bearer abc-123-xyz"
```

base64 Encoding 명려어

```
> echo -n 'mysecretpassword123!' ¦ base64
bXlzZWNyZXRwYXNzd29yZDEyMyE=
> echo -n 'Bearer abc-123-xyz' ¦ base64
QmVhcmVyIGFiYy0xMjMteHl6
```

base64 decoding 명려어

```
> echo -n 'bXlzZWNyZXRwYXNzd29yZDEyMyE=' ¦ base64 -d
mysecretpassword123!
> echo -n 'QmVhcmVyIGFiYy0xMjMteHl6' ¦ base64 -d
Bearer abc-123-xyz
```

kubernetes in action book 내용 참고

secret 유형 : kubernetes.io/dockerconfigjson

컨테이너 런타임이 비공개 레지스트리(Harbor, AWS ECR, Docker Hub 등)에서 이미지를 받아올 수 있도록 인증 토큰을 전달하는 역할(imagePullSecrets)로 사용

Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: example-dockerconfig
type: kubernetes.io/dockerconfigjson
stringData:
  .dockerconfigjson: |
    {
      "auths": {
        "<도커레지스트리주소>": {
          "username": "<계정명>",
          "password": "<비밀번호>"
        }
      }
    }
```

Pod / Deployment/Statefulset

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
  - name: my-container
    image: skala-ai.com/class-0/myapp:latest
  imagePullSecrets:
  - name: example-dockerconfig
```

kubernetes in action book 내용 참고

secret 유형 : kubernetes.io/basic-auth

HTTP 기본 인증(ID/비밀번호 방식)에 필요한 자격 증명을 표준화된 필드명으로 보관할 때 사용
주로 Ingress Controller(Ingress-NGINX 등)의 외부 접근 제한 인증(auth-type: basic) 설정시 참조

Secret

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: basic-auth-secret
type: kubernetes.io/basic-auth
stringData:
  username: "admin"
  password: "adminSecurePassword123"
```

Pod / Deployment/Statefulset

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-service-ingress
  annotations:
    # NGINX Ingress Controller에 Basic Auth 적용 지시
    nginx.ingress.kubernetes.io/auth-type: basic
    # Secret 이름 지정
    nginx.ingress.kubernetes.io/auth-secret: basic-auth-secret
    # 브라우저 로그인 창에 표시될 메시지
    nginx.ingress.kubernetes.io/auth-realm: "Authentication
Required"
spec:
  rules:
  - host: myapp.example.com
    http:
```

kubernetes in action book 내용 참고

secret 유형 : kubernetes.io/sshj-auth

프라이빗 Git 저장소(GitHub/GitLab)나 원격 서버에 접근하기 위한 SSH 개인키를 보관
CI/CD 파이프라인(Jenkins 에이전트, Tekton, ArgoCD 등)이 소스 코드를 git clone할 때 주로 마운트하여 사용

Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: git-ssh-secret
type: kubernetes.io/ssh-auth
stringData:
  ssh-privatekey: |
    -----BEGIN OPENSSH PRIVATE KEY-----
    b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAA... (실제 개인키 본문)
    -----END OPENSSH PRIVATE KEY-----
```

kubernetes in action book 내용 참고

secret 유형 : kubernetes.io/tls

HTTPS 통신을 위한 SSL/TLS 인증서 체인과 개인키를 묶어 관리

Ingress 리소스의 spec.tls 섹션에 등록하여 외부 트래픽을 HTTPS로 SSL Termination 처리 시 사용

Secret

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: myapp-tls-secret
type: kubernetes.io/tls
stringData:
  tls.crt: |
    -----BEGIN CERTIFICATE-----
    MIIDdzCCAl+gAwIBAgIEAgAA...
    -----END CERTIFICATE-----
  tls.key: |
    -----BEGIN RSA PRIVATE KEY-----
    MIIEowIBAAKCAQEA0... (비밀키 본문)
    -----END RSA PRIVATE KEY-----
```

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myfirst-ingress
  labels:
    app: myfirst-ingress
spec:
  ingressClassName: nginx
  rules:
  - host: sk000-ingress.class-0.skala-ai.com
    http:
…
  tls:
  - hosts:
    - sk000-ingress.class-0.skala-ai.com'
    secretName: myapp-tls-secret
```

kubernetes in action book 내용 참고

[참고] base64

Base64는 암호화(Encryption) 기술이 아니라,
바이너리 데이터를 64개의 ASCII 텍스트 문자로 변환하는 인코딩(Encoding) 방식

Base 64 Secret 을 사용하는 이유

바이너리 데이터의 안전한 저장:

- TLS 개인키(tls.key), SSH 키, 압축 파일, 이미지 바이너리 등 제어 문자나 개행 문자가 포함된 데이터가 존재
- 이를 YAML이나 JSON에 그대로 넣으면 문법 오류가 나거나 파일이 깨질 수 있어서 Base64는 이를 안전한 64개 영숫자/기호 문자열로 표준화를 위해 사용
- 예) {"user":"admin"} → eyJ1c2VyIjoiYWRtaW4ifQ==

YAML/JSON 파서 충돌 방지:

- 비밀번호에 포함될 수 있는 특수문자(:, #, 따옴표, 줄바꿈 등)가 YAML의 구조 문법과 충돌하는 문제를 원천 차단

실제 Secret의 보안 책임:

쿠버네티스에서 Secret의 실제 암호화는 Base64가 아니라,
etcd 저장소 암호화(Encryption at Rest)와 **RBAC 접근 제어**로 해결

[실습 코드] Opaque Secret

가장 일반적인 Secret 유형으로 Key-value 쌍 형태로 데이터를 저장한다.

[dev|prod]-key-secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: skala-db-secret
data:
  database.url:
4oCTbiDigJxqZGJjOm15c3FsOi8vbG9jYWxob3N0OjMzMDYvZGV2ZWxvcC1k
YuKAnQo=
  database.user: ZGV2ZWxvcGVy
  database.password: ZGV2ZWxvcGVy
```

secret에 데이터를 저장하기 위해서는 base64로 encoding 되어야 한다.
$echo –n “username” | base64 #-n은 \n 을 제거한다.

```
dXNlcm5hbWU=
```

```
$echo –n “developer” | base64
ZGV2ZWxvcGVy
```

마운트한 Container에서는 자동적으로 decoding 해서 환경 변수로 주입된다.

참고: data: 대신 → stringData: 로 변경하면 base64 encoding이 자동으로 실행

환경 변수로 주입

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-deploy-test
  namespace: skala-practice
…
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: skala-db-secret
              key: database.url
        - name: DATABASE_USER
          valueFrom:
            secretKeyRef:
              name: skala-db-secret
              key: database.user
        - name: DATABASE_PASSWORD
          valueFrom:
            secretKeyRef:
              name: skala-db-secret
              key: database.password
```

kubernetes in action book 내용 참고

[실습 코드] Key-Value 환경 변수 주입 (1/3)

위치: 12.secret/01.key-value

동일 Pod 이미지에 대해 Secret을 변경해서 환경 변수가 어떻게 바뀌는지 확인한다.

deploy.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-myfirst-api-server
  namespace: class-0
  labels:
    app: sk000-myfirst-api-server
spec:
…
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: sk000-db-secret
              key: database.url
        - name: DATABASE_USER
          valueFrom:
            secretKeyRef:
              name: sk000-db-secret
              key: database.user
        - name: DATABASE_PASSWORD
          valueFrom:
            secretKeyRef:
              name: sk000-db-secret
              key: database.password
```

dev-key-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: sk000-db-secret
stringData:
  database.url: jdbc:mysql://localhost:3306/develop-db
  database.user: developer
  database.password: developer
```

prod-key-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: sk000-db-secret
stringData:
  database.url: jdbc:mysql://localhost:3306/production-db
  database.user: productor
  database.password: productor
```

kubernetes in action book 내용 참고

[실습 코드] Key-Value 환경 변수 주입 (2/3)

위치: 12.secret/01.key-value

동일 Pod 이미지에 대해 secret을 변경해서 환경 변수가 어떻게 바뀌는지 확인한다.

```
> k apply -f dev-key-secret.yaml

> k apply -f deploy.yaml

> k get pod -l app=sk000-myfirst-api-server
NAME                                        READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-d9fbf5b5d-rv6z5   1/1     Running   0          19m

> k exec -it sk000-myfirst-api-server-d9fbf5b5d-rv6z5 -- /bin/bash

/app# echo $DATABASE_URL
jdbc:mysql://localhost:3306/develop-db

/app# echo $DATABASE_USER
developer

/app# echo $DATABASE_PASSWORD
developer

/app# exit
```

kubernetes in action book 내용 참고

[실습 코드] Key-Value 환경 변수 주입 (3/3)

위치: 12.secret/01.key-value

동일 Pod 이미지에 대해 secret을 변경해서 환경 변수가 어떻게 바뀌는지 확인한다.

```
> k apply -f prod-key-secret.yaml

# Rollout restart 명령을 통해 pod 를 restart
> k rollout restart -f deploy.yaml

> k get pod -l app=sk000-myfirst-api-server
NAME                                      READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-b9bddfb68-2dpv2   1/1     Running   0          3m28s

> k exec -it sk000-myfirst-api-server-b9bddfb68-2dpv2 -- /bin/bash

/app# echo $DATABASE_URL
jdbc:mysql://localhost:3306/production-db

/app# echo $DATABASE_USER
productor

:/app# echo $DATABASE_PASSWORD
productor

:/app# exit
```

kubernetes in action book 내용 참고

[후속 처리] 실습 자원을 삭제해보자

위치: 12.secret/01.key-value

```
# 11.configmap으로 이동

> k delete -f 01.key-value
```

[실습 코드] Secret File Mount (1/3)

위치: 12.secret/02.file

동일 Pod 이미지에 대해 secret을 변경해서 마운트되는 파일 정보가 어떻게 바뀌는지 확인한다.

deploy.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-myfirst-api-server
  namespace: class-0
  labels:
    app: sk000-myfirst-api-server
 spec:
     containers:
     - name: webserver
       image: xxx:1.0
       imagePullPolicy: Always

       volumeMounts:
       - name: config-volume
         mountPath: /etc/config
     volumes:
     - name: config-volume
       secretName:
         name: sk000-file-secret
```

dev-file-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: sk000-file-secret
stringData:
  config.json: |
  {
    "database.url": "jdbc:mysql://localhost:3306/develop-db",
    "database.user": "developer",
    "database.password": "developer"
  }
```

prod-file-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: sk000-file-secret
stringData:
  config.json: |
  {
    "database.url": "jdbc:mysql://localhost:3306/prod-db",
    "database.user": "productor",
    "database.password": "productor"
  }
```

kubernetes in action book 내용 참고

[실습 코드] Secret File Mount (2/3)

위치: 12.secret/02.file

동일 Pod 이미지에 대해 secret을 변경해서 마운트되는 파일 정보가 어떻게 바뀌는지 확인한다.

```
> k apply -f dev-file-secret.yaml

> k apply -f deploy.yaml

> k get pod -l app=sk000-myfirst-api-server
NAME                                        READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-d9fbf5b5d-rv6z5    1/1     Running   0          19m

> k exec -it sk000-myfirst-api-server-d9fbf5b5d-rv6z5 -- /bin/bash

/app# cd /etc/config

/app# ls
config.json

/app# cat config.json
{
  "database.url": "jdbc:mysql://localhost:3306/develop-db",
  "database.user": "developer",
  "database.password": "developer"
}
```

현재 Terminal 은 그대로 두고 새로운 터미널을 오픈한다

```
/app#
```

kubernetes in action book 내용 참고

[실습 코드] Secret File Mount (3/3)

위치: 12.secret/02.file

새로운 Terminal

```
> k apply -f prod-file-secret.yaml
```

앞의 기존 터미널 (k exec –it 실행을 통해 이미 열러있는 터미널)

```
/app# ls
config.json

# 일정 시간 지난후 (30초 내외)
/app# cat config.json
{
  "database.url": "jdbc:mysql://localhost:3306/production-db",
  "database.user": "productor",
  "database.password": "productor"
}
```

Volume 을 연결하는 방식은 Pod 재 기동과 상관없이 Pod에 반영됨

단, Application이 반영된 파일을 로딩하는 것은 별개의 문제

kubernetes in action book 내용 참고

[후속 처리] 실습 자원을 삭제해보자

위치: 12.secret/02.file

```
# 11.configmap으로 이동

> k delete -f 02.file
```

KDT Cloud – 5. 쿠버네티스 심화하기

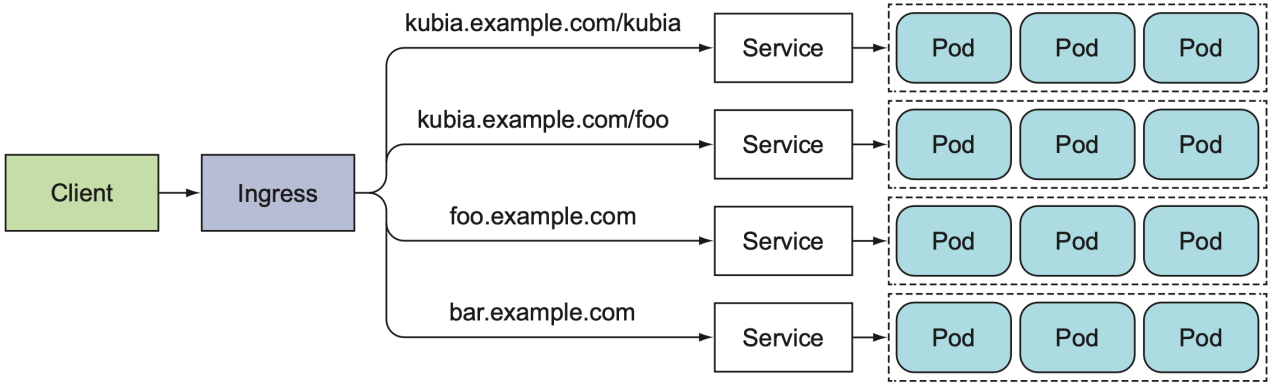
HTTPS 기반 ingress 구성

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Ingress

Ingress는 외부에서 들어오는 HTTP/HTTPS 요청을 어떤 Service로 보낼지 **L7 라우팅 규칙**만 선언하는 리소스
HTTP 및 HTTPS 요청을 처리하며, 도메인, 경로 기반 라우팅, SSL 종료(TLS), 호스트 헤더 처리 등의 기능을 제공



선언적 리소스

- **이렇게 라우팅하고 싶다**만 기술
- 어떻게 처리할지는 **Controller** 책임

L7 (Application Layer) 기반

- Host 기반 라우팅
- Path 기반 라우팅
- TLS 종료 설정

kubernetes in action book 내용 참고

Ingress 리소스

HTTPS GET request Header

```
GET /api/users HTTP/1.1
Host: myapp.example.com
User-Agent: curl/8.4.0
Accept: */*
```

Routing flow

- myapp.example.com/api → api-svc
- myapp.example.com/web → api-web

ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
spec:
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-svc
            port:
              number: 80
      - path: /web
        pathType: Prefix
        backend:
          service:
            name: web-svc
            port:
              number: 80
```

kubernetes in action book 내용 참고

Ingress Controller

Ingress 리소스를 “해석해서 실제 트래픽을 처리하는 실행 주체

Ingress Controller는 k8s Controller 역할과 L7 proxy/LoadBalancer 역할 수행

구현 방식

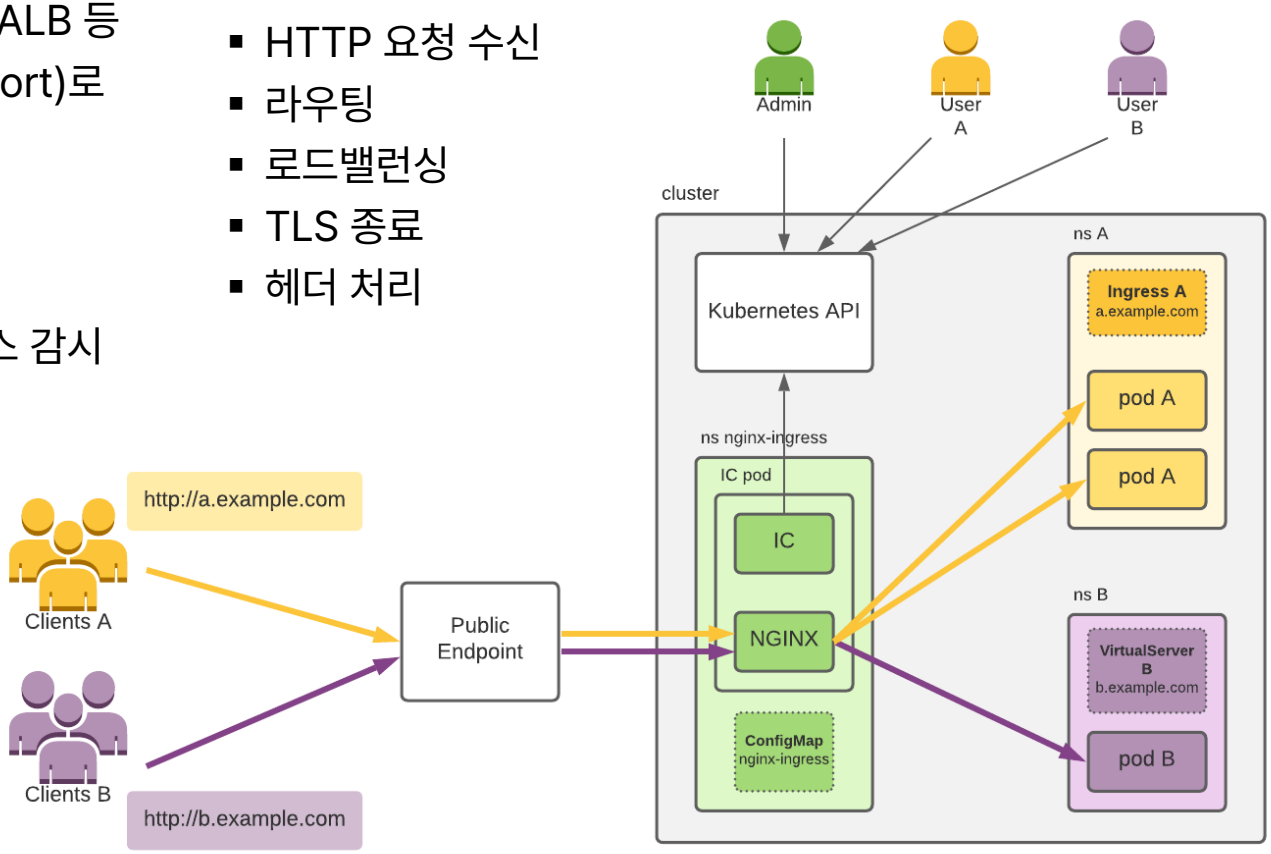
- NGINX, Envoy, HAProxy, AWS ALB 등
- Service(LoadBalancer / NodePort)로 외부 노출

실제 트래픽 처리 주체

- HTTP 요청 수신
- 라우팅
- 로드밸런싱
- TLS 종료
- 헤더 처리

Ingress 리소스를 Watch

- Kube API Server의 Ingress 리소스 감시
- 설정 변경 시 즉시 반영

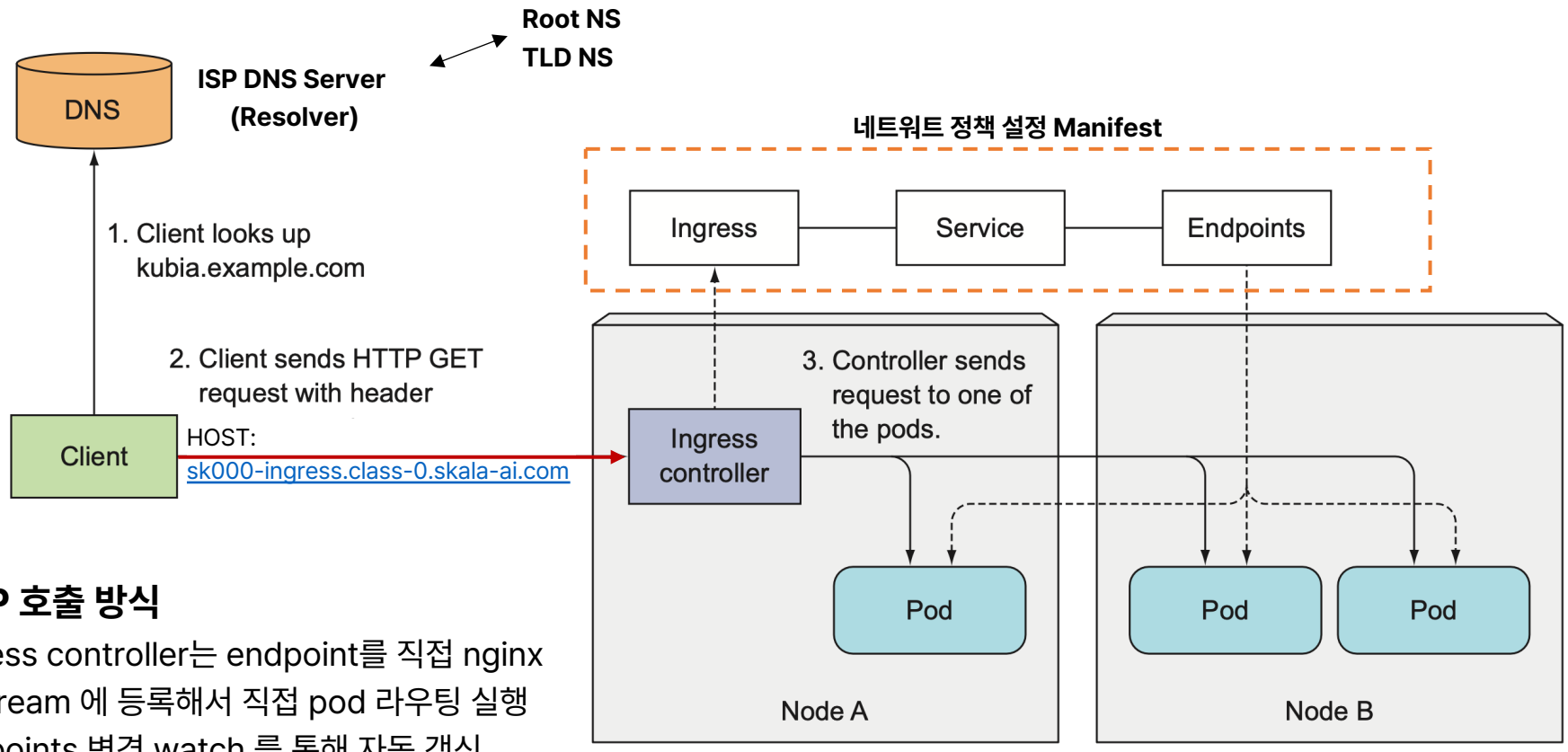


kubernetes in action book 내용 참고

Ingress 흐름

curl https://sk000-ingress.class-0.skala-ai.com 접속 시

Ingress 는 서비스를 호출하고 서비스는 동일 pod 중 하나로 배포



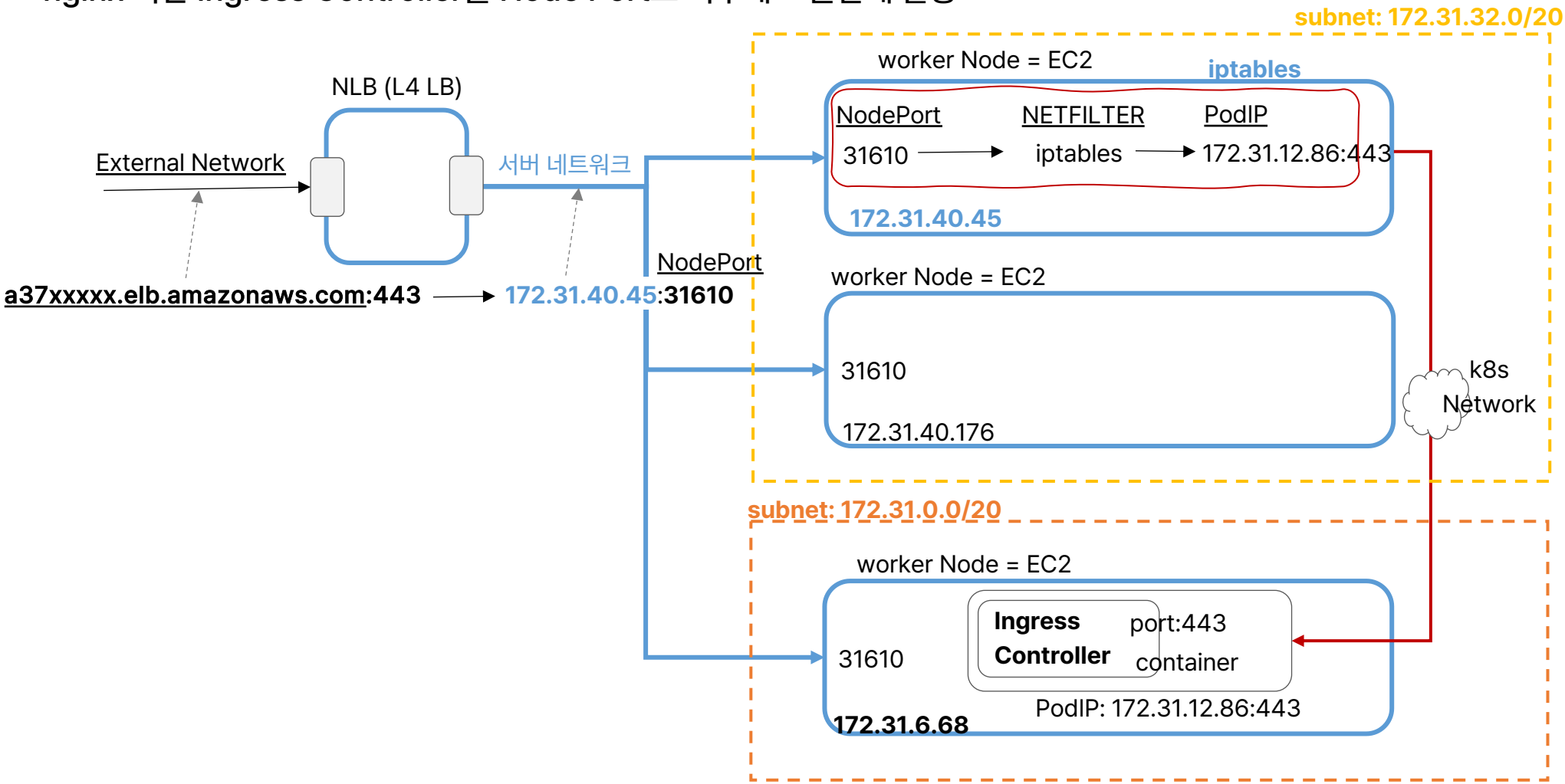
Pod IP 호출 방식

- ingress controller는 endpoint를 직접 nginx upstream 에 등록해서 직접 pod 라우팅 실행
- endpoints 변경 watch 를 통해 자동 갱신

kubernetes in action book 내용 참고

[참고] Ingress Controller 접속 흐름

Kubernetes Cluster 외부에서 특정 노드의 포트를 통해 서비스 접근을 지원. Private Cluster를 구성하는 경우 nginx 기반 Ingress Controller를 Node Port로 외부에 노출할때 활용



Ingress 작성 방식

동일 Hosts에 여러 서비스를 노출 여러 Path로

```
...
  - host: kubia.example.com
    http:
      paths:
      - path: /kubia
        backend:
          serviceName: kubia
          servicePort: 80
      - path: /foo
        backend:
          serviceName: bar
          servicePort: 80
```

Requests to kubia.example.com/kubia will be routed to the kubia service.

Requests to kubia.example.com/bar will be routed to the bar service.

다른 Hosts로 multiple service 를 노출하는 방식

```
spec:
  rules:
  - host: foo.example.com
    http:
      paths:
      - path: /
        backend:
          serviceName: foo
          servicePort: 80
  - host: bar.example.com
    http:
      paths:
      - path: /
        backend:
          serviceName: bar
          servicePort: 80
```

Requests for foo.example.com will be routed to service foo.

Requests for bar.example.com will be routed to service bar.

kubernetes in action book 내용 참고

[실습] HTTP 기반 Ingress

위치: 14.ingress/01.http

sk000-myfirst-api-server:1.0 기반으로 간단하게 인그레스를 구성해서 테스트해보기

```
> k apply -f deploy.yaml
> k apply –f service.yaml
> kubectl get pod -l app=sk000-myfirst-api-server
> kubectl get service ¦ grep sk000

ingress.yaml 확인하기
> k apply -f ingress.yaml
> k get ingress sk000-ingress -w
```

```
NAME                  CLASS          HOSTS                                ADDRESS
                                                PORTS     AGE
sk000-cm-tls-ingress  public-nginx   sk000-tls.skala25a.project.skala-ai.com  a55bf7a790b8e4b26b67a129d3263385-109
4816387.ap-northeast-2.elb.amazonaws.com   80, 443   15d
```

ADDRESS 가 할당되어야 함 (AWS ELB의 CNAME)

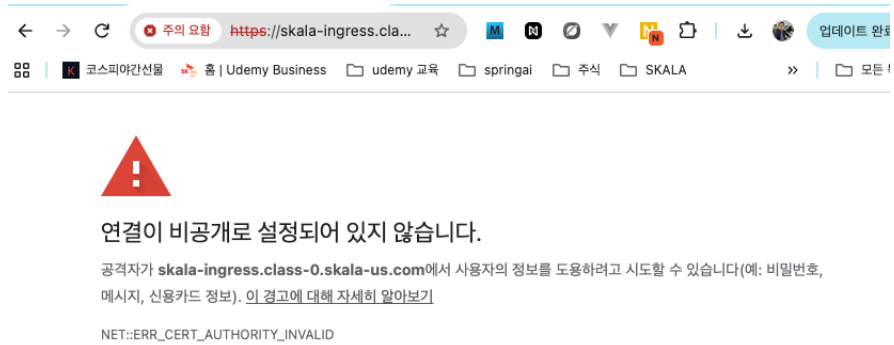
kubernetes in action book 내용 참고

[실습] Browser 접속하기

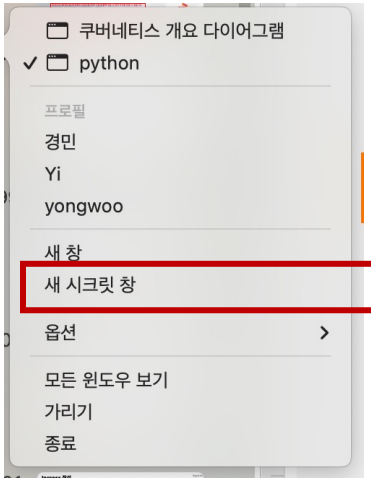
위치: 14.ingress/01.http

http://sk000-ingress.skala25a.project.skala-??.com

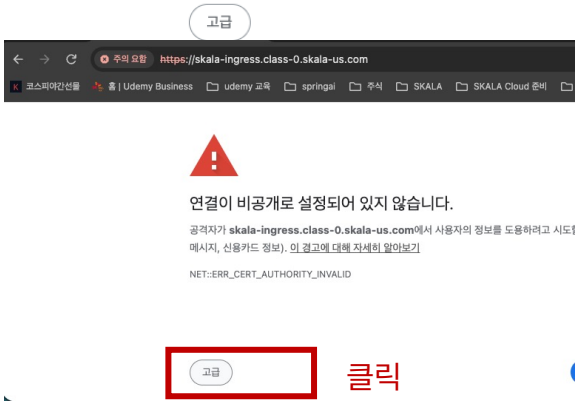
1. 접속시 정상적으로 되나?



2. 시크릿창으로 다시 접속해보기



클릭



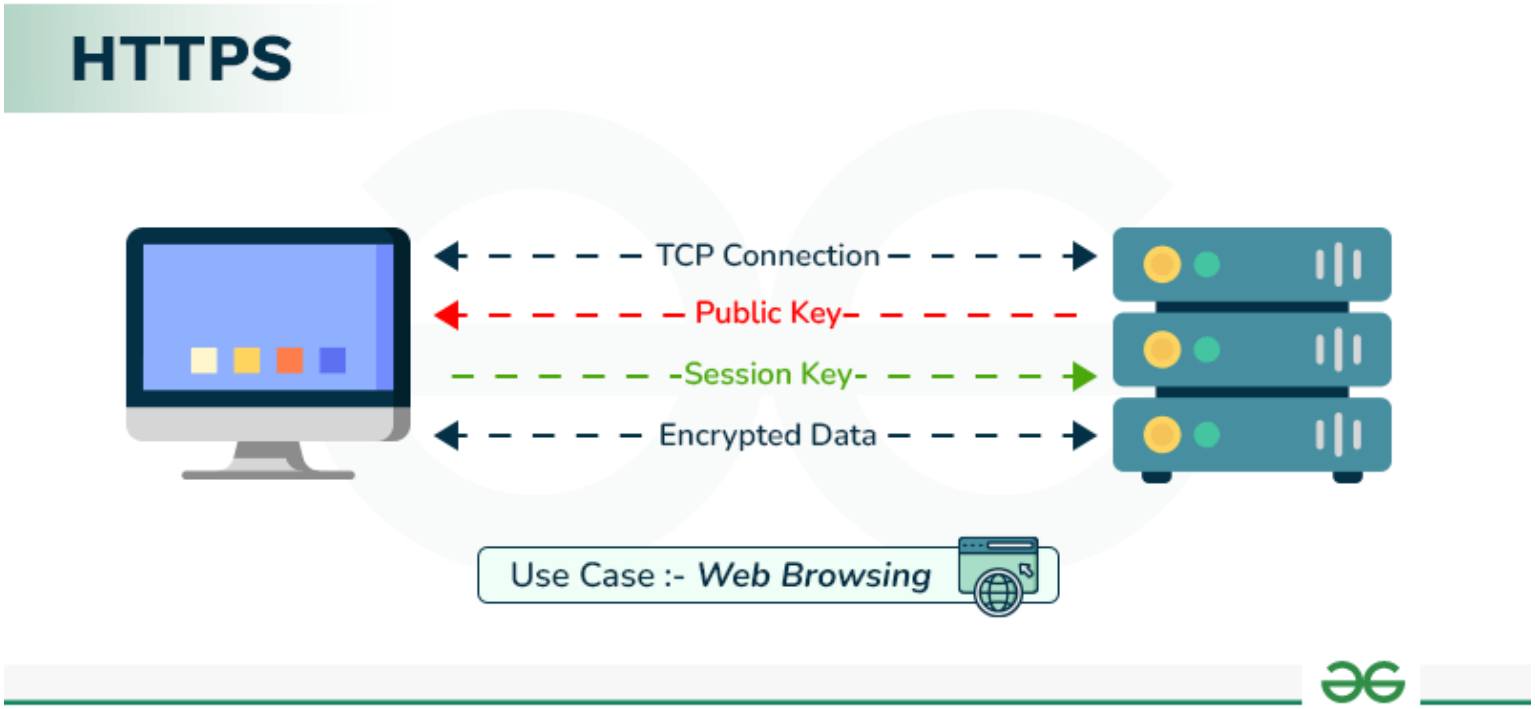
클릭

클릭

kubernetes in action book 내용 참고

HTTPS (HyperText Transfer Procotol Secure)

웹 브라우저와 웹사이트 간에 데이터를 전송하는 데 가장 일반적으로 사용되는 프로토콜
HTTPS는 사용자의 브라우저와 웹사이트 간의 통신에 사용되며 데이터 전송을 암호화하여 보안을 강화



- HTTPS는 기존 HTTP 프로토콜에 **SSL/TLS** 보안 계층을 추가한 것
- HTTP와 HTTPS의 워크플로는 동일 하며, 브라우저와 서버는 여전히 HTTP 프로토콜을 사용하여 서로 통신
- 하지만 이 과정은 안전한 SSL 연결을 통해 이루어짐
- SSL 연결은 **데이터 안전을 보장하기 위해 교환되는 데이터의 암호화 및 복호화**를 담당

kubernetes in action book 내용 참고

HTTPS 프로토콜을 이해하기 위한 암호화 기술

- HTTPS를 이해하기 위해 필요한 암호화 요소는 다음과 같다

	사용 기술	설명	
TLS 연결용	**Public/Private Key**	공개키로 암호화하고 개인키로 복호화	tls.crt, tls.key
	Certificate / CA	이 Public Key가 누구의 것인지 CA가 보증한다	ca.crt
키 인증용	**Hash**	원문에서 만든 요약값, 즉 데이터의 지문	
	전자서명	Private Key로 서명하고 Public Key로 검증한다	
데이터 암호화	**대칭키**	같은 키로 암호화하고 복호화	session.key

HTTPS Ingress : tls.crt, tls.key
Client가 사전에 알아야 할 Key: ca.crt

kubernetes in action book 내용 참고

CERT Manager 란?

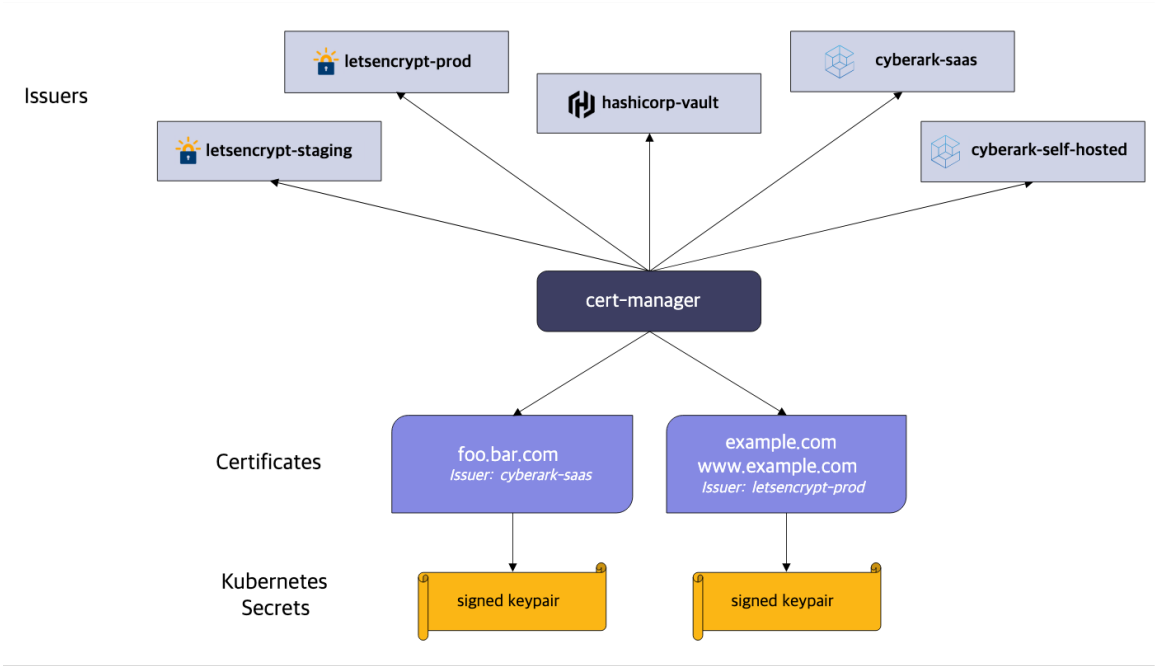
TLS를 위한 개인 키 tls.key를 생성하고 Let’s Encrypt 같은 공인 인증 기관(CA)을 통해
인증된 tls.crt를 발급받아서 kubernetes 기반의 kubernetes.io/tls Secret을 생성해주는 도구

- tls.key → **cert-manager가 생성**
- tls.crt → **Let's Encrypt 같은 CA가 서명해서 발급**
- public key + domain 정보 (*.class-0.skala-ai.com) 를 포함한 CSR 생성 및 공인인증기관 전달

Issuers
어떤 CA를 사용해 발급할지 정의하는 리소스

Certificate
어떤 도메인으로 발급받을지를 요청하는 리소스

Secrets
발행받은 tls.crt + tls.key



kubernetes in action book 내용 참고

[참고] CERT Manager 설치 방법

cert-manager는 Kubernetes 클러스터에서 TLS 인증서를 **자동으로 발급, 갱신, 관리**해주는 컨트롤러
Let's Encrypt, HashiCorp Vault, Venafi, 또는 내부 PKI 등 다양한 인증기관(CA)과 연동이 가능

helm 기반 설치 방법

```
helm repo add jetstack https://charts.jetstack.io
helm repo update

helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager \
  --create-namespace \
  --version v1.14.3
```

kubernetes in action book 내용 참고

[참고] tls.crt 발급 무료 인증기관 Let’s Encrypt (CA)

cert-manager에 의해 생성된 pub.key를 인증해주기 위한 기관

- ACME 프로토콜의 대표적인 구현체
- 비영리 단체인 [ISRG (Internet Security Research Group)]에서 운영
- 전 세계적으로 가장 널리 사용되는 무료 인증기관

CA 역할

역할	설명
발급 (Issue)	인증서 요청을 받고, 조건을 만족하면 인증서(tls.crt)를 발급
서명 (Sign)	인증서에 CA 자신의 비밀키로 전자서명하여 신뢰를 보장
인증 (Validate)	브라우저 또는 클라이언트가 접속 시 CA 서명 여부를 확인해 인증
갱신/폐기	인증서 만료 전 갱신하거나 유출·폐기 시 인증서를 취소 처리

ACME (Automated Certificate Management Environment): TLS 인증서를 자동으로 발급, 갱신, 취소할 수 있도록 만든 공개 표준 프로토콜

[참고] CERT Manager: Cluster Issuer

인증서를 어디에서, 어떤 방식으로 발급받을 것인가?"를 정의한 전역 설정

적용 범위에 따라 ClusterIssuer 와 Issuer로 구분

역할	설명
privateKey SecretRef	Let's Encrypt 계정을 식별하는 **핵심 키** (관리 계정 인증) • 인증서 발급 횟수 제한 • 동일 도메인 재발급 제한
server	cert-manager가 통신할 ACME 서버 주소 Let's Encrypt의 운영 환경 선택 필드 https://acme-v02.api.letsencrypt.org/directory
solvers	• 도메인 소유권을 어떻게 증명할지 정의하는 설정 • ACME Challenge 방식 선택 (HTTP-01, DNS-01 등) • 이 예제에서는 DNS-01 + AWS Route53 사용

```yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    # 인증서 갱신/오류 관련 알림을 받을 이메일 주소
    email: himang10@sk.com
    # 인증서 체인을 특정할 필요 없으면 비워둡니다
    preferredChain: ""
    # ACME 계정 키를 저장할 Secret 이름
    privateKeySecretRef:
      name: letsencrypt-prod
    # Let's Encrypt 프로덕션 서버 URL (실제 인증서 발급용)
    server: https://acme-v02.api.letsencrypt.org/directory
    # 도메인 소유 확인 방법 (여기서는 DNS-01, AWS Route53)
    solvers:
    - dns01:
        route53:
          region: ap-northeast-2
          accessKeyID: XXXXXXXXXXX
          secretAccessKeySecretRef:
            name: route53-secret
            key: secretAccessKey
```

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: route53-secret
  namespace: cert-manager
type: Opaque
stringData:
  secretAccessKey: yoa+eAaV3yc
```

kubernetes in action book 내용 참고

[참고] Cluster Issuer의 Solvers

Solver의 목적

ACME Challenge = **도메인 소유(제어)권 증명 방식**

[ACME CA (Let's Encrypt)] **-- Challenge 제시 ----→** **[cert-manager]**

" 이 Domain이 진짜 네 것 이라면,
내가 준 Token을 특정 경로에 올려라"

" 알겠다 내기 지정한 Solver로 이 문제를 풀겠다"

[Solver 실행]

- HTTP01: Pod/Ingress 임시 배포 되는지 확인
- DNS01: TXT 레코드를 dns에 자동 생성되는지 확인

kubernetes in action book 내용 참고

[참고] Cluster Issuer의 Solvers

ACME Challenge는 인증서 발급 요청자가 해당 도메인을 실제로 제어하고 있는지를
HTTP 방식으로 증명할지, DNS 방식으로 증명할지를 선택하는 것

1. HTTP-01

- Let’s Encrypt가 HTTP (80) 요청으로 검증
- 특정 URL에 검증용 토큰 응답이 가능한지 확인

전제 조건

- LoadBalancer에 80 포트는 노출 시켜야 함
  (Ingress Load Balancer에 80, 443모두 노출)
- Ingress / 웹 서버 접근 가능

2. DNS-01

- _acme-challenge.도메인에 TXT 레코드를 만들고
  , Let's Encrypt가 DNS를 조회해서 확인

전제 조건

- DNS API 접근 권한(Route53 등)

Wildcard와의 결정적 차이

- HTTP-01 → Wildcard (불가)
- DNS-01 → Wildcard (가능)

kubernetes in action book 내용 참고

[참고] Cluster Issuer Solver: http01

Ingress Loadbalancer 생성 시 80 포트도 노출후 ClusterIssuer 적용 가능

```yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: http01-letsencrypt-prod
spec:
  acme:
    email: skala@sk.com
    privateKeySecretRef:
      name: letsencrypt-prod
    server: https://acme-v02.api.letsencrypt.org/directory
    solvers:
    - http01:
        ingress:
          ingressClassName: nginx
```

대상 ingress controller 의 이름

별도 Aws Route53 권한 정보가 필요없음

80 노출되어야 함

```
NAME                       TYPE           CLUSTER-IP      EXTERNAL-IP                                PORT(S)                      AGE
ingress-nginx-controller   LoadBalancer   172.20.173.98   a927de98344[illegible]-2.elb.amazonaws.com   80:30454/TCP,443:31052/TCP   31m
```

kubernetes in action book 내용 참고

인증서 발급 요청: Certificate 리소스

Cert-manager에게 pub.key와 tls.key를 발급요청하고 pub.key를 CA로 부터 인증받아서 tls.crt를 만든후
wildcard-class-0-skala-us-com-tls secret (kubernetes.io/tls)를 생성하도록 요청

CluserIssuer with dns01

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: wildcard-class-0-skala-us-com
  namespace: class-0
spec:
  secretName: wildcard-class-0-skala-us-com-tls
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
  dnsNames:
    - '*.class-0.skala-us.com'
    - 'class-0.skala-us.com'
```

실행 절차

① cert-manager에게 tls.crt와 tls.key 생성을 요청
② cert-manager는 tls.key와 pub.key 생성
③ pub.key 인증을 위해 pub.key + domain 포함한 CSR (인증서 서명 요청)을 let's encrypt에 전달
④ let'sencrpyt는 도메인 소유권 확인 및 TLS 인증서 발급
⑤ cert-manager는
- 발급 받은 tls.crt 와 tls.key를
- wildcard-class-0-skala-us-com-tls에 저장

이 secret은 Ingress가 해당 Secret을 참조해 HTTPS 활성화

kubernetes in action book 내용 참고

HTTP Ingress 구성 절차 : Certificate 생성 (1/2)

Certificate 를 통해 Domain 에 대한 인증된 TLS Secret 생성

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: wildcard-class-0-sk000-us-com
  namespace: class-0
spec:
  secretName: wildcard-class-0-sk000-us-com-tls
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
  dnsNames:
    - '*.class-0.skala-us.com'
    - 'class-0.skala-us.com'
```

최종 생성되는 TLS Secret

허가되는 domain 등록

*.class-0.skala-us.com: class-0.skala-us.com인 모든 subdmain용

예) skasssss.class-0.skala-us.com, aaaa.class-0.skala-us.com

kubernetes in action book 내용 참고

HTTP Ingress 구성 절차 : Ingress 구성 (2/2)

Certificate를 통해 생성된 wildcard-class-0-sk000-us-com-tls을 이용해서 구성

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: skala-cert-ingress
  namespace: class-0
spec:
  ingressClassName: nginx
  rules:
  - host: sk000-cert-ingress.class-0.skala-us.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: sk000-my-app
            port:
              number: 8080
  tls:
  - hosts:
    - sk000-cert-ingress.class-0.skala-us.com
    secretName: wildcard-class-0-sk000-us-com-tls
```

Certificate를 통해 생성된
Secret (tls.crt/tls.key) 을 Ingress TLS로 사용

kubernetes in action book 내용 참고

[실습] Cert-manager 기반 Ingress 생성

위치: 14.ingress/04.cert-manager

기 배포되어 있는 Certificate를 기분으로 TLS 지원 Ingress를 구성해보자

1. 기 배포되어 있는 Certificate와 이를 통해 생성된 Secret을 확인해보자

```
> k get certificate
NAME                            READY   SECRET                                  AGE
wildcard-class-0-skala-us-com   True    wildcard-class-0-sk000-us-com-tls   17d

k get secret wildcard-class-0-skala-us-com-tls -o yaml
apiVersion: v1
data:
  tls.crt: LS0tLS1CRUdJT
  tls.key: LS0tLS1CRU
kind: Secret
…
```

2. ingress를 배포해보자

```
> k apply -f wild-cert-ingress.yaml
ingress.networking.k8s.io/skala-cert-ingress created

> k get ingress sk000-cert-ingress -w
NAME                CLASS   HOSTS                                    ADDRESS
sk000-cert-ingress   nginx   sk000-cert-ingress.class-0.skala-us.com   ac8c1a2a656274c8a90dbfebc5fb80c6-1990274969.ap-
northeast-2.elb.amazonaws.com   80, 443   51s
```

ADDRESS 부분에 URL이 표시될때 까지 대기 (이 주소는 NLB URL)

여기 주소로 Browser를 통해 접속해보자. 접속이되는지 확인

kubernetes in action book 내용 참고

[참고] CERT Manager – ingress 기반 Certificate 자동 생성

annoations에 cert-manager.io/cluster-issuer: letsencrypt-prod 설정 시 Certificate 설정없이 자동적으로 Certificate 생성 및 Ingress 실행 지원

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: sk000-cm-tls-ingress
  namespace: skala-practice
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: public-nginx
  tls:
  - hosts:
    - sk000-tls.skala25a.project.skala-ai.com
    secretName: sk000-cm-auto-tls-secret
  rules:
  - host: sk000-tls.skala25a.project.skala-ai.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: sk000-my-app
            port:
              number: 8080
```

annotation에 cluster-issuer 등록 시 :
Certificate 및 TLS Secret을 별도 생성 없이 자동적으로 sk000-cm-auto-tls-secret을 생성

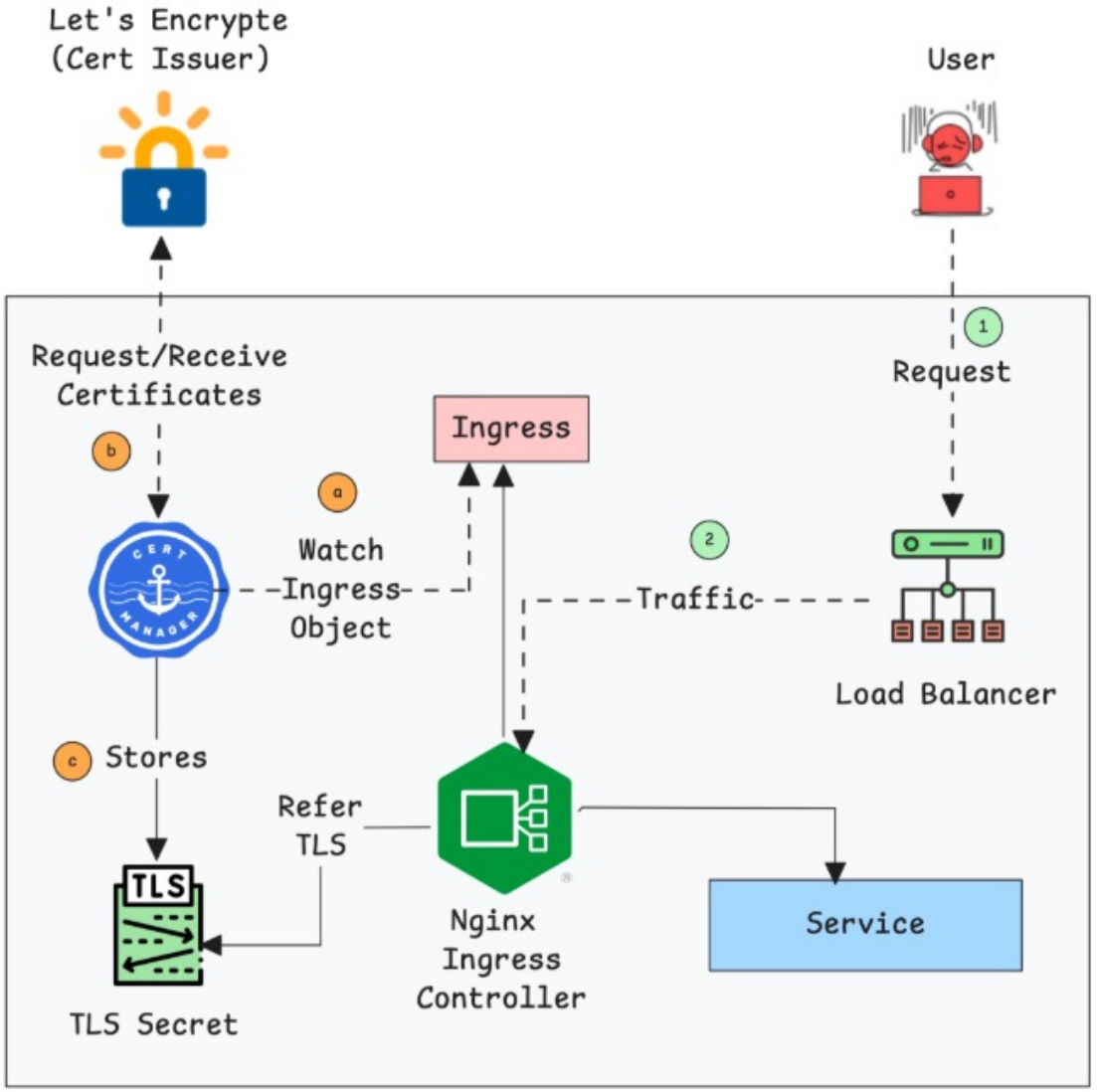
개별 클러스터에서 Domain 등록 후 사용 시
Route53에 별도 등록 과정 필요

your-subdomain.skala25a.cloud.skala-ai.com 제공
직접 등록 과정 필요

kubernetes in action book 내용 참고

[참고] CERT Manager – ingress 용 TLS 흐름

Ingress 생성 시 Domain 에 대한 TLS Secret (tls.crt, tls.key 포함)을 할당 받는 과정



1. Ingress 객체를 생성할 때,
   해당 객체에 Cert Manager Issuer에 대한 참조를 추가
2. **Ingress Controller**는 Ingress 객체에서 정보를 가져와
   Cert Manager에 인증서를 요청
3. **Cert-Manager**가 서버 개인키(tls.key)를 생성하고,
   이를 기반으로 CSR를 만들어 **CA(Let's Encrypt)**에 요청
4. CA는 서버 인증서(tls.crt)를 **Cert-Manager**에 반환
5. tls.crt 와 tls.key는 kubernetes.io/tls 타입의
   TLS Secret으로 저장
6. Ingress Controller는 TLS Secret를 사용하여 TLS 처리

kubernetes in action book 내용 참고

명시적으로 Certificate 구성해야 하는 경우

Certificate를 별도로 구성해야 할 경우

- Ingress가 아닌 다른 대체 Gateway 활용 시 인증서가 필요할 때
  - envoy Gateway, Traefik, kong, istio etc
- 인증서 요구사항을 더 정교하게 제어할 때 (정책/표준을 강제해야 하는 조직에 적합)
- **하나의 인증서를 여러 Ingress/서비스에서 공유할 때**
- GitOps/운영 관점에서 **명시적 리소스**가 필요할 때

KDT Cloud – 5. 쿠버네티스 심화하기

무중단 배포

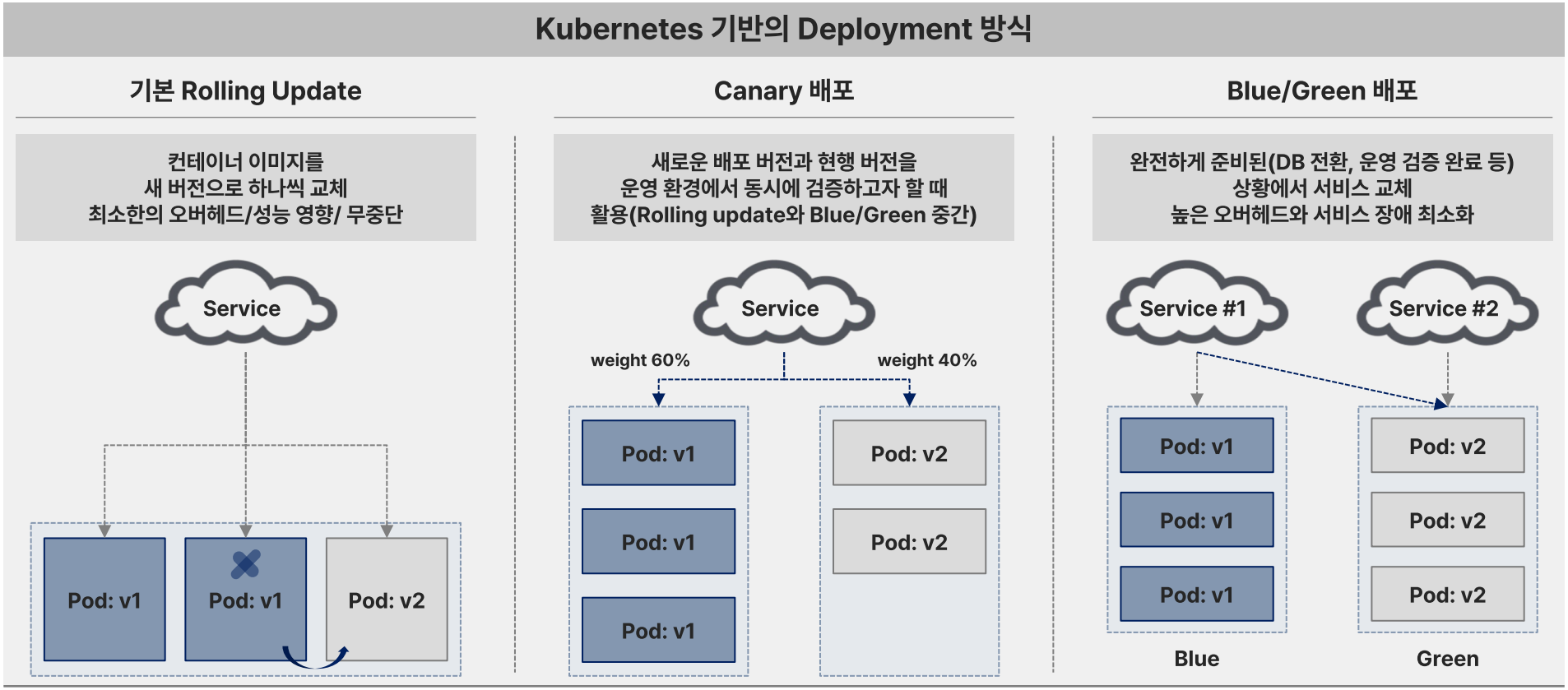
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

배포 방식

기본 개념

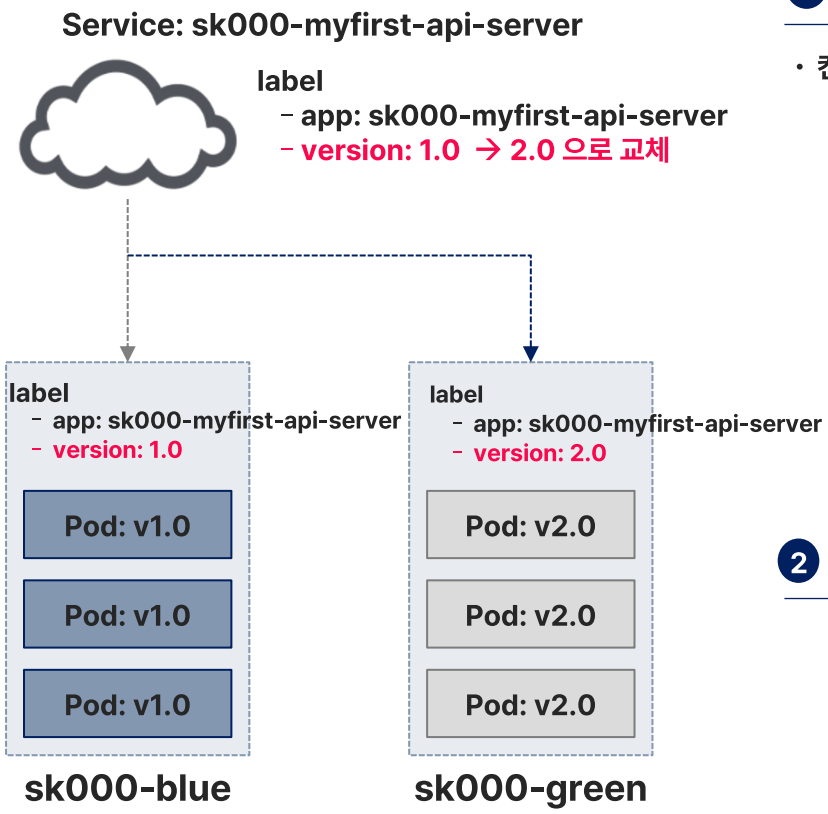
마이크로 서비스 수가 증가하게 됨에 따라 새로운 버전으로 지속적으로 업데이트하고 교체하는 것에 대한 부담이 커지고 있으며, 수동 개입으로 인한 작업자 오류를 최소화하면서 서비스 중단 없이 수시배포가 가능한 자동화 방식 필요



kubernetes in action book 내용 참고

Blue/Green 배포 : 서비스 기반

컨테이너는 자신의 데이터를 유지하거나 또는 Pod 내 컨테이너간 또는 Pod 간 데이터를 공유하기 위해 외부 디스크를 사용



1 현행 버전인 Blue Deploy와 신규 버전 Green Deploy의 배포

- 컨테이너 앱의 현행 버전인 Blue app와 신규 버전인 green app이 모두 배포되어 있어야 한다

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-blue
  namespace: class-0
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sk000-myfirst-api-server
      version: "1.0"
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-green
  namespace: class-0
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sk000-myfirst-api-server
      version: "2.0"
```

2 Service의 Label 변경을 통한 Traffic Routing 전환

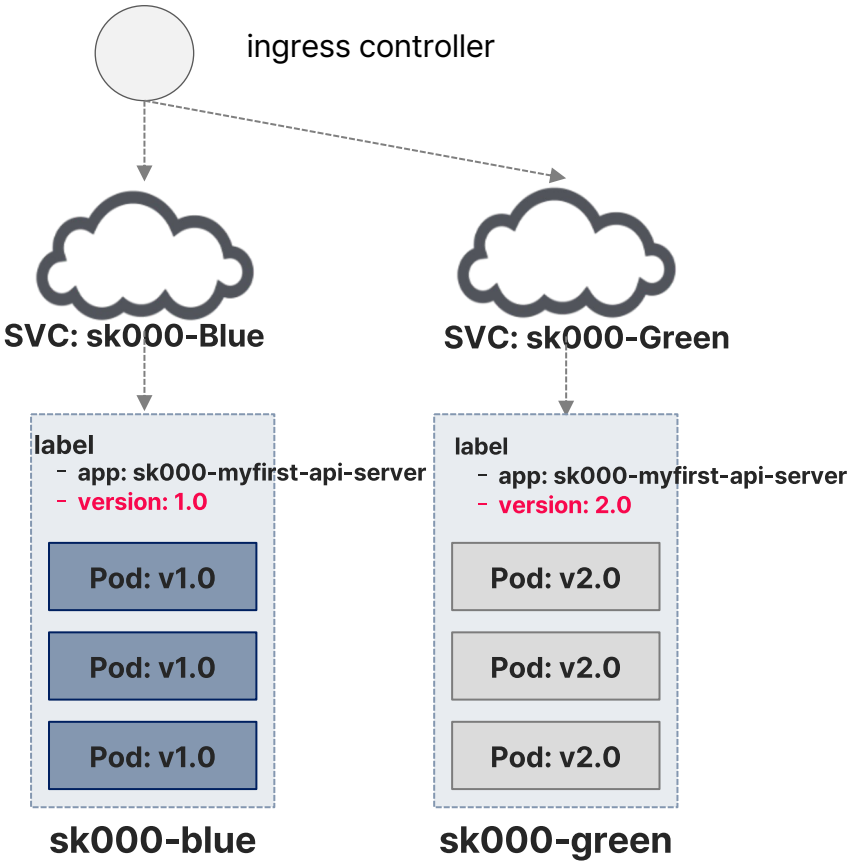
```yaml
apiVersion: v1
kind: Service
metadata:
  name: sk000-blue
  namespace: class-0
spec:
  selector:
    app: sk000-myfirst-api-server
    version: "1.0"
```

→ 2.0으로 교체 시 자동 라우팅 변경

kubernetes in action book 내용 참고

Blue/Green 배포: Ingress 기반

Ingress 기능을 통해 Blue/Green 지원



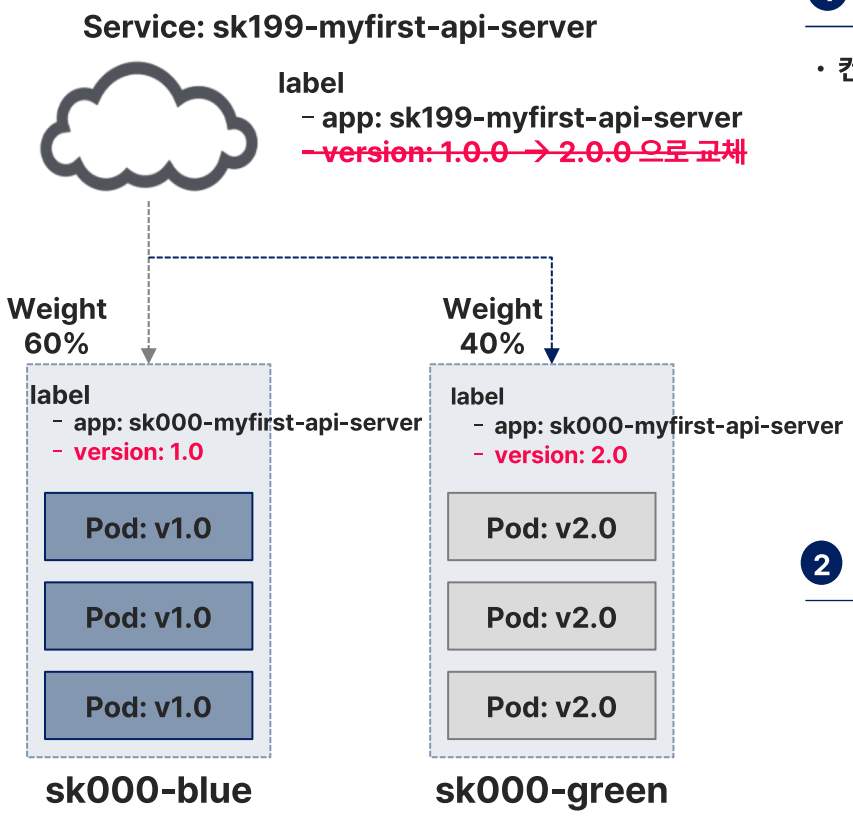

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: sk000-cert-ingress
  namespace: class-0
  labels:
    app: sk000-cert-ingress
spec:
  ingressClassName: nginx
  rules:
  - host: sk000-cert-ingress.class-0.skala-us.com
    http:
      paths:
      - backend:
          service:
            name: sk000-blue
            port:
              number: 8080
        path: /
        pathType: Prefix
```

sk000-green 으로 교체

kubernetes in action book 내용 참고

Carary 배포 : 서비스 기반

Kubernetes의 Service를 활용해서 Canary 배포를 지원 할 수 있음
참고: 별도의 API Gateway 또는 Istio를 통해 지원 가능



1 1.0 과 2.0을 함께 배포

- 컨테이너 앱의 현행 버전인 Blue app와 신규 버전인 green app이 모두 배포되어 있어야 한다

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-blue
  namespace: class-0
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sk000-myfirst-api-server
      version: "1.0"
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sk000-green
  namespace: class-0
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sk000-myfirst-api-server
      version: "2.0"
```

2 Service의 Label 변경을 통한 Traffic Routing 전환

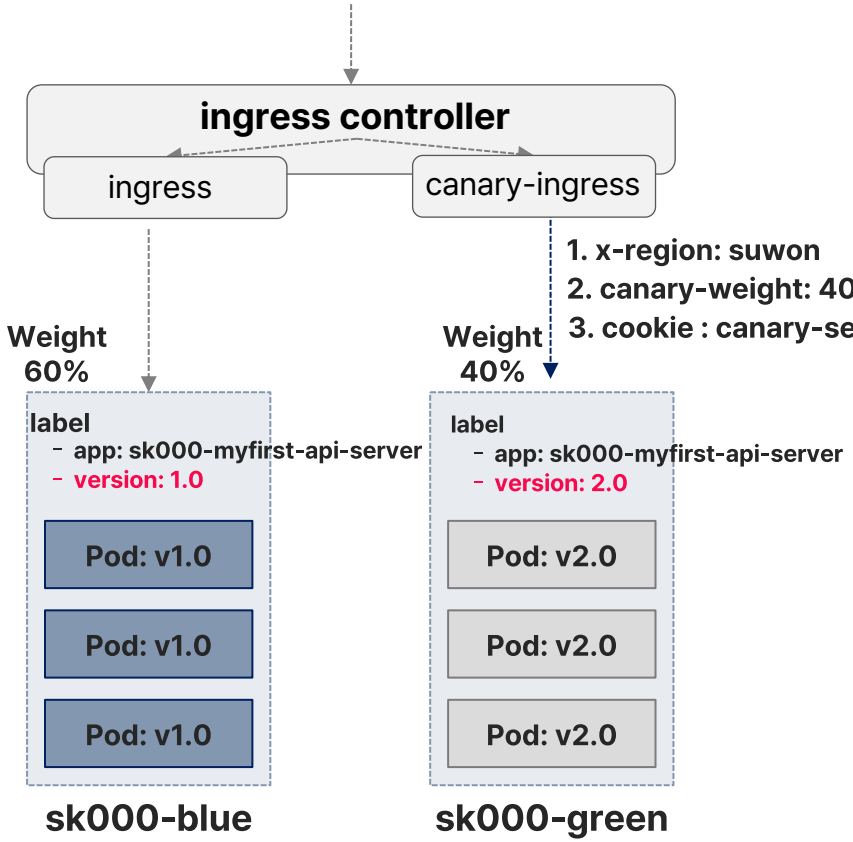
```yaml
apiVersion: v1
kind: Service
metadata:
  name: sk000-blue
  namespace: class-0
spec:
  selector:
    app: sk000-myfirst-api-server
    version: "1.0"
```

version 필드를 제거

kubernetes in action book 내용 참고

Carary 배포: Ingress 기반

kubernetes 의 nginx ingress controller을 이용해서 canray 배포를 지원하도록 구성할 수 있음



1 header 기반 canary

- -H "x-region: ap-suwon" 이 포함되어 있는 경우 라우팅

```
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-by-header: "x-region"
    nginx.ingress.kubernetes.io/canary-by-header-value: "ap-suwon"
```

2 cookie 기반 canary

- --cookie "canary-session=always" 이 포함되어 있는 경우 라우팅

```
annotations:
  cert-manager.io/cluster-issuer: letsencrypt-prod
  nginx.ingress.kubernetes.io/canary: "true"
  nginx.ingress.kubernetes.io/canary-by-cookie: "canary-session"
```

3 weight 기반 canary

- canary-ingress 생성 시 "canary-weight" 적용

```
annotations:
  cert-manager.io/cluster-issuer: letsencrypt-prod
  nginx.ingress.kubernetes.io/canary: "true"
  nginx.ingress.kubernetes.io/canary-weight: "40"
```

kubernetes in action book 내용 참고

[실습] Service 기반 Blue/Green (1/2)

위치: 15.bluegreen/01.bluegreen

사전 작업

자신이 배포한 자원을 삭제한다

Blue와 Green Pod 배포하기

```
> k apply -f  ./blue
> k apply -f  ./green

> k get pod -l app=sk000-myfirst-api-server
NAME                                        READY   STATUS    RESTARTS   AGE
sk000-blue-75686bb9bc-dj6lk    1/1     Running        0          50s
sk000-green-599fd5b8f9-fdnj8   1/1     Running       0          10s

> k apply -f ingress.yaml
> k get ingress sk000-cert-ingress -w
```

service 기반 Blue/Green 접속하기

```
> kubectl apply -f service.yaml
https://sk000-cert-ingress.class-0.skala-us.com/ 접속 해서 1.0 동작 여부 확인

> kubectl apply -f change-green-service.yaml
https://sk199-tls.skala25a.project.skala-ai.com/api/users 접속 해서 2.0 동작 여부 확인
```

kubernetes in action book 내용 참고

[실습] Service 기반 Blue/Green (1/2)

위치: 15.bluegreen/01.bluegreen

blue-service.yaml 적용 시

change-green-service.yaml 적용 시

서비스를 교체 할때 마다 Pod 대상이 변경되는 이유는 무엇인가?

blue-service.yaml과 change-green-service.yaml의 구조를 이해해보자

kubernetes in action book 내용 참고

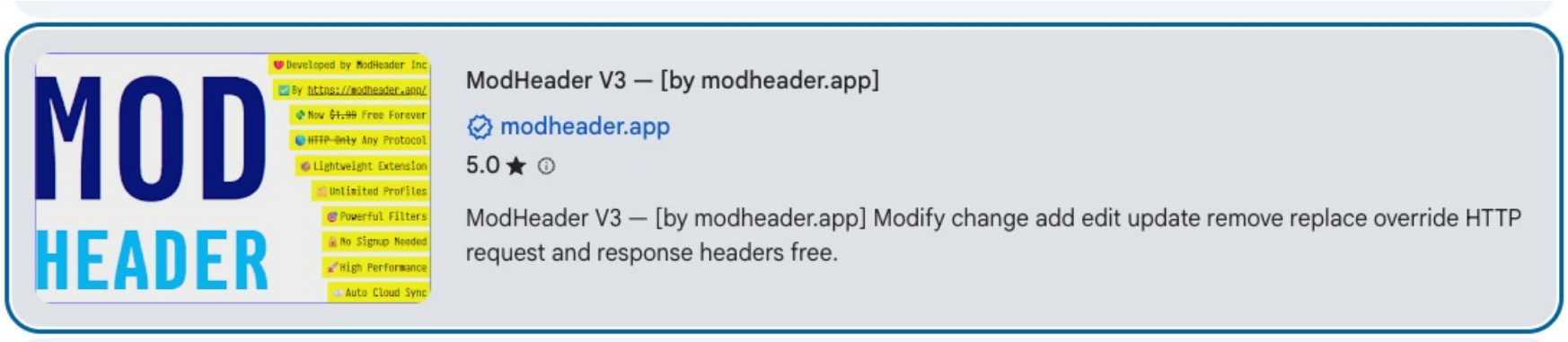
[실습] ModHeader Extension 설치

위치: 15.bluegreen/02.ingress-canary

크롬 확장 프로그램을 사용한 HTTP Header 설정

https://chromewebstore.google.com/

ModHeader 확장 프로그램
1. Chrome 웹 스토어에서 “ModHeader V3” 설치
2. 확장 프로그램 아이콘 클릭



[실습] Ingress 활용 canary

위치: 15.bluegreen/02.ingress-canary

Ingress를 활용한 canaray 적용해보기

환경 구성하기

1. blue Ingress 배포
   >kubectl apply –f blue-ingress.yaml
2. https://sk000-cert-ingress.class-0.skala-??.com 접속 해서 1.0 spring boot 접속 확인’

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: sk000-cert-ingress
  namespace: class-0
  labels:
    app: sk000-cert-ingress
spec:
  ingressClassName: nginx
  rules:
  - host: sk000-cert-ingress.class-0.skala-us.com
    http:
      paths:
      - backend:
          service:
            name: sk000-blue
            port:
              number: 8080
        path: /
        pathType: Prefix
  tls:
  - hosts:
    - 'sk000-cert-ingress.class-0.skala-us.com'
    secretName: wildcard-class-0-skala-us-com-tls
```

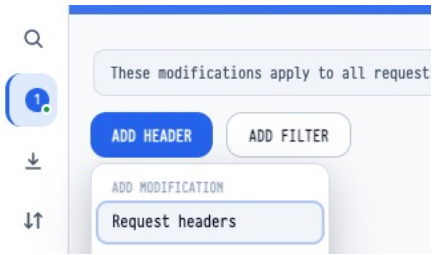
[실습] Ingress 활용 canary: HTTP Header 기반 (1/2)

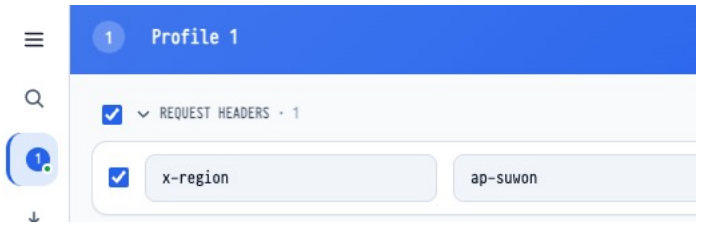
canary-header-ingress.yaml 확인 및 적용하기

```
> cat canary-header-ingress.yaml
> kubectl apply   -f canary-header-ingress.yaml
```

```yaml
metadata:
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-by-header: "x-region"
    nginx.ingress.kubernetes.io/canary-by-header-value: "ap-suwon"
```

1. ModHeader 에서 “Mode” 선택후 “Request Header” 선택
“x-region: ap-suwon” 적용





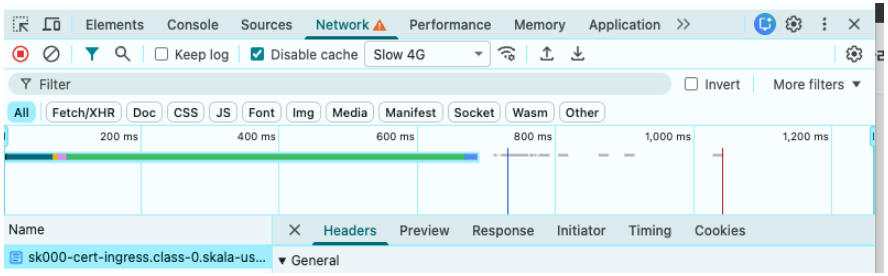
[실습] Ingress 활용 canary: HTTP Header 기반 (2/2)

2. Chrome Browser 접속

https://sk000-cert-ingress.class-0.skala-??.com/

개발자 도구 > 네트워크 내

users request 의 Header 정보에 x-Region ap-suwon 확인



3. 시크릿 모드 기반 Chrome Browser 접속

https://sk000-cert-ingress.class-0.skala-??.com/

[실습] Ingress 활용 canary: HTTP Cookie 기반 (1/2)

cookie 기반 canary 적용하기

: canary-cookie-ingresss.yaml

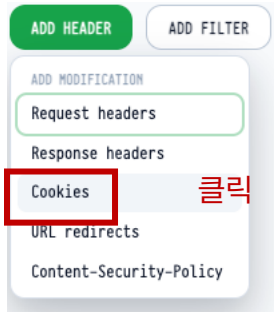
```yaml
annotations:
  cert-manager.io/cluster-issuer: letsencrypt-prod
  nginx.ingress.kubernetes.io/canary: "true"
  nginx.ingress.kubernetes.io/canary-by-cookie: "canary-session"
```

```
kubectl apply   -f canary-cookie-ingress.yaml
```

1. ModHeader 에서 Header 설정 제거

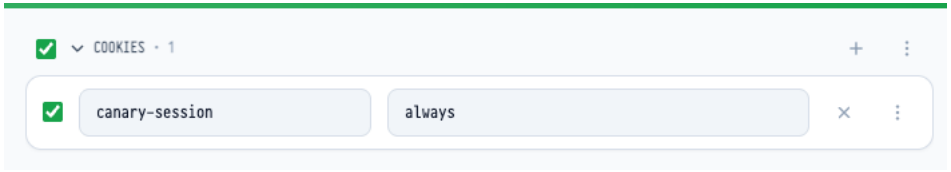
클릭

2.ModHeader의 Add Header 에서 Cookie 선택



클릭

3. Cookies에 “canary-session”, “always” 등록



[실습] Ingress 활용 canary: HTTP Cookie 기반 (2/2)

2. Chrome Browser 접속

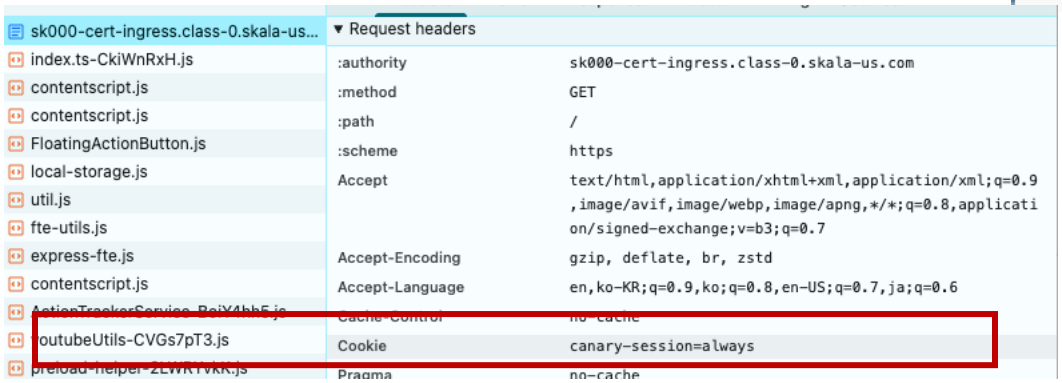
https://sk000-cert-ingress.class-0.skala-??.com/

3. 시크릿 모드 기반 Chrome Browser 접속

https://sk000-cert-ingress.class-0.skala-??.com/

개발자 도구 > 네트워크 내

users request 의 Header 정보에 Cookie canary-session=always 확인



[실습] Ingress 활용 canary: HTTP weight 기반

weight 기반 canary 적용하기

: canary-weight-ingresss.yaml

```
annotations:
  cert-manager.io/cluster-issuer: letsencrypt-prod
  nginx.ingress.kubernetes.io/canary: "true"
  nginx.ingress.kubernetes.io/canary-weight: "40"
```

```
kubectl apply   -f canary-header-ingress.yaml
```

1. Chrome Browser 접속

https://sk000-cert-ingress.class-0.skala-??.com/api/users

2. canary-weight의 설정 값이 40 (40%) 설정되어 있어서 6:4 로 라우팅 실행 되는지 확인

blue pod (1.0)


```
[
  {
    "id": 1,
    "name": "홍길동",
    "email": "hong@sk.com"
  },
  {
    "id": 2,
    "name": "김철수",
    "email": "kim@sk.com"
  },
  {
    "id": 3,
    "name": "이영희",
    "email": "lee@sk.com"
  }
]
```


green pod (2.0)


```
[
  {
    "id": 1,
    "name": "alice",
    "email": "alice@green.com"
  },
  {
    "id": 2,
    "name": "bob",
    "email": "bob@green.com"
  },
  {
    "id": 3,
    "name": "charlie",
    "email": "charlie@green.com"
  }
]
```

KDT Cloud – 5. 쿠버네티스 심화하기

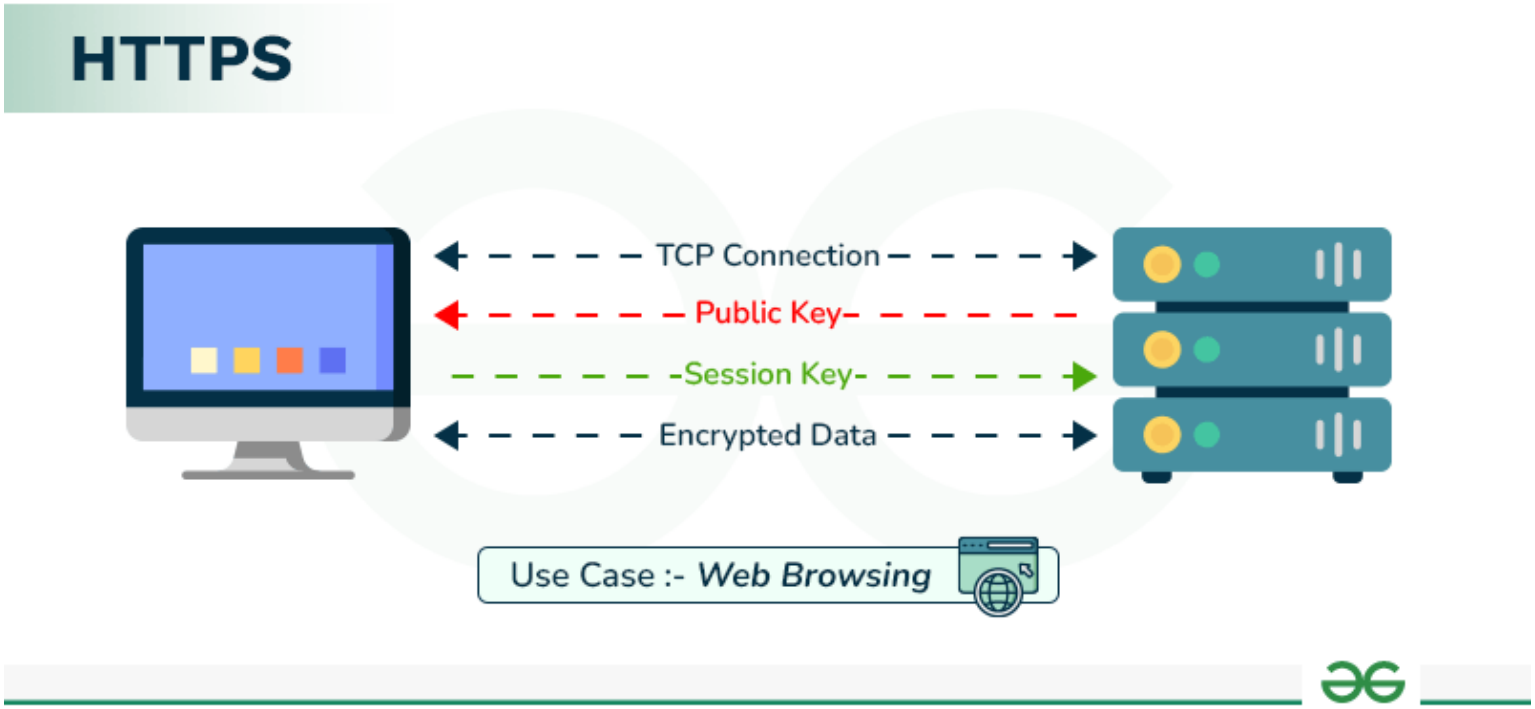
TLS 보안 메커니즘 이해

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

HTTPS (HyperText Transfer Procotol Secure)

웹 브라우저와 웹사이트 간에 데이터를 전송하는 데 가장 일반적으로 사용되는 프로토콜
HTTPS는 사용자의 브라우저와 웹사이트 간의 통신에 사용되며 데이터 전송을 암호화하여 보안을 강화



- HTTPS는 기존 HTTP 프로토콜에 **SSL/TLS** 보안 계층을 추가한 것
- HTTP와 HTTPS의 워크플로는 동일 하며, 브라우저와 서버는 여전히 HTTP 프로토콜을 사용하여 서로 통신
- 하지만 이 과정은 안전한 SSL 연결을 통해 이루어짐
- SSL 연결은 **데이터 안전을 보장하기 위해 교환되는 데이터의 암호화 및 복호화**를 담당

kubernetes in action book 내용 참고

HTTPS 프로토콜을 이해하기 위한 암호화 기술

- HTTPS를 이해하기 위해 필요한 암호화 요소는 다음과 같다

	사용 기술	설명	
TLS 연결용	**Public/Private Key**	공개키로 암호화하고 개인키로 복호화	tls.crt, tls.key
	Certificate / CA	이 Public Key가 누구의 것인지 CA가 보증한다	ca.crt
키 인증용	**Hash**	원문에서 만든 요약값, 즉 데이터의 지문	
	전자서명	Private Key로 서명하고 Public Key로 검증한다	
데이터 암호화	**대칭키**	같은 키로 암호화하고 복호화	session.key

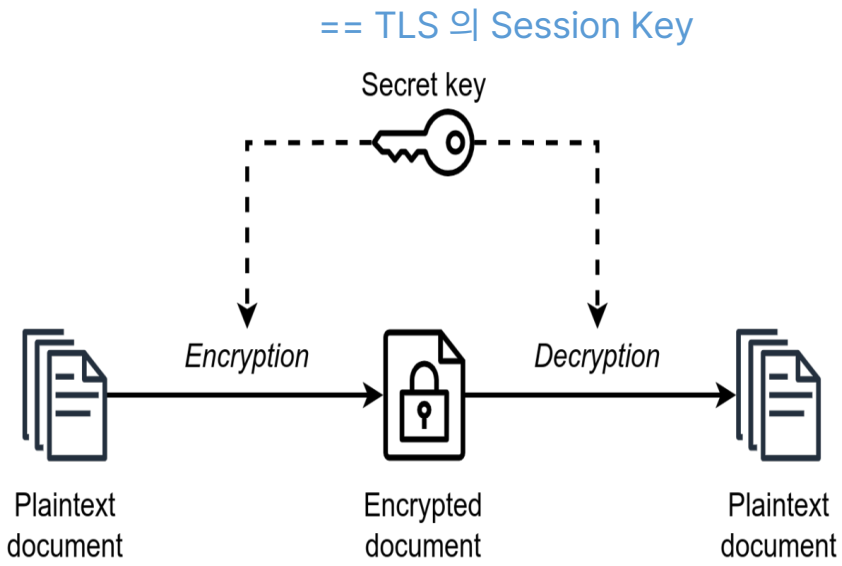
HTTPS Ingress : tls.crt, tls.key
Client가 사전에 알아야 할 Key: ca.crt

kubernetes in action book 내용 참고

대칭키 암호화 (symmetric key encryption)

위치: 14.ingress/secret-key/symmetric

- 대칭키 암호화란 **암호화(encryption)와 복호화(decryption)에 동일한 키(key)를 사용하는 암호화 방식**
- **보내는 쪽과 받는 쪽이 같은 비밀키(secret key)를 공유**해야만 데이터를 안전하게 주고받을 수 있**음**
  - ❖ 단일 키 사용 → 암호화와 복호화에 같은 키를 씀
  - ❖ 빠른 속도 → 공개키 암호화(비대칭키 방식)보다 계산이 간단해서 성능이 뛰어남
  - ❖ 키 관리 문제 → 키가 유출되면 누구든지 데이터를 복호화할 수 있음
  - ❖ 대표 알고리즘: AES(Advanced Encryption Standard) 등



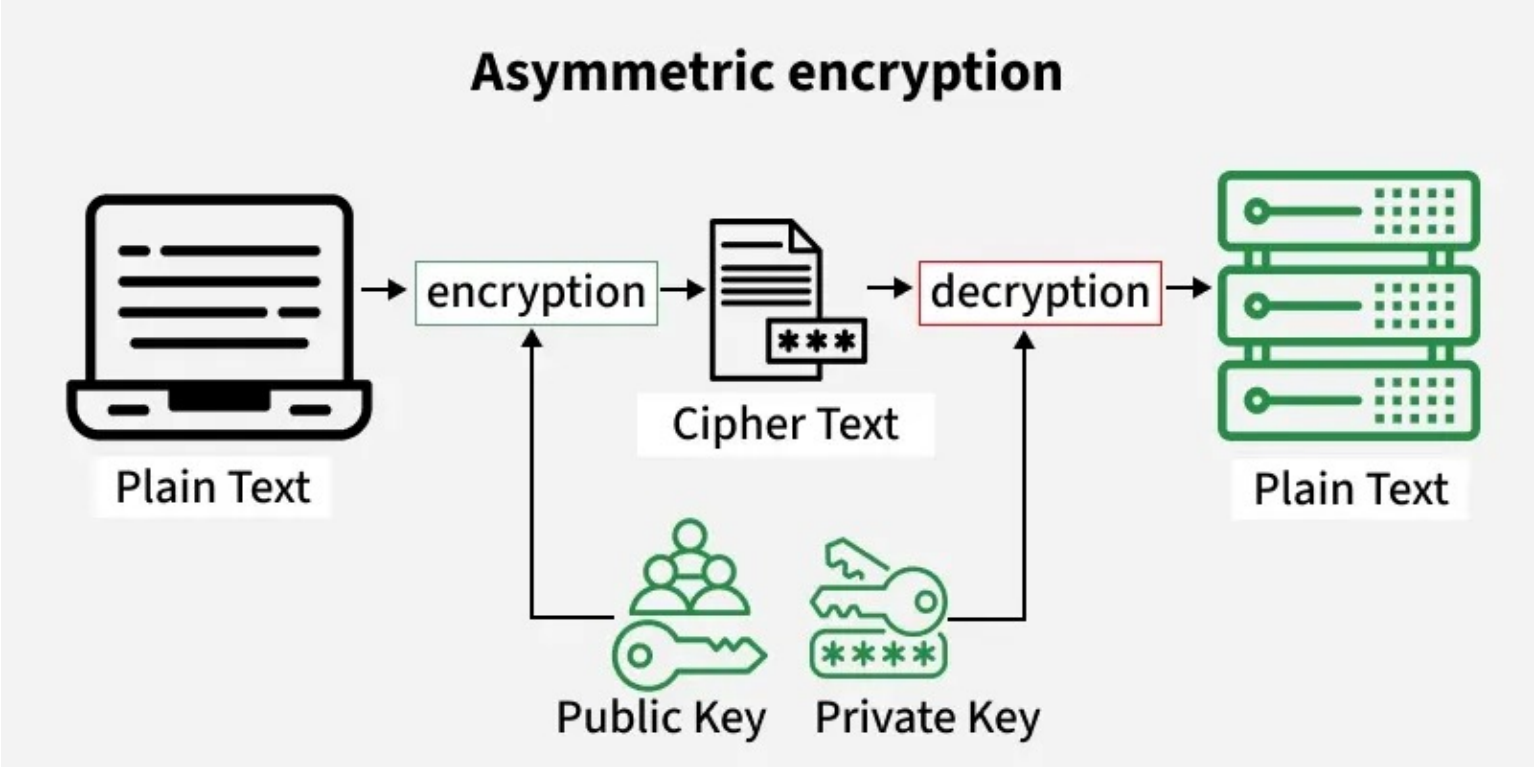

```
# Key 생성
openssl rand -base64 32 > secret.key

# Encryption
openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 \
  -pass file:secret.key \
  -in plain.txt \
  -out secret.enc

# Decryption
openssl enc -aes-256-cbc -d -pbkdf2 -iter 100000 \
  -pass file:secret.key \
  -in secret.enc \
  -out decrypted.txt
```

비 대칭키 암호화

위치: 14.ingress/secret-key/asymmetric

- **공개키(Public Key)와 개인키(Private Key)라는 서로 다른 두 개의 키**를 사용하는 암호화 방식
- 소유자가 개인키와 공개키를 생성해서 공개키를 모두에게 공유

❖ 대칭키와 달리 키 교환 문제를 해결할 수 있음 (비밀키를 직접 주고받지 않아도 됨)
❖ 계산량이 많아 속도가 느림 → 대용량 데이터 암호화에는 비효율적
❖ 실제 시스템에서는 보통실제 시스템에서 처리하는 방식
- 비대칭키로 세션 키(Session Key, 대칭키)를 안전하게 교환
- 이후 데이터 암호화는 대칭키로 수행 (하이브리드 암호화)



Hash Code SHA-256

위치: 14.ingress/secret-key/sha256.sh

목적: 데이터를 고정 길이의 해시값으로 요약하여 **무결성 확인, 동일성 비교** 등에 사용하는 해시 함수

방법: 임의의 데이터(문자열, 파일, 객체 등)를 고정된 길이의 짧은 값으로 바꾼 것

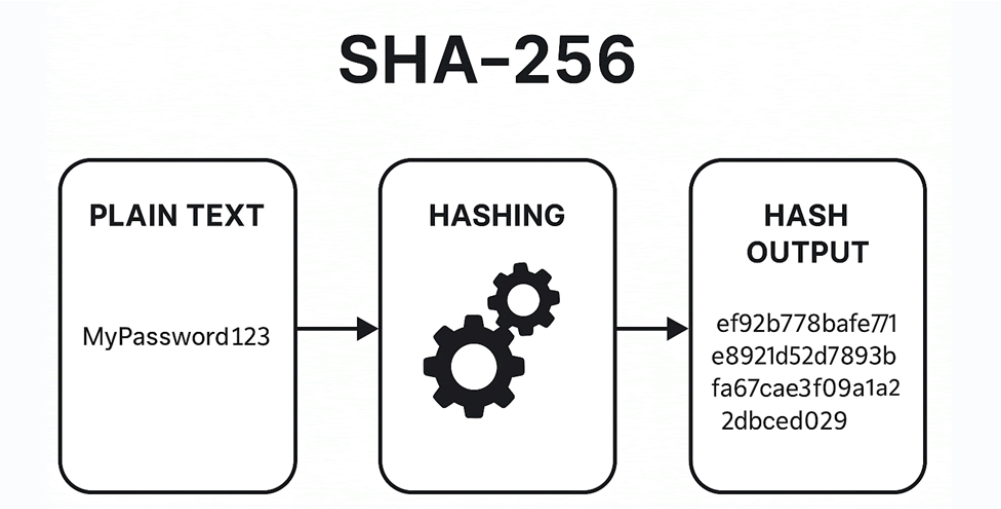
SHA-256 : 텍스트, 파일, 사진, 영상등의 데이터를 넣으면 항상 똑같은 **64자리 16진수 문자열(256비트)을 만드는 알고리즘**

Hash 의 특징

- Hash를 원본으로 다시 돌릴 수 없음
- 원문 복원 불가

사용처

- **파일 무결성 검증**
  다운로드한 파일이 원본과 같은지 확인
- **메시지/데이터 변경 여부 확인**
  전송 중 변조 여부 확인
- **전자서명 전 해시 생성**
  원문 전체 대신 해시값에 서명



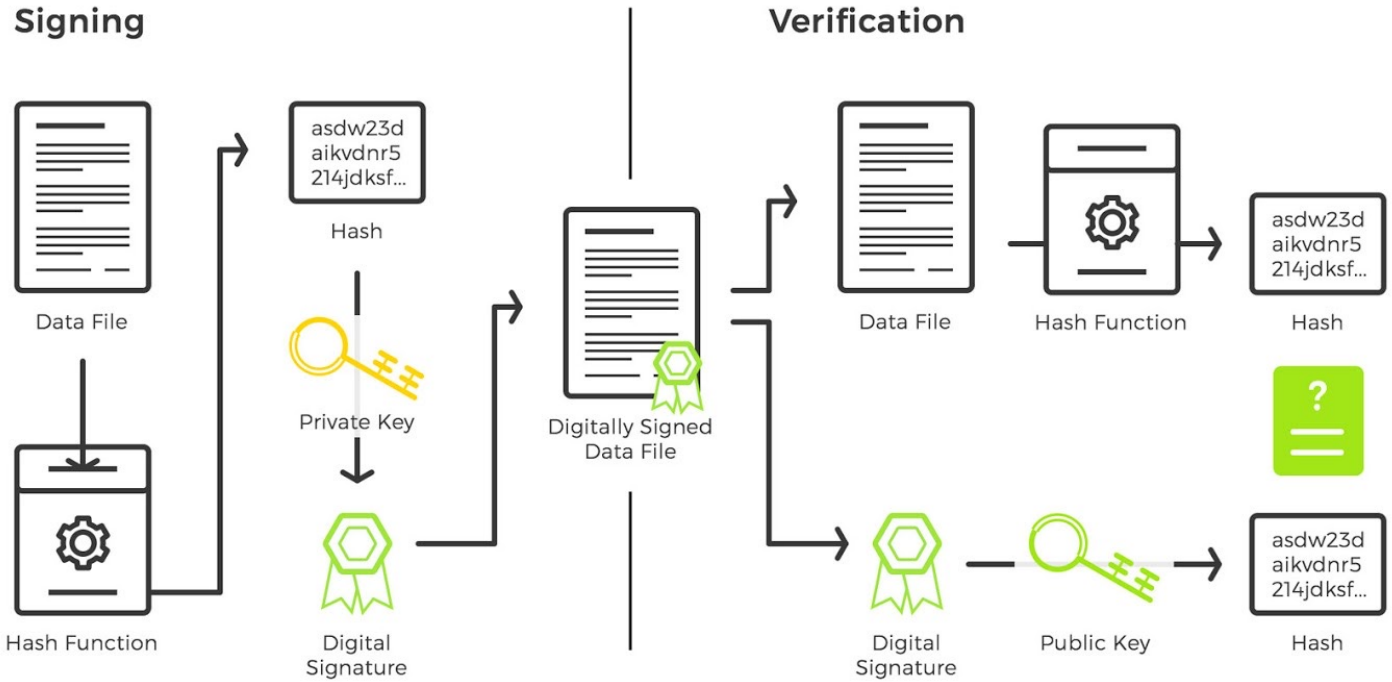

```
> echo "hello" | sha256
5891b5b522d5df086d0ff0b110fbd9d21bb4fc7163af34d08286a2e846f6be03

> cat deploy.yaml | sha256
242cda8de2a5a9f06d5d460a19d6f7c1155518c1bfd325862c2b19532f9cefb0
```

kubernetes in action book 내용 참고

전자서명 (Digital Signature)

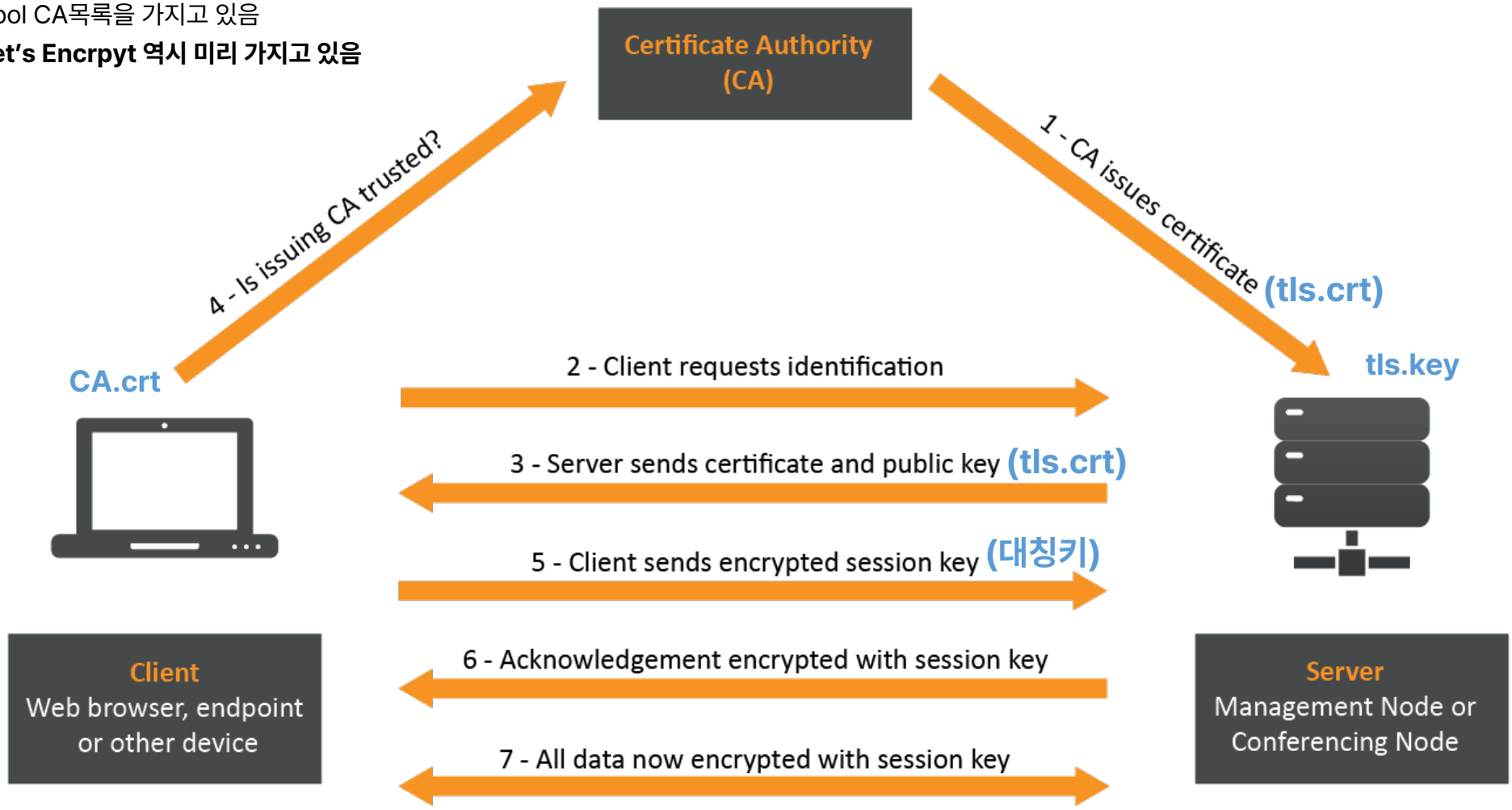
- 전자문서가 특정 사람에 의해 작성·승인되었고, 이후 내용이 변조되지 않았음을 증명하는 전자적 서명 방식
- **HashCode와 비대칭키 암호화**를 활용한 일종의 증명 메커니즘

보내는쪽 : Plain Text + **전자 서명 (Plain Text → Hashcode → Private Key 암호화)** 전달
받는 쪽 : Plain Text의 Hash code, Public Key로 복호화된 Hashcode 비교 하여 원본 확인



도메인 신원 확인: CA Process

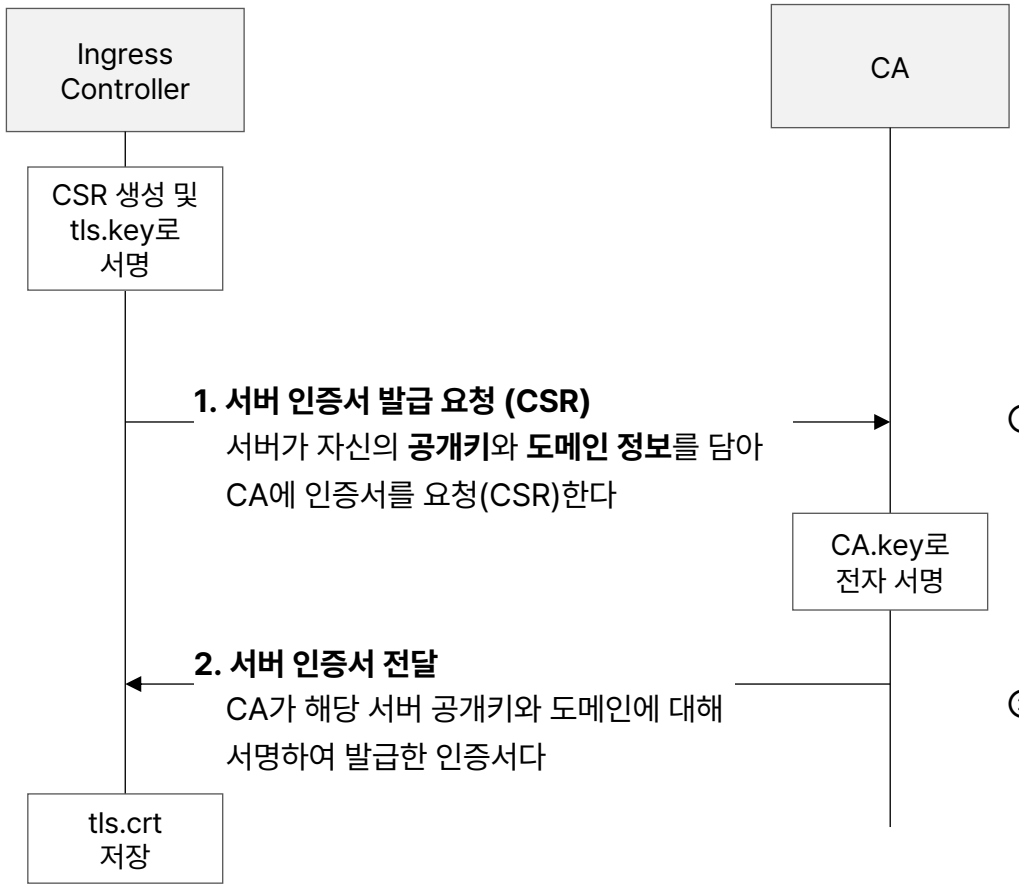
CA.crt는 브라우저가 설치될때 미리 신뢰할 수 있는
Rool CA목록을 가지고 있음
Let's Encrpyt 역시 미리 가지고 있음



kubernetes in action book 내용 참고

도메인 신원 확인: 서버 인증서 발급 흐름

클라이언트 (브라우저)가 서버가 제공하는 서버 인증서가 신뢰할 수 있는지를 CA 전자서명 확인을 통해 검증




```
Certificate (서버 인증서, tls.crt – X.509)
 ├─ tbsCertificate (To Be Signed)
 │    ├─ Subject (도메인 정보)
 │    ├─ Subject Public Key Info  ← 서버 공개키
 │    ├─ Validity (유효 기간)
 │    ├─ Extensions (SAN, Key Usage 등)
 │
 ├─ Signature Algorithm
 └─ Signature Value  ← CA.key(개인키)로 서명한 값
```

tbsCertificate는 CA가 서명하기 "직전"의 인증서 본문 데이터 전체

① 서버가 **CSR (Certificate Signing Request)** 생성 및 제출
- 서버 공개 키 포함 (public key)
- 도메인 정보 포함

② CA는 서버 공개키를 포함한 인증서 내용 전체에 대해
- **CA 개인키(CA.key)**로 전자 서명을 수행
- 서명 후 서버로 **서버 인증서(tls.crt)** 전달

kubernetes in action book 내용 참고

[참고] 인증서 (Certificate)

공개되는 공개 키 (public key)가 너가 접속한 서버에서 제공하는 것이 맞다고 전자 서명해서 보증한 전자 서명서
→ 이 인증키는 공인 인증 기관(CA: Certificated Authority)에서 인증한 것이다라고 표현

인증서에 담겨져 있는 내용

- **공개키** (public Key)
- **소유자 정보** (예: 도메인명, sk199-ingress.skala25a.project.skala-ai.com)
- 발급자 정보(CA)
- 유효기간 (언제까지 유효)
- **전자서명** (digital signature)

```
Certificate (서버 인증서, tls.crt – X.509)
 ├─ tbsCertificate (To Be Signed)
 │   ├─ Subject (도메인 정보)
 │   ├─ Subject Public Key Info  ← 서버 공개키
 │   ├─ Validity (유효 기간)
 │   ├─ Extensions (SAN, Key Usage 등)
 │
 ├─ Signature Algorithm
 └─ Signature Value  ← CA.key(개인키)로 서명한 값
```

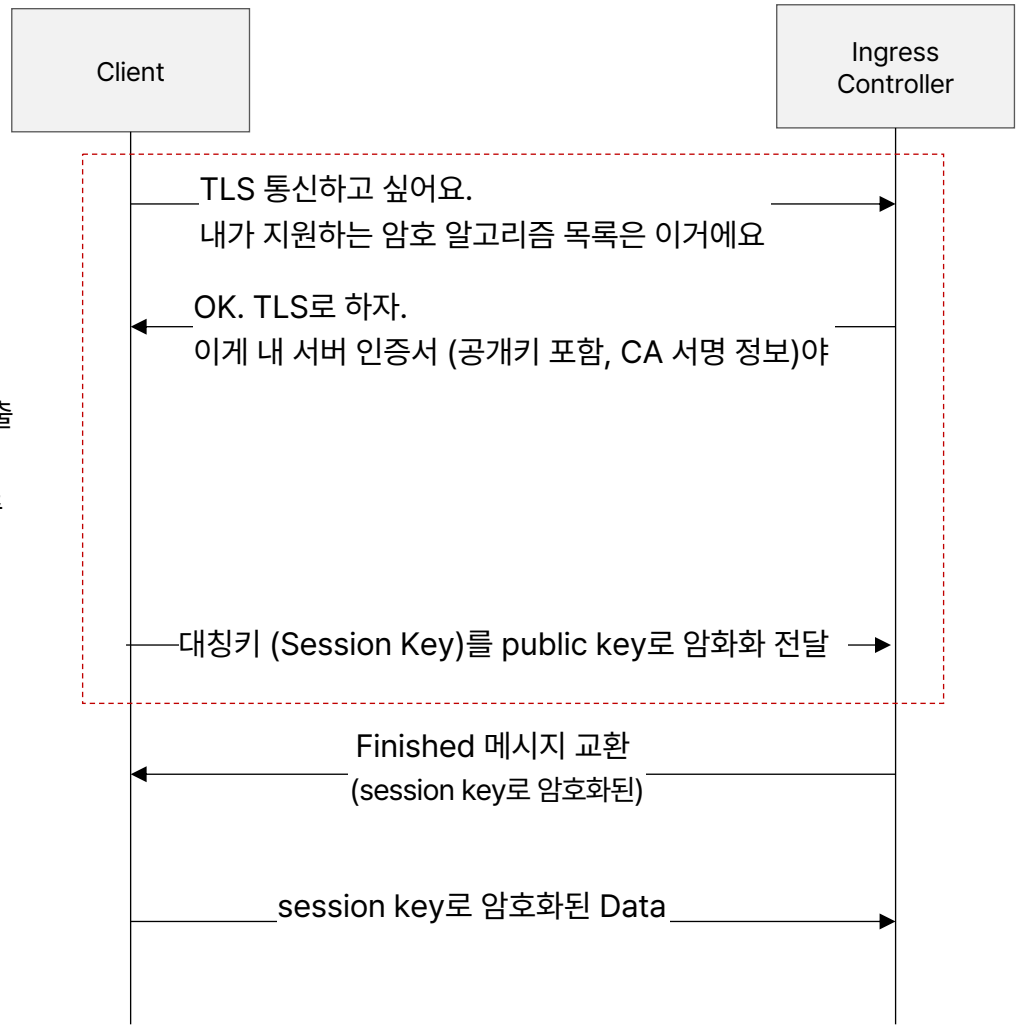
tbsCertificate는 CA가 서명하기 "직전"의 인증서 본문 데이터 전체

kubernetes in action book 내용 참고

키 공유 및 데이터 암호화: HTTPS TLS 흐름

TLS (Transport Layer Security)는 인터넷 상에서 데이터를 안전하게 암호화하여 전송하기 위한 보안 프로토콜로 HTTPS 443 포트 노출을 위해 반드시 TLS 인증 처리 필요

TLS 버전: TLSv1.2, TLSv1.3

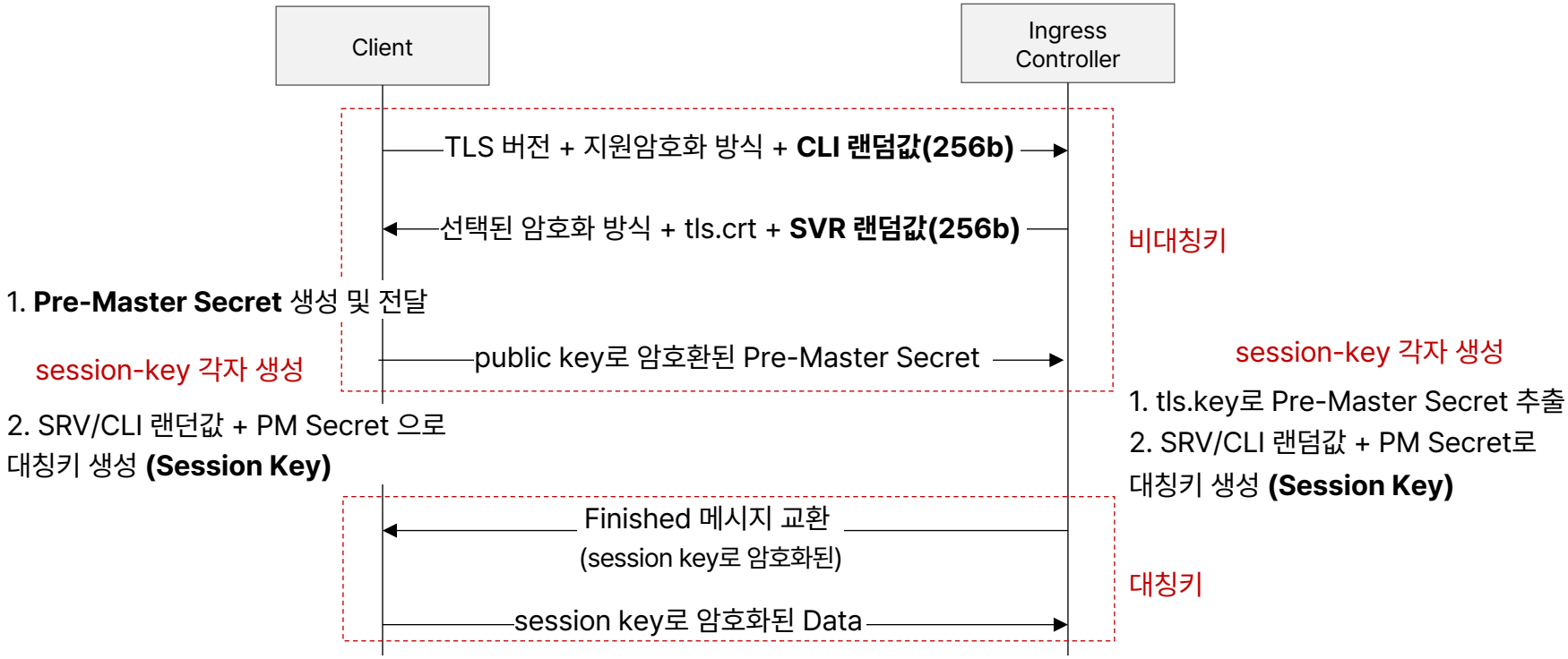


Client의 Certificate 검증 과정

① 서버 인증서에서 tbsCertificate 추출
② 동일한 해시 알고리즘으로 해시 계산
③ CA 공개키로 Signature Value 검증
④ 값이 일치하면:
이 내용은 CA가 서명한 게 맞다

kubernetes in action book 내용 참고

키 공유 및 데이터 암호화 : TLS v1.2 버전 상세 흐름

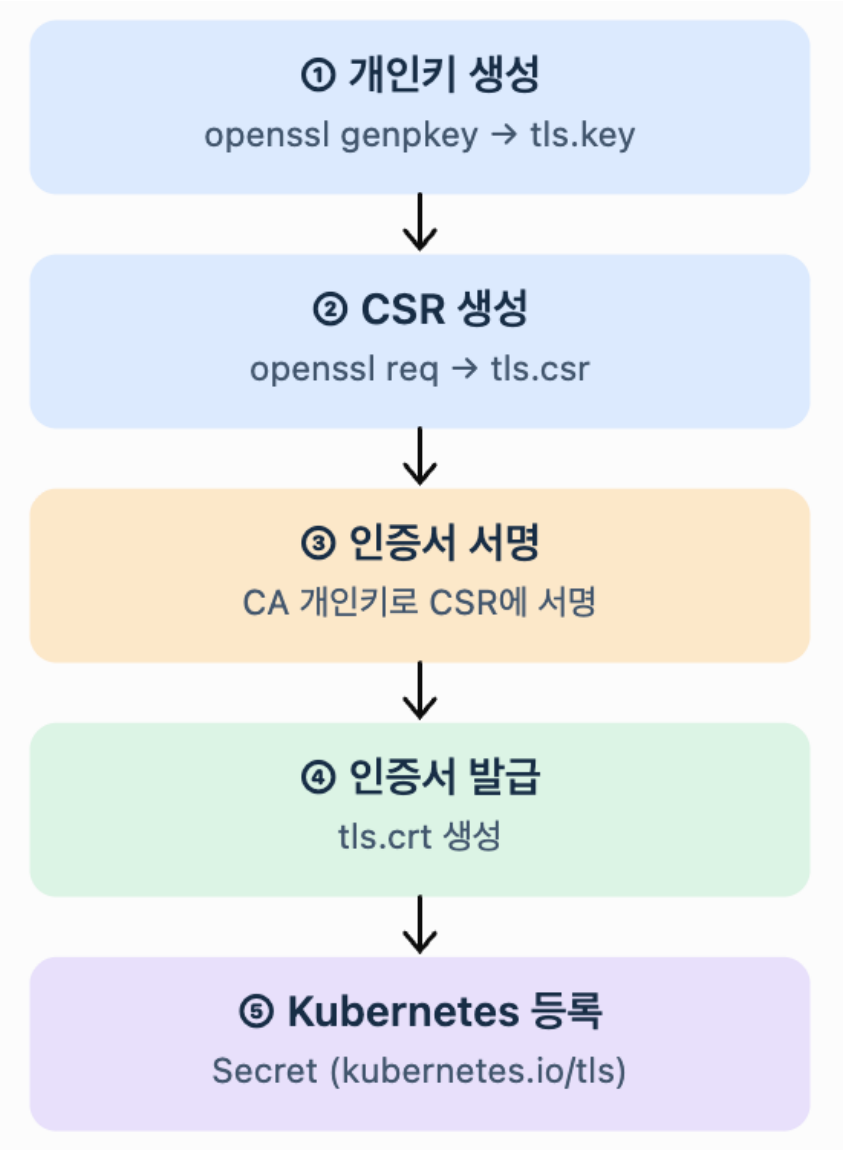


개념	설명
Client/Server Random	Session Key 만들기 위한 재료 (seed)
Pre-Master Secret	Session Key 만들기 위한 비밀 값
Session Key	클라이언트와 서버가 **동일하게 계산**하여 사용하는 대칭키
암호화/복호화 방식	모두 동일한 Session Key 사용, 랜덤값으로 직접 암호화하진 않음

kubernetes in action book 내용 참고

TLS 인증서 발급 과정

- tls.crt 발급을 위한 과정



kubernetes in action book 내용 참고

[실습] TLS 인증 키 발급 및 Secret 생성 (1/2)

위치: 05.ingress/03.tls/01.gen-cert.sh

CA (Certificate Authority) 를 거치지 않고 자기 자신이 서명한 (Self-signed) 서버 인증서를 발급

tls.crt(서버 인증서) 와 tls.key(개인키)를 생성해서 Ingress Controller 등록해보자

1. 공개 키 생성 (01.gen-cert.sh 참고)

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -subj "/CN=${STUDENT_NUM}-ingress.${CLASS_NAME}.skala-${LOCATION}.com/O=com.sk.skala" \
  -keyout tls.key \
  -out tls.crt
```

파라미터	설명
`req`	인증서 서명 요청(Certificate Signing Request, CSR)을 생성하거나 자체 서명 인증서를 만들기 위한 명령입니다.
`-x509`	CSR이 아니라 바로 X.509 형식의 자체 서명된 인증서를 생성하겠다는 의미입니다.
`-nodes`	프라이빗 키를 생성할 때 암호(passphrase)를 걸지 않겠다는 의미입니다. 서버 자동 구동을 위해 자주 사용됩니다.
`-days 365`	생성된 인증서의 유효기간을 365일로 지정합니다.
`-newkey rsa:2048`	새로 2048비트 RSA 키 쌍을 생성하겠다는 뜻입니다.
`-keyout tls.key`	생성된 개인 키를 `tls.key` 파일로 저장합니다.
`-out tls.crt`	생성된 인증서를 `tls.crt` 파일로 저장합니다.
`-subj "/CN=yourdomain.com/O=yourorg"`	인증서에 들어갈 주체 정보(subject)를 지정합니다. - `CN` : Common Name (예: 도메인 이름) - `O` : Organization (예: 조직 이름)

[실습] TLS 인증 키 발급 및 Secret 생성 (2/2)

위치: 05.ingress/03.tls/02.create-tls-secret.sh

CA (Certificate Authority) 를 거치지 않고 **자기 자신이 서명한 (Self-signed)** 서버 인증서를 발급

tls.crt(서버 인증서) 와 tls.key(개인키)를 생성해서 Ingress Controller 등록해보자

2. 생성된 인증서 확인

```
openssl x509 -in tls.crt -pubkey -noout
```

3. TLS Secret 생성

```
# 1. TLS 시크릿을 k8s에 생성합니다.
kubectl create secret tls ${STUDENT_NUM}-manual-tls-secret \
        --key tls.key \
        --cert tls.crt \
        -n ${CLASS_NAME}
```

또는 02.create-tls-secret.sh 실행

4. TLS Secret 생성 확인

```
# 2. 생성된 TLS 시크릿을 확인합니다.
kubectl get secret ${STUDENT_NUM}-manual-tls-secret -o yaml -n ${CLASS_NAME}
```

kubernetes in action book 내용 참고

TLS Ingress 등록

위치: 05.ingress/03.tls/ingress-tls.yaml

tls.crt (공개 키) 와 tls.key (개인키)를 생성해서 Ingress Controller에 등록

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: skala-manual-tls-ingress
  namespace: class-0
spec:
  ingressClassName: nginx  ¦ public-nginx
  rules:
  - host: skala-manual-tls-ingress.class-0.skala-us.com
    http:
      paths:
      - backend:
          service:
            name: skala-myfirst-api-server
            port:
…
tls:
  - hosts:
    - skala-manual-tls-ingress.class-0.skala-us.com
    secretName: skala-manual-tls-secret
```

metadata:

- **name**: Ingress의 이름
- **namespace**: Ingress가 속한 네임스페이스
- **ingressClassName**: 대상 ingress controller 지정

TLS: HTTPS Secret 설정

- **hosts**: HTTPS로 접근할 도메인 목록
- **secretName**: TLS 인증서와 키가 저장된 k8s Secret

rules: 도메인별 요청 처리 규칙을 정의

- **hosts.host**: 요청 도메인
- **paths**:
  - **path**: /는 모든 경로
  - **pathType**: Prefix | Extract | ImplementationSpecific
  - **backend**: 실제 트래픽을 받을 Kubernetes 서비스 정의
  - **name**: 서비스 이름 (sk000-my-app)
  - **port**: 해당 서비스의 포트 번호 (8080)

kubernetes in action book 내용 참고

[실습] HTTPS 기반 Ingress

위치: 14.ingress/03.tls

TLS Secret 을 수동으로 생성한후 이것을 이용해서 Ingress를 배포해보자

```
> kubectl get pod -l app=${STUDENT_NUM}-myfirst-api-server
> kubectl get service ¦ grep ${STUDENT_NUM}

ingress.yaml 배포하기
> k apply -f ingress-tls.yaml
> k get ingress ${STUDENT_NUM}-manual-tls-ingress -w
```

ADDRESS 가 할당되어야 함 (AWS ELB의 CNAME)

kubernetes in action book 내용 참고

[실습] HTTPS 기반 Ingress 접속하기

위치: 14.ingress/03.tls

https://sk000-manual-tls-ingress.class-0.skala-??.com/

kubernetes in action book 내용 참고

일회성 배치 작업 관리하기

Job과 CronJob

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Job

Job은 하나 이상의 Pod를 실행해서 **정해진 작업을 끝까지 완료**하도록 보장하는 Kubernetes 리소스

목적

- 한 번 실행 → 성공/실패로 종료
- 항상 떠있어야 하는 서비스가 아닌 일회성 작업
- 예시
  - ✓ DB 마이그레이션
  - ✓ 배치 계산
  - ✓ 파일 변환
  - ✓ 초기 데이터 적재
  - ✓ 백업/복구

필드	설명
completions	성공해야하는 Pod 실행 횟수 default=1
parallelism	동시 최대 실행 Pod 수 default=1
backoffLimit	Pod 실패 시 재시도 횟수
restartPolicy	보통 OnFailure Container 실패 시 재 시작

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: sk199-current-time
  namespace: skala-practice
spec:
  backoffLimit: 3
  completions: 3
  parallelism: 2
  template:
    spec:
      containers:
      - name: print-time
        image: busybox
        command:
          - sh
          - -c
          - 'echo "Current Time: $(date +%Y-%m-%d_%H:%M:%S)"'
      restartPolicy: Never
```

onFailure: 동일 Pod 내 컨테이너만 다시 실행 (동일 노드에서), Naver: Pod는 종료시키고, 새로운 Pod로 다시 실행

[실습] Job 실행해보기

위치: cloud/k8s-ddive/17.cronjob

동시에 두개의 Pod를 실행하고 Pod를 3개까지 실행하는 Job을 실행해보자

Job 실행하기

```
> kubectl apply –f job.yaml

> kubectl get job
NAME                 STATUS    COMPLETIONS   DURATION   AGE
sk199-current-time   Running   1/3           3s         3s
```

Pod 확인하기

```
> k get pod -l job-name=sk199-job
NAME              READY   STATUS      RESTARTS   AGE
sk199-job-6545b   0/1     Completed   0          2m14s
sk199-job-p9kcn   0/1     Completed   0          2m14s
sk199-job-zr92k   0/1     Completed   0          2m100s

> k logs -f sk199-job-6545b
```

삭제하기

```
> kubectl delete -f job.yaml
```

kubernetes in action book 내용 참고

CronJob

CronJob은 **Job을 정해진 시간 간격(스케줄)**으로 자동 생성하는 리소스

- 내부적으로는 CronJob → Job → Pod
- Linux crontab과 개념 동일

CronJob의 핵심 특징

주기적 실행

- 분 / 시 / 일 / 월 / 요일 단위 스케줄

실행 시점마다 Job 생성

- 실행 기록이 Job으로 남음

실패 정책 제어 가능

- 이전 Job이 아직 실행 중일 때 처리 방식 설정 가능

```
┌──────────── 분 (0-59)
│ ┌────────── 시 (0-23)
│ │ ┌──────── 일 (1-31)
│ │ │ ┌────── 월 (1-12)
│ │ │ │ ┌──── 요일 (0-7, 0=일요일, 7=일요일)
│ │ │ │ │
* * * * *
```

표현식	의미
*/5 * * * *	5분 간격으로
0 * * * *	매 정시마다 (1시간 마다)
0 */6 * * *	6시간 간격으로 (0시, 6시, 12시, 18시)
0 0 * * *	매일 자정
0 9 * * 1	매주 월요일 오전 9시
0 9 * * 1-5	평일 (월-금) 오전 9시

/ 는 **step 값**, 즉 **간격**

CronJob

CronJob은 **Job을 정해진 시간 간격(스케줄)**으로 자동 생성하는 리소스

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: ${USER_NAME}-current-time
  namespace: ${NAMESPACE}
spec:
  schedule: "*/3 * * * *"          # 매 3분마다 실행
  timeZone: "Asia/Seoul"           # KST 기준 스케줄
  concurrencyPolicy: Forbid        # 이전 Job 실행 중이면 새 Job 스킵
  successfulJobsHistoryLimit: 2    # 성공한 Job 보관 개수
  failedJobsHistoryLimit: 1        # 실패한 Job 보관 개수
  startingDeadlineSeconds: 30      # 스케줄 놓쳤을 때 허용 대기시간(초)
  jobTemplate:
    spec:
      backoffLimit: 3
      completions: 2
      parallelism: 1
      template:
        spec:
          containers:
          - name: print-time
            image: busybox
            command:
```

클러스터 과부하, CronJob 컨트롤러 중단, 또는 concurrencyPolicy: Forbid로 인한 대기

- **concurrencyPolicy**

Allow	이전 Job이 있어도 새 Job 실행
Forbid	이전 Job 실행 중이면 새 Job 스킵
Replace	이전 Job을 종료하고 새 Job 실행

…

[실습] CronJob 실행해보기

위치: 18.cronjob

3분마다 Job을 실행해보자

CronJob 실행하기

```
> kubectl apply –f job.yaml

> kubectl get cronjob
NAME                 STATUS    COMPLETIONS   DURATION   AGE
sk199-current-time   Running   1/3           3s         3s
```

Job 확인하기

```
> k get job ¦ grep sk000-cronjob
NAME                       STATUS     COMPLETIONS   DURATION   AGE
sk199-cronjob-29553453     Complete   2/2           8s         3m14s
sk199-cronjob-29553456     Complete   2/2           8s         14s

> k get pod ¦ grep sk199-cronjob

> k logs –f sk199-cronjob-29553456
```

삭제하기

```
> kubectl delete -f job.yaml
```

kubernetes in action book 내용 참고

KDT Cloud – 5. 쿠버네티스 심화하기

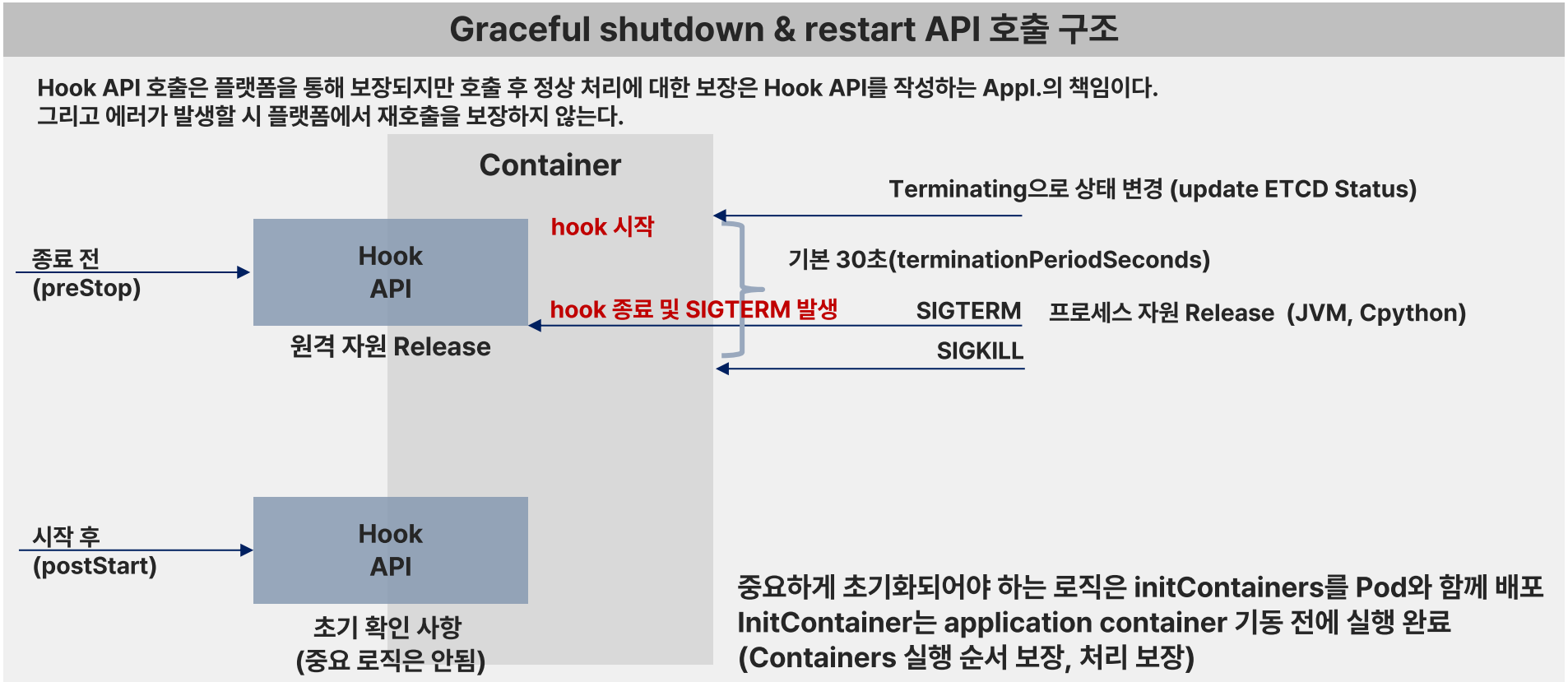
Graceful shutdown

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Graceful shutdown

Appl.의 실행 시 사전 작업을 실행할 수 있도록 하고 종료 시 연결된 자원 해지(세션, 연결 자원 해지 등) 등을 할 수 있도록 지원



spring boot은 @PreDestory 같은 hook을 지원

SIGINT, SIGTERM: Shutdown Hook 실행
SIGKILL: Shutdown Hook 미 실행

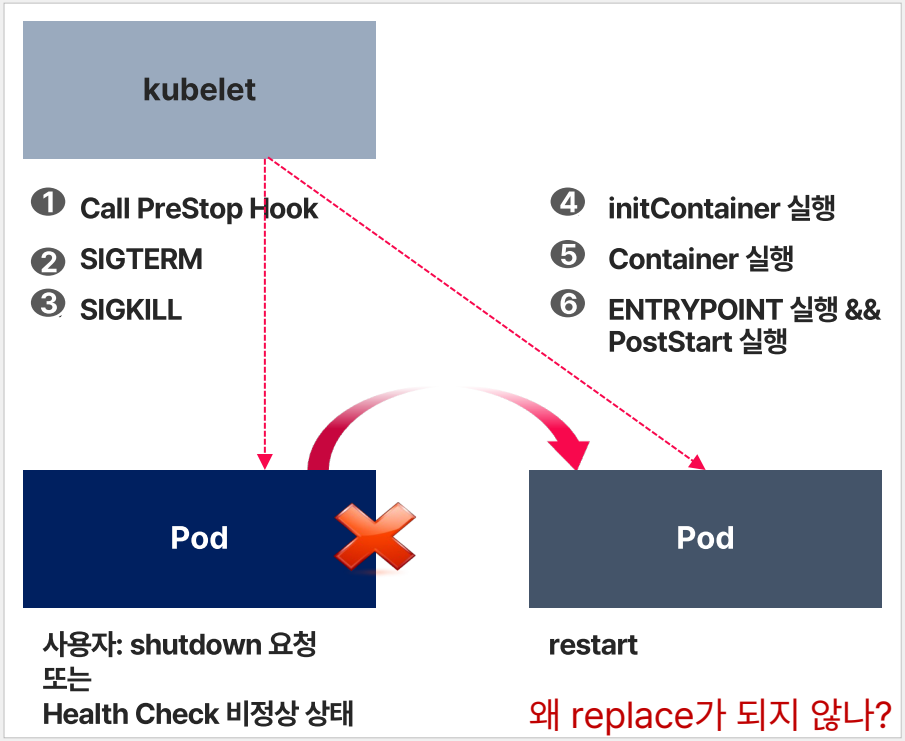
kubernetes in action book 내용 참고

Graceful shutdown

구성 방식

Kubelet은 shutdown 전 deployment resource의 spec.container[0].lifecycle.prestop을 실행하며,
재 실행 시 spec.container[0].lifecycle.postStart hoop을 호출

Graceful shutdown & restart API 호출 흐름



Deployment lifecycle 설정

```
lifecycle:
  postStart:
    exec:
      command:
      - sh
      - -c
      - sleep 30 && echo "Wake up!" > /opt/postStart-done
  preStop:
    httpGet:
      port: 8080
      path: /shutdown
```

- spec.container[0].lifecycle.prestop.httpGet
  shutdown전 Appl.의 /shutdown method를 호출
  /shutdown 은 DB connection release 등의 자원 해지 코드 를 포함한다.
- spec.container[0].lifecycle.postStart.exec
  sleep 30 초 후 실행 하도록 구성

kubernetes in action book 내용 참고

Graceful shutdown

Graceful shutdown 진행 순서

Pod shutdown 시 상태 전환, 시그널, preStop hook을 호출, Pod 종료 과정에 대한 흐름 설명

1. **사용자 요청 또는 Probe 실패**로 인해, **파드(Pod) 삭제 요청**이 API 서버에 전달
2. API 서버는 삭제 요청을 반영하여, etcd에서 해당 파드의 상태를 **Terminating**으로 업데이트
3. 해당 **파드 상태를 Terminating으로 표시**한다.
4. 파드는 **서비스(Endpoint 목록)에서 제거**되어, 더 이상 **외부 트래픽을 받지 않게 됨** (ready: false 상태)
   - 이 시점부터는 **로드밸런서(예: kube-proxy, ingress 등)에서도 제외**된다.
5. kubelet이 파드 종료를 감지하고, 파드 종료 절차를 시작한다:
   - **spec.terminationGracePeriodSeconds (기본 30초)** 동안 종료를 유예 (PreStop 실행 SIGTERM 처리 요청)
   - 이 유예 시간 동안 다음 작업이 **순차적**으로 진행
     1) 각 컨테이너에 lifecycle.preStop 훅이 정의되어 있다면, 컨테이너 내부에서 해당 커맨드를 실행
     2) **SIGTERM 시그널**이 파드 내 **모든 컨테이너에 동시**에 전달
     → 컨테이너 **종료 순서가 중요한 경우**, 개별 컨테이너마다 preStop 설정이 필요 **(hook 동기화 방식 적용)**
6. terminationGracePeriodSeconds 내에 종료되지 않으면, kubelet은 남아있는 프로세스에 **SIGKILL을 보내 강제 종료**
7. 파드가 완전히 종료되면, API 서버에서 파드 정보가 제거되고, kubectl에서는 더 이상 조회되지 않음

```
containers:
- name: app
  lifecycle:
    preStop:
      exec:
        command: ["sh", "-c", "sleep 5"]   # 요청 마무리 시간 확보
```

```
- name: proxy
  lifecycle:
    preStop:
      exec:
        command: ["sh", "-c", "sleep 15"]  # app이 먼저 끝나도록 더 늦게 종료
```

kubernetes in action book 내용 참고

[참고] Prestop 과 PostStart

postStop은 권고하지만 prestart는 initContainer를 활용하도록 권고

구분	권고 정도	핵심 팁
PreStop	**적극 권고**	terminationGracePeriodSeconds를 preStop 실행 시간보다 길게 설정 ▪ **서비스 등록 해제**: 유레카(Eureka)나 콘솔(Consul) 같은 서비스 디스커버리에 자신을 '종료 중'이라고 알릴 때 사용 ▪ **로드밸런서 전파 지연 (중요)**: Pod이 삭제될 때 쿠버네티스 엔드포인트에서 제거되는 시간과 실제 트래픽이 끊기는 시간 사이에 미세한 차이가 발생 이를 방지하기 위해 5~10초 정도 sleep을 주어 트래픽 누수 방지 ▪ **데이터 저장 및 로그 플러시**: 종료 전 버퍼에 쌓인 로그를 전송하거나, 진행 중인 작업을 안전하게 마무리하고 커넥션을 닫을 때 사용
PostStart	**선택적 사용**	애플리케이션 실행과 동시에 돌아가도 무방한 작업에만 사용

kubernetes 1.29 이후부터는
sleep action을 제공

```
lifecycle:
  preStop:
    sleep:
      seconds: 10
```

sleep을 해야 하는 이유

Terminating 으로 상태 변경 시 ETCD Endpoint에서는 PodIP가 제거되지만
실제 IpTables , coreDNS에 반영은 일정 시간이 갭이 발생하므로
이로 인해 SIGTERM으로 즉시 반영 되는 경우 traffic을 받지 못하는 문제 발생 (sleep을 통해 일정 시간 지연 후 SIGTERM 발생)

kubernetes in action book 내용 참고

[실습] PreStop, PostStart Hook

위치: 16.grace-shutdown/deploy.yaml

deploy.yaml

```yaml
lifecycle:
  postStart:
    exec:
      command: ["/bin/sh", "-c", "echo '[postStart] container started: ${USER_NAME}-${SERVICE_NAME}' > /proc/1/fd/1"]
  preStop:
    exec:
      command: ["/bin/sh", "-c", "echo '[preStop] container stopping: ${USER_NAME}-${SERVICE_NAME}' > /proc/1/fd/1 && sleep 10"]
```

/proc/1/fd/1 : PID 1 프로세스의 fd 1 (stdout) 의미

1. 배포하기

```
> kubectl apply -f deploy.yaml
```

2. 로그 확인

```
> k logs -f sk199-myfirst-api-server-57d49cf497-4wp8d
[postStart] container started: sk199-myfirst-api-server


 .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
```

3. pod restart

```
> k  rollout  restart  deployment sk199-myfirst-api-server
```

4. 로그 확인

```
[preStop] container stopping: sk199-myfirst-api-server
2026-03-11T02:48:06.889Z  INFO 1 --- [skalaspringboot] [ionShutdownHook] o.s.b.w.e.tomcat.GracefulShutdown       : Commencing graceful shutdown. Waiting for active requests to complete
...
```

Pod Lifecycle 관리

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

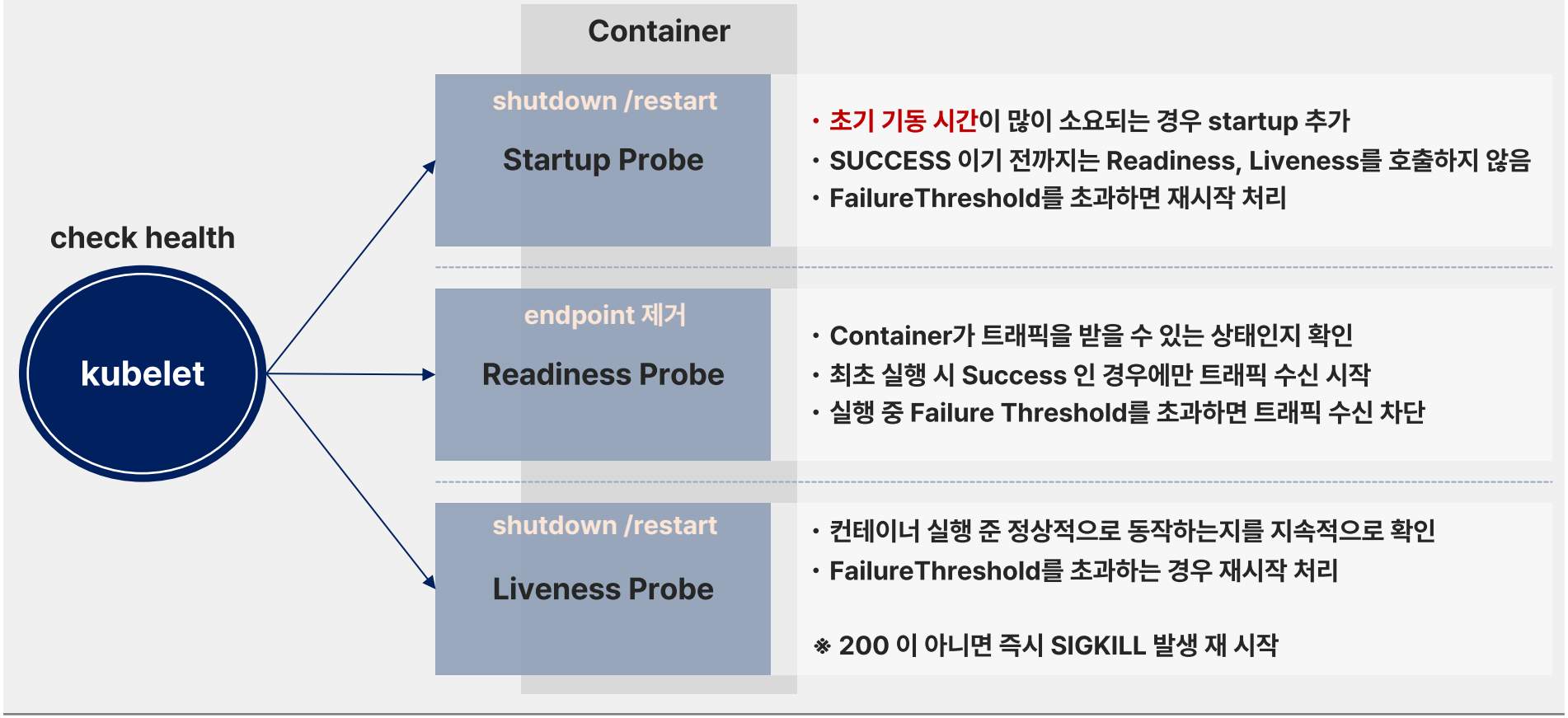
SK 주식회사 C&C

라이프사이클 관리

Appl의 Ready 상태, 서비스 불가 상태(Hang 상태) 또는 비정상 종료로 인한 장애 상태를 자동 감지하고 정상적으로 복구할 수 있도록 지원하기 위한 패턴

예) Appl.이 무한 루프, 교착상태 등

애플리케이션 Auto-Healing을 위한 3가지 Probe 기능

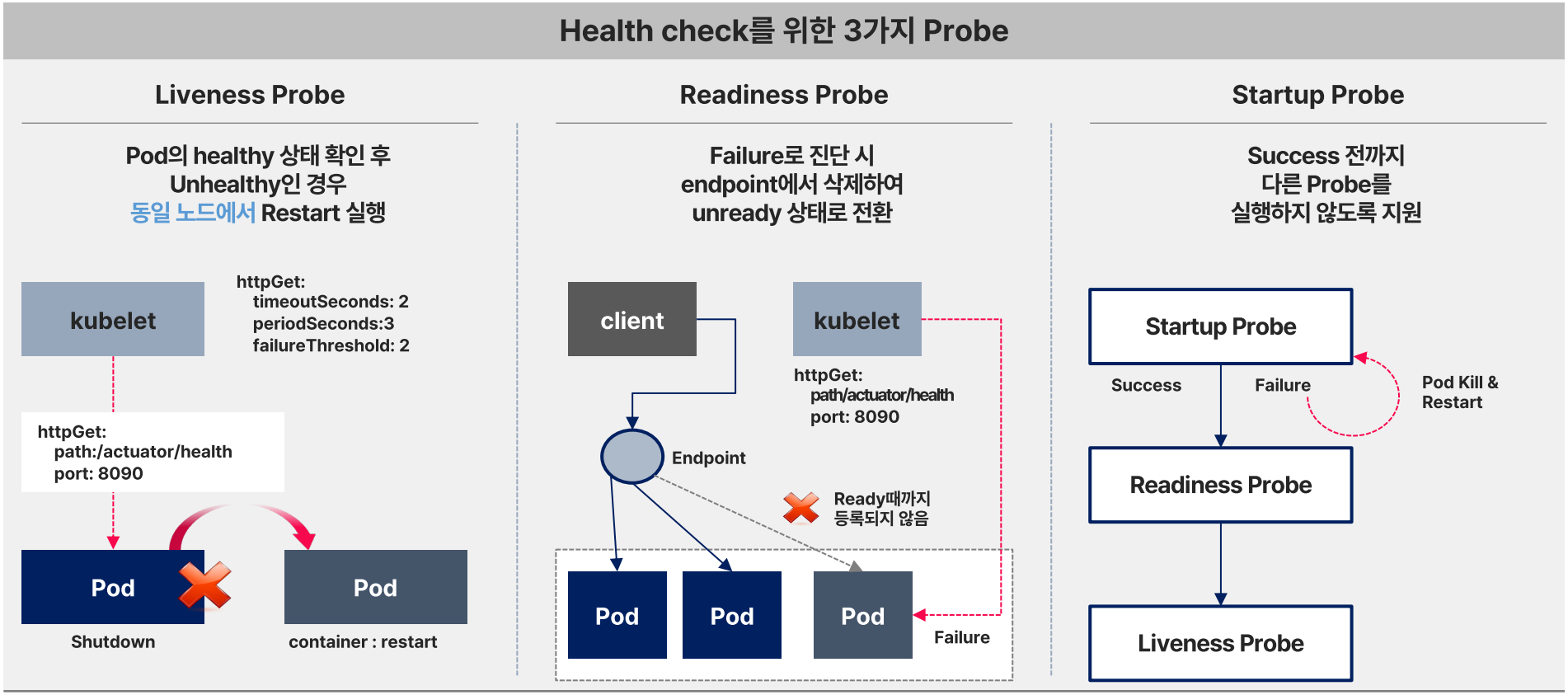


kubernetes in action book 내용 참고

라이프사이클 관리

Prob 동작 방식

Appl의 Ready 상태, 서비스 불가 상태(Hang 상태) 또는 비정상 종료로 인한 장애 상태를 자동 감지하고 정상적으로 복구할 수 있도록 지원하기 위한 패턴



kubernetes in action book 내용 참고

Probe Handler 호출 방식

Handler 호출 방식은 3가지로 나눠서 제공하고 있음

Handler 종류	설명	정상 판단
ExecAction	컨테이너 안에서 명령어를 실행하여 상태 판단	exit code 0
TCPSocketAction	특정 포트에 TCP 연결이 가능한지를 확인	TCP 연결 성공
HTTPGetAction	특정 엔드포인트에 HTTP 요청을 보내 응답 여부로 판단	응답코드 : 200 ~ 399

ExecAction

```yaml
livenessProbe:
  exec:
    command:
    - cat
    - /tmp/healthy
  initialDelaySeconds: 5
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
```

TcpSocketAction

```yaml
livenessProbe:
  tcpSocket:
    port: 3306
  initialDelaySeconds: 5
  periodSeconds: 5
  timeoutSeconds: 1
  failureThreshold: 2
```

HttpGetAction

```yaml
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
    httpHeaders:
    - name: Custom-Header
      value: Awesome
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3
  timeoutSeconds: 2
```

kubernetes in action book 내용 참고

deploy 리소스의 liveness/readiness 설정

위치: cloud/k8s-ddive/10.probe

```yaml
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 3
  failureThreshold: 2
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 3
  failureThreshold: 2
startupProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 3
  failureThreshold: 2
```

startupProbe

- pod 기동 후 30초 이후에 탐침 실행
- 3초 주기로 탐침을 진행하며, 2번 이상 시 실패 시 종료 실행
- SIGTERM 시 앱 종료될때까지 30초 대기 (terminationGracePeriodSeconds=30 sec)

readinessProbe

- startupProbe 처리 후 5초 뒤 탐침 실행
- 3초 주기로 탐침을 진행하며,
  → 2 번 내 정상이면 endpoint 등록
  → 2번 모두 실패 시, 비정상으로 endpoint 제거

livenessProbe

- startupProbe 처리 후 5초 뒤 탐침 실행
- 3초 주기로 탐침을 진행하며, 2번 이상 시 실패시 종료 실행
- SIGTERM 시 앱 종료될때까지 30초 대기 (terminationGracePeriodSeconds=30 sec)

deploy 리소스의 liveness/readiness 설정

spring boot의 readinessProbe와 livenessProbe를 통해 애플리케이션 상태에 대한 자동 복구 기능 지원

```
spec:
  containers:
    - name: my-spring-app
      image: amdp-registry.skala-ai.com/skala25a/sk000-myfirst-api-server:1.0.0
      ports:
        - containerPort: 8080
      livenessProbe:
        httpGet:
          path: /actuator/health/liveness
          port: 8080
        initialDelaySeconds: 5    # 컨테이너 시작 후 5초 뒤 체크 시작
        periodSeconds: 10         # 10초마다 체크
        timeoutSeconds: 3         # 응답 없으면 3초 후 실패 간주
        failureThreshold: 3       # 3번 연속 실패 시 컨테이너 재시작

      readinessProbe:
        httpGet:
          path: /actuator/health/readiness
          port: 8080
        initialDelaySeconds: 5    # 컨테이너 시작 후 5초 뒤 체크 시작
        periodSeconds: 10         # 10초마다 체크
        timeoutSeconds: 3         # 응답 없으면 3초 후 실패 간주
        failureThreshold: 3       # 3번 연속 실패 시 트래픽에서 제외
```

[실습] fastserver.py 내 Health 코드

위치: 00.sample-container/02.fastapi-backend-v2.0/fastserver.py

```python
# Initial state
data = {
    "server_status": "정상",
    "ready_status": "준비 완료"
}


  # ---------------- /api/* ----------------
  @python_router.get("/health/liveness")
  def healthz():
      if data["server_status"] == "정상":
          return JSONResponse({"status": "UP"}, status_code=200)
      else:
          return JSONResponse({"status": "DOWN"}, status_code=400)

  @python_router.get("/health/readiness")
  def ready():
      if data["ready_status"] == "준비 완료":
          return JSONResponse({"status": "READY"}, status_code=200)
      else:
          return JSONResponse({"status": "NOT READY"}, status_code=503)
```

[실습] Liveness & Readiness

1. 아래와 같이 Probe Field를 추가

실습 코드: 04.myfirst-api-server/06.probe

06.probe/fastapi-deploy.yaml 에 추가되어 있음

```
spec:
     containers:
     - name: webserver
       image: skala-registry.skala-ai.com/class-0/skala-webserver:2.0
       imagePullPolicy: Always
       livenessProbe:
         httpGet:
           path: /python/health/liveness
           port: 8080
         initialDelaySeconds: 10
         periodSeconds: 5
         failureThreshold: 3
         timeoutSeconds: 5
       readinessProbe:
         httpGet:
           path: /python/health/readiness
           port: 8080
         initialDelaySeconds: 10
         periodSeconds: 5
         failureThreshold: 3
         timeoutSeconds: 5
```

[실습] Liveness & Readiness

actuator가 제공하는 /actuator/health/readiness 와 liveness 기능을 이용해서 상태 변화에 따른 Pod의 lifecycle 매커니즘을 이해하도록한다.

사전 준비 사항

1. 파일 확인하기

04.myfirst-api-server/06.probe/ 아래에 deploy.yaml 이 있는지 확인한다.

```
> cd ~/workspace/04.myfirst-api-server/06.probe/
> ls fastapi-deploy.yaml
```

2. 배포하기

```
> kubectl apply –f fastapi-deploy.yaml

> kubectl get pod -l app=sk{{학번}}-myfirst-api-server-new
```

[실습] readiness: NotReady로 변경하기 (1)

1. 현재 상태 (터미널 1번)

```
> watch -n 1 "kubectl get pod -l app=skala-myfirst-api-server-new"
Every 1.0s: kubectl get pod -l app=skala-myfirst-api-server-new

NAME                                          READY   STATUS    RESTARTS   AGE
skala-myfirst-api-server-new-7d98bf695f-gftms   1/1     Running   0          114m
```

2. Service의 Endpoint 확인하기 (터미널 2번)

```
> watch -n 1 "kubectl get endpoints -l app=skala-myfirst-api-server-new"
Every 1.0s: kubectl get endpoints -l app=skala-myfirst-api-server-new
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                          ENDPOINTS                                      AGE
skala-myfirst-api-server-new   172.31.13.169:8080,172.31.13.169:8081          175m
```

[실습] readiness: NotReady로 변경하기 (2)

3. Ready 상태 (readiness Probe) 확인 상태 : UP --> DOWN (503 Error)

[실습] readiness: NotReady로 변경하기 (3)

1. 현재 상태 (터미널 1번)

```
> watch -n 1 "kubectl get pod -l app=skala-myfirst-api-server-new"
Every 1.0s: kubectl get pod -l app=skala-myfirst-api-server-new

NAME                                          READY   STATUS    RESTARTS   AGE
skala-myfirst-api-server-new-7d98bf695f-gftms   0/1     Running   0          114m
```

2. Service의 Endpoint 확인하기 (터미널 2번)

```
> watch -n 1 "kubectl get endpoints -l app=skala-myfirst-api-server-new"
Every 1.0s: kubectl get endpoints -l app=skala-myfirst-api-server-new
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                           ENDPOINTS   AGE
skala-myfirst-api-server-new               175m
```

[실습] Readiness=UP 상태로 복구하기 (1)

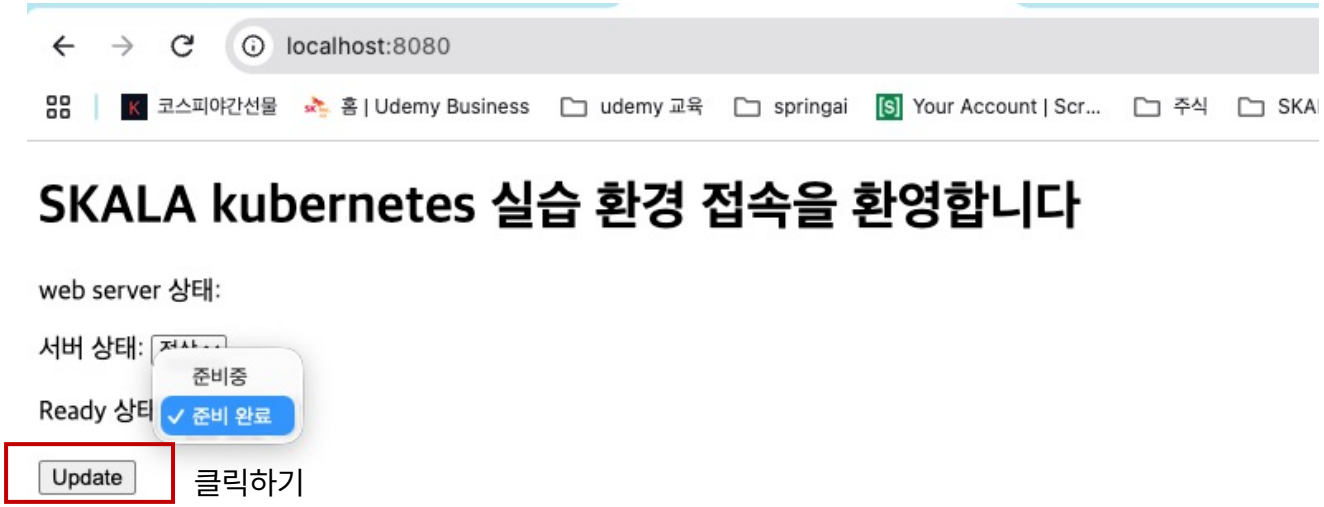
1. pod 의 port-forward 로 연결하기

```
> k port-forward pod/sk000-myfirst-api-server-new-7d98bf695f-gftms 8080:8080
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Handling connection for 8080
```

domain 으로 접근이 불가능한 이유는?

브라우저에서 sk000-ingress.class-?.skala-??.com 으로 접속하는 경우 접속이 되지 않는다.

2. 설정 변경



[실습] Readiness=UP 상태로 복구하기 (2)

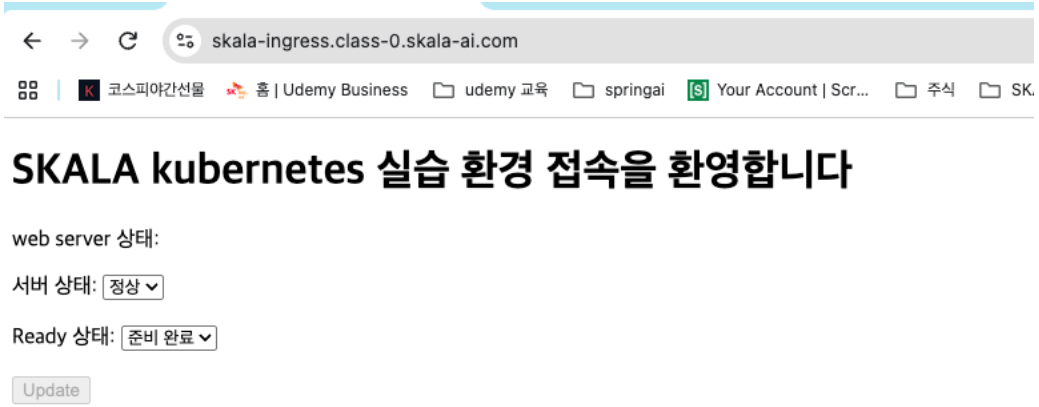
3. 현재 상태 (터미널 1번)

```
> watch -n 1 "kubectl get pod -l app=skala-myfirst-api-server-new"
Every 1.0s: kubectl get pod -l app=skala-myfirst-api-server-new

NAME                                          READY   STATUS    RESTARTS   AGE
skala-myfirst-api-server-new-7d98bf695f-gftms   1/1     Running   0          114m
```

4. Service의 Endpoint 확인하기 (터미널 2번)

```
> watch -n 1 "kubectl get endpoints -l app=skala-myfirst-api-server-new"
Every 1.0s: kubectl get endpoints -l app=skala-myfirst-api-server-new
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                          ENDPOINTS                                      AGE
skala-myfirst-api-server-new   172.31.13.169:8080,172.31.13.169:8081          175m
```

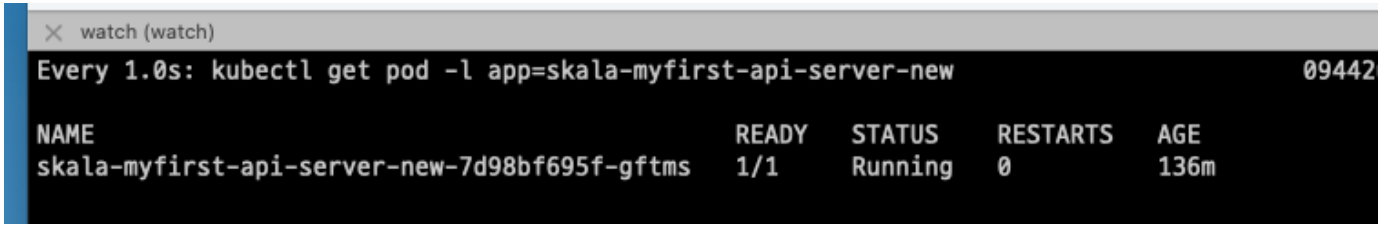
5.Domain 을 통해 접속이 됨

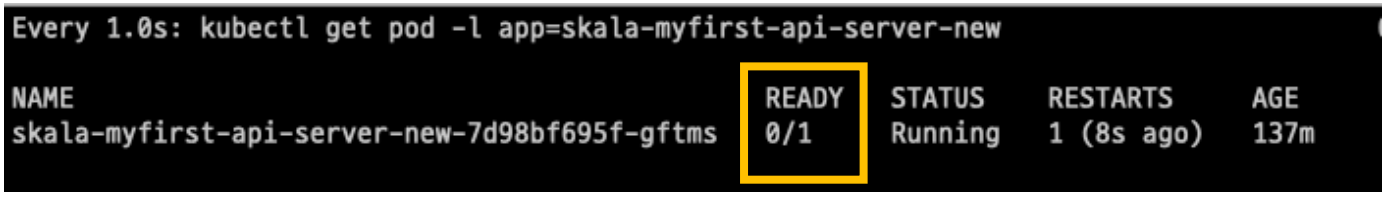


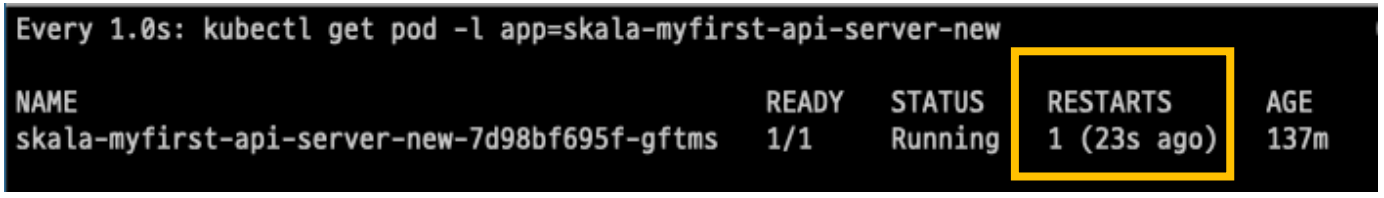
[실습] Readiness Probe : DOWN

1. 서버 상태 (Liveness Probe) 확인 상태 : UP --> DOWN (400 Error)

2. Liveness Probe 400 Error RETURN 시 : shutdown 후 재 시작







KDT Cloud – 5. 쿠버네티스 심화하기

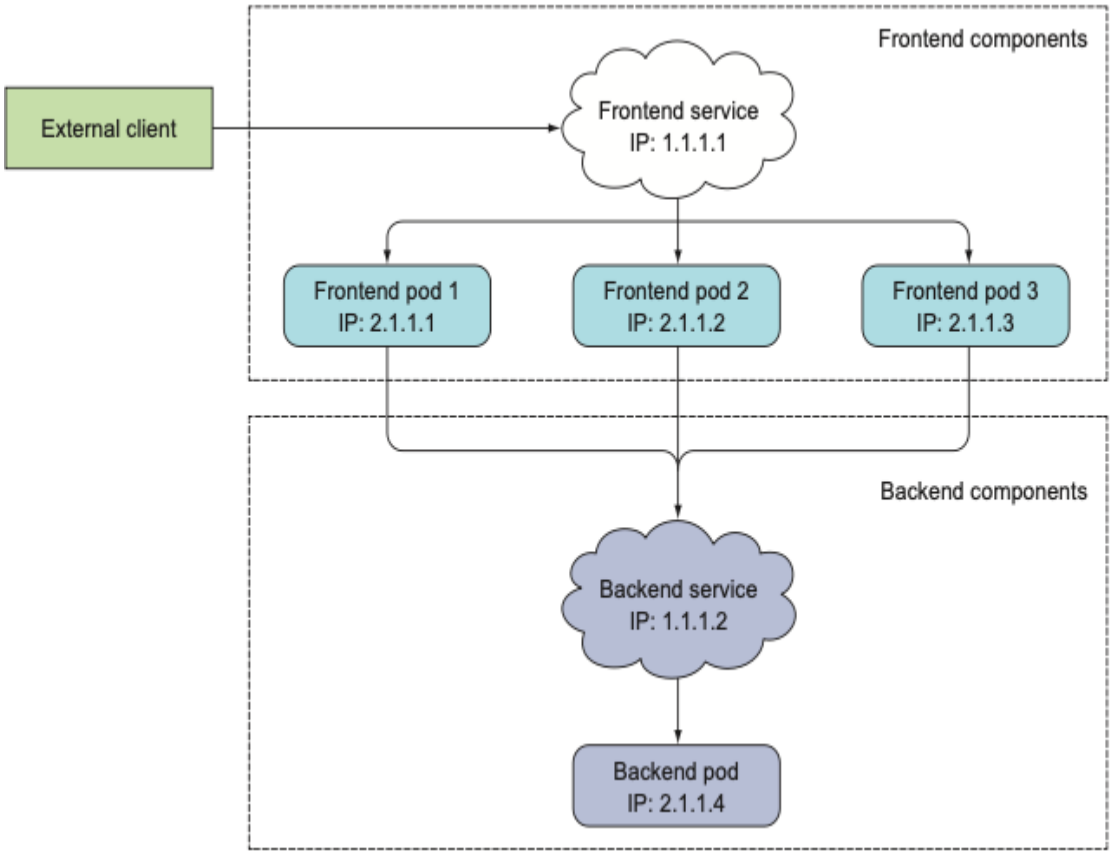
Service 깊이 이해하기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Service

Service는 동적으로 변하는 Pod 집합을 하나의 안정적인 네트워크 엔드포인트로 묶어주는 추상화 객체
Pod는 언제든지 생성·삭제·재시작되고 IP도 바뀌지만, Service는 변하지 않는 접근 지점(VIP/DNS)을 제공



정의

- 여러 Pod 통신을 위한 고정 IP 및 도메일 제공
  - Cluster IP (1.1.1.1)
  - Doman (http://frontend-service:8080)
- 부하 분산 (Load balancing)
- Label 을 통해 Pod 그룹핑

서비스 종류

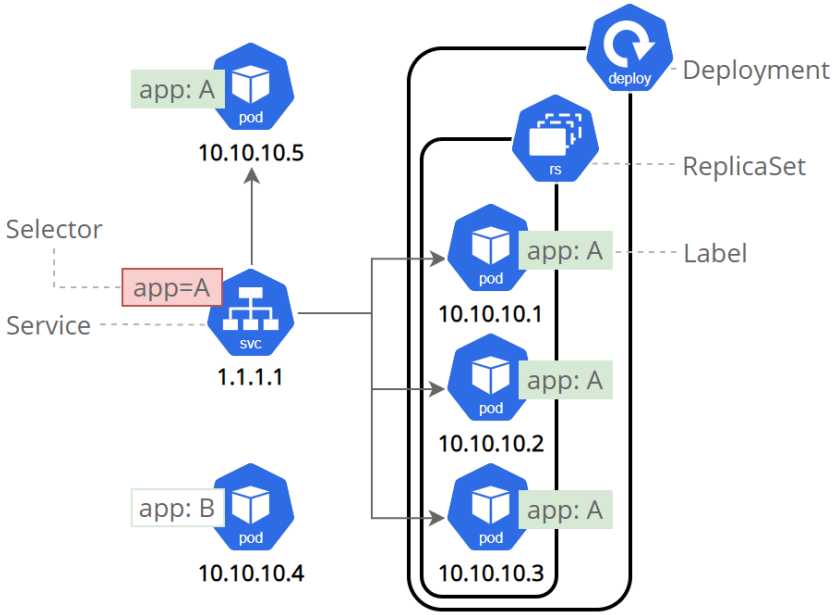
- Cluster IP (기본)
- NodePort
- Load Balancer
- External Name

kubernetes in action book 내용 참고

Service: cluster IP 통신

서비스의 타입을 지정하지 않거나 ClusterIP로 지정하게 되면 Cluster 내부에서 어느 곳에든지 호출 가능한 고정 ClusterIP를 할당

서비스 호출 시 ClusterIP로 호출하는 경우 대응 되는 Pod IP로 전환되어 Pod와 통신 실행




```yaml
apiVersion: v1
kind: Service
metadata:
  name: hell-service
  namespace: skala-practice
spec:
  type: ClusterIP
  selector:
    app: A
  ports:
    - name: http
      port: 8080
      targetPort: 8080
      protocol: TCP
    - name: mgmt
      port: 8081
      targetPort: 8081
      protocol: TCP
```

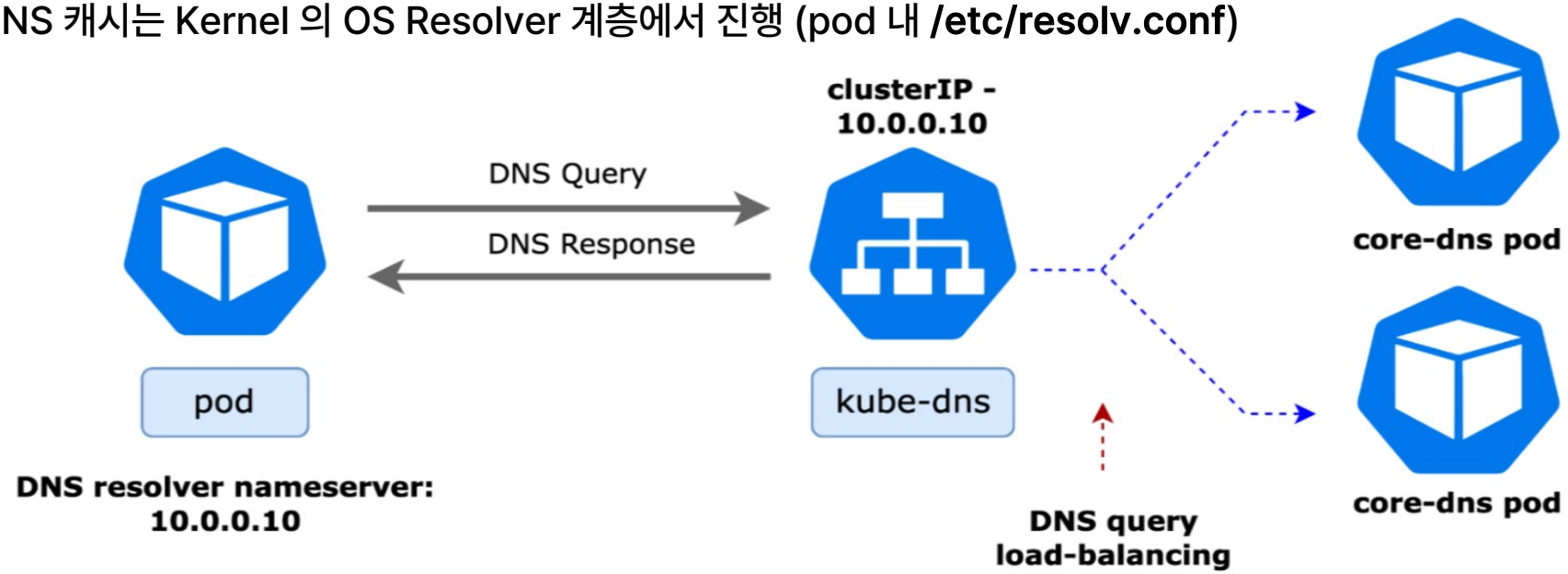
```
$ kubectl get service -o wide
NAME                  TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE   SELECTOR
himang10-deploy-test  ClusterIP   10.100.103.22   <none>        8080/TCP   9h    app=himang10-deploy-test
```

kubernetes in action book 내용 참고

Service : serviceName으로 통신

서비스 이름으로 호출 시에는 CoreDNS로 부터 Cluster IP를 획득

- 이름 구조: `{service-name}.{namespace}.svc.cluster.local:{port-number}`
- DNS 캐시는 Kernel 의 OS Resolver 계층에서 진행 (pod 내 **/etc/resolv.conf**)



호출

http://myfirst-api-server:8080 myfirst-api-server.class-0.svc.cluster.local → 10.100.152.148

http://myfirst-api-server.class-1:8080 myfirst-api-server.class-1.svc.cluster.local → 10.100.152.192

/etc/resolv.conf

```
search class-0.svc.cluster.local svc.cluster.local          # . 개수가 5개 미만이면 선택
nameserver 10.100.0.10                                      # coredns clusterIP
options ndots:5
```

-

kubernetes in action book 내용 참고

[실습] Service Name 실습

위치: 04.service

04.service 에서 service에 endpoints를 수동으로 등록해보기

1. deployment 배포하기

```
> k apply -f deploy.yaml
> k get pod -l app=sk000-myfirst-api-server
NAME                                      READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-86567f4d8c-pq85d   1/1     Running   0          10h
```

2. core-DNS 위치 (namespace = kube-system)

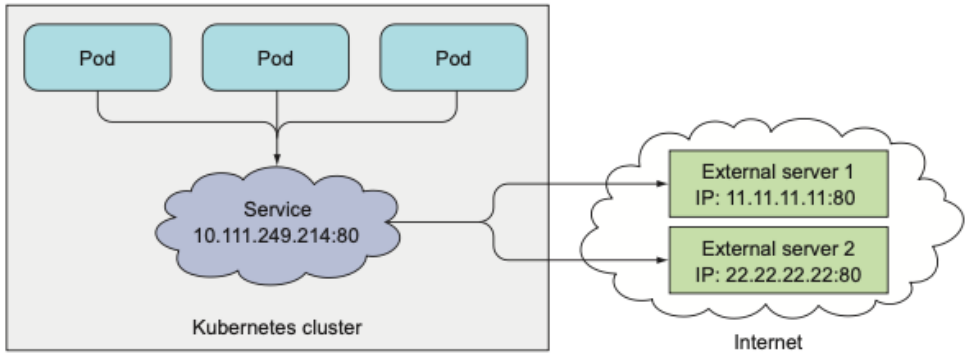
```
> k get pod -o wide -n kube-system ¦ grep coredns
coredns-5477bf6575-qnvl8              1/1     Running   0          9h    192.168.140.172

> k get service kube-dns -n kube-system
NAME       TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)                  AGE
kube-dns   ClusterIP   10.100.0.10   <none>        53/UDP,53/TCP,9153/TCP   45d
```

3. Application에서 Core DNS 찾는 방법

```
> k exec –it sk000-myfirst-api-server-86567f4d8c-pq85d   -- /bin/bash

> cat /etc/resolv.conf
search class-0.svc.cluster.local svc.cluster.local cluster.local ap-northeast-2.compute.internal
nameserver 10.100.0.10
options ndots:5
```

Endpoints

Service와 실제 트래픽을 처리하는 Pod들을 연결해 주는 일종의 동적 IP/포트 주소록




```yaml
apiVersion: v1
kind: Service
metadata:
  name: sk000-myfirst-api-server
  namespace: skala-practice
spec:
  type: ClusterIP
  selector:
    app: sk000-myfirst-api-server
  ports:
    - name: http
      port: 8080
      targetPort: 8080
      protocol: TCP
    - name: mgmt
      port: 8081
      targetPort: 8081
      protocol: TCP
```

```yaml
apiVersion: v1
kind: Endpoints
metadata:
  name: sk000-myfirst-api-server
  namespace: skala-practice
subsets:
- addresses:
  - ip: 172.31.45.106
    nodeName: ip-172-31-45-122.ap-northeast-
2.compute.internal
    targetRef:
      kind: Pod
      name: sk000-myfirst-api-server-5ddb555658-pdrdf
      namespace: skala-practice
  ports:
  - name: http
    port: 8080
    protocol: TCP
```

kubernetes in action book 내용 참고

EndpointSlices

EndpointSlice는 Endpoint의 성능 문제를 해결하기 위해 나온 개선된 구조
기존: service 1개, Pod 300 이면 Endpoint 객체 1개에 300개 IP → pod IP 변경 시 모든 Endpoint 갱신 (by Kube-proxy)
개선: service 1개, Pod 300이면 EndpointSlices 3개로 나눠짐 → Pod IP 변경 시 1/3 수준의 갱신 속도

Endpoints 사용 시

- Service 1개
- Pod 300개
- Endpoints 객체 1개에 300개 IP
- Pod 하나 바뀌면:
  - ✓ Endpoints 전체 업데이트
  - ✓ kube-proxy full rule rebuild

EndpointSlice 사용 시

- Service 1개
- EndpointSlice 3개 (100 + 100 + 100)
- Pod 하나 바뀌면:
  - ✓ 해당 Slice 1개만 변경
  - ✓ kube-proxy가 **변경분만 반영**
  - ✓ 전체 Endpoints 재계산 없음

```
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  generateName: sk000-myfirst-api-server-
  namespace: skala-practice
  labels:
    kubernetes.io/service-name: sk199-myfirst-api-server
addressType: IPv4
ports:
  - name: http
    protocol: TCP
    port: 8080
  - name: mgmt
    protocol: TCP
    port: 8081
endpoints:
  - addresses:
      - "172.31.45.106"
    conditions:
      ready: true
    nodeName: ip-172-31-45-122.ap-northeast-
2.compute.internal
    targetRef:
      kind: Pod
      name: sk199-myfirst-api-server-5ddb555658-pdrdf
      namespace: skala-practice
```

참고: generateName은 kubectl apply에는 사용할 수 없으면 create에만 사용 가능

kubernetes in action book 내용 참고

EndPoints 수동 지정하기

Service에서 Selector 를 지정하지 않고 Endpoint를 수동으로 생성하는 경우 수동으로 Pod 대신 다른 서비스로 라우팅 가능

pod selector가 없는 서비스

```yaml
apiVersion: v1
kind: Service
metadata:
  name: sk000-myfirst-api-server
  namespace: class-0
spec:
  ports:
    - name: http
      protocol: TCP
      port: 8080
      targetPort: 8080
    - name: mgmt
      protocol: TCP
      port: 8081
      targetPort: 8081
```

수동으로 Service의 end-point 지정

```yaml
apiVersion: v1
kind: Endpoints
metadata:
  name: sk000-myfirst-api-server
  namespace: class-0
subsets:
  - addresses:
      - ip: 10.244.1.23   # 실제 Pod IP
      - ip: 10.244.2.45   # 다른 Pod IP
    ports:
      - name: http
        port: 8080
        protocol: TCP
      - name: mgmt
        port: 8081
        protocol: TCP
```

kubernetes in action book 내용 참고

[실습] 수동 endpoints 등록하기 (1/3)

위치: 10.service

1. deploy.yaml 에 replicas=2로 변경하기

```
> k scale --replicas=2 -f deploy.yaml
```

2. 배포 실행

```
> k get pod -l app=sk000-myfirst-api-server -o wide
NAME                                        READY   STATUS    RESTARTS   AGE   IP
skala-myfirst-api-server-5df6c96558-g9m9f   1/1     Running   0          90s   192.168.148.42
skala-myfirst-api-server-5df6c96558-ss89d   1/1     Running   0          50m   192.168.142.230
```

3. endpoints에 podIP 수동 등록하기

```
> vi sk000-myfirst-api-server
apiVersion: v1
kind: Endpoints
metadata:
  name: sk000-myfirst-api-server
  namespace: class-cloud
subsets:
  - addresses:
      - ip: 192.168.148.42        # 실제 Pod IP
      - ip: 192.168.142.230       # 다른 Pod IP
…
```

[실습] 수동 endpoints 등록하기 (2/3)

위치: 10.service

4. service 등록

```
> k apply -f no-selector-service.yaml
> k get endpointslices
> k exec –it
```

5. 앞서 배포했던 curl-pod를 통해 Pod를 호출해보자

(아래의 IP 와 Service Name은 자신이 배포한 Pod와 Service 정보로 호출해야 합니다 반드시)

```
> k exec -it skala-curl-86567f4d8c-pq85d -- /bin/sh

# container 내부 (curl-deploy)
# Cluster IP로 호출하기 (실패)
~ $ curl http://10.100.72.160:8080/api/users ¦ jq
curl: (7) Failed to connect to 10.100.72.160:8080 after 0 ms: Could not connect to server

# Service Name으로 호출하기 (실패)
~ $ curl http://skala-myfirst-api-server:8080/api/users ¦ jq
curl: (7) Failed to connect to skala-myfirst-api-server:8080 after 0 ms: Could not connect to server

# Pod IP로 호출하기 (성공)
~ $ curl http://192.168.148.42:8080/api/users ¦ jq
[{"id":1,"name":"홍길동","email":"hong@sk.com"},{"id":2,"name":"김철수
","email":"kim@sk.com"},{"id":3,"name":"이영희","email":"lee@sk.com"}]

~ $ exit
```

[실습] 수동 endpoints 등록하기 (3/3)

위치: 10.service

6. endpoints 수동 등록

```
> k apply –f no-selector-endpoints.yaml
> k get -f no-selector-endpoints.yaml
NAME                      ENDPOINTS                                                  AGE
skala-myfirst-api-server 192.168.148.42:8080,192.168.142.230:8080,192.168.148.42:8081 + 1 more...   64s
```

7. 앞서 배포했던 curl-pod를 통해 Pod를 호출해보자

(아래의 IP 와 Service Name은 자신이 배포한 Pod와 Service 정보로 호출해야 합니다 반드시)

```
> k exec -it skala-curl-86567f4d8c-pq85d -- /bin/sh

# container 내부 (curl-deploy)
# Cluster IP로 호출하기 (성공)
~ $ curl http://10.100.72.160:8080/api/users ¦ jq
[{"id":1,"name":"홍길동","email":"hong@sk.com"},{"id":2,"name":"김철수","email":"kim@sk.com"}

# Service Name으로 호출하기 (성공)
~ $ curl http://skala-myfirst-api-server:8080/api/users ¦ jq
[{"id":1,"name":"홍길동","email":"hong@sk.com"},{"id":2,"name":"김철수","email":"kim@sk.com"}

# Pod IP로 호출하기 (성공)
~ $ curl http://192.168.148.42:8080/api/users ¦ jq
[{"id":1,"name":"홍길동","email":"hong@sk.com"},{"id":2,"name":"김철수
","email":"kim@sk.com"},{"id":3,"name":"이영희","email":"lee@sk.com"}]

~ $ exit
```

[실습] kubernetes.default.svc.cluster.local ?

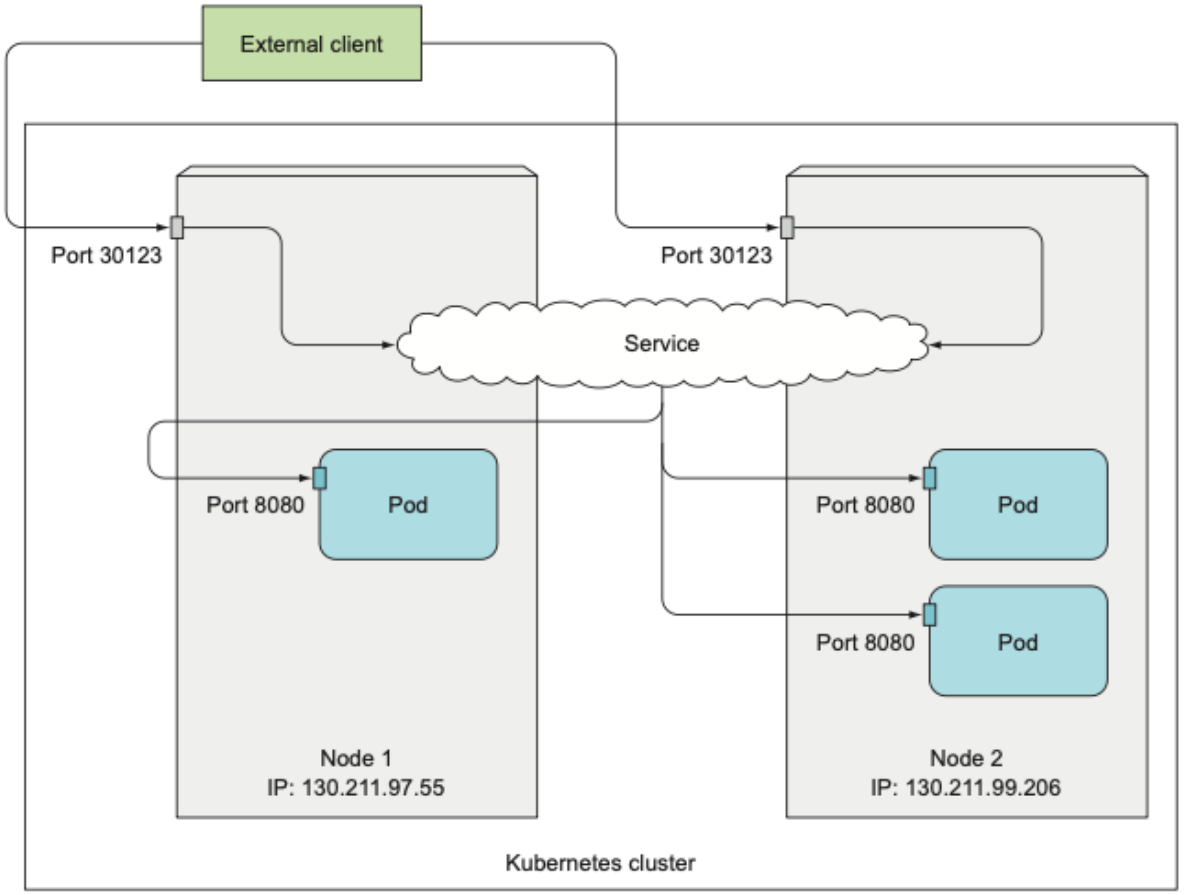
kubernetes.default.svc.cluster.local은 cluster 내부에서 API Server를 호출하기 위한 방법

```
# kubernetes service를 확인하기
> k get service kubernetes -o yaml -n default
## selector 가 없다??

# kubernetes endpoints를 확인하기
> k get endpoints kubernetes -o yaml -n default
apiVersion: v1
kind: Endpoints
metadata:
  creationTimestamp: "2026-08-16T12:05:43Z"
  labels:
    endpointslice.kubernetes.io/skip-mirror: "true"
  name: kubernetes
  namespace: default
  resourceVersion: "14729090"
  uid: 2e4f3f3f-747e-43ac-96b4-9b22f18539e0
subsets:
- addresses:
  - ip: 192.168.166.159
  - ip: 192.168.99.188
  ports:
  - name: https
    port: 443
    protocol: TCP
```

Master Node 내 kube-API 접속을 위한 IP 정보를 수동을 등록

Service Type: Node Port

노드 네트워크과 Container Network 간 연결을 지원하는 방식




```yaml
apiVersion: v1
kind: Service
metadata:
  name: api-server
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30123
```

미 지정시 자동 할당

지원 노드 포트 범위
30000 ~ 32767

Kubernetes Cluster 외부에서 특정 노드의 포트를 통해 서비스 접근 하도록 지원
AWS CLB 기반 또는 Private Cluster를 구성하는 경우 nginx 기반 Ingress Controller를 Node Port로 외부에 노출할때 활용

kubernetes in action book 내용 참고

[참고] Node Port 흐름

NodePort로 들어온 패킷은 L4(Netfilter NAT) 단계에서 Pod IP로 DNAT 실행

아래의 흐름은 Kube-proxy에 의해 사전 설정

```
External Client
  ↓
<NodeIP>:30123
  ↓
Node 커널 (netfilter)
  ↓
PREROUTING (nat)
  ↓
KUBE-NODEPORTS
  ↓
KUBE-SVC-XXXX   (Service 체인)
  ↓
KUBE-SEP-YYYY   (Endpoint 체인)
  ↓
DNAT → PodIP:8080
```

1. 패킷 유입 (L3)

- 목적지 IP: Node IP
- 목적지 Port: NodePort (30123)

```
Client → NodeIP:30123
```

2. Netfilter PREROUTING

- Node 커널의 netfilter nat 테이블에서 처리
- 여기서 kube-proxy가 만든 룰이 동작

```
-A PREROUTING -p tcp --dport 30123 -j KUBE-NODEPORTS
```

3. KUBE-NODEPORT 체인

- Node Port를 기준으로 직접 SVC Chain 호출
- ClusterIP 대신 Node Port 기반으로 Pod 라우팅

```
-A KUBE-NODEPORTS -p tcp --dport 30123 -j KUBE-SVC-XXXX
```

※ DNAT (Destination Network Address Translation)

[참고] Node Port 와 Host Port

Node Port는 커널(Netfilter)이 가로채는 포트이고, Host Port는 노드에서 실제 프로세스가 점유하는 포트

NodePort의 특징

- 노드에 어떤 프로세스도 리스닝하지 않음
- kube-proxy가 만든 iptables / ipvs 규칙이 처리
- DNAT으로 Pod IP:Port로 변환

```
<NodeIP>:30123
  ↓ (netfilter DNAT)
PodIP:8080
```

EC2에서 30123 포트를 listen하는 프로세스는 없다

HostPort의 특징

- 노드의 8080 포트를 실제로 점유
- 해당 노드에 1개의 Pod만 사용 가능
- Docker의 -p 8080:80 과 동일한 개념

```
NodeIP:8080
  ↓ (process listen)
Container:80
```

EC2에서 30123 포트를 listen하는 프로세스는 없다

연결 방식

동일하게 Netfilter를 통과하지만 NodePort는 KUBE-NODEPORTS Filter로 통과하고
HostPort는 별도 Filter를 통과하지 않고 Process 호출로 연결

[실습] NodePort 실습

위치: 10.service

1. nodeport-service.yaml 배포하기

```
> k apply -f  nodeport-service.yaml
>kubectl get service sk000-myfirst-api-server-nodeport
NAME                                 TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)                         AGE
sk000-myfirst-api-server-nodeport    NodePort   10.100.224.229   <none>        8080:30733/TCP,8081:32555/TCP   18s
```

2. node IP로 접속하기

```
> k get node
NAME                                               STATUS   ROLES    AGE    VERSION
ip-172-31-12-19.ap-northeast-2.compute.internal    Ready    <none>   5h13m  v1.31.3-eks-59bf375
ip-172-31-12-7.ap-northeast-2.compute.internal     Ready    <none>   5h13m  v1.31.3-eks-59bf375
```

3. pod 내에서 node Port를 접속하기

```
> k exec -it sk000-curl-86567f4d8c-pq85d -- /bin/sh
# 첫번째 node로 호출해보기
> curl http://ip-172-31-12-19.ap-northeast-2.compute.internal:30733/api/users ¦ jq

# 다른 노드로 호출해보기
> curl http://ip-172-31-12-7.ap-northeast-2.compute.internal:30733/api/users ¦ jq
```

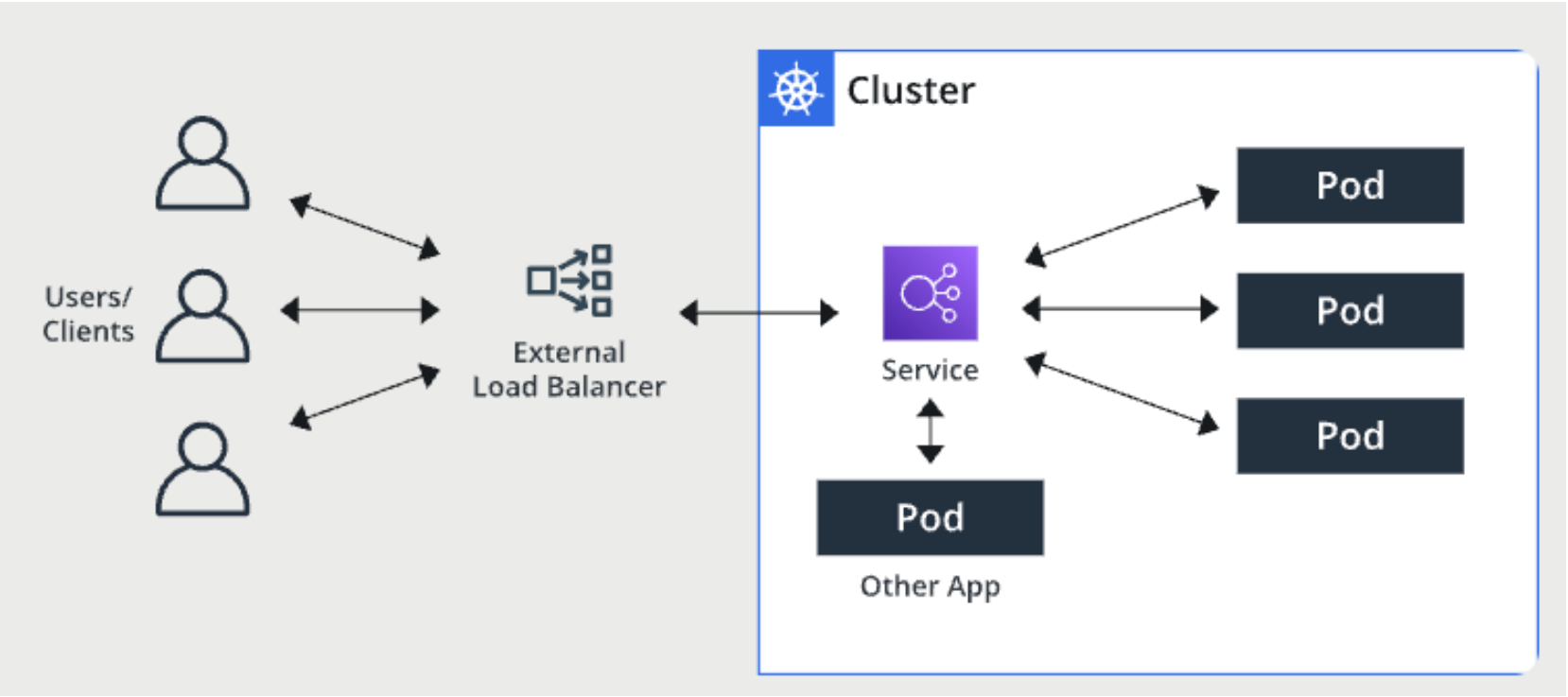
4. node-port delete 하기

```
> kubectl delete -f nodeport-service.yaml
```

Service Type: LoadBalancer

LoadBalancer(L4)는 클라우드 제공자의 외부 로드밸런서를 자동으로 생성하고,
그 로드밸런서가 Service(→ Pod 집합) 로 트래픽을 전달하도록 연결해주는 타입

외부에서 접근 가능한 공식 진입점(External IP)을 만들어주는 Service



kubernetes in action book 내용 참고

외부 로드 밸런스 연동 방식

NLB/ALB가 k8s workload(pod)로 트래픽을 전달하는 방식은 보통 **Target Type**으로 구분

구분	Instance Target Type	IP Target Type
Target Group에 등록되는 대상	**EC2 Worker Node**	**VPC 내 Pod IP**
특징	Kubernetes 기본 구조	AWS 전용 구조
트래픽 경로	NLB → NodeIP:NodePort → Pod	NLB → Pod IP 직접
NodePort 사용	O	X
Fargate 사용	불가	가능
AWS 명칭	instance	ip
지원 대상	NLB, CLB	NLB only
default	O	X

```
# target Type: instance ¦ ip
metadata:
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: instance
```

kubernetes in action book 내용 참고

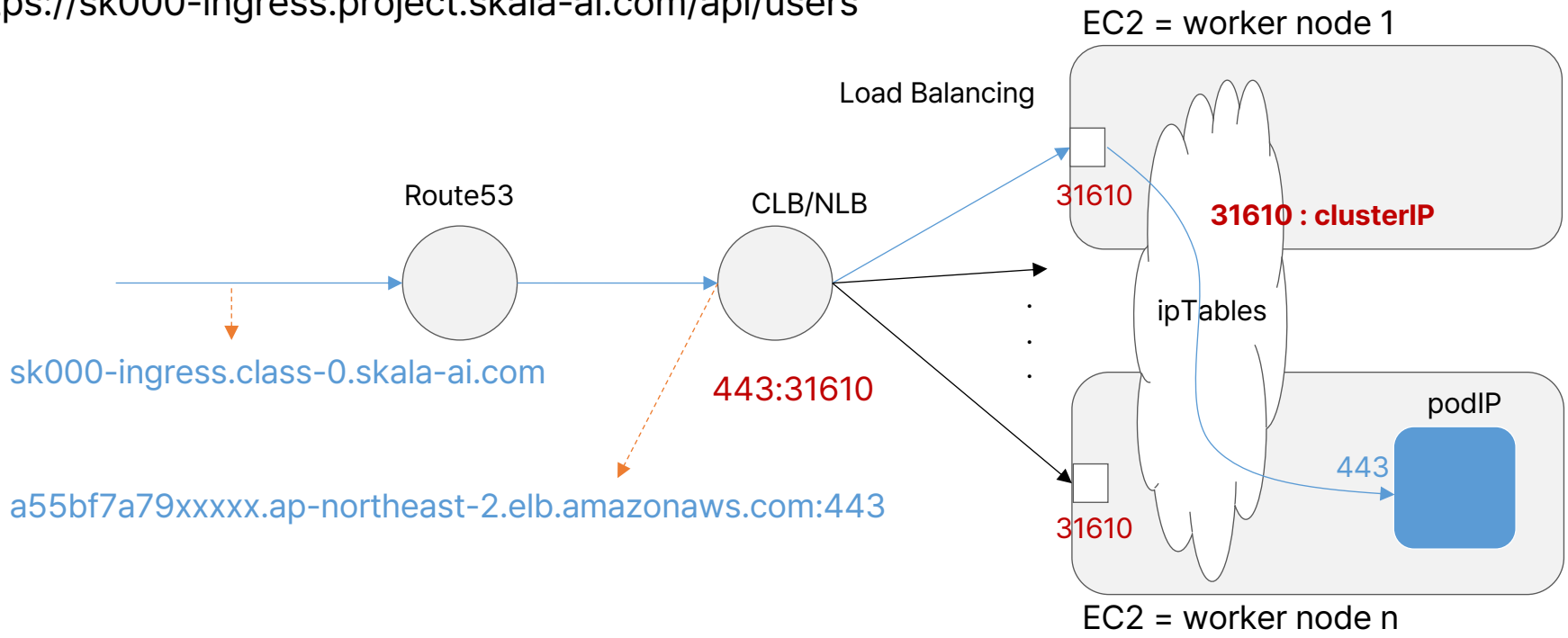
CLB/NLB LoadBalancer 흐름

Load Balancer Type 설정 시 Cloud Controller Manager를 통해 외부 노출을 위한 External IP 및 DNS Name 할당 및 CLB 의 Forwarding 정보 업데이트

Route53 → CLB → EC2:NodePort → Netfliter → Pod IP:443 → ingress Controller (ALB 역할)

chrome browser Call

https://sk000-ingress.project.skala-ai.com/api/users



kubernetes in action book 내용 참고

LoadBalancer 선택 시 주의 사항

주의 사항

비용

- Service 하나당 LB 1개 생성 (비용 증가)
- 대규모 서비스를 외부 노출 시 Ingress 권장

한계

- 경로 기반 라우팅 미지원
- TLS 종료 미 지원

언제 사용하나?

적합

- 외부에서 직접 접근해야 하는 단일 서비스
- HTTP/HTTPS 외 특정 포트 (22, 로 접근 필요시
- TCP/UDP 기반 서비스 (FTP 등)

비적합

- 다수의 HTTP 서비스
- 경로/도메인 기반 라우팅 필요
- 비용 최적화가 중요한 경우

서비스	일반 포트
Ingress Controller	80/443
SSH / SFTP	22
MySQL / MariaDB	3306
PostgreSQL	5432
Oracle DB	1521
Redis	6379
MongoDB	27017
Kafka	9092

kubernetes in action book 내용 참고

[실습] Loadbalancer 확인하기

실제 적용되어 있는 Loadbalancer를 확인해보자

ingress controller 를 위한 loadbalancer 확인하기

```
>  k get service ingress-nginx-controller -n ingress-nginx
NAME                       TYPE           CLUSTER-IP       EXTERNAL-IP          PORT(S)                      45d
ingress-nginx-controller   LoadBalancer   10.100.208.215   xxxxx                80:31677/TCP,443:30923/TCP
```

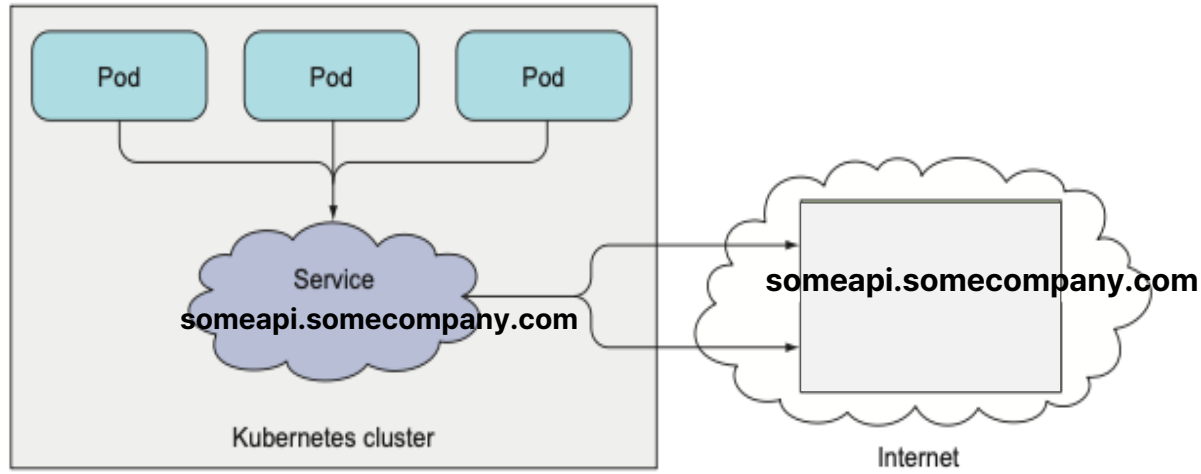
Service Type: ExternalName

클러스터 내부 Service 이름을 외부 DNS 이름에 그대로 매핑해주는 Service 타입

네트워크 포워딩 없이, DNS 레벨에서 이름만 바꿔주는 Service

ExternalNames

```
apiVersion: v1
kind: Service
metadata:
  name: external-service
spec:
  type: ExternalName
  externalName: someapi.somecompany.com
  ports:
  - port: 80
```



kubernetes in action book 내용 참고

Service Type: ExternalName

클러스터 내부 Service 이름을 외부 DNS 이름에 그대로 매핑해주는 Service 타입

특징

- Service 생성 시 ClusterIP 없음
- Endpoints / kube-proxy / iptables 사용 안 함
- CoreDNS가 CNAME 응답만 반환
- 로드 밸런싱 없음

사용처

- 외부 DB / 외부 API를 내부 Service 이름처럼 쓰고 싶을 때
- 코드나 설정을 바꾸지 않고 대상만 교체하고 싶을 때
- 네트워크 경로는 그대로 두고 이름 추상화만 하고 싶을 때

type: ExternalName

```yaml
apiVersion: v1
kind: Service
metadata:
  name: skala-external-name
  namespace: class-0
spec:
  type: ExternalName
  externalName: skala-fileserver.skala-ai.com
  ports:
    - port: 443
```

호출 흐름

```
Pod
  ↓ DNS 질의
skala-external-name
  ↓
CoreDNS
  ↓ (CNAME 응답)
skala-fileserver.skala-ai.com
  ↓
외부 DNS 해석
  ↓
외부 서버 IP로 직접 연결
```

kubernetes in action book 내용 참고

[실습] ExternalName 서비스 등록하기 (1/2)

위치: 10.service

서비스 이름으로 skala-fileserver.skala-ai.com 을 호출해보자

1. externalName.yaml을 확인해보자

```
> cat externalName.yaml
apiVersion: v1
kind: Service
metadata:
  name: skala-external-name
  namespace: class-0
spec:
  type: ExternalName
  externalName: skala-fileserver.skala-ai.com
  ports:
    - port: 443
```

2. 배포 실행

```
> k apply -f externalName.yaml
k get service sk000-external-name
NAME                   TYPE           CLUSTER-IP   EXTERNAL-IP                      PORT(S)   AGE
sk000-external-name    ExternalName   <none>       skala-fileserver.skala-ai.com    443/TCP   13m
```

[실습] ExternalName 서비스 등록하기 (2/2)

위치: 10.service

3. externalName으로 외부 API를 호출해보자

```
> k exec -it skala-curl-86567f4d8c-pq85d -- /bin/sh

# container 내부 (curl-deploy)
/ # curl -k -H "Host: skala-fileserver.skala-ai.com" https://skala-external-name:443
<!doctype html>
<html lang="ko">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>skala File Server</title>
<link rel="stylesheet" href="/style.css" />
</head>
<body>
  <header class="topbar">
…
~ $ exit
```

[후속 처리] 실습 자원을 삭제해보자

위치: 10.service

```
> cd workspace/skala-ddive-kube
> k delete -f 10.service
```

KDT Cloud – 5. 쿠버네티스 심화하기

Pod Security Context & Isolation

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

SecurityContext

Pod 또는 Container가 Linux에서 어떤 사용자·권한·보안 제약을 가지고 실행될지를 정의하는 Kubernetes 설정

Linux Kernel이 제공하는 **UID/GID, capabilities, seccomp, SELinux, privilege** 등의 보안 기능을 컨테이너에 적용하도록 설정하는 인터페이스

Security Context에서 설정하는 것

- 컨테이너의 프로세스에서 실행 할 수 있는 UID/GID 지정
- 컨테이너가 Root로 실행되는 것을 방지
- 컨테이너를 privileged mode로 실행해서 노드의 커널에 대한 모든 액세스 권한을 부여
- privileged mode 중 특정 권한 부여
- 컨테이너를 강력하게 잠그기 위해 SELinux 옵션 설정하기
- 프로세스가 컨테이너의 파일 시스템에 쓰지 못하게 하기

[실습] SecurityContext

위치: 19.security/pod-as-nonroot.yaml

Pod에 UID/GID 설정

조건	설명
runAsUser	컨테이너 프로세스를 실행할 사용자 ID
runAsGroup	프로세스를 실행할 기본 그룹 ID
runAsNonRoot=true	Non root 사용자만 실행 허용

포드에 UID를 설정하면 모든 컨테이너에 적용 확인

```
> k apply -f pod-as-nonroot.yaml
> k exec -it skala-security-context -- /bin/sh
#/ id
uid=1000 gid=1000 groups=1000
#/ ps -eaf
   PID  USER                  TIME  COMMAND
     1  1000                  0:00  sleep 1h
    10  1000                  0:00  sh
    16  1000                  0:00  ps -eaf

#/ exit
> k delete -f pod-as-nonroot.yaml
```

pod-as-nonroot.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sk000-pod-as-nonroot
  namespace: class-0
spec:
  containers:
    - name: main
      image: alpine
      command: ["/bin/sh", "-c", "exec tail -f /dev/null"]
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        runAsGroup: 1000
```

[실습] SecurityContext : privileged

노드의 커널에 대한 모든 액세스 권한을 얻기 위한 설정

위치: 19.security/pod-as-privileged.yaml

예를 들어 kube-proxy Pod는 iptables 규칙 수정을 위한 동작 필요

조건	설명
privileged: true	커널 기능 접근 허용 (cap_net_admin 포함) 커널/컨테이너 격리 측면에서 제약을 완화
hostNetwork: true	호스트 네트워크 인터페이스에 접근 가능
iptables 명령 사용	Alpine이면 apk add iptables 필요
(선택) hostPID: true	호스트 프로세스 접근 (디버깅 시 유용)

Node의 iptables 검색하기

```
> k apply -f pod-run-as-privileged.yaml
> k exec -it skala-security-context -- /bin/sh
#/ iptables -L -n -v
#/ iptables-save > rules.txt
#/ cat rules.txt
#/ exit
> k delete -f pod-run-as-privileged.yaml
```

pod-run-as-privileged.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: skala-security-context
  namespace: class-0
spec:
  hostNetwork: true
  containers:
    - name: main
      image: alpine
      command:
        - /bin/sh
        - -c
        - |
          apk add iptables
          exec tail -f /dev/null
      securityContext:
        privileged: true
```

SecurityContext: Capability

위치: 19.security/pod-as-capability.yaml

컨테이너에 필요한 Linux Kernel 권한만 선택적으로 부여하기 위한 설정

예를 들어 iptables 규칙을 변경하려면 CAP_NET_ADMIN 권한 필요

조건	설명
capabilities.add: NET_ADMIN	네트워크 관리 권한 추가 (iptables 등)
hostNetwork: true	호스트 네트워크 인터페이스에 접근 가능
iptables 명령 사용	Alpine이면 apk add iptables 필요
privileged: false	모든 권한을 주지 않고 필요한 Capability만 부여

Node의 iptables 검색하기

```
> k apply -f pod-run-as-privileged.yaml
> k exec -it skala-security-context -- /bin/sh
#/ iptables -L -n -v
#/ iptables-save > rules.txt
#/ cat rules.txt
#/ exit
> k delete -f pod-run-as-privileged.yaml
```

pod-run-as-capability.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: skala-linux-capability
  namespace: class-0

spec:
  hostNetwork: true
  containers:
  - name: main
    image: alpine
    command:
      - /bin/sh
      - -c
      - |
        apk add iptables
        exec tail -f /dev/null
    securityContext:
      privileged: false
      capabilities:
        add:
          - NET_ADMIN
```

SecurityContext: Capability

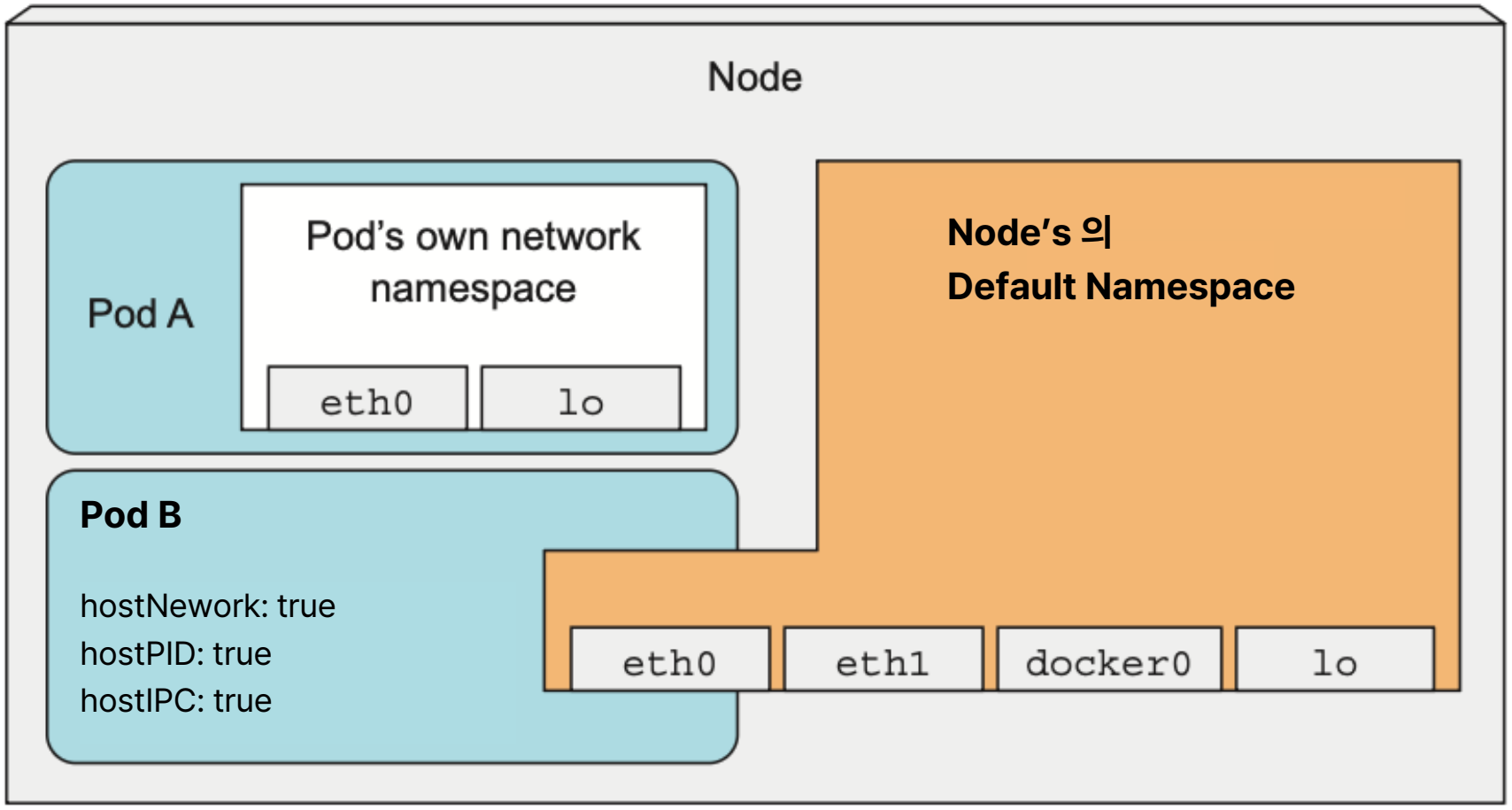
리눅스에서 root만 할 수 있었던 작업들을 **세분화된 권한(capabilities) 단위로 나누어,**
필요한 권한만 컨테이너나 프로세스에 부여할 수 있도록 한 기능

```yaml
securityContext:
  capabilities:
    add: ["NET_ADMIN", "SYS_TIME"]
    drop: ["ALL"]
```

Capability	설명	위험도
CHOWN	파일 소유자 변경	낮음
DAC_OVERRIDE	파일 권한 무시 (모든 파일 읽기/쓰기)	높음
DAC_READ_SEARCH	읽기 권한 우회	높음
FOWNER	자신이 소유하지 않은 파일도 변경	중간
CAP_FSETID	setuid/setgid 비트 유지	중간
CAP_KILL	다른 프로세스 종료	중간
CAP_NET_BIND_SERVICE	1024 이하 포트 바인딩	낮음
NET_ADMIN	네트워크 설정 변경 (iptables, interface 설정 등)	**높음**
NET_RAW	RAW 소켓 사용 (ping, nmap 등)	**높음**
SYS_ADMIN	**거의 root 수준의 권한** (mount, umount, hostname 변경 등)	**매우 높음**
SYS_TIME	시스템 시간 변경	높음
SYS_MODULE	커널 모듈 로딩/언로딩	**매우 높음**
SETUID / CAP_SETGID	UID/GID 변경	높음

Node Level 특권 부여

컨테이너의 고유한 네임스페이스를 사용하는 것이 아닌 Host Network, PID, IPC를 사용하는 방법



[실습] Pod 내에서 HostNode의 namespace 사용하기

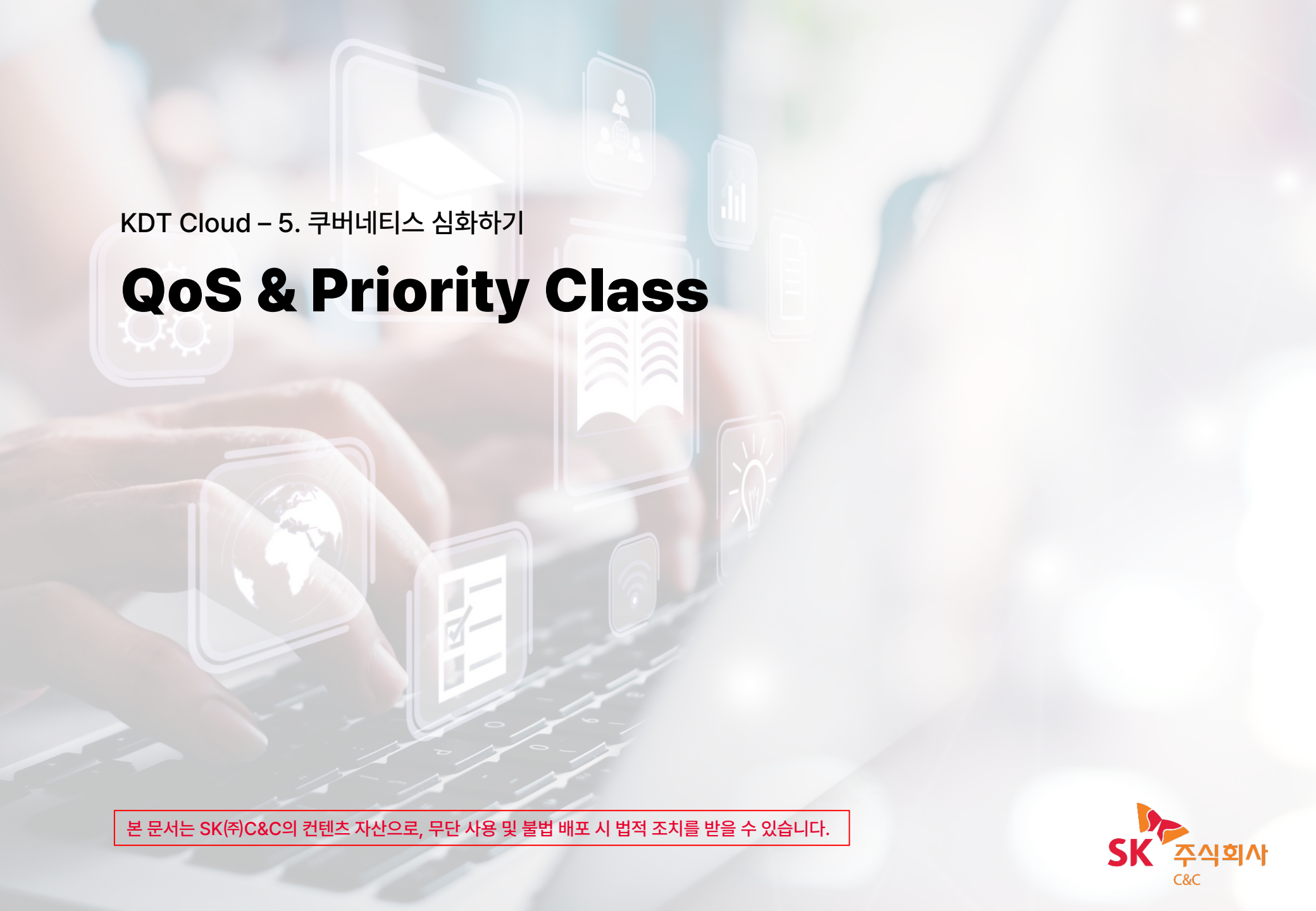
위치: 19.security/host-viewer.yaml

컨테이너의 고유한 네임스페이스를 사용하는 것이 아닌 Host Network, PID, IPC를 사용하는 방법

접속 하기

```
> kubectl apply -f host-viewer.yaml
> kubectl exec -it skala-security-context -- /bin/sh

# 호스트 이름확인하기
#/ hostname
#/ id

# 호스트의 프로세스 목록 확인
#/ ps aux

# 호스트의 컨테이너 런타임 확인
#/ ps aux ¦ grep -E 'containerd¦crio'

# 호스트의 루트 파일시스템 접근
#/ ls -la /host

# systemd 서비스 상태 확인 (필요한 경우)
#/ chroot   /host  systemctl status kubelet
#/ chroot   /host  systemctl status containerd

#/ exit
> k delete -f host-viewer.yaml
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: skala-security-context
spec:
  hostPID: true
  hostIPC: true
  hostNetwork: true
  restartPolicy: Always
  containers:
    - name: log-viewer
      image: ubuntu:24.04
      securityContext:
        privileged: true
      command:
        - /bin/sh -c
        - ¦
          apt-get update && apt-get install -y procps && \
          exec tail -f /dev/null
      volumeMounts:
        - name: host-root
          mountPath: /host
          readOnly: true
…
```



KDT Cloud – 5. 쿠버네티스 심화하기

QoS & Priority Class

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

QoS Class

QoS(Quality of Service) 클래스는 각 Pod에 대해 리소스 보장 수준을 정의하는 분류 체계
이는 노드에 자원이 부족할 때 어떤 Pod가 우선적으로 제거(eviction) 대상이 되는지를 결정하는 중요한 기준

QoS 클래스	requests	limits	보장 수준	우선순위
Guaranteed	O (동일)	O (동일)	높음	가장 높음
Burstable	O	O (다름) 또는 일부만 O	중간	중간
BestEffort	X	X	없음	가장 낮음

노드에 리소스 부족 발생 시,

kubelet은 QoS에 따라 순차적으로 Pod를 종료

먼저

BestEffort → Burstable → 마지막으로 Guaranteed

리소스 우선순위 설정을 통해 중요한 서비스의 안정성 보장 가능

QoS Class

QoS(Quality of Service) 클래스는 각 Pod에 대해 리소스 보장 수준을 정의하는 분류 체계

이는 노드에 자원이 부족할 때 어떤 Pod가 우선적으로 제거(eviction) 대상이 되는지를 결정하는 중요한 기준

1. Guaranteed (보장됨)

조건:
모든 컨테이너가 requests와 limits를 모두 지정하고, 두 값이 같을 때.

```yaml
resources:
  requests:
    cpu: "500m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "128Mi"
```

특징:

- 가장 높은 우선순위를 가짐.
- 자원 부족 시 가장 마지막에 evict 됨.
- 완전한 리소스 보장

1. Bustable

조건:
requests와 limits가 둘 다 있지만 값이 다르거나, requests만 설정한 경우

```yaml
resources:
  requests:
    cpu: "250m"
    memory: "64Mi"
  limits:
    cpu: "500m"
    memory: "128Mi"
```

특징:

- 리소스는 어느 정도 보장되나, 필요 시 limits까지 사용
- Guaranteed보다는 낮은 우선순위
- 자원 부족 시 중간 우선순위로 evict 대상

3. BestEffort

조건:
컨테이너에 requests와 limits 모두 설정하지 않은 경우.

```yaml
resources: {}
```

특징:

- 자원 보장이 없음.
- CPU/메모리를 많이 사용할 수 있으나, 자원 부족 시 가장 먼저 evict됨.
- 테스트나 부가적인 작업에 사용됨.

[리뷰] QoS Class

위치: 20.qos/deploy.yaml

Request/Limit 을 설정해서 BestEffort → Burstable → Guaranteed로 전환하면서 상태를 확인해보자

1. BestEffort 배포

```
kubectl apply -f deploy.yaml
kbuectl describe pod -l app=sk000-myfirst-api-server
```

```
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Topology Spread Constraints: skala.io/cloud:DoNotSchedule when max skew 1 is exceeded for
```

2. Burstable 배포

```
kubectl apply -f resource-deploy.yaml
kbuectl describe pod -l app=sk000-myfirst-api-server
```

```
QoS Class:                   Burstable
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=
                             node.kubernetes.io/unreachable:NoExecute
```

3. Guaranteed 배포

resource-deploy.yaml에서 request/limit의 cpu/memory 설정 값을 동일하게 지정 **(limit 을 request 에 맞춰라)**

```
kubectl apply -f resource-deploy.yaml
kbuectl describe pod -l app=sk000-myfirst-api-server
```

```
QoS Class:                   Guaranteed
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-re
                             node.kubernetes.io/unreac
```

```
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "100m"
    memory: "128Mi"
```

Priority Class

Priority Class는 Pod의 우선순위를 정하는 메커니즘
클러스터에 리소스가 부족할 때 어떤 Pod을 먼저 유지하고 어떤 Pod을 제거(evict)할지를 판단하는 기준으로 사용

특징:

- **스케줄링 대기열 정렬:**
  스케쥴 대기중인 여러 파드가 있는 경우 우선순위 수치가 높은 Pod를 앞으로 이동
- **Preemption(선점) 프로세스 (자원이 부족한 경우):**
  - 스케쥴러는 하나의 노드를 선택후 우선순위가 낮은 파드 검색
  - 낮은 우선순위 파드 추출 (Evition) : SIGTERM 발송 및 강제 종료
  - 자원 확보 및 스케쥴링
  - 추출된 Pod는 다시 스케쥴링 대기 상태(Pending)에서 스케쥴링 대기

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: important-app
spec:
  priorityClassName: high-priority
  containers:
  - name: app
    image: nginx
```

QoS Class 의 우선 순위는 동일 PriorityClass 에 대해 QoS 기준으로 우선 순위 선정
(priority class를 지정하지 않는 Pod 의 priority = 0)

Pod 이름	PriorityClass	Value	결과
A	low-priority	1000	리소스 부족 시 가장 먼저 제거
B	high-priority	100000	리소스 부족해도 유지 시도
C	없음	(기본값)	globalDefault가 있다면 그 값 사용

value 필드의 **정수 값이 클수록 더 높은 우선순위**를 의미합니다 (최대 10^9, 10억까지 설정 가능).

Priority Class 정의

Priority Class는 명시적으로 아래와 같이 지정 가능하며, 미 적용 시 globalDefault Priority Class 적용

```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 100000
globalDefault: false
description: "PriorityClass는 중요한 워크로드에 사용됩니다."
```

필드명	설명
value	우선순위 값 (정수). 값이 클수록 더 높은 우선순위
globalDefault	true로 설정하면, PriorityClass가 명시되지 않은 Pod에 기본으로 적용
description	PriorityClass의 설명으로, 사람이 읽기 쉽게 작성

[리뷰] PriorityClass

위치: 20.qos/priority-deploy.yaml

cloud/k8s-ddive/11.qos

만들어져 있는 prority Class 를 검색하고, deploy.yaml에 priorityClassName을 지정해보자

1. 현재 priorityClass 검색해보기

```
kubectl get priorityclass
```

2. Burstable 배포

```
kubectl apply -f priority-deploy.yaml
kubectl describe pod –l app= sk199-myfirst-api-server
```

```
Name:                 sk199-myfirst-api-server-6dbd559f98-pcb98
Namespace:            skala-practice
Priority:             100000
Priority Class Name:  high-priority
Service Account:      default
Node:                 ip-172-31-33-60.ap-northeast-2.compute.internal/172.31.33.60
```

priority Class Name을 지정하지 않은 deployment는 Priority =0

3. 전체 priorityClass 할당 정보 확인하기

```
kubectl get pods -A -o custom-columns=NAMESPACE:.metadata.namespace,NAME:.metadata.name,\
PRIORITY_CLASS:.spec.priorityClassName,PRIORITY:.spec.priority
```

[참고] Node Pressure 판단 기준

자원 종류	임계값 (기본값)	의미
memory.available	< 100Mi	사용 가능한 메모리가 100MiB 미만일 때
nodefs.available	< 10%	루트 파일 시스템 남은 용량이 10% 미만일 때
imagefs.available	< 15%	컨테이너 이미지가 저장된 디스크 용량이 15% 미만일 때
nodefs.inodesFree	< 5%	루트 파일 시스템의 Inode가 5% 미만일 때

kubelet은 Node Pressure 발생 시 QoS + prioity 에 따라 Evict 실행

Evict 결정 기준

QoS를 기준으로 우선 선별하여 축출 그룹을 선정하고 여기서 Priority Class 기준으로 우선 순위 선정

축출 순위	자원 사용 상태 (QoS 기준)	해당 QoS 클래스	그룹 내 축출 판별 기준 (우선순위)
1순위 (최우선 축출)	**Request 초과 사용** 자원을 과도하게 쓰는 그룹	**BestEffort** Request가 0이므로 항상 초과 **Burstable** Request 초과 사용 중인 경우	**1.** PriorityClass가 **낮은** 파드 우선 **2.** 우선순위가 같다면 Request 대비 초과 사용량이 큰 파드 우선
2순위 (차순위 축출)	**Request 이내 사용** 1순위 후에도 자원이 부족	**Burstable** Request 이하 사용 중인 경우	**1.** PriorityClass가 **낮은** 파드 우선
3순위 (최후 보호)	**보장된 자원 범위 내 사용**	**Guaranteed** Request = Limit 설정	최후까지 보호 시스템 컴포넌트를 제외하고 최후의 순간에만 Evict 고려

수고하셨습니다.

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C